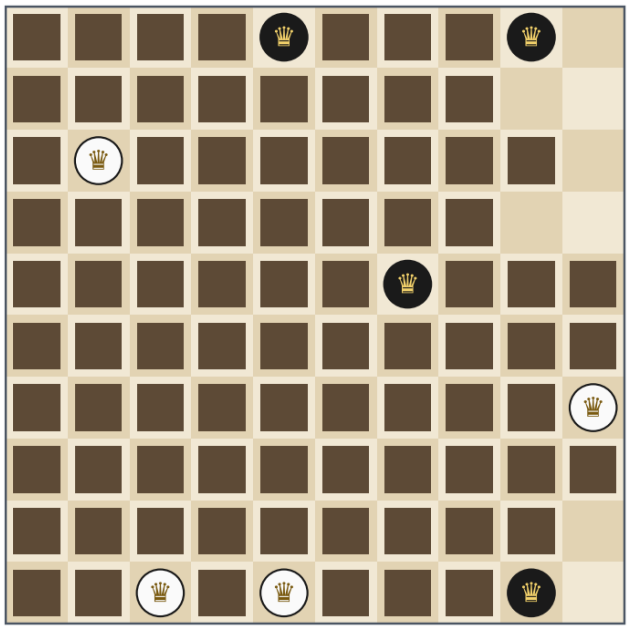
人工智能基础导论 · 第三次课程作业

亚马逊棋博弈搜索算法研究报告

（人工智能算法）

α-β 搜索、蒙特卡洛树搜索与自对弈学习的实现与比较



课程名称： 人工智能基础导论

学　　院： 信息科学与工程学院

班　　级： 自动化 2405 班

学　　号： 20242524

姓　　名： 宋宇浩

2026 年 11 月

代码仓库：https://github.com/NEUMelon/amazon-ai

在线试玩：https://neumelon.github.io/amazon-ai/

摘 要

亚马逊棋（Game of the Amazons）规则简单，但每一步由“移动一个后”和“射出一支箭”两部分组成，开局一步就有 2176 种合法走法，是研究大分支因子博弈搜索的典型平台。本文以 10×10 标准亚马逊棋为对象，在统一的引擎与统一的领域知识（Lieberum 式领地评估、本文设计的“可分离评估”着法排序、分隔后的填格策略）之上，实现并比较了两类经典的人工智能方法：带迭代加深、主变例搜索、置换表、杀手与历史启发和束宽剪枝的 α-β 博弈树搜索[1]，以及采用 UCT/PUCT 选择、渐进展开与评估截断模拟的蒙特卡洛树搜索（MCTS）[2,3]。此外还实现了两种轻量级学习方法：以自对弈棋谱训练的线性 softmax 走子策略（作为 MCTS 的先验），以及用 Texel 方法拟合的评估函数权重。所有算法都被打包进一个可直接在浏览器中运行的单文件网页游戏，引擎在 Web Worker 中运行，界面实时展示双方领地、候选着法、搜索统计、主变例与局势曲线。

实验部分共进行了 2144 局主实验对局（循环赛、α-β 消融、MCTS 消融、算力缩放与同预算对比）、384 局独立种子的重复实验，以及决策质量、搜索效率与随机性测试。主要结果如下：（1）在 200 ms/步的循环赛中，完整 α-β（+427 Elo，相对一步贪心）与 MCTS-PUCT（+362）处于同一档，含重复实验在内的 160 局直接对局中 α-β 得 60.0%；不使用领域知识的纯随机 MCTS 比一步贪心还弱 771 Elo。（2）α-β 中作用最大的是着法排序与评估函数，束宽加倍显著更强，而置换表、杀手启发与 PVS 没有显著影响。（3）MCTS 的强度主要来自先验与短模拟：换成 UCT 选择损失约 390 Elo，随机模拟到终局几乎全负。（4）两类方法的强弱随时间预算翻转：50—100 ms 时 α-β 显著占优，800 ms 时 MCTS 显著占优。（5）学习得到的走子策略在离线指标上优于手工排序，但作为先验的棋力提升不稳定（5 批共 160 局合计得分 46.2%，与 50% 无显著差别）；Texel 权重降低了胜负预测的交叉熵，却让 α-β 变弱了约 290 Elo。失败案例分析中发现了第一版程序“分隔判定”的逻辑漏洞（漏掉敌对的后相邻的情形），本文修复后重跑了全部实验，并保留第一版数据，如实报告该漏洞对结果的影响。

本文没有提出新算法，贡献在于对已有方法的复现、在同一平台上的系统比较、可分离评估这一工程设计及决策过程的可视化。所有实验数据均来自真实运行，原始记录随代码一并提交；文中对 AI 工具的使用、未完成与无法核实的部分均作了说明。全部代码、数据与报告源文件公开于 https://github.com/NEUMelon/amazon-ai ，网页游戏可在 https://neumelon.github.io/amazon-ai/ 在线试玩。

关键词：亚马逊棋；α-β 剪枝；蒙特卡洛树搜索；博弈树搜索；束宽剪枝；Texel 调参

Abstract

The Game of the Amazons has simple rules, but every move combines a queen move and an arrow shot, giving 2176 legal moves in the opening position; this makes it a standard testbed for game-tree search with a very large branching factor. Using 10×10 Amazons and a single engine with shared domain knowledge (a Lieberum-style territory evaluation, a "separable evaluation" move ordering designed in this work, and a common filling policy after separation), this report implements and compares two classical families of methods: alpha-beta search with iterative deepening, principal variation search, a transposition table, killer and history heuristics and beam pruning; and Monte Carlo tree search (MCTS) with UCT/PUCT selection, progressive widening and evaluation-cutoff playouts. Two lightweight learning methods are added: a linear softmax move policy trained on self-play games (used as the MCTS prior) and evaluation weights fitted with Texel tuning. Everything is packaged as a single-file browser game; the engine runs in a Web Worker, and the interface shows territory, candidate moves, search statistics, principal variations and a win-rate curve.

The evaluation comprises 2144 games (a round robin, alpha-beta and MCTS ablations, time scaling and equal-budget comparisons), 384 replication games with fresh seeds, and decision-quality, efficiency and variance tests. At 200 ms per move, full alpha-beta (+427 Elo relative to a one-ply greedy player) and PUCT-MCTS (+362) are in the same tier, while MCTS with pure random playouts is 771 Elo below greedy. Move ordering and the evaluation function matter most for alpha-beta, and doubling the beam width is significantly stronger, whereas the transposition table, killer moves and PVS show no significant effect. MCTS relies on its prior and on short playouts. The ranking flips with the time budget: alpha-beta is significantly stronger at 50–100 ms and MCTS at 800 ms. The learned policy beats hand-made ordering on offline metrics but its gain in play is unstable (46.2% over 160 games, not significantly different from 50%), and the Texel weights predict outcomes better yet make alpha-beta about 290 Elo weaker. Failure analysis revealed a logic bug in the first version's separation test (adjacent enemy queens were missed); it was fixed, all experiments were rerun, and the first-version data are kept to report the bug's impact. No new algorithm is claimed; all data come from real runs and are released with the code at https://github.com/NEUMelon/amazon-ai (playable online at https://neumelon.github.io/amazon-ai/).

Keywords: Game of the Amazons; alpha-beta pruning; Monte Carlo tree search; game-tree search; beam search; Texel tuning

编写说明

一、报告内容。 本报告是《人工智能基础导论》第三次作业，主题为“以人工智能算法为核心的游戏项目”。本报告选择 10×10 亚马逊棋（Game of the Amazons），实现了 α-β 博弈树搜索（PVS、迭代加深、置换表、杀手与历史启发、束宽剪枝）、蒙特卡洛树搜索（UCT/PUCT、渐进展开、评估截断）以及两种轻量级学习方法，开发了可在浏览器中直接运行、能展示 AI 决策过程的游戏程序，并通过两千余局对局实验对两类搜索算法及其组件进行了系统比较。正文按老师模板的六个一级章节组织。

二、提交材料。 （1）本报告（Word 与 PDF）；（2）游戏程序 `amazons.html`（单文件，双击即可运行）；（3）源代码包（引擎、网页、实验、学习、分析、测试与报告排版脚本）；（4）运行说明 `README.md`（与附录 C 一致）。

三、数据来源。 报告中的所有实验数据均来自本项目程序在作者使用的云端容器中的实际运行，原始对局记录（JSONL 格式）随源代码包一并提交；表格由分析脚本直接从原始记录生成，图表由绘图脚本生成，均未手工修改数值。凡是基于有限数据、有限计算量得出的结论，正文均给出置信区间或说明其局限。少量参数（MCTS 的先验温度与探索系数）在与正式实验分开的 16 个开局上做过试调，1.5 节与 5.1.3 节如实说明。

四、贡献界定。 本报告使用的搜索与学习算法均来自已有文献，本报告没有提出新的搜索算法。1.5 节逐项说明了哪些是已有方法、哪些是本人复现实现、哪些是本项目自己设计的部分（“可分离评估”着法排序、统一比较实验与可视化界面）。其中可分离评估是在开发中针对排序瓶颈设计的近似方法，作者未见到完全相同的公开描述，但思想与已有的“两阶段选择”做法相近，因此不宣称其为原创算法。

五、参考文献。 参考文献按 GB/T 7714—2015 顺序编码制著录，共 421 条，全部经过在线数据库检索核验，著录信息取自检索结果；未能核验的候选文献一律未引用。核验方法与记录见附录 D。需要说明的是，核验只能确认文献存在且著录正确，部分通过主题检索补充的文献，正文的引用停留在其研究主题层面，作者未逐篇阅读全文（附录 D 末尾说明）。

六、AI 工具使用。 本项目在选题建议、代码编写、实验运行、文献检索核验与报告起草中大量使用了生成式 AI 工具（Anthropic 公司的 Claude）。使用方式、AI 犯过的错误与验证手段在 3.7 节如实说明。第六章“个人学习和实践心得”亦应本人要求由 AI 根据项目的实际过程撰写。

七、未完成或无法核实的部分。（1）没有与外部亚马逊棋强程序（如文献中的 Invader 等）对弈，所有 Elo 仅有相对意义；（2）没有实现深度神经网络与 AlphaZero 式自对弈训练；（3）分隔后的填格阶段使用一个简单的贪心策略，没有实现精确的残局求解；（4）大部分对比实验每组只有 32 局，统计效力有限，只有四组关键对比做了重复验证（5.6.3 节）；（5）参考文献中通过主题检索补充的条目未逐篇阅读全文；（6）只在 Chromium 浏览器中测试了网页游戏。以上各项均在正文相应位置标注。

八、符号与排版。主要符号与术语见附录 A。图、表、公式、代码清单按章编号（如“图 5-3”“表 2-1”“式 2-7”“代码 4-2”）。棋盘坐标采用列字母 a—j、行数字 1—10 的记法（白方在下方），着法记为“出发格-目的格/射箭格”，例如 d1-d7/g7。

目 录

摘 要.....I
Abstract.....III
编写说明.....V
一、问题描述和需求分析.....1
1.1 选题背景与动机.....1
1.1.1 棋类博弈在人工智能中的位置.....1
1.1.2 为什么选择亚马逊棋.....1
1.1.3 亚马逊棋与计算机亚马逊棋简述.....2
1.2 游戏规则与形式化定义.....3
1.2.1 规则.....3
1.2.2 形式化定义.....4
1.2.3 领地、分隔与填格.....4
1.3 问题难点分析.....5
1.3.1 状态空间与博弈树规模.....5
1.3.2 分支因子.....5
1.3.3 着法的“两段式”结构.....6
1.3.4 评估函数的不完美.....6
1.3.5 先手优势与评价偏差.....6
1.3.6 计算预算与运行环境.....7
1.4 需求分析.....7
1.4.1 课程任务要求.....7
1.4.2 功能需求.....7
1.4.3 非功能需求.....8
1.4.4 算法与实验需求.....9
1.4.5 报告需求.....9
1.5 本文工作与贡献界定.....10
1.5.1 已有方法、复现内容与本人工作的区分.....10
1.5.2 本人工作.....10
1.5.3 不在本文范围之内的内容.....11
1.6 本报告的结构.....11
二、算法原理和技术路线.....12
2.1 技术路线总览.....12
2.1.1 总体思路.....12
2.1.2 为什么同时研究 α-β 与 MCTS.....13

2.2 棋盘表示与领地评估.....14
2.2.1 棋盘表示与走法生成.....14
2.2.2 后距离与王距离.....14
2.2.3 领地评估函数.....15
2.2.4 分隔后的评估.....16
2.2.5 着法排序：廉价分数与可分离评估.....16
2.3 α-β 博弈树搜索.....17
2.3.1 极小极大与 α-β 剪枝.....17
2.3.2 迭代加深与时间控制.....18
2.3.3 主变例搜索.....18
2.3.4 置换表、杀手着法与历史启发.....19
2.3.5 束宽剪枝.....19
2.4 蒙特卡洛树搜索.....19
2.4.1 从蒙特卡洛评估到 UCT.....19
2.4.2 四个阶段.....20
2.4.3 展开顺序与渐进展开.....20
2.4.4 PUCT 与先验.....21
2.4.5 模拟策略与评估截断.....21
2.4.6 本文的 MCTS 配置.....22
2.5 学习方法.....22
2.5.1 线性 softmax 走子策略.....23
2.5.2 评估函数的 Texel 式调参.....23
2.5.3 学习组件与深度强化学习的关系.....24
2.6 分隔后的填格策略.....24
2.7 棋力评价的统计方法.....25
2.7.1 Elo 模型与 Bradley–Terry 模型.....25
2.7.2 置信区间与显著性.....25
2.7.3 其他评测.....26
2.8 相关研究综述.....26
2.8.1 博弈树搜索与 α-β 的增强.....26
2.8.2 证明数搜索与博弈求解.....27
2.8.3 蒙特卡洛树搜索及其改进.....28
2.8.4 MCTS 与极小极大的混合.....29
2.8.5 机器学习与深度强化学习在博弈中的应用.....29
2.8.6 亚马逊棋的理论与程序.....30
2.8.7 其他棋类中的评估与学习.....31
2.8.8 评价方法与可复现性.....32
2.8.9 可解释性、人机交互与可视化.....32

2.8.10 本文的定位.....33
2.9 技术路线中的关键设计决策.....33
2.10 本章小结.....34
三、智能问题求解分析（包括使用 AI 工具解决方法）.....35
3.1 问题建模.....35
3.1.1 智能体与环境.....35
3.1.2 求解目标的分层.....35
3.1.3 终局条件与“分隔”这一子目标.....36
3.2 搜索空间的压缩.....36
3.2.1 为什么全宽搜索不可行.....36
3.2.2 三级压缩.....37
3.2.3 动作分解的合理性.....37
3.3 知识表示：距离与领地.....37
3.3.1 为什么用“距离”表示局面.....37
3.3.2 后距离与王距离的分工.....38
3.3.3 表示的局限.....38
3.4 争夺阶段的权衡.....38
3.4.1 进攻与防守：移动性的作用.....38
3.4.2 后的分工.....39
3.4.3 射箭的双重作用.....39
3.5 近似与不确定性.....39
3.5.1 三类近似误差.....39
3.5.2 评估值与胜率.....39
3.5.3 时间管理.....40
3.6 算法选择分析与研究问题.....40
3.6.1 α-β 与 MCTS 在亚马逊棋上的预期.....40
3.6.2 研究问题.....40
3.7 使用 AI 工具解决问题的方法.....41
3.7.1 使用的工具与范围.....41
3.7.2 工作方式：生成—验证循环.....42
3.7.3 AI 在本项目中犯过的错误.....42
3.7.4 对课程学习的影响与反思.....43
3.8 本章小结.....44
四、流程图和程序代码.....45
4.1 系统架构.....45
4.1.1 总体结构.....45
4.1.2 模块与接口.....46
4.2 核心数据结构与代码.....47

4.2.1 棋盘与走法生成 47
4.2.2 距离计算 48
4.2.3 评估函数 49
4.2.4 可分离评估 51
4.3 α-β 搜索的流程与代码 52
4.3.1 根结点流程 52
4.3.2 候选生成与排序 53
4.3.3 负极大值搜索主体 54
4.3.4 迭代加深与根结点 55
4.4 MCTS 的流程与代码 57
4.4.1 主循环 57
4.4.2 候选、展开与选择 59
4.4.3 模拟策略 60
4.5 填格策略与智能体 61
4.5.1 共用的填格策略 61
4.5.2 智能体预设 61
4.6 网页界面的实现 62
4.6.1 Web Worker 架构 62
4.6.2 人机交互 65
4.6.3 决策过程的可视化 65
4.7 实验系统的实现 66
4.7.1 对局器与任务执行 66
4.7.2 公平性措施 67
4.8 测试 68
4.8.1 单元测试 68
4.8.2 界面自动化测试 69
4.9 本章小结 69
五、算法结果和界面呈现 70
5.1 实验设置 70
5.1.1 硬件与软件环境 70
5.1.2 参赛智能体 70
5.1.3 开局、配对设计与参数试调 71
5.1.4 实验清单 71
5.1.5 评价指标 72
5.2 游戏界面与决策过程展示 72
5.2.1 界面总览 72
5.2.2 AI 决策过程的可视化 73
5.2.3 人机对弈与提示 76

5.2.4 局势曲线、棋谱与结果 77
5.2.5 稳定性测试 80
5.3 循环赛：各算法的总体棋力 82
5.3.1 Elo 排名 82
5.3.2 交叉胜率 83
5.3.3 对局行为统计 85
5.3.4 先后手、对局长度与分隔 86
5.4 α-β 搜索的消融实验 87
5.4.1 棋力 87
5.4.2 为什么经典增强没有帮助 89
5.5 MCTS 的消融实验 89
5.5.1 棋力 89
5.5.2 主要发现 90
5.5.3 两类方法的对照 91
5.6 算力缩放 92
5.6.1 对自身的缩放 92
5.6.2 相同预算下的 α-β 对 MCTS 93
5.6.3 重复实验：结论有多稳定 94
5.7 学习组件的评估 96
5.7.1 训练数据 96
5.7.2 走子策略 96
5.7.3 评估函数的 Texel 调参 98
5.8 单步决策质量 100
5.9 搜索效率 102
5.9.1 着法排序的质量 102
5.9.2 固定深度下的搜索量 103
5.9.3 固定预算下的搜索量 103
5.9.4 浏览器中的速度 105
5.9.5 决策的随机性 105
5.10 失败案例分析 106
5.10.1 案例一：分隔判定的漏洞 106
5.10.2 案例二：分隔之后的近似 108
5.10.3 案例三：强智能体输给弱智能体 109
5.11 局限与有效性威胁 110
5.11.1 结论的适用范围 110
5.11.2 统计效力 110
5.11.3 参数选择 110
5.11.4 计时与并行的影响 110

5.11.5 开局的代表性 111
5.11.6 实现层面的近似 111
5.12 本章小结 111
六、个人学习和实践心得 113
6.1 对博弈搜索算法的理解 113
6.2 对实验结果的思考 114
6.3 一个程序漏洞带来的教训 114
6.4 关于使用 AI 工具的体会 115
6.5 与自动化专业的联系 115
6.6 不足与今后的打算 116
参考文献 117
附录 A 符号与术语 139
附录 B 实验数据明细 141
各实验组概况 141
循环赛逐对结果 141
各开局的统计 143
修复分隔判定前后的循环赛 Elo 143
自对弈数据 144
数据文件清单 144
附录 C 运行说明 145
代码仓库与在线试玩 145
直接运行游戏 145
开发环境 145
复现步骤 145
关于可重复性的说明 146
附录 D 参考文献核验记录 148
核验流程 148
核验结果 148
局限说明 150
附录 E 源代码 151
engine/core.js（246 行） 151
engine/eval.js（173 行） 155
engine/movegen.js（181 行） 157
engine/search.js（234 行） 160
engine/mcts.js（179 行） 164
engine/policy.js（40 行） 167

engine/agents.js（132 行）.....167
engine/index.js（10 行）.....170
game/src/app.js（566 行）.....170
game/src/worker.js（37 行）.....180
game/build.js（39 行）.....180
experiments/arena.js（28 行）.....181
experiments/runner.js（59 行）.....182
experiments/make_jobs.js（81 行）.....183

一、问题描述和需求分析

本章说明为什么选择亚马逊棋作为人工智能算法的研究载体，给出游戏规则与问题的形式化定义，分析它在分支因子、评估与时间预算上的难点，然后从课程任务出发整理功能、性能、实验与报告四个方面的需求，最后界定本报告中“已有方法”“复现内容”与“本人工作”的边界。

1.1 选题背景与动机

1.1.1 棋类博弈在人工智能中的位置

棋类游戏是人工智能最早、也最持久的试验场。1950 年 Shannon 发表《为计算机编写下棋程序》，第一次把下棋表述为在博弈树上做有限深度的极小极大搜索，并区分了“全宽搜索”（A 型）与“选择性搜索”（B 型）两种策略[4]；Turing 几乎同时在纸上手工执行了一个国际象棋程序[5]。随后 Newell、Shaw 与 Simon 讨论了下棋程序与问题复杂性之间的关系[6]，Samuel 的西洋跳棋程序把“机器学习”一词带入了计算机科学[7,8]。在之后的半个多世纪里，博弈树搜索、启发式评估、机器学习三条技术线反复在棋类上交汇：α-β 剪枝的分析[1]、置换表与 Zobrist 哈希[9]、历史启发[10]，以及 Deep Blue[11,12]、Chinook[13,14]、TD-Gammon[15,16]，再到 AlphaGo、AlphaGo Zero 与 AlphaZero[17–19]，几乎每一代代表性方法都在棋类上得到检验。

棋类适合做算法研究，原因并不神秘。第一，规则完全明确、胜负可判定，算法好坏可以通过对局结果客观地度量，而不依赖主观评分；第二，状态空间巨大但结构清晰，既不能穷举，又允许人们设计和验证领域知识；第三，同一个问题可以同时容纳“搜索”“推理”“学习”三类方法，适合横向比较[20,21]。这些性质与本课程第三次作业“将人工智能算法用于游戏决策、博弈、搜索或学习”的要求高度契合。

1.1.2 为什么选择亚马逊棋

在众多棋类中，本文选择 10×10 的亚马逊棋（Game of the Amazons）。选题时也考虑过五子棋，但五子棋是课程作业中非常常见的题目，而且它的强弱主要取决于“冲四”“活三”等战术棋型，研究重点容易落到棋型工程上。亚马逊棋在下面四个方面更适合本次作业。

一是**分支因子极大，搜索算法的取舍更尖锐**。亚马逊棋每一步由“移动一个后”和“射出一支箭”两部分组成，开局时合法着法多达 2176 个（本文程序的实际计数，4.8 节单元测试），中局仍常有数百个，远大于国际象棋（约 35）和五子棋（候选点约 100）[22,23]。在这样的分支因子下，全宽的 α-β 搜索两三层就耗尽时间，而蒙特卡洛

树搜索（MCTS）恰恰被认为适合大分支因子的游戏。亚马逊棋上的 α-β 与 MCTS 之争也确实是文献中的一个经典话题：Lorentz 在 2008 年报告，他的程序 Invader 改用 MCTS 后，新版本在对弈中胜过了原来的 α-β 版本[24]，Kloetzer 等人研究了 MCTS 在亚马逊棋开局与残局中的应用[25,26]，而国内全国大学生计算机博弈大赛中的亚马逊棋程序则大量采用 PVS 等 α-β 变体[27–29]。在同一平台上公平地比较两者，本身就是一个有意义的问题。

二是**评估函数有清晰的几何意义**。亚马逊棋的胜负取决于双方最后各自“圈住”多少空格，因此评估函数可以用“谁先到达某个空格”的距离概念来定义领地[23,30,31]。这种评估不依赖人工总结的棋型，既容易解释，也便于在界面上直观展示（5.2 节的“领地”图层）。

三是**终局可以精确分析，便于检验算法**。当双方的后被箭完全隔开后，游戏退化为“各自在自己的区域里填格子”，胜负主要由区域大小决定[32–34]。这让本文能够明确地区分“争夺阶段”和“填格阶段”，把算法比较集中在真正需要智能决策的争夺阶段（2.6 节）。

四是**计算资源可承受、题材少见**。亚马逊棋最多 92 手结束，单局对弈在 200 ms/步的预算下约需 10 秒，一台普通电脑或浏览器即可完成几千局的统计实验；同时它在课程作业中较少见，与本课程另一份报告（决策树故障诊断）在问题类型、算法和评价方式上也都不重叠。

1.1.3 亚马逊棋与计算机亚马逊棋简述

一般认为亚马逊棋由阿根廷人 Walter Zamkauskas 于 1988 年发明，规则极其简洁：每方四个“后”，走一步、射一箭，谁先无路可走谁输。它很快引起了组合博弈论学者的兴趣：Berlekamp 分析了 N×2 窄棋盘上亚马逊棋局面的和[32]，Snatzke 用穷举搜索得到了小棋盘上的新结果[35]，Buro 证明了一类简单亚马逊棋残局与三次子网格图上的哈密顿回路问题之间的联系，从而说明即使是这类简单残局的精确求解也是 NP 完全的[33]；Hearn 证明了广义亚马逊棋（任意大小棋盘）是 PSPACE 完全的[36,37]，与国际象棋、围棋、五子棋等游戏的广义版本同属一类复杂度[38–40]。

计算机亚马逊棋的研究从 1990 年代末开始活跃。1998—2000 年间举办了开放式计算机亚马逊棋锦标赛[41]，此后亚马逊棋多次列入计算机奥林匹克（Computer Olympiad）的比赛项目。Müller 与 Tegos 系统地实验了亚马逊棋程序中的评估与搜索技术[23]，Avetisyan 与 Lorentz 研究了选择性搜索[42]，Lieberum 提出了被后来大多数程序采用的评估函数框架[30,31]。2007 年后 MCTS 进入亚马逊棋：Lorentz[24]与 Kloetzer 等人[25,26]的工作都表明了“蒙特卡洛模拟 + 评估截断”在这里的有效性，Song 与 Müller 则改进了亚马逊棋残局的精确求解器[34]。在国内，亚马逊棋是全国大学生计算机博弈

大赛的比赛项目之一，中国控制与决策会议（CCDC）等会议上有大量相关论文，涉及搜索算法[43–46]、区域划分[47]、UCT 与 PVS 的混合[29,48]、评估函数优化[49–51]、强化学习与神经网络[52–55]、并行与 GPU 加速[27,28,56]以及 MCTS 的改进[57,58]；Sun 等人对这些方法做了综述[59]。第二章 2.8 节将系统梳理这些工作。

1.2 游戏规则与形式化定义

1.2.1 规则

本文实现的是标准 10×10 亚马逊棋规则（图 1-1）：

1. 棋盘为 10×10 的方格。白方的四个后位于 a4、d1、g1、j4，黑方的四个后位于 a7、d10、g10、j7。**白方先行**，双方轮流走棋；
2. 每一步包括两个动作：先选择己方的一个后，像国际象棋的后一样沿横、竖或斜线移动任意格（不能越过或停在被占据的格子上）；然后从**移动后的位置**像后一样射出一支箭，箭落在任一可达的空格上（可以射回刚刚离开的格子）；
3. 箭落下的格子被永久封死，此后任何后和箭都不能经过或停留；
4. 轮到某一方走棋时，如果它的四个后都无法移动，则该方判负。由于每一步都恰好封死一个格子，全局最多进行 $100-8=92$ 步，且**不存在和棋**。

棋谱记法采用通行的“出发格-到达格/箭的落点”格式，例如 `d1-d7/g7` 表示把 d1 的后移到 d7，再从 d7 向 g7 射箭。列用字母 a—j 表示，行用数字 1—10 表示，第 10 行在棋盘上方（黑方一侧）。

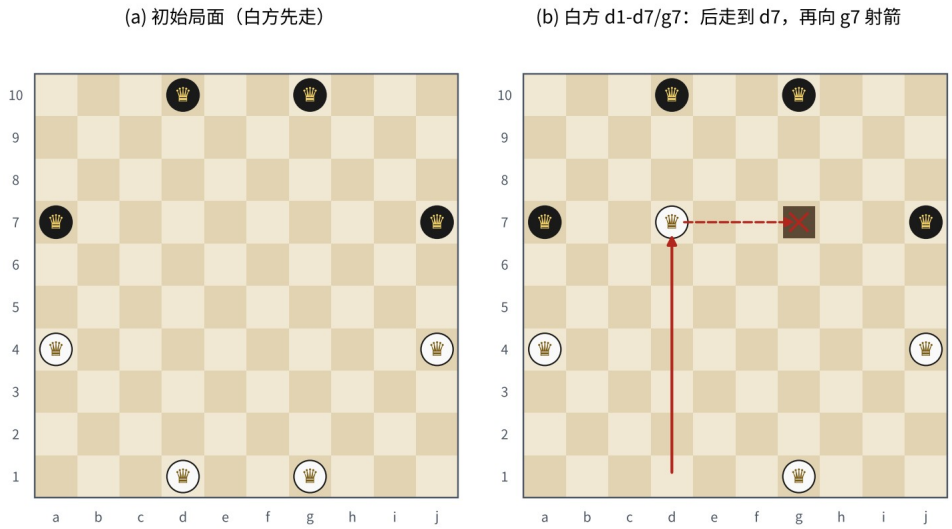


图 1-1 亚马逊棋的初始局面与一步着法示例

注：作者绘制。左：初始局面；右：白方走 d1-d7/g7，实线箭头为后的移动，虚线箭头为射箭方向，深色方块为箭。

1.2.2 形式化定义

亚马逊棋是二人、零和、确定性、完全信息的回合制博弈[20,60]。用 S 表示所有合法局面的集合，局面 $s \in S$ 包括棋盘内容（每格为空、白后、黑后或箭）与轮到哪一方走棋。记行棋方为 $\rho(s) \in \{白, 黑\}$，合法着法集合为 $A(s)$，转移函数 $T(s,a)$ 给出走棋后的新局面。终局集合 $S_T = \{s : A(s) = \varnothing\}$，此时行棋方负。

在负极大值（negamax）约定下，局面的博弈论值满足

$$V(s) = \begin{cases} -1, & A(s) = \varnothing \\ \max\limits_{a \in A(s)} \left[-V(T(s,a)) \right], & 其他 \end{cases} \quad (1\text{-}1)$$

其中 $V(s)$ 是从行棋方视角看的值。式 1-1 是 von Neumann 极小极大定理在有限完全信息博弈中的递归形式[60,61]。10×10 亚马逊棋的博弈论值至今未知；即使知道，它也不能告诉程序在具体局面下怎样走。实际的程序只能在有限时间内**近似**求解式 1-1，这正是本文研究的问题。

本文把“下一步怎么走”的决策问题写成：给定局面 s 与计算预算 B（毫秒数、结点数或模拟次数），算法 π_B 输出一个着法 $a = \pi_B(s) \in A(s)$，目标是在对手同样受预算约束的条件下最大化期望得分。由于真正的 V 不可得，算法的好坏只能用**对局结果**来衡量，这决定了本文的主要评价方法是大规模对局统计与 Elo 等级分（5.1 节）。

1.2.3 领地、分隔与填格

亚马逊棋的战略语言建立在“领地”之上。本文采用文献中通行的概念[23,30]，并在 2.2 节给出精确定义。这里先给出直观说明：

- **后距离**：一方的后从当前位置出发，最少需要走几步（每步按后的走法、不越过障碍）才能到达某个空格；
- **领地**：某空格离哪一方的后更近（后距离更小），就把它看作哪一方的“潜在领地”；距离相等的格子是双方争夺的“均势格”；
- **分隔**：当不存在任何双方都能到达的空格，并且双方的后也没有紧挨在一起时（严格地说：把箭以外的格子按 8 邻域连通，没有一个连通分量同时含有双方的后），称双方已被完全分隔。此后两方互不干扰，各自只能在自己的区域里走棋。后一个条件容易被忽略：两个敌对的后紧挨着、中间没有空格时，任何一方把后移开，空出的格子就会把两块区域连通。本项目的第一版程序正是漏掉了这一条件，5.10 节报告了它的影响；
- **填格**：分隔后，每一步都会在自己的区域中消耗一个格子（射出的箭），谁的可走格子先用完谁输。因此分隔后的胜负大体上由双方区域的大小决定，但区

域的形状也有影响——某些格子可能因为“走进去就出不来”而无法全部利用，这类区域在文献中称为“有缺陷的区域”[33,34]。

一盘亚马逊棋通常经历“开局—中局争夺—分隔—填格”几个阶段。本文的对局数据显示，双方平均在第 61 手左右完全分隔（5.3.4 节），此后的填格阶段平均约占全局手数的 27%（分隔时刻最早为第 44 手，最晚为第 90 手）。

1.3 问题难点分析

1.3.1 状态空间与博弈树规模

10×10 亚马逊棋每格有空、白后、黑后、箭四种状态，并受“各四个后”的约束。不考虑可达性，局面数的粗略上界是 $\binom{100}{4}\binom{96}{4}\cdot 2^{92}\approx 10^{40}$。博弈树规模可以用平均分支因子 $\acute{b}$ 与平均对局长度 $\acute{L}$ 估计为 $\acute{b}^{\acute{L}}$。用本文自对弈数据中的实测值（$b≈362、L≈84）估计，博弈树复杂度约为 $362^{84}\approx 10^{214}$；若改用逐手合法着法数的几何平均（分支因子在终盘迅速变小，算术平均会高估），则约为 10^{175}。无论哪种估计，都远远超出了任何穷举能力（作为参照，van den Herik 等人的综述中国际象棋的博弈树复杂度约为 10^{123}[22]）。理论上，广义亚马逊棋的判定问题是 PSPACE 完全的[36]，这意味着不能指望找到多项式时间的精确算法[37,40,62]。

1.3.2 分支因子

亚马逊棋最突出的特点是分支因子极大。本文统计了 400 局 α-β 自对弈棋谱中每个局面的合法着法数（图 1-2）：开局时为 2176 个，前 10 手的中位数仍在 1450 左右，此后随着棋盘被箭逐渐填满而缓慢下降，双方分隔前的局面平均约有 457 个合法着法。作为对比，国际象棋的平均合法着法数通常被认为在 35 左右。

如此大的分支因子对两类算法都构成挑战。对 α-β 而言，全宽搜索 3 层就需要访问 $2000^{3}\approx 10^{10}$ 个结点，在 200 ms 内完全不可能，必须依靠**着法排序与束宽剪枝**只搜索极少数候选（2.3.5 节）；而“只搜索哪几步”的选择本身就是最大的误差来源。对 MCTS 而言，根结点的每个子结点哪怕只模拟一次也需要上千次模拟，必须依靠**渐进展开**等技术控制树的宽度（2.4.3 节）。

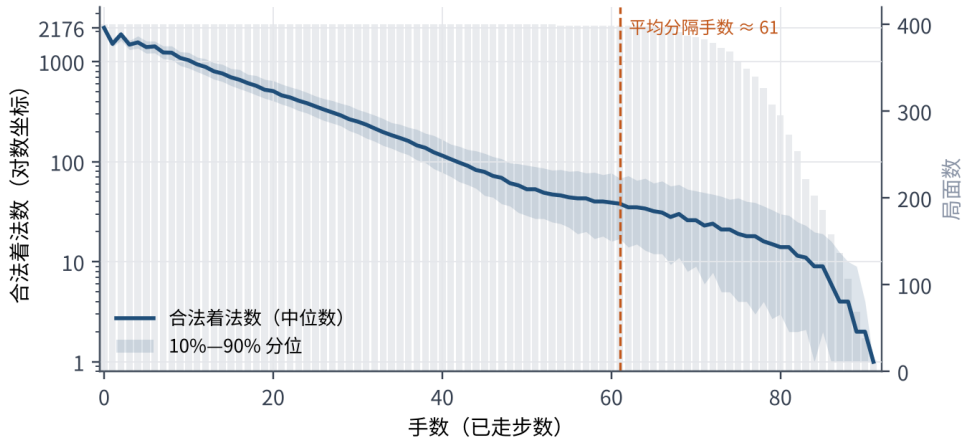


图 1-2　自对弈棋谱中合法着法数随手数的变化

注：数据：本文 400 局 α-β 自对弈棋谱（5.7.1 节），实线为中位数，阴影为 10%—90% 分位区间；柱状为各手数的局面数。

1.3.3　着法的“两段式”结构

亚马逊棋的一步由“移后”和“射箭”组成，一个好的着法往往需要两者配合：移后是为了占据有利位置或逃离包围，射箭则用来封堵对方的后或者保护自己的领地。这种结构使得着法的数量是“后的走法数 × 射箭位置数”，同时也给了算法设计一个机会：可以先分别评价“移后”与“射箭”，再把它们组合起来。本文的 α-β 与 MCTS 都依赖这一思想设计着法排序（2.2.5 节的“可分离评估”）。

1.3.4　评估函数的不完美

非终局局面的价值只能用启发式评估函数近似。基于距离的领地评估虽然直观，但存在明显的系统误差：开局阶段几乎所有格子都“可争夺”，领地的划分很不稳定；“谁先到达”也没有考虑对方可以用箭阻断道路；被困住的后、有缺陷的区域都可能让领地估计失真[23,30]。评估误差会被搜索放大或修正，这正是“搜索与知识的权衡”这一经典问题[63–65]。本文在 2.5.2 节用 Texel 方法从对局数据中学习评估权重，并在 5.7 节检验学习到的权重是否真正提升了棋力。

1.3.5　先手优势与评价偏差

亚马逊棋的先手优势没有五子棋那样极端，但也不能忽略：如果两个确定性程序反复从同一局面开始对弈，结果会完全重复；如果总让同一方先走，结果又会混入先手效应。因此评价方案必须引入多样化且相对均衡的开局，并交换先后手，才能得到有意义的统计结论[66,67]。本文所有对比实验都采用“16 个均衡开局 × 交换先后手”的配对设计（5.1.3 节）。

1.3.6 计算预算与运行环境

本项目要求在网页中展示游戏与 AI 决策过程。浏览器中的 JavaScript 是单线程的，若在主线程中运行搜索，界面会在 AI 思考期间完全卡住。因此必须把引擎放进 Web Worker 中运行，并通过消息传递把搜索进度推送给界面（4.6 节）。同时，网页端的计算速度与 Node.js 环境下大致相当（5.9 节），意味着网页中的 AI 与实验中测量的 AI 是“同一个强度”。

1.4 需求分析

1.4.1 课程任务要求

依据老师的作业模板与任务说明，本项目需要满足以下硬性要求：

表 1-1 课程任务要求与本项目的对应关系

编号	任务要求	本项目的落实方式	对应章节
R1	选择适合深入研究的游戏	10×10 亚马逊棋	1.1、1.2
R2	将 AI 算法用于决策、博弈、搜索或学习	α-β 搜索、MCTS、领地评估、策略学习与评估学习	第二章
R3	提交可运行程序，HTML 展示即可	单文件网页 `amazons.html`，双击即可在浏览器运行	4.6、5.2、附录 C
R4	AI 实际参与对弈	人机对弈、机机对弈两种模式，AI 实时计算着法	5.2
R5	界面美观稳定并展示决策过程	领地图、候选着法箭头、主变例、迭代过程、访问次数表、胜率曲线	5.2
R6	相关工作、基线、消融、性能评估、失败案例与局限	2.8 节综述；5.3—5.11 节实验	第二、五章
R7	数据来自真实运行，区分已有方法、复现与自己的贡献	所有数据由附带脚本生成；1.5 节界定贡献边界	1.5、第五章、附录 B
R8	如实说明 AI 工具使用	3.7 节专门说明	3.7
R9	封面题目、六个一级章节与模板一致	按模板排版	全文
R10	参考文献真实可核验、与正文一一对应	全部经过在线检索核验，未通过者不引用	参考文献、附录 D

1.4.2 功能需求

从使用者角度，游戏程序需要支持以下功能（用例关系见图 1-3）：

1. **对弈模式**：人执白对 AI、人执黑对 AI、AI 对 AI（观战）、人对人；
2. **走子交互**：用“点后—点目的地—点射箭位置”三次点击完成一步，每一步都只高亮合法的选项，可随时取消；

3. **AI 选择**：每一方可独立选择算法（α-β、MCTS 的若干配置以及贪心、随机等基线），并调节每步思考时间；
4. **开局设置**：标准初始局面，或从均衡开局库中随机选择一个开局；
5. **对局控制**：新局、悔棋、提示（由 AI 为人类推荐一手）、暂停/继续 AI 对弈；
6. **决策过程展示**：领地图、双方后距离、α-β 根结点得分最高的候选着法与主变例、迭代加深过程、MCTS 各候选的访问次数/胜率/先验、胜率曲线、领地统计与各后的机动性；
7. **棋谱功能**：显示棋谱、导出与导入、复制；
8. **算法说明**：在界面内用简短文字解释规则与各算法原理，便于演示。

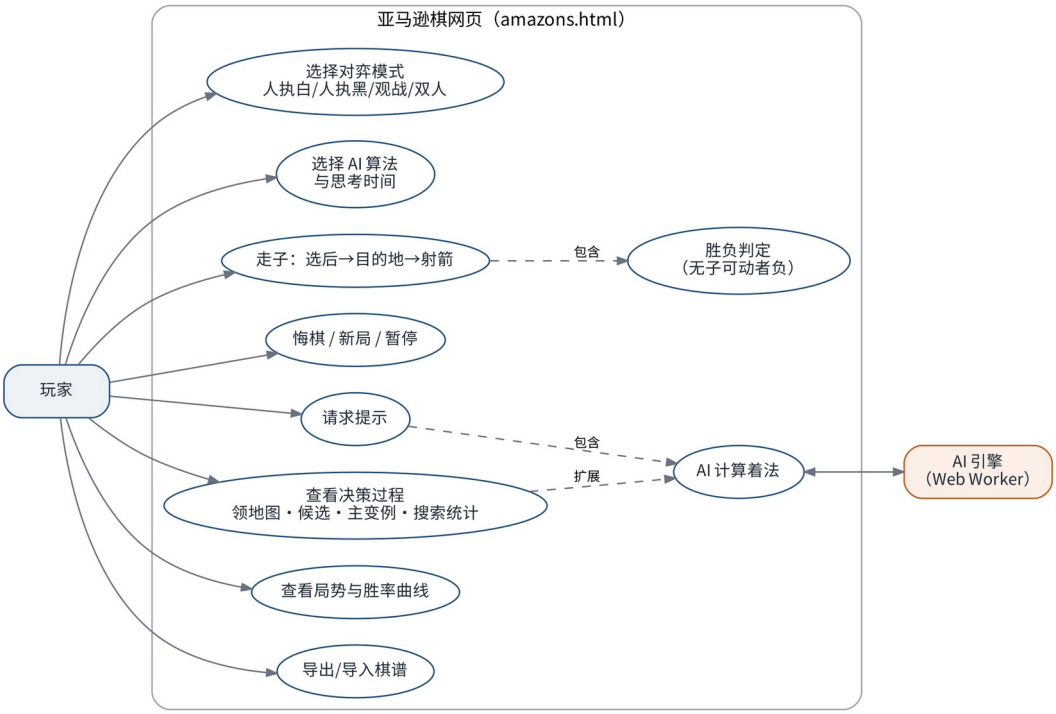


图 1-3 游戏程序用例图

注：作者绘制（Graphviz）。

1.4.3 非功能需求

表 1-2 非功能需求及其验收方式

类别	需求	验收方式
可运行性	不依赖服务器、网络或安装，单个 HTML 文件即可运行	以 `file://` 方式在 Chromium 中打开并完成整局对弈（5.2.5 节）
响应性	AI 思考期间界面不冻结；用时不超过设定时间的 1.2 倍	Web Worker 架构；对局日志中统计每步用时（5.9 节）
正确性	走法生成、胜负判定、增量哈希、距离计算与	自动化单元测试（4.8 节）

类别	需求	验收方式
	参考实现一致	
稳定性	连续机机对弈不出现异常、控制台无报错	Playwright 自动化界面测试（5.2.5 节）
一致性	网页中的 AI 与实验中的 AI 为同一份代码	构建脚本把同一引擎源码打包进网页（4.6 节）
可复现性	实验可以完整重跑，随机过程固定种子	任务清单与种子写入数据文件（附录 B、C）
可解释性	每一手都能看到“为什么这样走”	着法理由字段与可视化图层（5.2 节）

1.4.4 算法与实验需求

为了满足严谨实验的要求，算法与实验部分需要做到：

- **至少两类本质不同的算法**：深度优先的 α-β 搜索与基于采样的 MCTS，并加入学习组件；
- **合理的基线**：随机走子、一步贪心、未加增强的朴素 α-β、纯随机模拟的 MCTS；
- **消融实验**：对每个主要组件（置换表、杀手/历史启发、PVS、着法排序方式、束宽、评估函数各项、MCTS 的选择公式/展开顺序/渐进展开/模拟方式等）逐一替换或关闭，测量其对棋力的影响；
- **算力缩放**：测量棋力随每步思考时间的变化，并在相同预算下直接比较 α-β 与 MCTS；
- **学习效果评估**：学习得到的模型既要用离线指标（交叉熵、命中率）评价，也要用对局结果评价；
- **统计严谨**：报告置信区间与显著性，对关键结论做独立种子的重复实验；
- **失败案例与局限**：展示算法在具体局面中犯错的例子并分析原因。

1.4.5 报告需求

报告需要沿用第一次作业报告的版式（封面、摘要、编写说明、目录、页眉页脚、图表编号等），正文严格保留模板的六个一级章节，篇幅 150—200 页，参考文献 400—500 条且全部真实可核验、按 GB/T 7714 顺序编码制著录。第六章为个人学习和实践心得。

1.5 本文工作与贡献界定

1.5.1 已有方法、复现内容与本人工作的区分

本项目使用的核心算法全部来自已有文献，**没有提出新的算法**。为了避免夸大，表 1-3 逐项说明每个组成部分属于哪一类。“复现”指依据文献描述自行编写代码实现，而不是复制他人代码；本项目没有使用任何第三方博弈引擎或代码库。

表 1-3 本项目各组成部分的来源界定

组成部分	性质	依据的主要文献	说明
负极大值 α-β 搜索、迭代加深	已有方法，自行实现	[1,68,69]	标准算法
主变例搜索（PVS/ NegaScout）	已有方法，自行实现	[70,71]	标准算法
置换表与 Zobrist 哈希	已有方法，自行实现	[9,72]	直接替换策略
杀手着法、历史启发	已有方法，自行实现	[10,73]	标准增强
领地评估（t1、t2、c1、c2、w、机动性）	已有方法，自行实现	[23,30,31]	阶段权重改为可学习的线性形式，系数由本人设定
束宽剪枝 / 选择性搜索	已有思想，自行实现	[42,74]	束宽参数由本人设定
“可分离评估”着法排序	本人为本项目设计的近似	思想与“先评价移后、再评价射箭”的做法相近[23]	未见完全相同的公开描述，但不排除已有程序采用；本文不称之为创新
MCTS、UCT	已有方法，自行实现	[2,3,75]	标准算法
渐进展开	已有方法，自行实现	[76,77]	标准形式
评估截断模拟	已有方法，自行实现	[24,78,79]	亚马逊棋 MCTS 的常用做法
PUCT 选择与先验	已有方法，自行实现	[18,80]	先验来自启发式或线性模型，不是神经网络
线性 softmax 走子策略学习	已有方法（条件 logit / 走子预测）	[77,81,82]	特征设计由本人完成
Texel 式评估权重调参	已有方法	[83,84]	逻辑回归形式
Elo/Bradley–Terry 评估、自助法	已有统计方法	[85–87]	用于实验分析
网页界面与可视化	本人设计实现	—	单文件、Web Worker 架构
统一框架下的对比与消融实验	本人设计并实际运行	—	全部数据来自本项目运行

1.5.2 本人工作

在上述界定之下，本项目的工作可以概括为以下四点：

1. **一个完整、可玩、可解释的亚马逊棋 AI 系统**。用 JavaScript 实现了棋盘与走法生成、领地评估、α-β、MCTS、学习模型推断等全部组件，打包成一个无需安装的网页，并把搜索过程以领地图、候选着法箭头、主变例、访问次数表、胜率曲线等形式实时展示出来。
2. **统一预算下的系统比较**。所有算法共享同一棋盘表示、同一评估知识与同一填格策略，在完全相同的时间预算、开局与先后手设计下对弈，共完成主实验 2144 局（另有重复实验 384 局），从而把“算法框架的差异”与“领域知识的差异”尽量分离开。
3. **细粒度的消融与缩放实验**。对 α-β 的 11 个变体和 MCTS 的 15 个变体逐一进行配对对局，测量了每个组件的实际贡献，并测量了棋力随思考时间的变化。第五章如实报告了与预期不符的结果并分析了原因。为避免“在测试集上调参”，MCTS 的先验温度与探索系数只在另外 16 个试调开局上做过少量比较（5.1.3 节），正式实验的开局从未用于调参。
4. **轻量级学习组件的实证评估**。从自对弈数据中学习了线性走子策略与评估权重，用离线指标与对局结果两方面评价，并报告了离线指标与实际棋力之间的关系。

1.5.3 不在本文范围之内的内容

为了控制工作量并保证每一部分都能被真实完成与验证，以下内容明确**不做**：深度神经网络与 AlphaZero 式的大规模自对弈训练（需要的算力远超本项目条件，2.8 节只做文献讨论）；分隔后残局的精确求解（本文用一个简单的填格策略代替，2.6 节）；开局库；多线程/并行搜索；以及与已有的强亚马逊棋程序的对比（这些程序的接口与运行环境不同，本文无法在可控条件下进行公平对比，5.11 节将其列为局限）。

1.6 本报告的结构

全文共六章。第二章介绍棋盘表示、领地评估、α-β 搜索、MCTS 和学习方法的原理，并综述相关工作；第三章从“智能问题求解”的角度分析亚马逊棋决策问题的建模、搜索空间压缩、知识表示与算法选择，并在 3.7 节说明 AI 工具在本项目中的使用情况；第四章给出系统架构、流程图与关键代码；第五章报告界面呈现和全部实验结果，包括失败案例与局限；第六章是个人学习与实践心得。附录给出符号表、完整数据表、运行说明、参考文献核验记录与源代码。

二、算法原理和技术路线

本章先给出全文的技术路线，再依次介绍棋盘表示与领地评估、α-β 博弈树搜索及其增强、蒙特卡洛树搜索、两种轻量级学习方法、分隔后的填格策略以及棋力评价的统计方法，最后综述相关研究。每一节都先给出算法的一般原理和文献来源，再说明本项目实现时的具体选择，以便读者区分“标准算法”与“本文的实现细节”。

2.1 技术路线总览

2.1.1 总体思路

本项目的技术路线可以概括为“**一个知识层、两类搜索框架、两种学习组件、一套评价体系**”（图 2-1）。

- **知识层**：用后距离与王距离描述局面，在此基础上定义领地评估函数、廉价着法分数与“可分离评估”着法排序。所有算法共享这一层，从而保证比较的是“搜索/学习框架”本身，而不是各自携带的不同知识。
- **搜索框架一：α-β 搜索**。深度优先、确定性，配合迭代加深、PVS、置换表、杀手与历史启发，并用束宽剪枝把每个结点的候选限制在排序最靠前的少数几个。
- **搜索框架二：MCTS**。基于采样的最佳优先搜索，提供 UCT 与 PUCT 两种选择公式，子结点的展开顺序可以是随机、廉价分数、可分离评估或学习策略，配合渐进展开；模拟阶段可以随机下到终局、随机走几步后用评估函数截断，或者直接评估。
- **学习组件**：从 α-β 自对弈棋谱中学习（1）线性 softmax 走子策略，作为 MCTS 的先验与展开顺序；（2）评估函数权重（Texel 方法），替换 α-β 的手工权重。
- **评价体系**：以“均衡开局 + 交换先后手”的配对对局为基本单元，用 Bradley–Terry/Elo 模型估计相对棋力，用 Wilson 区间与自助法给出不确定性，再辅以决策质量、搜索效率、离线学习指标等专项评测。

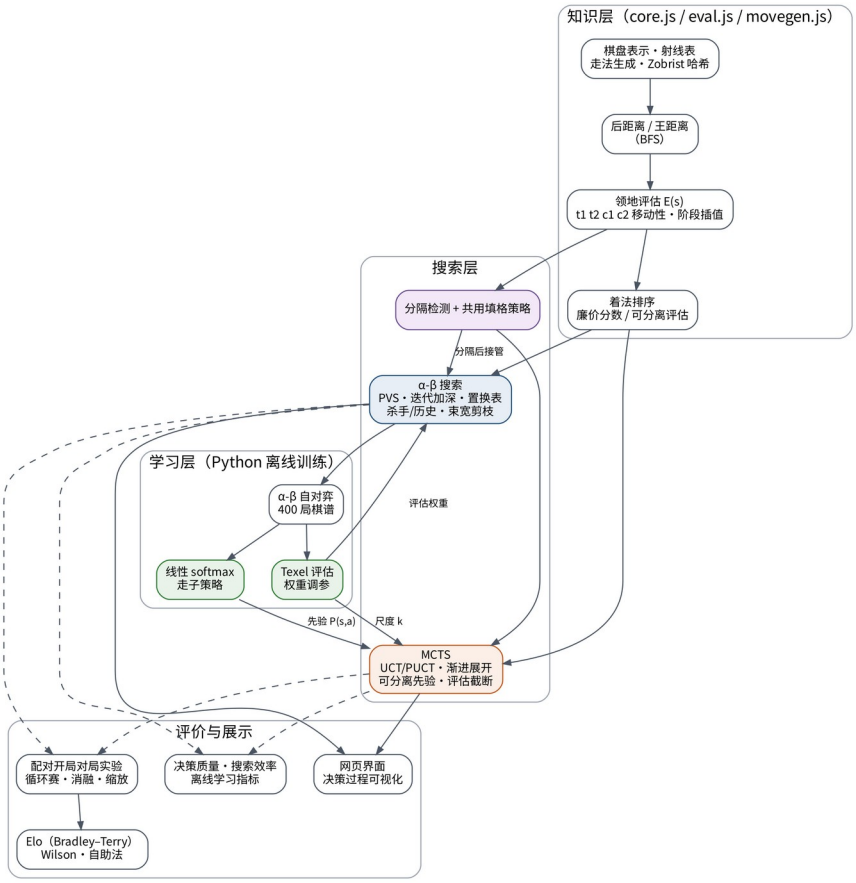


图 2-1 本项目技术路线

注：作者绘制（Graphviz）。实线为数据/调用关系，虚线为评价关系。

2.1.2 为什么同时研究 α-β 与 MCTS

α-β 与 MCTS 代表了博弈搜索中两种不同的哲学。α-β 依赖评估函数，对“短而尖锐”的战术非常有效，在国际象棋、跳棋等游戏中长期占据统治地位[11,14,71]；MCTS 不依赖或较少依赖评估函数，在评估困难、分支因子大的游戏（如围棋）中更有优势[3,88]，但在存在大量“陷阱”（浅层就能决定胜负的着法）的游戏中可能表现较差[89,90]。

亚马逊棋恰好处在两者的交界上：一方面它的分支因子比围棋还大，按“经验法则”应该更适合 MCTS；另一方面它有一个相当不错的领地评估函数，这又是 α-β 的强项。文献中两类方法在亚马逊棋上都有成功的程序[23–25,27]，也有人提出了两者的混合[29,48]，但在**相同知识、相同预算**下的系统比较并不多见。已有的混合方案（如 MCTS 中嵌入极小极大、隐式极小极大回传）[91–94]也提示我们：两类框架的差别，可能更多地在于它们**怎样使用同样的知识**。本文希望通过统一实现，对“在亚马逊棋上，两类框架各自的优势在哪里、各个组件到底贡献多少”给出有数据支持的回答。

2.2 棋盘表示与领地评估

2.2.1 棋盘表示与走法生成

棋盘用长度为 100 的一维数组表示，下标 $i=10r+c$，其中 r 为从上到下的行号（0 对应第 10 行），c 为从左到右的列号。每格取值为空、白后、黑后或箭之一，另用两个长度为 4 的数组记录双方四个后的位置。为了加速走法生成，程序预先为每个格子计算出 8 个方向上的“射线”（从该格出发、沿该方向直到棋盘边缘的格子序列），后与箭的移动都只需沿射线扫描到第一个非空格为止。

生成全部合法着法的过程是一个三重循环：对己方每个后 f，沿 8 条射线枚举目的地 t；对每个 t，再沿 8 条射线枚举箭的落点 a。需要注意的细节是：射箭时后已经离开了 f，因此生成箭的落点时必须先把 f 视为空格——这正是“箭可以射回原处”的原因。一个着法用整数 $f\cdot 10^4+t\cdot 10^2+a$ 编码，悔棋时按相反顺序恢复三个格子。

判断某方是否还能走棋不需要生成全部着法：只要它的某个后旁边（8 邻域）有一个空格，它就可以走一步到这个空格，再把箭射回原处。因此终局判断只需检查 32 个邻格，代价可以忽略。

局面哈希采用 Zobrist 方法：为每个（格子，内容）对分配一个随机数，局面哈希为所有非空格对应随机数的异或，再异或一个“行棋方”随机数[9]。一步棋改变三个格子，因此只需三次（加行棋方一次）异或即可增量更新。本文使用两个 32 位整数（共 64 位）作为哈希值：低位用作表下标，高位用于校验，碰撞概率可以忽略（有关散列与通用哈希的理论见文献[95,96]）。随机数由固定种子的 Mulberry32 生成器产生，保证不同运行之间完全可复现（关于伪随机数生成器参见文献[97,98]）。

2.2.2 后距离与王距离

领地评估的基础是两种距离[23,30]。对一方 p 和空格 x：

- **后距离** $D_p^{Q}(x)$：p 的任一后到达 x 所需的最少“后的走步”数，每一步可沿 8 个方向直线移动任意格，不能越过非空格；
- **王距离** $D_p^{K}(x)$：每一步只能移动到 8 邻域中的一个空格时的最少步数。

两种距离都用多源广度优先搜索计算：把 p 的四个后作为距离 0 的起点，后距离的扩展沿射线进行，王距离的扩展沿 8 邻域进行。不可达的格子距离记为 ∞（程序中用 99 表示）。后距离衡量“谁能更快占据”，王距离衡量“谁能更稳地控制”——后距离 2 的格子可能在棋盘另一端，王距离则更接近“填格”时真正的步数。

2.2.3 领地评估函数

在两种距离之上，Lieberum 定义了如下几项特征[30,31]（下面都以白方视角、对全部空格 x 求和）：

$$t_1=\sum_x \Delta\left(D_W^Q(x),D_B^Q(x)\right),t_2=\sum_x \Delta\left(D_W^K(x),D_B^K(x)\right) \tag{2-1}$$

其中

$$\Delta(n,m)=\begin{cases}0, & n=m=\infty\\ \kappa, & n=m<\infty\\ 1, & n<m\\ -1, & n>m\end{cases}$$

κ 是“距离相同的格子”给行棋方的分值（行棋方为白时 $\kappa=+0.2$，为黑时 $\kappa=-0.2$），反映“轮到谁走，谁就能先到”。t_1、t_2 统计的是“领地”：谁先到达就记谁的格子。

$$c_1=2\sum_x\left(2^{-D_W^Q(x)}-2^{-D_B^Q(x)}\right),c_2=\sum_x clip\left(\frac{D_B^K(x)-D_W^K(x)}{6},-1,1\right) \tag{2-2}$$

c_1、c_2 是“位置”项：它们不只看谁先到，还看领先了多少，在开局阶段比 t_1、t_2 更平滑。

$$w=\sum_{x:D_W^Q(x),D_B^Q(x)<\infty}2^{-\left|D_W^Q(x)-D_B^Q(x)\right|} \tag{2-3}$$

w 衡量局面中“还剩多少争夺”：双方距离越接近的格子越多，w 越大。开局时 $w\approx 70$，随着双方逐渐分开而减小，完全分隔时为 0。

此外，本文加入一项**机动性**特征：

$$mob=\frac{1}{4}\left(\sum_{q\in W}min\left(m_q,12\right)-\sum_{q\in B}min\left(m_q,12\right)\right) \tag{2-4}$$

其中 m_q 是后 q 一步能到达的格子数。截断到 12 是为了让特征主要反映“被困”的后（机动性很低），而不是奖励已经很自由的后。Lieberum 的原始评估中也有类似的机动性项，但形式不同[30]。

Lieberum 用一组依赖于 w 的非线性函数组合上述各项。为了能用 Texel 方法学习（2.5.2 节），本文改用**阶段线性插值**的形式：令阶段系数 $p=min\left(1,w/W_0\right)$（$W_0=60$），评估值为

$$E(s)=\sum_{k\in\{t_1,t_2,c_1,c_2,mob\}}\left[p\,a_k^{\mathrm{E}}+(1-p)\,a_k^{\mathrm{L}}\right]f_k(s) \tag{2-5}$$

其中 a_k^{E} 与 a_k^{L} 分别是“开局”与“终局”权重，共 10 个参数，评估值以“格”为单位，最后按行棋方取正负号。手工权重（表 2-1）遵循 Lieberum 的定性规律：开局时位置项 c_1, c_2 与机动性更重要，终局时领地项 t_1 占主导。

表 2-1 评估函数的手工权重

特征	含义	开局权重 a^{E}	终局权重 a^{L}
t_1	后距离领地	0.20	1.00
t_2	王距离领地	0.20	0.20
c_1	后距离位置	0.30	0.05
c_2	王距离位置	0.30	0.05
mob	机动性	0.25	0.05

这组权重是作者依据文献中的定性描述设定的，并没有经过系统调参；5.7 节将其与 Texel 调参得到的权重比较。评估函数的一次计算需要四次 BFS 与一次全盘扫描，在本文的实现中约需 8 微秒（5.9 节）。

2.2.4 分隔后的评估

当双方被完全分隔（不存在双方都能到达的空格，且双方的后不相邻，见 1.2.3 节）时，t_1 等特征失去了意义，本文改用**区域格数**评估：

$$E_{\mathrm{sep}}(s) = sgn(\delta)\cdot 200 + \delta,\ \delta = R_\rho - R_{\dot\rho} - 1/2 \quad (2\text{-}6)$$

其中 R_p 是 p 的后能到达的空格数，ρ 为行棋方。减去 $1/2$ 是因为两方格数相同时行棋方先用完；加上 ± 200 的常数是为了让搜索明确区分“已分隔且领先”与“尚未分隔”的局面。R_p 只是可走步数的**上界**：区域形状不好时，有些格子可能无法全部利用[33,34]。因此式 2-6 仍是近似。

2.2.5 着法排序：廉价分数与可分离评估

在亚马逊棋中，着法排序比在任何其他棋类中都更重要：束宽剪枝只搜索排序最靠前的几个着法，排序一旦漏掉了好棋，搜索再深也找不回来。本文实现了两种排序。

廉价分数。对每个着法 $(f \to t, a)$，用只需常数次查表的规则打分：

$$cheap(f,t,a) = 0.5(m_t - m_f) + 2n_{\mathrm{adj}}^{\mathrm{op}}(a) - n_{\mathrm{adj}}^{\mathrm{me}}(a) + 1\left[D_{\dot\rho}^{Q}(a) \le D_{\rho}^{Q}(a)\right] + 2^{1 - D_{\dot\rho}^{Q}(a)} \quad (2\text{-}7)$$

其中 m_t, m_f 为后在目的地与出发地的机动性（截断到 12），$n_{\mathrm{adj}}(a)$ 为与箭相邻的己方/对方后的个数。这些规则表达的是常见的经验：把后走到开阔处；用箭贴住对方

的后、封住对方的路；不要用箭堵自己；箭射在对方的领地或均势格上。给全部着法打分只需约 50 微秒。

可分离评估。廉价分数与真实的静态评估相关性不高：5.9 节的测试显示，静态评估最好的着法只有约一半排在廉价分数的前 24 名。另一个极端是对全部着法都走一步再评估，这在初始局面需要 2176 次评估，在本文开局阶段的测试局面上平均也要约 15 毫秒（5.9.1 节），作为 α-β 内部结点的排序太昂贵。本文在两者之间采用一种近似：把一步棋拆成“移后”和“射箭”两部分，分别评估再相加，

$$sep(f,t,a)=Q(f,t)+A(a),Q(f,t)=E(s\oplus(f\to t)),A(a)=E(s\oplus a)-E(s)\tag{2-8}$$

其中 $s\oplus(f\to t)$ 表示只移动后、不射箭的局面，$s\oplus a$ 表示只在 a 放一支箭、不移动后的局面（均以行棋方视角评估）。这样只需要“后的走法数 + 空格数”次评估（中局约 150 次，约 1—2 毫秒），就能给全部着法打分。它忽略了移后与射箭之间的相互作用（例如箭射在后刚离开的格子、或者箭的射线经过后的新位置），但实验显示它与真实静态评估高度一致：静态评估最好的着法有 72% 落在近似排序的前 8 名、92% 落在前 30 名（5.9.1 节）。

这一近似是本文为本项目设计的，其思想与亚马逊棋程序中常见的“先选后的走法、再选射箭位置”的两阶段做法相近[23,42]。由于没有见到完全相同的公开描述，本文把它作为一个实现选择而不是创新；它在消融实验中被证明是 α-β 最关键的组件（5.4 节）。

2.3 α-β 博弈树搜索

2.3.1 极小极大与 α-β 剪枝

对式 1-1 做深度为 D 的截断，用评估函数 E 代替叶结点的真值，就得到 Shannon 提出的有限深度极小极大搜索[4]。α-β 剪枝在不改变搜索结果的前提下跳过不可能影响根结点决策的分支：维护区间 (α,β)，若某结点的值已不可能落在区间内就停止搜索其余子结点[1,6]。Knuth 与 Moore 证明，在着法排序最优时，深度为 D、分支因子为 b 的均匀树上 α-β 需要访问的叶结点数为

$$b^{\lceil D/2\rceil}+b^{\lfloor D/2\rfloor}-1\tag{2-9}$$

约为极小极大的平方根[1]；Pearl 进一步分析了随机排序下的渐近分支因子[99,100]。因此，**着法排序的质量直接决定了 α-β 的效率**。在亚马逊棋中，即使 b 被束宽限制到 8，深度 4 的极小极大也需要 $8^4=4096$ 个叶结点，而最优排序下的 α-β 只需约 $2\times 8^2=128$ 个。本文的负极大值 α-β 写成算法 2-1 的形式（实际代码见 4.3 节）。

代码 2-1 负极大值 α-β 搜索（含本文的主要增强，伪代码）

```
function NEGAMAX(s, depth, α, β, ply):
    if 行棋方无子可动: return −(WIN − ply)              // 胜负分随距离递减
    if depth ≤ 0: return E(s)                          // 叶结点静态评估
    (ttMove, 截断) ← 查询置换表(s, depth, α, β)
    if 截断: return 置换表值
    if 已分隔: return E_sep(s)                         // 区域格数评估，不再向下搜索
    if depth = 1: moves ← 廉价分数前 P 个      // 子结点即叶结点，全部评估
    else:         moves ← 可分离评估前 W 个     // 束宽剪枝
    按 置换表着法 > 杀手着法 > 原排序 + 历史分 重新排列
    best ← −∞
    for i, m in enumerate(moves):
        s.play(m)
        if i = 0: v ← −NEGAMAX(s, depth−1, −β, −α, ply+1)
        else:                                          // PVS 零窗口试探
            v ← −NEGAMAX(s, depth−1, −α−ε, −α, ply+1)
            if α < v < β: v ← −NEGAMAX(s, depth−1, −β, −α, ply+1)
        s.undo()
        best ← max(best, v);  α ← max(α, v)
        if α ≥ β: 更新杀手与历史表; break
    存入置换表(s, depth, best, 边界类型, 最佳着法)
    return best
```

2.3.2 迭代加深与时间控制

迭代加深依次以深度 1, 2, 3, … 搜索，直到时间用完[68,69]。它的额外开销很小（每层的结点数大致按有效分支因子增长，前面各层的总和只占一小部分），却带来三个好处：任何时刻都有一个可用的最佳着法；上一层的最佳着法和置换表信息可以改进下一层的着法排序[71]；可以自然地控制时间。本文的时间策略是：若某一层完成时已用时间超过预算的一半，就不再开始下一层；若中途超时，则采用当前层已搜完部分中的最佳着法——由于上一层最佳着法总是排在第一个先搜，只要它已搜完，这一做法就是安全的。搜索每访问 8 个结点检查一次时间，实际用时一般不超过预算 5 毫秒。

2.3.3 主变例搜索

主变例搜索（PVS，又称 NegaScout）假设第一个着法就是最佳着法，对其余着法只用“零窗口”$(\alpha, \alpha+\varepsilon)$ 做试探：若试探结果不超过 α，说明该着法不优于当前最佳，直接跳过；只有试探失败时才用完整窗口重新搜索[70,71,101]。本文的评估值是实数，零窗口宽度取 $\varepsilon = 10^{-6}$。在着法排序较好时，绝大部分着法都能通过廉价的零窗口试探排除。与此相关的还有 SSS* 与 MTD(f) 等最佳优先或纯零窗口方法[102–104]，本文没有实现，但在 2.8 节中讨论。

2.3.4 置换表、杀手着法与历史启发

亚马逊棋中不同的着法顺序经常到达相同的局面（例如两个后各走一步的先后顺序互换）。置换表把已搜索局面的结果（值、深度、边界类型、最佳着法）存入哈希表，再次遇到时直接复用或用于排序[69,71]。本文的置换表有 2^{18} 项，采用“直接替换”策略（新结果总是覆盖旧结果），替换策略的比较见文献[72,105]。

除了置换表中的最佳着法外，本文还使用两种通用的排序增强：

- **杀手着法**：记录每一层最近两个引起 β 截断的着法，在同层其他结点优先尝试[73]；
- **历史启发**：每当着法在剩余深度 d 处引起截断，就把它的“出发格—目的地”对的历史分增加 d^2，排序时作为附加项[10]。由于完整的着法（含箭）数量太多、很少重复，历史表按“后的移动”而不是按完整着法索引。

排序时的优先级为：置换表着法 > 杀手着法 > 原排序（可分离评估或廉价分数）+ 历史分的微小加成。Schaeffer 的实验表明，在国际象棋中历史启发与置换表是贡献最大的两项增强[10]；在亚马逊棋中，由于束宽剪枝后每个结点只剩 8 个候选，这些增强的边际作用有多大，是 5.4 节要回答的问题。

2.3.5 束宽剪枝

本文采用**束宽剪枝**：在每个内部结点只搜索排序后的前 W 个着法（默认 $W=8$）；剩余深度为 1 的结点（子结点就是叶结点）则取廉价分数前 $P=24$ 个，全部做静态评估；根结点取可分离评估前 60 个候选，逐个做静态评估后搜索前 30 个。这是一种激进的前向剪枝，可能剪掉真正的最佳着法，但它是在亚马逊棋中让 α-β 达到 3—4 层深度的唯一办法。前向剪枝在国际象棋中有空着裁剪[106–108]、Futility 剪枝[109]等多种形式；在亚马逊棋中，Avetisyan 与 Lorentz 专门研究了选择性搜索[42]，国内的比赛程序也普遍只搜索评估排序前若干个着法[43,45,49]。5.4 节比较了不同束宽、不同排序方式的效果。

2.4 蒙特卡洛树搜索

2.4.1 从蒙特卡洛评估到 UCT

蒙特卡洛方法用随机对局的平均结果来估计局面价值，这一想法可以追溯到 Abramson 的“期望结果”模型[110]，并在双陆棋的在线策略改进[111]和蒙特卡洛围棋[112]中得到应用。2006 年前后，Coulom 提出把蒙特卡洛模拟与树搜索结合[75]，Kocsis 与 Szepesvári 提出了 UCT 算法[2]，把树中每个结点的着法选择看作多臂老虎机问题，使用 UCB1 公式[113]

$$a^{\star}=\arg\max_{a}\left[\dot{Q}(s,a)+c\sqrt{\frac{\ln N(s)}{N(s,a)}}\right] \quad (2\text{-}10)$$

其中 $\dot{Q}(s,a)$ 为着法 a 的平均奖励，$N(s)$、$N(s,a)$ 为访问次数，c 为探索常数。UCB 系列方法在随机老虎机问题上有对数级的遗憾界[113–116]，Kocsis 与 Szepesvári 证明了 UCT 在树上的一致性。此后 MCTS 迅速成为围棋程序的主流[88,117–120]，并被推广到 Hex[121]、Lines of Action[122]、亚马逊棋[24,25]、一般博弈[123,124]、不完全信息博弈[125]、实时策略游戏[126]等众多领域，Browne 等人与 Świechowski 等人先后给出了系统综述[3,127]。

2.4.2 四个阶段

MCTS 的每次模拟包括四个阶段[3,76]（流程图见图 4-3）：

1. **选择**：从根结点出发，按选择公式逐层选择子结点，直到到达一个还可以展开新子结点的结点；
2. **扩展**：为该结点展开一个新的子结点（按展开顺序取下一个候选着法）；
3. **模拟**：从新结点开始，用模拟策略得到一个结果（胜负或胜率估计）；
4. **回传**：把结果沿路径回传，更新每个结点的访问次数与累计奖励。

本文中，结点的累计奖励 W 以“**走入该结点的一方**”的视角累计，回传时每上一层取一次 $1-v$。最终着法选择访问次数最多的根子结点。若某个子结点走完后对方已无子可动（一步制胜），则该子结点被标记为终局，选择时直接选它；在开始模拟前，程序还会检查根结点是否存在一步制胜的着法。

2.4.3 展开顺序与渐进展开

亚马逊棋的分支因子使标准 UCT 很难工作：标准 UCT 要求每个子结点至少访问一次才开始比较，而根结点就有上千个子结点，200 ms 的模拟次数（约 2000—3000 次）只够把每个子结点访问一两次，树几乎无法向下生长。解决办法是**渐进展开**（progressive widening/unpruning）[76,77]：结点被访问 n 次时，最多只允许展开

$$k(n)=\lfloor C_{\mathrm{pw}}\, n^{\alpha}\rfloor \quad (2\text{-}11)$$

个子结点（本文 $C_{\mathrm{pw}}=2$，$\alpha=0.5$），子结点按某种**展开顺序**依次加入。这样搜索会先集中在排序靠前的少数着法上，随着访问次数增加再逐步考虑更多着法。展开顺序决定了先考虑哪些着法，本文实现了四种：

- **随机**：把全部合法着法随机打乱（不使用渐进展开时即为标准 UCT）；
- **廉价分数**：按式 2-7 取前 60 个；
- **可分离评估**：按式 2-8 取前 30 个；

- **学习策略**：按 2.5.1 节的线性策略取前 30 个。

每个结点的候选列表在第一次需要展开时计算并缓存。可分离评估的代价约 1—2 毫秒，相当于几十次模拟，因此“用更好的排序”与“做更多的模拟”之间存在取舍，5.5 节的消融实验专门测量了这一点。

2.4.4 PUCT 与先验

AlphaGo 系列使用的 PUCT 公式在 UCB 中引入策略先验 $P(s,a)$[17,18,80]：

$$a^{*}=\arg\max_{a}\left[\hat{Q}(s,a)+c_{\text{puct}}P(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)}\right] \tag{2-12}$$

与 UCT 不同，PUCT 中未访问着法的探索项与先验成正比，先验高的着法会被优先、更多地访问，先验极低的着法几乎不被访问。本文的先验有两种：

- **启发式先验**：对展开顺序所用的分数（可分离评估）做 softmax，$P(a)\propto\exp\left((sep(a)-max\,sep)/\tau\right)$，温度 $\tau=1$ 格；
- **学习先验**：2.5.1 节学习得到的线性 softmax 策略。

未访问结点的 $\hat{Q}$ 取 0.5。默认 $c_{\text{puct}}=1.5$，UCT 的默认 $c=0.5$；这些参数在独立于正式实验的 16 个开局上做了少量试调（5.1.3 节）。

2.4.5 模拟策略与评估截断

模拟阶段使用的策略决定了蒙特卡洛估计的质量。本文实现三种模拟方式：

- **随机模拟到终局**：双方随机走子直到一方无子可动。随机着法的生成不需要枚举全部着法：随机选一个后、随机选一个目的地、再随机选一个射箭位置即可，约 1 微秒一步；
- **评估截断**：随机走 L 步（默认 $L=4$）后，用评估函数估计胜率

$$\hat{p}=\sigma\left(k\cdot E(s)\right) \tag{2-13}$$

其中 $E(s)$ 以“格”为单位，$k=0.18$，取自第一版程序的自对弈数据上 Texel 拟合得到的缩放系数（2.5.2 节；修复分隔判定后重新拟合为 0.26，见 5.7.3 节，MCTS 中没有随之调整）；

- **直接评估**：不模拟，直接用式 2-13 评估新展开的结点（$L=0$）。

评估截断是亚马逊棋 MCTS 的标准做法：Lorentz 的 Invader 与 Kloetzer 的程序都发现，随机下到终局的结果几乎没有信息量，而短模拟加评估要好得多[24,25,78]。把评估函数引入 MCTS 的其他形式还有隐式极小极大回传、用浅层 α-β 代替模拟等[79,93,94,128]；Matsuzaki 讨论了评估函数是否真的能改进 MCTS[129]。5.5 节比较了不同的截断长度与模拟方式。

此外，若在树中到达一个已分隔的局面，程序直接用区域格数（式 2-6）判断胜负，不再模拟（“纯随机”配置除外）。

2.4.6 本文的 MCTS 配置

表 2-2 列出第五章使用的几种 MCTS 配置。它们按“从纯随机到加入领域知识”逐步变化，便于观察每类知识的作用。

表 2-2 实验中使用的 MCTS 配置

名称	选择	展开顺序	渐进展开	模拟	分隔判定	根结点一步制胜检查
MCTS（纯随机）	UCT，$c=0.5$	随机，全部着法	否	随机到终局	否	否
MCTS（UCT+截断）	UCT，$c=0.5$	随机，全部着法	否	截断 $L=4$	是	是
MCTS（廉价排序+截断）	UCT，$c=0.5$	廉价分数前 60	是	截断 $L=4$	是	是
MCTS（可分离排序，UCT）	UCT，$c=0.5$	可分离评估前 30	是	截断 $L=4$	是	是
MCTS（PUCT）	PUCT，$c=1.5$	可分离评估前 30	是	截断 $L=4$	是	是
MCTS（学习先验）	PUCT，$c=1.5$	学习策略前 30	是	截断 $L=4$	是	是

2.5 学习方法

本项目没有条件训练深度神经网络，但仍然可以从数据中学习。本节介绍两种轻量级学习方法：一是学习“在某个局面下强程序会走哪一步”的走子策略，二是学习“某个局面最终谁会赢”的评估函数。两者的训练数据都来自 α-β 的自对弈棋谱（5.7.1 节）。

2.5.1 线性 softmax 走子策略

由于合法着法太多，先用可分离评估取前 40 个作为候选集合 $M(x)$，再在候选集合上建立条件 logit（多项 logit）模型[81,130]：

$$\pi_\theta(m\mid x)=\frac{\exp\left(\theta^{\top}\phi(x,m)\right)}{\sum_{m'\in M(x)}\exp\left(\theta^{\top}\phi(x,m')\right)} \tag{2-14}$$

其中 $\phi(x,m)\in R^{12}$ 为着法特征（表 2-3）。这一模型等价于广义 Bradley–Terry 模型[85,131]，与 Coulom 在围棋中计算“着法模式 Elo”的方法[77]、Araki 等人的最大熵走子预测[82]、Stern 等人的贝叶斯模式排序[132]属于同一类，也可以看作深度策略网络[17,133,134]的线性特例。

表 2-3 线性策略的着法特征

编号	特征	含义
1	mobDelta	后在目的地与出发地的机动性之差（÷4）
2	mobT	后在目的地的机动性（÷12）
3	oppAdj	与箭相邻的对方后的个数
4	ownAdj	与箭相邻的己方后的个数（走子后）
5	terr	箭落在对方领地或均势格上
6—7	dOpT1、dOpT2	目的地到对方后的后距离为 1 / 为 2
8	arrowNearOpp	箭落点到对方后的后距离为 1
9	arrowDist	箭与目的地的切比雪夫距离（÷9）
10	qDist	后移动的切比雪夫距离（÷9）
11	opMobLoss	箭使对方各后机动性减少的总量（÷8）
12	sepScore	可分离评估分数与候选中最高分之差（÷4）

给定训练集 $\{(x_i,m_i)\}$（m_i 为棋谱中实际走的着法，只保留落在候选集合内的样本），以带 L2 正则的负对数似然为目标：

$$L(\theta)=-\frac{1}{n}\sum_{i=1}^{n}\log\pi_\theta(m_i\mid x_i)+\frac{\lambda}{2}\lVert\theta\rVert^2 \tag{2-15}$$

该目标是凸函数（多项 logit 与正则化的一般理论见文献[135–137]），本文用 Adam 优化器[138]（学习率 0.05，2000 步，$\lambda=10^{-3}$）求解。训练好的模型作为 MCTS 的先验与展开顺序。

2.5.2 评估函数的 Texel 式调参

评估函数的权重（表 2-1）是凭经验设定的。一个自然的问题是：能否从对局结果中学到更好的权重？这一思路在棋类中有很长的历史：Samuel 的跳棋程序就在自动

调整评估权重[7]，Buro 用统计方法为黑白棋组合特征[83,139,140]，Tesauro 的比较训练[141]、时间差分学习[16,142–145]、自举学习[146]、Hoki 与 Kaneko 的大规模优化[84]，以及基于进化算法或粒子群的调参[50,51,147–149]都属于这一方向。国际象棋社区中流行的“Texel 调参”方法用逻辑回归把评估值映射为胜率，再最小化与实际对局结果之间的交叉熵。

本文采用这一方法。由于式 2-5 对 10 个权重是线性的，可以把每个局面写成 10 维特征 x_i（前 5 维为 pf_k，后 5 维为 $(1-p)f_k$，白方视角），模型为

$$P(\text{白方获胜}\mid x_i)=\sigma\left(K\,a^{T}x_i\right)\tag{2-16}$$

按 Texel 方法的惯例分两步：先固定手工权重 a_0，只拟合缩放系数 K；再固定 K，以 a_0 为起点，用 L-BFGS[150] 最小化训练集交叉熵加上向手工权重收缩的弱正则项 $\lambda\,\|a-a_0\|^2$（$\lambda=10^{-4}$）。拟合得到的 K 同时用作 MCTS 中评估值到胜率的换算系数（式 2-13）。

2.5.3 学习组件与深度强化学习的关系

需要说明，上面两种方法都属于**监督学习**（模仿 α-β 的走法、拟合对局结果），而不是 AlphaZero 意义上的“自对弈强化学习”[18,19,151]：后者需要“用当前模型自对弈—用搜索结果训练模型—再自对弈”的迭代循环，以及表达能力强得多的深度网络。本项目在算力上无法完成这样的循环（AlphaZero 训练使用了数千个 TPU[19]，即使是加速版本的 KataGo 也需要数十 GPU 日[152]），因此只把它作为相关工作讨论（2.8 节）。国内也有研究尝试把神经网络与强化学习用于亚马逊棋[52–55]，但规模同样有限。这是本文在学习部分的一个明确局限（5.11 节）。

2.6 分隔后的填格策略

双方被完全分隔后，局面变成两个互不相干的“单人游戏”：每方只需尽可能多地走棋。理论上，即使是这种单人填格问题，求最优解也是 NP 困难的[33]；专门的残局求解器需要复杂的区域分解与搜索[26,34,35]。本文不研究残局求解，而是让除随机走子以外的**所有智能体在分隔后使用同一个简单的填格策略**：

1. 枚举己方全部合法着法，计算走完后己方仍能到达的空格数（用王距离 BFS）；
2. 选择使该数最大的着法；若有多个，优先把箭射在周围空格最少的位置（先填死角，减少对自己区域的切割）。

这一策略是贪心的，在形状复杂的区域中并不最优，但它对所有智能体都相同。这样做的目的有两个：一是让对比实验的胜负主要取决于**分隔之前的争夺阶段**，这是本文关心的部分；二是大幅缩短实验时间（填格阶段每步只需几毫秒）。代价是：本

文的结论不涉及残局能力，若某种算法在残局上更强，这一优势在本文的实验中被抹去了（5.11 节）。

2.7 棋力评价的统计方法

2.7.1 Elo 模型与 Bradley–Terry 模型

衡量博弈程序棋力的标准做法是让它们相互对弈，再用等级分模型估计相对强度。Elo 模型假设选手 i 对 j 的期望得分为[86]

$$E\left[S_{ij}\right]=\frac{1}{1+10^{-(R_i-R_j)/400}} \quad (2\text{-}17)$$

这就是 Bradley–Terry 配对比较模型[85]在对数优势比尺度上的换算。给定全部对局结果，用最大似然（或带弱先验的最大后验）估计各选手的等级分 R_i[131,153,154]，本文用 L-BFGS 方法[150]求解；TrueSkill 等贝叶斯模型则进一步处理时间变化与多人对局[155]。等级分只有差值有意义，本文把“一步贪心”固定为 0 作为参照点。亚马逊棋没有和棋，这简化了模型。关于 Elo 体系的统计性质与局限，可参见文献[156–160]，其中 Hamilton 讨论的“非传递性”问题在 5.3 节中也会遇到。

两个选手之间的得分率 $\hat{s}$ 可以换算为 Elo 差

$$\Delta R=-400\log_{10}\left(\frac{1}{\hat{s}}-1\right) \quad (2\text{-}18)$$

例如得分率 64% 约相当于 +100 Elo，76% 约相当于 +200 Elo。

2.7.2 置信区间与显著性

对局数有限时，得分率的抽样误差很大：32 局的得分率，其 95% 置信区间半宽约为 ±17 个百分点。本文对每组配对对局报告两种区间：

- **Wilson 得分区间**：二项比例的近似区间，在小样本下比正态近似更可靠[161,162]（精确的 Clopper–Pearson 区间[163]偏保守）；
- **按开局配对的自助法区间**：以开局为单位（每个开局包含交换先后手的两局）重采样 4000 次，取 2.5% 与 97.5% 分位数[87,164]。这种区间考虑了同一开局两局结果之间的相关性。

显著性检验采用配对符号检验：每个开局上，若变体两局得分之和大于 1 记为“+”，小于 1 记为“−”，等于 1 不计，再做二项检验。Elo 等级分的区间同样用自助法得到：在每个对阵组合内按开局重采样后重新拟合模型，重复 300 次。

在多组比较中，即使所有变体都与基准一样强，也会有约 5% 的比较“显著”。本文的消融实验包含 20 余组比较，因此对单个 p 值在 0.01—0.05 之间的结论保持谨

慎，主要关注效应量与区间，并对关键结论做独立种子的重复实验（相关讨论参见文献[66,67,165,166]）。

2.7.3 其他评测

对局结果是“最终指标”，但它把许多因素混在一起。为了更细致地理解算法，本文还使用三类专项评测：

- **决策质量测试**：在 60 个测试局面上，用更长时间的 α-β 搜索作为参考，测量各智能体在 200 ms 内所选着法的“遗憾值”（5.8 节）；
- **搜索效率**：着法排序的质量、固定深度下的结点数与首着截断率，以及 MCTS 每秒模拟次数（5.9 节）；
- **学习的离线指标**：测试集上的负对数似然、Top-k 命中率与交叉熵（5.7 节），校准评价参见文献[167–171]。

2.8 相关研究综述

本节按主题梳理与本项目相关的研究。为了避免“只列文献不做比较”，每一小节最后都说明这些工作与本文的关系。文献的选择以“与亚马逊棋 AI 的搜索、学习、评价直接相关”为原则；对于更一般的人工智能背景，读者可参考教材[20,21,172,173]与综述[174–176]。

2.8.1 博弈树搜索与 α-β 的增强

基础与理论。 博弈树搜索的基础是 von Neumann 的极小极大定理[60,61]与 Shannon 的下棋程序设计[4]。α-β 剪枝的早期历史与分析见 Knuth 与 Moore 的经典论文[1]，其平均情形的效率由 Pearl 给出[99,100]，Marsland 等人对剪枝技术与相对效率做了系统比较[71,101,177]。关于极小极大本身是否“有益”，存在著名的“病态”（pathology）问题：在某些随机树模型中，搜索越深反而决策越差[64,178,179]；Sadikov 等人用国际象棋残局实证考察了搜索与知识的关系[63]，Althöfer 构造了对 α-β 特别容易的博弈树[180]。Michie 从信息论角度讨论了国际象棋的复杂性[65]，Thompson 研究了计算机国际象棋棋力与搜索深度的关系[181]。

α-β 的增强。 迭代加深的思想来自 CHESS 4.5[69]，Korf 给出了一般的深度优先迭代加深分析[68]，Fink 提出了迭代 α-β 的改进[182]。着法排序方面有杀手启发[73]、历史启发[10]，以及基于模式与着法影响的排序方案[183,184]和“守卫启发”[185]。窗口技术方面有 Scout/NegaScout[70]与基于零窗口的 MTD(f)[103]，de Bruin 等人以“解树”为基础统一了这些方法[104]。置换表方面，Zobrist 哈希[9]、替换策略[72]、多次探测[105]、利用置换

表“从经验中学习”[186]、在随机博弈树中的使用[187]，以及并行搜索中的无锁实现[188]都有专门研究。

选择性搜索。 为了让搜索把时间花在关键变化上，人们提出了静态搜索[189–191]、单一着法延伸[192]、空着裁剪及其变体[106–108,193]、Futility 剪枝等前向剪枝方法（Hoki 与 Muramatsu 比较了它们在将棋中的效率[109]），以及基于实现概率的搜索[194]。另一条路线是放弃固定深度，采用最佳优先的方法：启发式最佳优先搜索 A[195] *在博弈中的对应物 B* 算法[196]、SSS*[102]、阴谋数搜索[197,198]、极小极大近似[199]、最佳优先极小极大[200]、偏序界[201]等。Junghanns 讨论了这些方法能否真正替代 α-β[202]，Mańdziuk 研究了实值评估函数下的 α-β 增强[203]。Adelson-Velsky 等人很早就讨论了控制搜索的多种方法[204]，Ballard 将极小极大推广到含机会结点的树[205]。

并行与系统。 并行博弈树搜索有大量研究[206–209]。工程上最具代表性的是 Deep Blue：它使用专用芯片和大规模并行 α-β 搜索[11,12]。Chinook 结合深度搜索、开局库与残局库成为跳棋世界冠军级程序，并最终证明了跳棋是和棋[13,14]，其中残局库依赖逆向分析[210]。国际象棋程序的发展史与入门介绍可见文献[5,211–213]，关于搜索深度与棋力关系的实证研究见文献[214,215]。

与本文的关系。 本文的 α-β 采用其中最成熟的一组技术（迭代加深、PVS、置换表、杀手、历史），再加上激进的束宽剪枝，没有实现空着裁剪、静态搜索等国际象棋特有的技术：亚马逊棋没有“吃子”，局面不存在明显的“不稳定”状态，静态搜索的必要性不大；空着裁剪则与束宽剪枝的作用重叠。本文的贡献不在于提出新的增强，而在于在亚马逊棋上逐一测量这些增强的实际作用（5.4、5.9 节）。

2.8.2 证明数搜索与博弈求解

证明数搜索由 Allis 等人提出[216]，并与威胁空间搜索一起用于求解五子棋[217]。Kishimoto 等人的综述总结了它在头二十年的发展[218]；van den Herik 与 Winands 也对 PNS 及其变体做了综述[219]。在算法层面，研究者提出了两层 PNS[220]、PN* 与深度优先 df-pn[221]、处理置换的方法[222]、解决“图历史交互”（GHI）问题的方法[223]、关于 df-pn 完备性的分析[224]、处理无限循环与估计偏差的技术[225]、新的证明数计算方法[226]、减少“跷跷板效应”的深度证明数搜索[227]、并行 df-pn[228,229]以及基于概率的证明数搜索[230]。Gao 等人用卷积神经网络为 df-pn 提供启发[231]。在应用方面，PNS 被用于求解 Breakthrough[232]等游戏；PNS 与 MCTS 的结合也是近年的研究热点：蒙特卡洛证明数搜索[233]、MCTS 与 PNS 的组合[234]、基于证明数的 MCTS[235]。威胁空间搜索方面，λ 搜索把“威胁的阶数”形式化[236]，抽象证明搜索利用目标相关着法缩小搜索空间[237]。

在亚马逊棋中，“求解”主要发生在分隔后的残局：每个区域都是一个单人或双人的小游戏，可以用组合博弈论把整个局面分解为若干子游戏的和[32,238]。Kloetzer 等人比较了多种残局求解器（包括 PNS 与基于组合博弈论的方法）[26]，Song 与 Müller 提出了增强的残局求解器[34]，Snatzke 则在小棋盘上做了穷举[35]。

与本文的关系。 本文没有实现 PNS 或残局求解器。分隔后的局面由一个简单的贪心填格策略处理（2.6 节），这是本文的一个明确简化。

2.8.3 蒙特卡洛树搜索及其改进

起源与理论。 MCTS 的思想来源包括期望结果模型[110]、在线策略改进[111]、稀疏采样规划[239]和蒙特卡洛围棋[112]。Coulom 的“选择性与回传算子”[75]和 Kocsis 与 Szepesvári 的 UCT[2]标志着现代 MCTS 的诞生，其理论基础是多臂老虎机[113–116,240]。Rosin 提出带上下文先验的老虎机算法[80]，成为 PUCT 的理论来源；Silver 与 Veness 把 MCTS 推广到部分可观测环境[241]；Vodopivec 等人讨论了 MCTS 与强化学习的关系[242]，Grill 等人把 AlphaZero 式的 MCTS 解释为正则化的策略优化[243]。教程与分类可见文献[244,245]。

在围棋中的发展。 MCTS 在围棋中经历了大量改进：结合离线与在线知识[117]、RAVE[118]、渐进策略[76]、专家知识与探索[246]、交叉熵调参[247]、上下文模式[248]、“共同命运图”模式[249]、动态贴目[250]、时间管理[251,252]等，Gelly 等人的综述[88]和 Fuego 等开源框架[119]总结了这一阶段的成果。也有研究分析了模拟本身[253]、MCTS 在围棋中的局限[254]、静态知识偏置的效率[255]、激励学习[256]以及 MCTS 在围棋中的评价[120,257]。在深度学习时代，卷积网络被引入 MCTS[258,259]，并最终发展为 AlphaGo[17]。

选择公式与探索。 除了 UCT 和 PUCT，研究者还提出了改进的 UCB[260,261]、面向“简单遗憾”的探索调节[262]、多臂老虎机探索的改进[263]、新颖性奖励[264]，以及用进化或蒙特卡洛搜索自动发现探索项的方法[265–267]。参数的随机化与自适应在一般博弈中也有研究[268–270]，RAVE 的各种变体在一般博弈中被比较[271]。

模拟策略。 模拟策略的设计影响巨大：在线学习模拟策略[272–274]、学习非随机着法[275]、模拟策略自适应[276]、模拟优化[277]等。Teytaud 指出决定性着法规则带来巨大收益[278]。

树结构与并行。 置换与着法分组[279,280]、分数有界 MCTS[281]、祖先 α-β 界[282]、批量 MCTS[283]、束 MCTS[74]等改进了树的组织方式。并行化方面有根并行、树并行与虚拟损失[284–288]以及分布式 MCTS[289]。

在其他游戏中的应用。 MCTS 已被用于 Hex[121]、Lines of Action[122]、亚马逊棋（2.8.6 节）、“突然死亡”类游戏[290]、一般博弈[123,124]、不完全信息博弈[125,291]、实时策略游戏[126]，也被用于评估玩家策略的复杂性[292]。Berthier 等人讨论了自动调参 MCTS 的一致性问题[293]。综述[3,127]对此有全面总结。

与本文的关系。 本文实现的 MCTS 吸收了其中的几项关键思想：渐进展开、PUCT 先验、评估截断模拟与一步制胜检查。没有实现 RAVE：亚马逊棋的着法由“移后 + 射箭”组成，同一着法在不同局面中几乎不会重复出现，“所有着法同时更新”的统计量很难积累（2.9 节）。

2.8.4 MCTS 与极小极大的混合

由于 MCTS 在“陷阱”多的游戏中表现不佳[89,90]，许多研究尝试把极小极大搜索的优势引入 MCTS：MCTS-Solver 在树内传播胜负证明[294]；隐式极小极大回传把评估函数值与模拟结果结合起来[93]；Baier 与 Winands 系统研究了在选择、扩展、模拟各阶段嵌入浅层极小极大搜索的混合方案[91,92]；Winands 与 Björnsson 在模拟中使用 α-β[94]；Lorentz 讨论了评估函数在 MCTS 中的用法[79]；Matsuzaki 实证检验了评估函数是否真正改进 MCTS[129]；Winands 等人把基于评估函数的 MCTS 用于 Lines of Action[128]。

与本文的关系。 本文的 MCTS 以“评估截断”的方式使用评估函数，属于其中最常见的一类；没有实现隐式极小极大回传或在模拟中嵌入 α-β。国内也有在亚马逊棋上把 UCT 与 PVS 结合的尝试[29,48]。5.3 与 5.6 节在相同知识、相同时间下直接比较了两类框架。

2.8.5 机器学习与深度强化学习在博弈中的应用

早期学习方法。 Samuel 的跳棋程序[7,8]开创了“从对局中学习评估函数”的方向。此后有时间差分学习[15,16,142–144]、比较训练[141]、统计特征组合[83,139,140]、自举学习[146]、大规模优化[84]等。Fürnkranz[295]和 Mańdziuk[296]分别综述了国际象棋中的机器学习和“无知识/基于学习”的博弈方法。Lucas 与 Runarsson 比较了时间差分学习与协同进化[145]，Szubert 与 Jaśkowski 在 2048 中使用 n 元组网络[297]。

走子预测。 在围棋中，从棋谱学习走子模型的方法包括 Bayes 模式排序[132]、着法模式 Elo[77]、最大熵模型[82]，以及后来的深度卷积网络[133,134,259]。本文的线性策略正是这一路线在亚马逊棋上的轻量版本。

深度强化学习。 DQN 在 Atari 游戏上达到人类水平[298]，相关评测平台包括 ALE[299,300]、Gym[301]、OpenSpiel[302]与 GVGAI[303]。在棋类中，AlphaGo 结合策略网络、价值网络与 MCTS 击败了人类顶尖棋手[17]；AlphaGo Zero 和 AlphaZero 完全通过自对

弈学习[18,19,304]；Anthony 等人同时提出了“专家迭代”[151]；MuZero 进一步学习环境模型[305]，并被扩展到复杂动作空间[306]。开源复现包括 ELF OpenGo[307] 与 KataGo[152]。DeepChess[308]、Giraffe[309]等则用深度学习改进国际象棋评估。强化学习的一般理论见 Sutton 与 Barto 的教材[310]，常用算法包括 Q 学习[311]、REINFORCE[312]与 PPO[313]，Silver 等人更提出“奖励即足够”的观点[314]。

更广泛的博弈 AI。 深度强化学习与搜索的结合也被用于扑克[315–317]、星际争霸 II[318]、Dota 2[319]、外交[320]、西洋陆军棋[321]，以及统一完全与不完全信息的“Student of Games”[322]；同样的“搜索 + 学习”范式还被用于发现矩阵乘法与排序算法[323,324]。

AlphaZero 的改进与分析。 针对 AlphaZero，研究者提出了多种效率改进[325–328]、替代损失函数[329]、超参数作为搜索偏置的分析[330]、在非确定性游戏中的应用[331]、在四子棋上的实现[332]、“精确胜利策略”[333]，以及神经缩放规律[334]。自对弈本身也有专门的综述与研究[335–338]。概念层面的分析表明，AlphaZero 内部确实习得了人类可理解的棋类知识[339–341]。然而，超人类水平的程序也并非无懈可击：对抗策略可以击败超人类围棋 AI[342]，AlphaZero 式智能体对对抗扰动也不鲁棒[343]。

缩放规律。 近年的研究开始系统地测量“计算量—棋力”关系：Jones 在棋盘游戏上研究了训练与测试计算之间的权衡[344]；Neumann 与 Gros 研究了多智能体强化学习模型的缩放规律[345]；语言模型的缩放规律[346]和测试时计算缩放[347,348]也引起了广泛关注；也有工作研究用大规模 Transformer 进行“摊销规划”[349]，或让语言模型通过内外部规划下棋[350]。早期的国际象棋研究同样观察到搜索深度增加带来的“收益递减”[214,215]。

与本文的关系。 本文的学习组件在规模上与 AlphaZero 不可同日而语，属于“特征工程 + 线性模型”的经典路线。5.6 节的算力缩放实验在小尺度上呼应了缩放规律研究的问题：在固定算法下，增加思考时间能换来多少棋力？

2.8.6 亚马逊棋的理论与程序

组合博弈论与复杂性。 亚马逊棋的残局是组合博弈论的经典研究对象。Berlekamp 等人的《Winning Ways》奠定了“游戏的和”的理论基础[238]，Berlekamp 分析了 N×2 窄棋盘上亚马逊棋的和[32]，Snatzke 用穷举得到了小棋盘上的新结果[35]。复杂性方面，Buro 证明了一类简单亚马逊棋残局与三次子网格图上哈密顿回路问题的联系，说明残局判定是 NP 完全的[33]；Hearn 证明了广义亚马逊棋、Konane 与 Cross Purposes 都是 PSPACE 完全的[36]，Hearn 与 Demaine 在专著中系统总结了这类结果[37]。类似的结论还有广义五子棋[38]、广义围棋[39]、黑白棋[351]、广义国际象棋[352]以及一般的两人完全信息博弈[40,62]。与亚马逊棋同属“组合博弈论友好”的游戏还有

Konane[353]、Clobber[354–356]与 Domineering 等，它们常被用来检验“组合博弈论 + 搜索”的结合方法。

评估与 α-β 程序。 Müller 与 Tegos 的实验研究是计算机亚马逊棋的早期代表，系统比较了多种评估特征与搜索技术[23]；Avetisyan 与 Lorentz 研究了亚马逊棋中的选择性搜索[42]；Lieberum 提出的“后距离 + 王距离 + 阶段权重”评估函数成为后来大多数程序的基础[30,31]。Iida 与 Müller 报告了早期的计算机亚马逊棋锦标赛[41]。国内的亚马逊棋研究多与全国大学生计算机博弈大赛相关，主要发表在中国控制与决策会议等会议上，内容包括搜索算法的分析与改进[43–46]、区域划分搜索[47]、评估函数的优化（包括粒子群与并行遗传算法调参）[49–51]、基于 PVS 的并行优化与开局库[27,28]。

MCTS 程序。 Lorentz 在 2008 年报告，他的亚马逊棋程序改用“蒙特卡洛模拟 + 评估截断”后胜过了原来的 α-β 版本[24]，后来又系统讨论了“提前终止模拟”与评估函数在 MCTS 中的用法[78,79]。Kloetzer 等人研究了 MCTS 的亚马逊棋开局库[25]。Kato 等人分析了 MCTS 在变体“骑士亚马逊棋”上的表现[357]。国内工作包括 UCT 算法在亚马逊棋人机对弈中的应用[48]、UCT 与 PVS 的混合[29]、ε-贪心 MCTS[58]、MCTS 的效率优化[57]以及 MCTS 与评估函数的 GPU 加速[56]；Sun 等人对亚马逊棋的博弈算法做了综述[59]。

学习方法。 用神经网络和强化学习下亚马逊棋的尝试包括：神经网络结合分布式计算[52]、基于强化学习的对弈系统与评估函数设计[53,54]、强化学习的应用[55]。这些工作的规模都远小于 AlphaZero，报告的棋力也多以与课程或比赛程序的对局为准。

与本文的关系。 本文的领地评估直接采用 Lieberum 的特征[30]，评估截断的 MCTS 沿用 Lorentz 等人的做法[24]，束宽剪枝与 Avetisyan 等人的选择性搜索[42]、国内程序的常见做法一致。与这些工作不同的是：（1）本文在**同一知识层、同一时间预算、同一填格策略**下对两类框架做了系统比较，而不是各自报告与不同对手的结果；（2）报告了细粒度消融、缩放与置信区间；（3）提供了可解释的网页界面。本文的程序没有开局库、残局求解器与并行搜索，在绝对棋力上很可能弱于这些比赛程序，这一点在 5.11 节中说明。

2.8.7 其他棋类中的评估与学习

评估函数的构造与学习在许多棋类中都有研究，可以为亚马逊棋提供借鉴。黑白棋中有神经网络评估[358]、小型网络的时间差分学习[359]、基于胜率的时间差分学习[360]、前瞻深度对学习的影响[361]、资源受限纳什记忆[362]、偏好学习[363]，以及对固定对手学习与自对弈学习的比较[337]。将棋中有基于棋谱的评估学习[364]、自对弈学习[365]、均匀性正则网络与多任务学习[366,367]、小规模深度学习压缩评估函数[368]，Iida 等人综述了计算机将棋[369]，Takeuchi 等人估计了程序等级分并比较了不同棋类的搜索行为

[370,371]。国际象棋中有序相关调参[372]、差分进化与进化算法调参[147–149]。中国象棋中有混合评估函数[373]，Yen 等人综述了计算机中国象棋[374]。Lines of Action[375]、四子棋[376,377]中同样有评估函数设计的研究，Chang 等人用残局库验证与微调评估函数[378]。Hex 中有死点分析[379]、虚连接与局面化简[380]、作业级 UCT 求解开局[381]，Gale 证明了 Hex 与 Brouwer 不动点定理的联系[382]。一般博弈（GGP）中的评估函数自动构造[383,384]、策略自适应[385]与国际 GGP 比赛[386]，则代表了“不依赖特定游戏知识”的方向。

与本文的关系。 本文的 Texel 调参属于“从对局结果拟合线性评估”的路线，与 Buro、Hoki 等人的方法同源；5.7 节同时用离线交叉熵与对局结果评价学到的权重，并讨论两者之间的关系（参见“评估精度与搜索棋力之间并非单调关系”的已有观察[129,372]）。

2.8.8 评价方法与可复现性

等级分。 Elo 体系[86]及其统计基础 Bradley–Terry 模型[85]、Luce 选择公理[81]和 Plackett–Luce 模型[130]是对局评价的基础。Hunter 给出了广义 Bradley–Terry 模型的 MM 算法[131]，Glickman 处理了动态配对比较[154]，Coulom 提出了“全历史等级分”[153]，Herbrich 等人提出了 TrueSkill[155]。也有研究用等级分刻画局部模式[387]。统计方法方面，本文用到了 Wilson 区间[161–163]和自助法[87,164]。

实验方法与可复现性。 Cohen 的《人工智能的经验方法》[66]系统讨论了 AI 实验设计。Henderson 等人[67]和 Agarwal 等人[166]指出深度强化学习实验中普遍存在的统计问题，Demšar 讨论了多数据集上的统计比较[165]，Lipton 与 Steinhardt 批评了机器学习文献中的不良趋势[388]。可复现性方面，Gundersen 与 Kjensmo 调查了 AI 研究的可复现性现状[389]，Pineau 等人报告了 NeurIPS 可复现性计划[390]，Peng[391]、Sandve 等人[392]与 Wilson 等人[393]给出了计算研究可复现的实践建议。

与本文的关系。 本文遵循这些建议：所有实验由脚本生成、固定随机种子、报告置信区间与效应量、明确说明哪些比较不显著。

2.8.9 可解释性、人机交互与可视化

博弈 AI 的可解释性既包括理解模型学到了什么[339,394]，也包括解释单个决策（例如用特征显著性解释智能体的着法[395]）。人机交互方面，Amershi 等人提出了人机交互设计准则[396]，Shneiderman 提出“以人为中心的 AI”[397]；McIlroy-Young 等人研究如何让超人类 AI 与人类行为对齐[398]，Shin 等人发现超人类 AI 促使人类棋手做出更具新意的决策[399]。可视化设计方面，本文参考了图形感知[400,401]与可视化分析设计[402]的研究来设计界面中的热力图与图表。

与本文的关系。 本文的网页界面把“AI 为什么这样走”拆解为领地图、候选着法与搜索统计、主变例三个层次，属于“局部解释”的朴素实现；它主要服务于课程演示与教学，而不是严格意义上的可解释性研究。

2.8.10 本文的定位

综上，本文所用算法均来自已有研究，本文的定位是：**在一个可控、可复现的统一框架中，对亚马逊棋上的经典搜索与轻量学习方法做系统的实证比较，并把算法的决策过程以可交互的方式呈现出来**。表 2-4 从几个维度把本文与代表性工作做了对比。

表 2-4 本文与代表性相关工作的比较

工作	游戏	搜索方法	学习	多算法统一比较	可交互界面
Müller 与 Tegos[23]	亚马逊棋	α-β	无	部分（评估特征比较）	否
Lieberum[30]	亚马逊棋	α-β	无	否（评估函数设计）	否
Lorentz[24]	亚马逊棋	MCTS + 评估截断	无	α-β 与 MCTS 两个版本	否
Kloetzer 等[25]	亚马逊棋	MCTS	无（开局库）	—	否
Baier 与 Winands[92]	多种棋类	MCTS–极小极大混合	无	是	—
Li 等[29]	亚马逊棋	UCT–PVS 混合	无	—	—
Zhang 等[57]	亚马逊棋	MCTS 优化	—	—	—
本文	亚马逊棋	α-β、6 种 MCTS	线性策略、Texel	是（8 个智能体循环赛 + 24 组消融）	是

2.9 技术路线中的关键设计决策

最后，把本章介绍的方法与本项目的具体选择总结为表 2-5。每一项都列出了备选方案与取舍理由，第三章将从“问题求解”的角度对其中几项做更深入的分析。

表 2-5 关键设计决策汇总

决策点	本文选择	备选方案	取舍理由
棋盘	标准 10×10	6×6、8×8 小棋盘	标准棋盘是文献与比赛的通用设置，结论更有参照意义
走法表示	整数编码 + 预计算射线	位棋盘	实现简单，开局 2176 个着法的生成只需约 10 微秒

决策点	本文选择	备选方案	取舍理由
评估函数	Lieberum 特征 + 阶段线性插值	神经网络；原始非线性阶段函数	线性形式可以直接用 Texel 方法学习，速度快、可解释
α-β 框架	PVS + 迭代加深 + 置换表	MTD(f)、SSS*	PVS 实现简单且足够高效
着法排序	可分离评估（移后 + 射箭分别评估）	廉价规则分数；逐一完整评估	前者质量差，后者太慢；可分离评估兼顾两者
前向剪枝	束宽 8（根结点 30）	全宽；空着裁剪	分支因子上千，不剪枝只能搜 2 层
MCTS 展开	渐进展开 + 可分离评估顺序	标准 UCT（全部展开）	根结点上千个子结点，标准 UCT 的树长不起来
MCTS 选择	PUCT（启发式先验）	UCT、RAVE	先验提供软剪枝；RAVE 的统计量在亚马逊棋中很难积累
模拟策略	随机 4 步 + 评估截断	随机下到终局	随机终局结果的信息量很低（Lorentz 等人的经验）
残局	共用的贪心填格策略	残局求解器	不在本文研究范围；保证比较集中在争夺阶段
学习	线性策略 + Texel	深度网络自对弈	算力限制；可在单机分钟级训练
运行环境	浏览器 Web Worker	服务器端引擎	满足“HTML 展示、无需安装”的要求
评价	配对开局 + Elo + 自助法	标准初始局面对弈	消除先手优势与确定性带来的偏差

2.10 本章小结

本章给出了“共享知识层 + 两类搜索框架 + 两种学习组件 + 统计评价”的技术路线。知识层用后距离与王距离定义领地评估，并用“可分离评估”在 1—2 毫秒内给全部着法排序；α-β 搜索以 PVS、迭代加深、置换表、杀手与历史启发提高效率，用束宽剪枝应对上千的分支因子；MCTS 以渐进展开、PUCT 先验与评估截断适应亚马逊棋的大分支因子；分隔后的残局交给一个所有智能体共用的填格策略；学习组件分别从自对弈数据中学习走子策略与评估权重；评价方面以配对开局、Bradley–Terry 模型与自助法为核心。相关研究综述表明，本文用到的算法都有成熟的文献基础，本文的价值在于统一实现与系统比较。下一章将从“智能问题求解”的角度，进一步分析这些设计背后的问题建模与权衡。

三、智能问题求解分析（包括使用 AI 工具解决方法）

第二章介绍了“用什么算法”，本章讨论“为什么这样用”。我们把亚马逊棋决策看作一个智能问题求解任务，依次分析问题建模、搜索空间压缩、知识表示、领地争夺中的权衡、近似与不确定性和算法选择，并提出第五章实验要回答的研究问题。3.7 节如实说明本项目使用 AI 工具的方式、AI 犯过的错误以及核验手段。

3.1 问题建模

3.1.1 智能体与环境

按照 Russell 与 Norvig 的智能体框架[20]，亚马逊棋 AI 是一个在“棋盘环境”中感知局面、选择着法的智能体。表 3-1 从环境性质的角度刻画这一问题。

表 3-1 亚马逊棋决策问题的环境性质

性质	亚马逊棋	对算法设计的含义
可观测性	完全可观测	不需要信念状态，可直接在局面上搜索
确定性	确定性	不需要机会结点，可用极小极大
智能体数量	两人对抗、零和	采用负极大值形式；对手被假设为最优
回合性	回合制、离散	可在每一步之间“思考”
静态性	思考期间环境不变，但有时间限制	需要“随时可停”的算法（迭代加深、MCTS）
已知性	规则完全已知	可以使用基于模型的搜索，无需学习环境模型
回合数	有限（每手放一支箭，最多 92 手）	博弈值存在且唯一，且没有和棋
动作空间	复合动作（移后 + 射箭），开局 2176 个	着法生成与排序是主要开销，需要分解动作

与五子棋、国际象棋相比，最后两行是亚马逊棋的特殊之处：一方面，每一手都会永久地减少一个空格，对局长度有严格上界，也不存在循环与和棋，这让“谁先无棋可走”成为唯一的终局条件；另一方面，一个“着法”是由两个子动作组合而成的，动作空间是两个子空间的笛卡儿积的子集。在 PEAS 描述中：性能度量是对局胜负；环境是棋盘与对手；执行器是“移动一个后并射出一支箭”；传感器是当前局面与对局历史。

3.1.2 求解目标的分层

“下好亚马逊棋”可以分解为三个层次的目标，它们需要不同的求解方法：

1. **战略目标（争夺阶段）**：在双方的后还相互接触时，选择能扩大己方“势力范围”、压缩对方活动空间的着法。这只能依靠领地评估与有限深度搜索来**近似**，是本文比较 α-β 与 MCTS 的主战场。
2. **计数目标（分隔之后）**：一旦双方的后被箭完全隔开，对局就分解为若干互不影响的区域，胜负取决于各方能在自己的区域里走多少步。这是一个组合优化问题，在一般情况下也很难（Buro 证明了相关的填格问题是 NP 完全的[33]），但它与对手无关，可以用一个确定的策略处理。本文把它交给所有智能体共用的贪心填格策略（2.6 节），把比较集中在第一层目标上。
3. **资源目标**：在给定时间内把计算分配给最有价值的变化，即“想得准”与“想得深”之间的权衡。在亚马逊棋中，这一权衡尤其突出，因为仅仅“给全部着法排一次序”就要花费 1—2 毫秒。

第二章的各个组件正是分别针对这三个层次：领地评估、可分离排序、α-β 或 MCTS 解决战略目标；分隔检测与填格策略解决计数目标；迭代加深、束宽、渐进展开与时间控制解决资源目标。

3.1.3 终局条件与“分隔”这一子目标

亚马逊棋的终局判定是“轮到某方而他无棋可走”。在实际计算中，这一条件出现得很晚：分隔之后，双方往往还要各自填格十几到几十手。如果搜索只在真正的终局才给出确定的价值，那么在分隔后的大部分局面中搜索仍然要依赖启发式评估。本文的处理是把“分隔”当作一个**可判定的中间终点**：一旦检测到双方不再共享任何可达区域，就用“己方可达格数 − 对方可达格数”给出近似确定的价值（式 2-6），α-β 在这样的结点直接返回，MCTS 也在这样的结点停止模拟。这一近似在区域形状简单时与真实结果一致，在形状复杂（例如有“缺口”或“死格”）时可能高估某一方，5.10 节给出了相关的失败案例。

3.2 搜索空间的压缩

3.2.1 为什么全宽搜索不可行

如 1.3.2 节所述，亚马逊棋开局时有 2176 个合法着法，双方分隔前的局面平均约有 457 个。若进行深度为 D 的全宽 α-β 搜索，即使排序完美，按 Knuth–Moore 的最好情况公式，$D=3$ 时也需要约 $b^{\lceil D/2 \rceil}+b^{\lfloor D/2 \rfloor}\approx 2000^2=4\times10^6$ 个叶结点；而本文的静态评估每次约需 8 微秒，仅这些叶结点就要 30 秒以上。也就是说，在 200 毫秒的预算下，**全宽 α-β 连 2 层都很难完成**。MCTS 也面临同样的问题：标准 UCT 要求在一个结点的每个子结点都被访问一次之后才开始“选择”，而 200 毫秒只够做两千多次模拟，根结点的子结点都还没访问完。

3.2.2 三级压缩

本文用三级机制压缩搜索空间，它们在 α-β 与 MCTS 中以不同的形式出现：

- **第一级：动作分解**。把“移后 + 射箭”分成两个子动作分别评估，即可分离评估（2.2.5 节）。它不删去任何着法，而是把“评估全部着法”的代价从“着法数次评估”（开局 2176 次）降为“后的走法数 + 空格数”次（约 150 次），从而能用接近静态评估质量的排序给**全部**着法排序。
- **第二级：排序后截断**。α-β 只搜索排序后的前 W 个着法（束宽剪枝），MCTS 只为前 K 个着法建子结点（候选上限），并通过渐进展开逐步扩大。这一级是“有损”的：被截掉的着法永远不会被搜索。
- **第三级：概率化的软剪枝**。MCTS 的 PUCT 先验把模拟集中在先验高的少数着法上，但不完全排除其他着法；α-β 中的杀手与历史启发也起类似作用，它们不剪掉着法，而是改变着法的搜索顺序，使截断更早发生。

第一级是本文的设计重点。试调阶段的观察（5.1.3 节）显示，如果只用廉价规则分数排序，α-β 对贪心基线的胜率从 15/16 降到 7/16，说明在亚马逊棋中“排序质量”几乎就是“棋力”。这一发现促成了可分离评估的设计。

3.2.3 动作分解的合理性

可分离评估的近似前提是：“移后”与“射箭”对局面价值的贡献近似可加。这一假设显然不严格：射箭的最佳位置取决于后移到了哪里（例如后移动后，原来的位置空出来，可能成为新的射箭目标；射箭也可能挡住刚走的后自己的路）。但在大多数局面中，这种交互只影响一小部分格子。5.9.1 节的测试中，静态评估最好的着法有 72% 排在可分离评估的前 8 名、92% 排在前 30 名，而在廉价分数中只有约一半（53%）排在前 24 名，说明这一近似虽然粗糙，但对排序已经足够好。从问题求解的角度看，这是一种经典的**问题分解**：把一个大的联合决策拆成两个较小的、近似独立的子决策，再用代价较低的“组合”恢复联合决策。类似的思想在亚马逊棋程序中以“先选后、再选箭”的两阶段搜索形式出现过[23,42]，在大动作空间的强化学习中也有“动作分解”的说法。

3.3 知识表示：距离与领地

3.3.1 为什么用“距离”表示局面

一个直接的局面表示是 100 个格子的状态。但这种表示离“决策”太远：棋手看到的是“这片区域是我的”“对方的后快被围住了”，而不是 100 个数字。本文采用

的中间表示是**每个空格到双方后的距离**：后距离 d^Q（按后的走法计算的最少步数）与王距离 d^K（按国际象棋王的走法计算的最少步数）。这一表示有三个优点：

1. **与领地直接对应**：一个格子“归谁”由双方距离的比较决定，领地图（图 5-3）可以直接画在棋盘上，是界面中“AI 怎么看局面”的核心可视化；
2. **天然包含分隔检测**：如果某个格子只有一方能到达，它就已经是“确定领地”；如果所有格子都只有一方能到达，双方就已经分隔；
3. **可组合**：同一组距离同时服务于评估函数、着法排序（可分离评估）、分隔判定与填格策略，四个模块只依赖一次 BFS 的结果。

3.3.2 后距离与王距离的分工

为什么需要两种距离？后距离反映“谁能更快地占领一个格子”，它在开阔的局面中区分度很低：大部分格子都能在 1—2 步内到达，双方往往相等。王距离反映“谁在一个区域里有更细致的控制”，它在空格较多时更能区分局部的优劣。Lieberum 的评估把二者分别用于 t_1（后距离领地）与 t_2（王距离领地），再用阶段系数 w 调节它们的权重[30,31]：开局时双方距离差异小，w 大，偏重 c_1, c_2 这类“连续”的位置项；分隔临近时 w 变小，偏重 t_1 这类“离散”的领地项。本文把原始的非线性组合改为阶段线性插值（2.2.3 节），使权重可以用 Texel 方法学习。

3.3.3 表示的局限

这一表示有两类已知的盲区。第一，**距离不等于控制**：一个格子离己方的后只有 1 步，并不意味着己方能真正占住它，因为对方可能在下一手用箭封住通道。第二，**区域的“可填性”**：一块区域的格数并不等于能在其中走的步数，形状狭长或有“死角”的区域实际可走的步数会少于格数，Kloetzer 等人用组合博弈论分析过这类情况[26]。本文的评估与分隔价值都没有处理这一点，5.10 节的失败案例中能看到它的影响。

3.4 争夺阶段的权衡

3.4.1 进攻与防守：移动性的作用

在亚马逊棋中，“进攻”通常意味着用箭压缩对方后的活动空间，“防守”意味着保持己方后的出路。一个后如果被围住（四周八个方向都无法移动），它对之后的领地争夺就毫无贡献。Lieberum 的评估中加入了**移动性**特征，本文的实现对每个后的可走步数取 $min(m, 12)$ 后求和（2.2.3 节）。截断在 12 的原因是：移动性在“快被困住”时极为重要，在后已经很自由时则边际价值很小。廉价着法分数中也有类似的项：

射箭打在对方后旁边的格子（$2\cdot\text{oppAdj}$）加分，打在己方后旁边的格子（ownAdj）扣分。5.4 节的消融实验测量了去掉移动性特征后的棋力变化。

3.4.2 后的分工

四个后的分工也是一个权衡：把四个后都留在中央可以保持影响力，但也容易被对方用少量箭同时限制；把一个后派去“守住”一块边角领地，则可能在中央争夺中处于数量劣势。本文的算法没有显式建模这种分工，而是通过评估函数中所有后的距离与移动性隐式地反映它。

3.4.3 射箭的双重作用

一支箭同时具有“占领”（把格子变成障碍、减少对方可达格）与“自伤”（也可能挡住己方通道）两种效果。可分离评估中的 $A(a)$ 项把“只放箭不移后”的局面评估一次，恰好衡量了一支箭在当前局面中的净作用。这也是可分离评估比廉价规则分数更准确的原因之一：廉价分数只看箭周围的局部，而 $A(a)$ 考虑了箭对全盘距离的影响。

3.5 近似与不确定性

3.5.1 三类近似误差

本文的每个算法都在做近似，误差来源可以分为三类：

1. **评估误差**：静态评估与真实博弈值之间的差距。α-β 通过向前搜索减小它，MCTS 通过模拟与平均减小它；
2. **剪枝误差**：束宽、候选上限与渐进展开可能剪掉最佳着法；可分离评估的排序误差会被剪枝放大成剪枝误差；
3. **分隔近似误差**：分隔后的价值用“可达格数差”近似，忽略了区域的可填性；而填格策略本身也是贪心的。

这三类误差相互作用。例如，束宽越窄，搜索越深，评估误差越小，但剪枝误差越大；MCTS 的候选上限越小，统计越集中，但最佳着法被排除的概率越大。哪一种组合最好，没有先验答案，只能通过实验确定，这是第五章消融实验的出发点。

3.5.2 评估值与胜率

MCTS 的截断评估与 Texel 调参都需要把评估值映射为胜率。本文用自对弈数据为手工权重拟合了 logistic 映射的尺度 K，并把它直接用作 MCTS 截断模拟中的 k。这样做的好处是：MCTS 中的“胜率”与自对弈数据中的实际胜率在统计上大致一致，不需要再单独调一个超参数。需要说明的是，MCTS 使用的 $k=0.18$ 来自第一版程序

的自对弈数据；修复分隔判定后重新拟合得到 $K \approx 0.26$（5.7.3 节），说明修复后评估值与胜负的关联更强（同样的评估差对应更确定的胜负），但 MCTS 中没有随之调整。即使尺度合适，线性评估给出的“胜率”仍然只是粗略的，这也是截断模拟的一个误差来源。

3.5.3 时间管理

在固定时间预算下，α-β 的迭代加深在“完成一层”时才会更新答案，若在一层的中途超时，只能使用部分结果。本文的策略是：已用时间超过 50% 时不再开始新一层；若最后一层中途超时，但根结点至少已有一个着法完整搜索完毕，则采用这一层的部分结果（2.3.2 节）。MCTS 则可以在任何时刻停止，这是它的天然优势。更精细的时间管理方法在国际象棋与围棋中有专门研究[251,252]，本文没有采用。

3.6 算法选择分析与研究问题

3.6.1 α-β 与 MCTS 在亚马逊棋上的预期

从理论上看，两类算法在亚马逊棋上各有优劣：

- α-β 的优势在于**最坏情况分析**：在搜索深度内，它考虑对手的最强应着，不会被“看起来不错但有一个好应着”的着法欺骗。但大分支因子迫使它只能搜索 2—4 层，且严重依赖着法排序与束宽，排序中的系统误差会直接变成盲点。
- MCTS 的优势在于**选择性**与**随时可停**：它把模拟集中在有希望的变化上，并通过平均来平滑评估噪声。亚马逊棋的局面价值通常是“渐变”的（一支箭很少立即决定胜负），陷阱远不如五子棋或国际象棋密集，这正是 MCTS 擅长的环境。Ramanujan 等人的研究表明，在“陷阱稀疏”的搜索空间中 UCT 的表现可以与 α-β 相当甚至更好[89,90]；亚马逊棋的程序比赛中，MCTS 程序在 2000 年代后期也逐渐占据了优势[24,25]。

因此可以预期：在相同的知识与时间下，两类算法的差距会比五子棋这类战术密集的游戏小得多；MCTS 的表现会高度依赖它如何使用评估函数（截断、先验）；而两类算法的差距可能随时间预算变化。这些预期并不一定成立，需要实验检验。

3.6.2 研究问题

基于上述分析，第五章围绕以下研究问题（Research Question，RQ）组织实验：

- **RQ1**：在相同的知识与时间预算下，α-β 与 MCTS 哪一个更强？它们分别比随机与贪心基线强多少？（5.3 节）

- **RQ2**：α-β 的各项增强（置换表、杀手、PVS、着法排序、束宽、评估特征）分别贡献多少棋力？（5.4 节）
- **RQ3**：MCTS 的各项设计（选择公式、先验与候选排序、渐进展开、候选上限、模拟策略与截断深度、探索系数）分别有多重要？（5.5 节）
- **RQ4**：增加思考时间能带来多少棋力提升？两类算法的缩放规律是否不同，二者的强弱关系是否随时间预算变化？（5.6 节）
- **RQ5**：从自对弈数据中学到的走子策略与评估权重，能否在离线指标和对局结果上改进手工知识？（5.7 节）
- **RQ6**：各算法单步决策的质量与搜索效率如何？它们在哪些局面中会失败、为什么？（5.8—5.10 节）

需要说明的是，这些问题是在设计实验时确定的，第五章对每个问题都如实报告结果，包括与预期不一致的结果。

3.7 使用 AI 工具解决问题的方法

本节按照作业要求如实说明 AI 工具在本项目中的使用情况。

3.7.1 使用的工具与范围

本项目在完成过程中**大量使用**了生成式 AI 工具。具体来说，使用的是 Anthropic 公司的 Claude（以 Claude Code 编程智能体的形式，在云端隔离容器中运行）。本人向它提出作业要求（包括老师的模板、篇幅与参考文献要求，以及老师“五子棋太常见、建议换题”的意见），它在容器中完成了以下工作：

1. **选题与方案设计**：在老师建议换题后，提出几个备选主题，由本人选定“亚马逊棋（α-β 对比 MCTS）”；之后由 AI 提出整体技术路线与第五章的实验设计；
2. **代码编写**：引擎（棋盘、走法生成、领地评估、可分离排序、α-β、MCTS、填格策略、学习模型推断）、网页界面、实验脚本、学习脚本、统计分析与绘图脚本，全部由 AI 生成；
3. **测试**：编写并运行单元测试与浏览器自动化测试；
4. **实验运行**：在容器中实际运行全部对局实验（自对弈 400 局、主实验 2144 局，以及效率、决策质量、方差测试等），本报告中的所有数据都来自这些运行；
5. **文献检索与核验**：先根据模型“记忆”列出候选文献，再编写脚本逐条在 Crossref、arXiv 与 OpenLibrary 等在线数据库上检索核验，另通过 Crossref 按主题检索补充文献（亚马逊棋相关的检索词见附录 D），并只引用核验通过的条目；

6. **报告撰写与排版**：起草本报告全部章节与附录，编写 Markdown→Word→PDF 的排版脚本，沿用前两次作业报告的样式。

第六章“个人学习和实践心得”也是应本人要求，由 AI 根据本项目的实际过程与实验结果撰写的。

本人在项目中的工作是：提出作业的全部要求与约束（包括老师的模板、换题意见、篇幅与参考文献要求，以及“数据必须来自真实运行、不得编造结果与引用、如实说明 AI 使用”等原则），在过程中做出关键选择（例如从备选主题中选定亚马逊棋），并审阅最终成果、决定提交。

3.7.2　工作方式：生成—验证循环

生成式模型会产生看似合理但错误的内容，这一现象在代码生成[403–405]和文献引用[406–409]中都有大量报道。因此本项目采用的基本原则是：**AI 生成的任何内容，都必须经过可执行的验证才能被采用**。具体手段包括：

- **代码**：每个模块都配有自动化测试，并且尽量采用“独立实现交叉验证”的方式——例如用逐格扫描验证走法生成的计数、用朴素 BFS 验证距离计算、用不带任何剪枝的朴素极小极大验证 α-β 在小深度上的结果。这类“差分测试”与“蜕变测试”的思想见文献[410–412]；
- **实验数据**：所有数字由脚本从原始对局记录计算得到，报告中的表格由分析脚本直接生成（`report/tables/`），不手工抄写；试调使用的 16 个开局与正式实验的 16 个开局严格分开；
- **参考文献**：只有在线检索到标题相似度不低于 0.88、第一作者姓氏吻合、年份相差不超过 2 年的条目才视为“已核验”，著录信息取自检索结果而不是模型记忆；
- **排版结果**：生成 PDF 后转为图片逐页抽查。

3.7.3　AI 在本项目中犯过的错误

如实记录 AI 犯过的错误，既是诚信要求，也有助于理解 AI 工具的能力边界。表 3-2 列出了开发过程中被验证手段发现的主要错误。

表 3-2　开发过程中发现的 AI 错误（节选）

类别	错误内容	发现方式	处理
算法设计	最初的着法排序只用廉价规则分数，α-β 对贪心基线的试调胜率只有 7/16，远低于预期	试调对局（独立的 16 个开局）	设计可分离评估排序，同条件下胜率升至 15/16
算法	候选生成函数中引用了未定义	运行时报错	改为按层数判断是否为根结点

类别	错误内容	发现方式	处理
实现	的变量 `root`，导致根结点的候选数设置无效		
算法实现	α-β 只在每层结束时检查时间，200 ms 的预算实际用到约 213 ms	对局日志中的用时统计	每 8 个结点检查一次时间，并允许采用不完整的最后一层
算法实现	分隔判定只检查“有没有双方都能到达的空格”，漏掉了“敌对的后紧挨着”的情形，导致程序过早认为对局已分隔、交给填格策略，此后局面又可能重新连通	第一轮主实验后复盘失败对局：α-β 在“已分隔”后的评估从 +249.5 跌到 +31，并最终输给纯随机 MCTS；统计发现第一轮 1728 局非随机对局中有 488 局出现“分隔后又恢复接触”	修正判定并增加回归测试，用修复后的引擎重跑自对弈、学习与全部实验（第一轮数据保留，用于 5.10 节的对比）
测试用例	暴力参考实现没有在“已分隔”的局面停止，也没有按层数调整胜负分，与 α-β 的结果不一致	单元测试失败后人工分析	修改参考实现，使其与 α-β 的终止条件一致；α-β 代码不变
测试用例	填格策略测试假设了一个具体的着法，但等价着法不止一个	单元测试失败	改为检查“所选着法使可达格数最大”
参考文献	起草时使用了几个不存在或未核验的文献键（例如虚构的作者年份组合），并出现了重复的文献键	引用检查脚本	替换为已核验条目或删除相应引用
报告文字	草稿中对某些文献结论的表述比原文更强（例如把“经验观察”写成“已被证明”），对个别程序名称的记忆也不可靠	人工对照检索结果与摘要	改为有保留的表述，删去无法核实的细节

这些错误说明：AI 工具能够高效地生成大量代码与文字，但在细节正确性与“设计是否真的有效”上并不可靠；**真正保证质量的是测试、交叉验证、实验与核验流程，而不是生成本身**。第一行与分隔判定那一行的错误尤其值得注意：前者的代码本身没有 bug，但一个“看起来合理”的设计在实验中被证明效果很差；后者的代码通过了当时的全部单元测试（因为测试与实现基于同一个不完整的定义），直到复盘失败对局才暴露出来。这类错误只有通过真实运行与对结果的追问才能发现。这一点与已有关于 AI 编程助手的研究结论一致：它们能显著提高开发速度[413,414]，但在真实工程任务上的成功率仍然有限[415]，使用者需要具备判断与验证的能力[404,405]。

3.7.4 对课程学习的影响与反思

在教育领域，生成式 AI 带来的机遇与挑战已有大量讨论[416–419]，大模型的能力边界也是研究热点[420,421]。就本项目而言，AI 工具使得在作业周期内完成“完整系统 + 数千局实验 + 近两百页报告”成为可能，但这也带来一个风险：如果本人不理解生成的内容，作业就失去了学习的意义。因此，报告中的每一个算法都配有原理说明、流

程图和对应的代码清单（第二、四章），每一项结论都给出了数据来源与不确定性（第五章），并提供了一键运行的游戏与完整的复现脚本（附录 C），以便读者——包括本人——能够逐项检查、理解和复现。

3.8 本章小结

本章把亚马逊棋 AI 看作一个分层的智能问题求解任务：争夺阶段只能近似，分隔之后可以交给确定的填格策略，资源层需要权衡。围绕这一分析，本文用动作分解（可分离评估）、排序后截断与软剪枝三级压缩搜索空间，用“到达距离”作为连接局面与决策的中间表示，讨论了移动性、射箭与后的分工等权衡，并提出了六个研究问题。3.7 节说明了 AI 工具在本项目中的使用方式：AI 承担了大部分生成工作，但质量依靠测试、交叉验证、实验与文献核验来保证；本人负责提出要求、做出关键选择并审阅成果。

四、流程图和程序代码

本章介绍系统的整体架构、模块划分、核心算法的流程图与关键代码，以及测试方案。代码清单均由排版脚本从项目源文件中按行号直接截取，与提交的程序完全一致；完整源代码见附录 E。

4.1 系统架构

4.1.1 总体结构

项目由五个部分组成（图 4-1）：

1. **引擎**（`engine/`，约 1150 行 JavaScript）：棋盘与走法生成、距离计算与领地评估、着法排序（廉价分数与可分离评估）、α-β 搜索、MCTS、线性策略模型、填格策略与统一的智能体接口。引擎不依赖浏览器或 Node.js 的任何 API，可以同时在网页、Web Worker 和 Node.js 中运行。
2. **网页游戏**（`game/`）：界面（HTML/CSS/JavaScript）、Web Worker 包装层和打包脚本。打包脚本把引擎源码、模型参数与界面代码内联为单个 HTML 文件。
3. **实验系统**（`experiments/`）：对局器、开局生成、任务清单生成、多进程任务执行器、测试局面抽取，以及效率、决策质量与方差测试脚本。
4. **学习与分析**（`learning/`、`analysis/`，Python 与少量 JavaScript）：训练数据抽取、策略训练、Texel 调参、统计分析与绘图。
5. **测试**（`tests/`）：引擎单元测试与浏览器自动化测试。

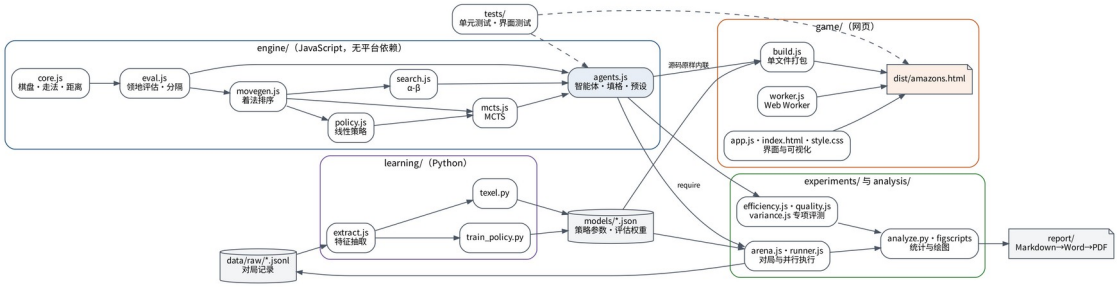


图 4-1 系统总体架构

注：作者绘制（Graphviz）。同一份引擎源码同时被网页（经打包）与实验系统（直接 require）使用。

这一结构的核心设计原则是“**网页里运行的 AI 与报告中测量的 AI 是同一份代码**”。实验脚本通过 `makeAgent(name)` 创建智能体，网页界面通过同一个函数和同一组预设创建智能体，打包脚本只是把源文件原样拼接，不做任何改写。这样，第五章测得的棋力就是用户在网页中实际面对的棋力。

4.1.2 模块与接口

表 4-1 列出主要模块及其接口。

表 4-1 主要模块与接口

文件	行数	主要内容	对外接口
`engine/core.js`	239	棋盘、射线表、走法生成与合法性检查、Zobrist 哈希、后距离与王距离、坐标转换	`Board`、`queenDist`、`kingDist`、`parseMove` 等
`engine/eval.js`	152	领地特征、阶段插值评估、分隔判定与区域计分、Texel 特征、领地图	`evaluate`、`separated`、`features`、`territoryMap`
`engine/movegen.js`	181	廉价着法分数、可分离评估、部分排序、策略特征	`topMoves`、`sepTopMoves`、`moveFeatures`
`engine/search.js`	234	α-β 搜索器（迭代加深、PVS、置换表、杀手、历史、束宽）	`AlphaBeta.search(board)`
`engine/mcts.js`	179	MCTS（UCT/PUCT、渐进展开、先验、模拟与截断）	`MCTS.search(board)`
`engine/policy.js`	40	线性 softmax 策略的特征与推断	`LinearPolicy.rank(board, k)`
`engine/agents.js`	132	填格策略，随机、贪心、α-β、MCTS 智能体与预设	`makeAgent(name, overrides)`、`PRESETS`、`fillerMove`
`game/src/app.js`	566	界面逻辑、绘制、交互、面板	—
`game/src/worker.js`	37	Worker 消息处理，转发搜索进度	`postMessage` 协议
`game/build.js`	39	打包为单文件 HTML	`node game/build.js`
`experiments/*.js`	约 430	对局、开局、任务、执行器、局面抽取、效率与质量测试	命令行
`learning/*`、`analysis/*`	约 900	数据抽取、训练、调参与分析	命令行
`tests/*.js`	约 650	单元测试、界面测试、浏览器性能测试与截图	`node tests/run_tests.js`

所有智能体都实现同一个接口 `think(board) → {move, reason, ...}`，返回的除了着法之外还有诊断信息：着法理由（`search`、`mcts`、`win`、`filler`、`greedy`、`random` 等）、搜索深度、结点数、模拟次数、根结点各候选的分数或访问次数与先验、主变例、迭代加深的逐层记录、截断与置换表统计等。界面与实验系统都使用这些信息：界面用来可视化，实验系统用来统计。

4.2 核心数据结构与代码

4.2.1 棋盘与走法生成

棋盘用长度为 100 的 `Uint8Array` 表示（0 空、1 白后、2 黑后、3 箭），另用两个长度为 4 的数组记录双方后的位置。程序在加载时为每个格子预计算 8 个方向的“射线”（沿该方向依次经过的格子列表），走法生成只需沿射线前进、遇到非空格停止。一个着法编码为整数 $f\cdot 10^4+t\cdot 10^2+a$，其中 f,t,a 分别是出发格、目的格与射箭格；这样着法可以直接作为置换表与杀手表中的键，也便于在 Worker 与主线程之间传递。

走法生成（代码 4-1）有一个容易出错的细节：射箭时后已经离开出发格，因此出发格是空的，箭可以射回出发格，也可以“穿过”出发格。程序在为某个后生成走法前先把出发格临时置空，生成完再恢复。单元测试用“随机编码 20 万次，比较 `isLegal` 与 `genMoves` 的结果”的方式验证了这一点（4.8 节）。

代码 4-1 走法生成与计数（engine/core.js）

```
  canMove(side = this.side) {
    for (const q of this.queens[side]) for (const n of NEIGH[q]) if (this.cells[n]
=== EMPTY) return true;
    return false;
  }
  isOver() { return !this.canMove(this.side); }
  get winner() { return this.isOver() ? 3 - this.side : 0; }
  // 一个后从 q 出发一步能到达的空格（目的地）
  queenTargets(q, out = []) {
    const rays = RAYS[q], cells = this.cells;
    for (let d = 0; d < 8; d++) { const ray = rays[d]; for (let k = 0; k <
ray.length; k++) { const x = ray[k]; if (cells[x] !== EMPTY) break; out.push(x); }
}
    return out;
  }
  // 全部合法走法（整数编码）。开局时约2176 步
  genMoves(side = this.side) {
    const out = [], cells = this.cells;
    for (const f of this.queens[side]) {
      cells[f] = EMPTY;                                   // 射箭时原位置已空出
      const rays = RAYS[f];
      for (let d = 0; d < 8; d++) {
        const ray = rays[d];
        for (let k = 0; k < ray.length; k++) {
          const t = ray[k]; if (cells[t] !== EMPTY) break;
          const base = f * 10000 + t * 100, rt = RAYS[t];
          for (let e = 0; e < 8; e++) { const r2 = rt[e]; for (let j = 0; j <
r2.length; j++) { const a = r2[j]; if (cells[a] !== EMPTY) break; out.push(base +
a); } }
        }
      }
      cells[f] = side;
    }
```

```
    return out;
  }
  countMoves(side = this.side) {
    let n = 0; const cells = this.cells;
    for (const f of this.queens[side]) {
      cells[f] = EMPTY;
      for (const ray of RAYS[f]) for (const t of ray) { if (cells[t] !== EMPTY)
break; for (const r2 of RAYS[t]) for (const a of r2) { if (cells[a] !== EMPTY)
break; n++; } }
      cells[f] = side;
    }
    return n;
  }
```

4.2.2 距离计算

后距离与王距离都用广度优先搜索（BFS）计算（代码 4-2）。后距离的 BFS 有一个特点：从一个格子出发，沿每条射线前进时，遇到已经标记为“更近”的格子不能停下，因为射线可以穿过它到达更远的格子；只有遇到障碍物才停止。代码中的 `else if (out[y] < d) continue` 就是为此而写的。四次 BFS（双方各两种距离）构成了评估函数的主要开销。

代码 4-2 后距离、王距离、“争夺格”计数与分隔判定（engine/core.js）

```
// ---------------------------------------------------------------- 距离计算
// 后距离（queen distance）：从某方任一后出发，最少需要几步“后的移动”能到达该格（不考虑对方阻挡的动态变化）
function queenDist(b, side, out = new Uint8Array(NN)) {
  out.fill(INF);
  const cells = b.cells, q = [];
  for (const s of b.queens[side]) { out[s] = 0; q.push(s); }
  let h = 0;
  while (h < q.length) {
    const x = q[h++], d = out[x] + 1, rays = RAYS[x];
    for (let k = 0; k < 8; k++) {
      const ray = rays[k];
      for (let j = 0; j < ray.length; j++) {
        const y = ray[j]; if (cells[y] !== EMPTY) break;
        if (out[y] > d) { out[y] = d; q.push(y); }
        else if (out[y] < d) continue;
      }
    }
  }
  return out;
}
// 王距离（king distance）：每步只走一格（8 邻域）
function kingDist(b, side, out = new Uint8Array(NN)) {
  out.fill(INF);
  const cells = b.cells, q = [];
  for (const s of b.queens[side]) { out[s] = 0; q.push(s); }
  let h = 0;
  while (h < q.length) {
```

```
    const x = q[h++], d = out[x] + 1;
    for (const y of NEIGH[x]) if (cells[y] === EMPTY && out[y] > d) { out[y] = d;
q.push(y); }
  }
  return out;
}
// 双方的后距离都有限的空格数；为0 表示双方已被完全分隔（进入“填格”阶段）
function contestedCount(b, dW = queenDist(b, WHITE), dB = queenDist(b, BLACK)) {
  let n = 0;
  for (let i = 0; i < NN; i++) if (b.cells[i] === EMPTY && dW[i] < INF && dB[i] <
INF) n++;
  return n;
}
// 已分隔：没有争夺格，且双方的后不在同一个“非箭格子”的8 邻域连通分量中（敌对的后紧挨着时，移开
一方即会重新连通）
function isSeparated(b) {
  if (contestedCount(b) !== 0) return false;
  const seen = new Uint8Array(NN), st = b.queens[WHITE].slice();
  for (const q of st) seen[q] = 1;
  while (st.length) { const x = st.pop(); for (const y of NEIGH[x]) { if (seen[y]
|| b.cells[y] === ARROW) continue; if (b.cells[y] === BLACK) return false; seen[y]
= 1; st.push(y); } }
  return true;
}
```

4.2.3 评估函数

代码 4-3 实现了 2.2 节的评估函数：一次遍历全部空格同时累加 t_1,t_2,c_1,c_2、阶段系数 w、争夺格数与双方可达格数，再加上移动性。若争夺格数为 0、并且从白后出发在非箭格子上做 8 邻域洪泛碰不到黑后（queensTouch，即双方已分隔，见 5.10.1 节），直接返回式 2-6 的区域计分；否则按阶段系数 p 对两组权重线性插值。评估值以白方视角计算，最后按行棋方取符号。

代码 4-3 领地特征、分隔计分、评估函数与分隔判定（engine/eval.js）

```
function features(b, out = new Float64Array(NF)) {
  const cells = b.cells, qW = b.queens[WHITE], qB = b.queens[BLACK];
  queenDist(cells, qW, dW); queenDist(cells, qB, dB);
  kingDist(cells, qW, kW); kingDist(cells, qB, kB);
  const kap = b.side === WHITE ? KAPPA : -KAPPA;
  let t1 = 0, t2 = 0, c1 = 0, c2 = 0, w = 0, contested = 0, rW = 0, rB = 0;
  for (let i = 0; i < NN; i++) {
    if (cells[i] !== EMPTY) continue;
    const a = dW[i], c = dB[i];
    if (a < INF) rW++;
    if (c < INF) rB++;
    if (a === INF && c === INF) continue;
    if (a < c) t1 += 1; else if (a > c) t1 -= 1; else t1 += kap;
    const ka = kW[i], kc = kB[i];
    if (ka < kc) t2 += 1; else if (ka > kc) t2 -= 1; else if (ka < INF) t2 += kap;
    c1 += 2 * (POW2[a] - POW2[c]);
    let v = (kc - ka) / 6; if (v > 1) v = 1; else if (v < -1) v = -1;
```

```
    c2 += v;
    if (a < INF && c < INF) { contested++; const diff = a > c ? a - c : c - a; w
+= POW2[diff]; }
  }
  let mob = 0;
  for (let i = 0; i < 4; i++) { const m = queenMobility(cells, qW[i]); mob += m <
12 ? m : 12; }
  for (let i = 0; i < 4; i++) { const m = queenMobility(cells, qB[i]); mob -= m <
12 ? m : 12; }
  out[0] = t1; out[1] = t2; out[2] = c1; out[3] = c2; out[4] = mob / 4;
  const p = w / W0 > 1 ? 1 : w / W0;
  return { f: out, p, w, contested, regionW: rW, regionB: rB };
}

// 分隔后的区域格数：用王距离可达的空格数（剩余步数的上界）
function separatedScore(b, info) {
  // info.regionW/B 用后距离可达性统计；分隔后与王距离可达性一致（同一个连通区域）
  const own = b.side === WHITE ? info.regionW : info.regionB;
  const opp = b.side === WHITE ? info.regionB : info.regionW;
  const diff = own - opp - 0.5;            // 步数相同时行棋方先用完，所以减0.5
  return (diff > 0 ? SEP_BONUS : -SEP_BONUS) + diff;
}

const fbuf = new Float64Array(NF);
/** 行棋方视角的评估值*/
function evaluate(b, weights = DEFAULT_WEIGHTS) {
  if (!b.canMove(b.side)) return -WIN;
  const info = features(b, fbuf);
  if (info.contested === 0 && !queensTouch(b.cells, b.queens[WHITE])) return
separatedScore(b, info);
  const p = info.p, f = info.f;
  let s = 0;
  for (let k = 0; k < NF; k++) s += (p * weights[k] + (1 - p) * weights[NF + k]) *
f[k];
  return b.side === WHITE ? s : -s;
}

/**
 * 判断是否已分隔：在“空格+ 后所在格”构成的8 邻域连通分量中，没有任何一个分量同时含有双方的后。
 * 注意：只检查“没有空格同时可被双方到达”是不够的——两个敌对的后紧挨着、彼此之间没有空格时，
 * 任何一方把后移开，空出的格子就会把两块区域连通（第一版实现正是这样判断的，见5.10 节）。
 */
const stk = new Int8Array(NN), seen = new Uint8Array(NN);
function separated(b) {
  const cells = b.cells;
  queenDist(cells, b.queens[WHITE], dW); queenDist(cells, b.queens[BLACK], dB);
  for (let i = 0; i < NN; i++) if (cells[i] === EMPTY && dW[i] < INF && dB[i] <
INF) return false;
  return !queensTouch(cells, b.queens[WHITE]);
}
// 从白方的后出发，在非箭的格子上做8 邻域洪泛，若能碰到黑方的后则返回true
function queensTouch(cells, qW) {
  seen.fill(0); let top = 0;
  for (const q of qW) { seen[q] = 1; stk[top++] = q; }
  while (top) {
```

```
    const x = stk[--top];
    for (const y of C.NEIGH[x]) {
      if (seen[y]) continue;
      const v = cells[y];
      if (v === ARROW) continue;
      if (v === BLACK) return true;
      seen[y] = 1; stk[top++] = y;
    }
  }
  return false;
}
```

4.2.4 可分离评估

代码 4-4 是可分离评估（式 2-8）的实现。它先对每个空格计算“只放一支箭”时评估值的变化 $A(a)$，再对每个后的每个目的地计算“只移后”的评估值 $Q(f,t)$，最后在生成走法的同一个循环中把二者相加作为该着法的分数。与 `genMoves` 一样，生成射箭位置时出发格已被置空；当箭恰好射回出发格时，$A(f)$ 的计算前提（出发格被后占据）不成立，程序简单地取 $A=0$。所有分数写入预分配的缓冲区，再由 `selectTop` 做部分排序取前 k 个。

代码 4-4 可分离评估：一次给全部着法打分（engine/movegen.js）

```
function sepScoreAll(b, weights) {
  const s = b.side, o = 3 - s, cells = b.cells, qs = b.queens[s];
  // A：只放箭（视角仍为行棋方，评估时轮到对方走，故取负）
  b.side = o;
  const e0 = -E.evaluate(b, weights);
  for (let a = 0; a < NN; a++) {
    if (cells[a] !== EMPTY) { Atab[a] = 0; continue; }
    cells[a] = C.ARROW; Atab[a] = -E.evaluate(b, weights) - e0; cells[a] = EMPTY;
  }
  b.side = s;
  let n = 0;
  for (let qi = 0; qi < 4; qi++) {
    const f = qs[qi];
    cells[f] = EMPTY;
    const rf = RAYS[f];
    for (let d = 0; d < 8; d++) {
      const ray = rf[d];
      for (let k = 0; k < ray.length; k++) {
        const t = ray[k]; if (cells[t] !== EMPTY) break;
        cells[t] = s; qs[qi] = t; b.side = o;
        const q = -E.evaluate(b, weights);
        b.side = s; qs[qi] = f; cells[t] = EMPTY;
        const base = f * 10000 + t * 100, rt = RAYS[t];
        for (let e = 0; e < 8; e++) {
          const r2 = rt[e];
          for (let j = 0; j < r2.length; j++) {
            const a = r2[j]; if (cells[a] !== EMPTY) break;
            bufM[n] = base + a; bufS[n] = q + (a === f ? 0 : Atab[a]); n++;
          }
```

```
        }
      }
    }
    cells[f] = s;
  }
  return n;
}
```

4.3 α-β 搜索的流程与代码

4.3.1 根结点流程

α-β 智能体每一步的决策流程如图 4-2 所示：先检查是否已分隔（若是则交给填格策略），再进入迭代加深循环；每一层在根结点用可分离评估取前 60 个着法、逐一静态评估后保留前 30 个，再依次以 PVS 方式搜索；每层结束后检查时间，已用时超过一半则停止。

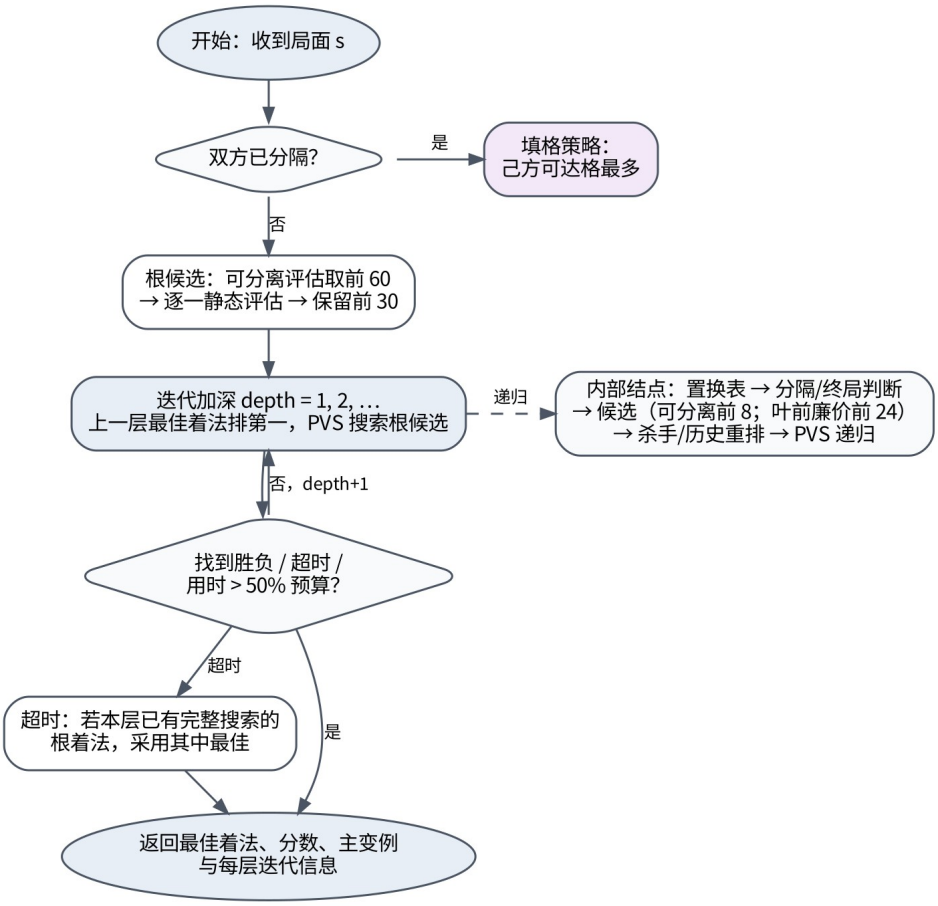


图 4-2 α-β 智能体的决策流程

注：作者绘制（Graphviz）。

4.3.2 候选生成与排序

候选生成（代码 4-5）根据结点类型使用不同的排序：根结点用“可分离评估预选 + 静态评估精排”；剩余深度为 1 的结点用廉价分数取前 24 个（因为它们的子结点都要做静态评估，排序精度的收益不大）；其他内部结点用可分离评估取前 8 个。随后，杀手着法与历史分高的着法被稳定地提前，置换表着法放在最前面——若置换表着法不在束宽之内，也会被补充进来。

代码 4-5 候选生成、束宽截断与排序增强（engine/search.js）

```
  candidates(b, depth, ply, ttMove) {
    const o = this.o;
    let list;
    if (ply === 0) {
      const top = (o.order === 'sep' ? M.sepTopMoves(b, o.rootPre, this.w) :
M.topMoves(b, o.rootPre)).moves;
      const sc = new Array(top.length);
      for (let i = 0; i < top.length; i++) { b.play(top[i]); sc[i] = -
this.evaluate(b); b.undo(); }
      const idx = top.map((_, i) => i).sort((x, y) => sc[y] - sc[x] || x - y);
      list = idx.slice(0, o.rootWidth).map(i => top[i]);
      this.rootStatic = idx.map(i => ({ move: top[i], score: sc[i] }));
    } else if (depth === 1 && o.leafOrder === 'cheap') {
      list = M.topMoves(b, o.preWidth).moves;
    } else if (o.order === 'sep') {
      list = M.sepTopMoves(b, o.width, this.w).moves;
    } else {
      const top = M.topMoves(b, o.preWidth).moves;
      if (depth >= 2) {
        const sc = new Array(top.length);
        for (let i = 0; i < top.length; i++) { b.play(top[i]); sc[i] = -
this.evaluate(b); b.undo(); }
        list = top.map((_, i) => i).sort((x, y) => sc[y] - sc[x] || x -
y).slice(0, o.width).map(i => top[i]);
      } else list = top;
    }
    // 杀手着法与历史启发：稳定地把它们提前
    if (ply !== 0 && (o.useKillers || o.useHistory)) {
      const k = this.killers[ply] || [];
      const pri = new Map();
      list.forEach((m, i) => {
        let p = -i;                                          // 原顺序
        if (o.useHistory) p += Math.min(4, this.history[(C.mFrom(m) * 100 +
C.mTo(m)) * 2 + (b.side - 1)] / 1e3);
        if (o.useKillers && (m === k[0] || m === k[1])) p += 1000;
        pri.set(m, p);
      });
      list.sort((x, y) => pri.get(y) - pri.get(x));
    }
    if (ttMove > 0) {
      const j = list.indexOf(ttMove);
      if (j > 0) { list.splice(j, 1); list.unshift(ttMove); }
      else if (j < 0 && b.isLegal(ttMove)) list.unshift(ttMove);
```

```
    }
    return list;
  }
```

4.3.3 负极大值搜索主体

代码 4-6 是搜索的主体。与教科书中的 α-β 相比，它有以下几处针对本问题的处理：

1. **时间检查**：每 8 个结点检查一次时间，超时则抛出异常，由迭代加深的外层捕获；
2. **终局与分隔**：行棋方无子可动时返回 $-\mathrm{WIN}+\mathrm{ply}$（越早获胜分数越高）；静态评估落在“已分隔”的分数段时直接返回，不再向下搜索；
3. **置换表**：用 64 位 Zobrist 哈希的低 32 位定位槽位、高 32 位校验，按深度优先替换；
4. **PVS**：第一个子结点用完整窗口，其余用零宽窗口试探，失败高时再用完整窗口重搜；
5. **截断统计**：记录截断次数与“第一个着法即截断”的次数，用于 5.9 节的效率分析。

代码 4-6 负极大值 α-β 搜索（engine/search.js）

```
  negamax(b, depth, alpha, beta, ply) {
    const o = this.o;
    if ((++this.nodes & 7) === 0 && Date.now() > this.deadline) throw new
TimeUp();
    if (!b.canMove(b.side)) return -WIN + ply;
    if (depth <= 0) return this.evaluate(b);
    const a0 = alpha;
    let ttMove = 0, slot = -1;
    if (o.useTT) {
      slot = b.hashLo & this.ttMask;
      if (this.ttKey[slot] === b.hashLo && this.ttKey2[slot] === b.hashHi &&
this.ttDepth[slot] >= 0) {
        ttMove = this.ttMove[slot];
        if (this.ttDepth[slot] >= depth) {
          const v = this.ttVal[slot], f = this.ttFlag[slot];
          this.ttHits++;
          if (f === EXACT) return v;
          if (f === LOWER && v >= beta) return v;
          if (f === UPPER && v <= alpha) return v;
        }
      }
    }
    const sv = this.evaluate(b);
    if (Math.abs(sv) >= E.SEP_BONUS - 60 && Math.abs(sv) < WIN_BOUND) return sv;
// 已分隔：区域计数即为估计
    const moves = this.candidates(b, depth, ply, ttMove);
```

```
    this.interior++;
    let best = -INF, bestM = moves[0];
    for (let i = 0; i < moves.length; i++) {
      const m = moves[i];
      b.play(m);
      let v;
      if (i === 0 || !o.usePVS) v = -this.negamax(b, depth - 1, -beta, -alpha, ply
+ 1);
      else {
        v = -this.negamax(b, depth - 1, -alpha - 1e-6, -alpha, ply + 1);
        if (v > alpha && v < beta) v = -this.negamax(b, depth - 1, -beta, -alpha,
ply + 1);
      }
      b.undo();
      if (v > best) { best = v; bestM = m; }
      if (v > alpha) alpha = v;
      if (alpha >= beta) {
        this.cutoffs++; if (i === 0) this.firstCutoffs++;
        if (o.useKillers) {
          const k = this.killers[ply] || (this.killers[ply] = [0, 0]);
          if (k[0] !== m) { k[1] = k[0]; k[0] = m; }
        }
        if (o.useHistory) this.history[(C.mFrom(m) * 100 + C.mTo(m)) * 2 + (b.side
- 1)] += depth * depth;
        break;
      }
    }
    if (o.useTT) {
      this.ttKey[slot] = b.hashLo; this.ttKey2[slot] = b.hashHi;
this.ttDepth[slot] = depth;
      this.ttVal[slot] = best; this.ttMove[slot] = bestM;
      this.ttFlag[slot] = best <= a0 ? UPPER : best >= beta ? LOWER : EXACT;
    }
    return best;
  }
```

4.3.4 迭代加深与根结点

迭代加深的外层循环（代码 4-7）每完成一层就记录本层的最佳着法、分数、累计结点数与用时，并通过回调推送给界面；若某一层中途超时，而根结点已有至少一个着法完整搜索完毕，就采用这些着法中最好的一个（因为根结点的第一个着法总是上一层的最佳着法，这一做法不会比只用上一层的结果更差）。

代码 4-7 迭代加深的外层循环与根结点搜索（engine/search.js）

```
  search(b0) {
    const o = this.o;
    const b = b0.clone();
    this.t0 = Date.now(); this.deadline = this.t0 + o.timeMs;
    this.nodes = 0; this.cutoffs = 0; this.firstCutoffs = 0; this.ttHits = 0;
this.interior = 0; this.killers = []; this.history.fill(0); this.lastBest = 0;
this.rootStatic = null;
    if (o.useTT) this.ttDepth.fill(-1);
```

```
    const iters = [];
    let best = -1, bestScore = 0, depthDone = 0, pv = [], rootScores = [],
partialUsed = false;
    for (let d = 1; d <= o.maxDepth; d++) {
      this.rootDepth = d;
      try {
        const r = this.root(b, d);
        best = r.move; bestScore = r.score; depthDone = d; rootScores = r.scores;
        pv = this.extractPV(b, best, d);
        iters.push({ depth: d, move: C.moveToStr(best), score:
+bestScore.toFixed(2), nodes: this.nodes, ms: Date.now() - this.t0 });
        if (o.onIteration) o.onIteration(Object.assign({ pv:
pv.map(C.moveToStr) }, iters[iters.length - 1]));
        if (Math.abs(bestScore) > WIN_BOUND) break;          // 已找到胜负
        const el = Date.now() - this.t0;
        if (el > o.timeMs * 0.5) break;                      // 剩余时间不足一半时不再
开始新的一层
      } catch (e) {
        if (!(e instanceof TimeUp)) throw e;
        // 未完成的一层：若上一层的最佳着法已在本层搜完，则可采用本层已搜完部分中的最佳着法
        const p = this.partial;
        if (p && p.count >= 1 && p.move !== best) {
          best = p.move; bestScore = p.score; partialUsed = true;
          iters.push({ depth: d, move: C.moveToStr(best), score:
+bestScore.toFixed(2), nodes: this.nodes, ms: Date.now() - this.t0, partial:
p.count });
        }
        break;
      }
    }
    if (best < 0) best = this.rootStatic && this.rootStatic.length ?
this.rootStatic[0].move : b.genMoves()[0];
    return {
      move: best, score: bestScore, depth: depthDone, nodes: this.nodes, timeMs:
Date.now() - this.t0,
      reason: 'search', pv: pv.map(C.moveToStr), iterations: iters, partialUsed,
      stats: { cutoffs: this.cutoffs, firstCutoffs: this.firstCutoffs, ttHits:
this.ttHits, interior: this.interior },
      rootMoves: rootScores.slice(0, 12).map(x => ({ move: x.move, score:
x.score })),
    };
  }

  root(b, depth) {
    const o = this.o;
    const moves = this.candidates(b, depth, 0, depth > 1 ? this.lastBest : 0);
    let alpha = -INF, beta = INF, best = moves[0], bestV = -INF;
    const scores = [];
    this.partial = null;
    for (let i = 0; i < moves.length; i++) {
      const m = moves[i];
      b.play(m);
      let v;
      if (i === 0 || !o.usePVS) v = -this.negamax(b, depth - 1, -beta, -alpha, 1);
      else {
```

```
        v = -this.negamax(b, depth - 1, -alpha - 1e-6, -alpha, 1);
        if (v > alpha) v = -this.negamax(b, depth - 1, -beta, -alpha, 1);
      }
      b.undo();
      scores.push({ move: m, score: v });
      if (v > bestV) { bestV = v; best = m; }
      if (v > alpha) alpha = v;
      this.partial = { move: best, score: bestV, count: i + 1 };
    }
    this.lastBest = best;
    scores.sort((x, y) => y.score - x.score);
    return { move: best, score: bestV, scores };
  }
```

4.4 MCTS 的流程与代码

4.4.1 主循环

MCTS 的一次模拟分为选择、扩展、模拟与回传四步（图 4-3）。代码 4-8 是主循环：在树中下降时，若当前结点“允许展开”（渐进展开的条件满足且仍有候选），就按候选顺序展开下一个子结点并停止下降；否则用 UCT 或 PUCT 选择子结点继续下降。到达叶结点后，若它是终局则价值为 1（走入该结点的一方获胜）；若已分隔则用区域计分直接判断胜负；否则调用模拟策略。回传时价值在每一层翻转。

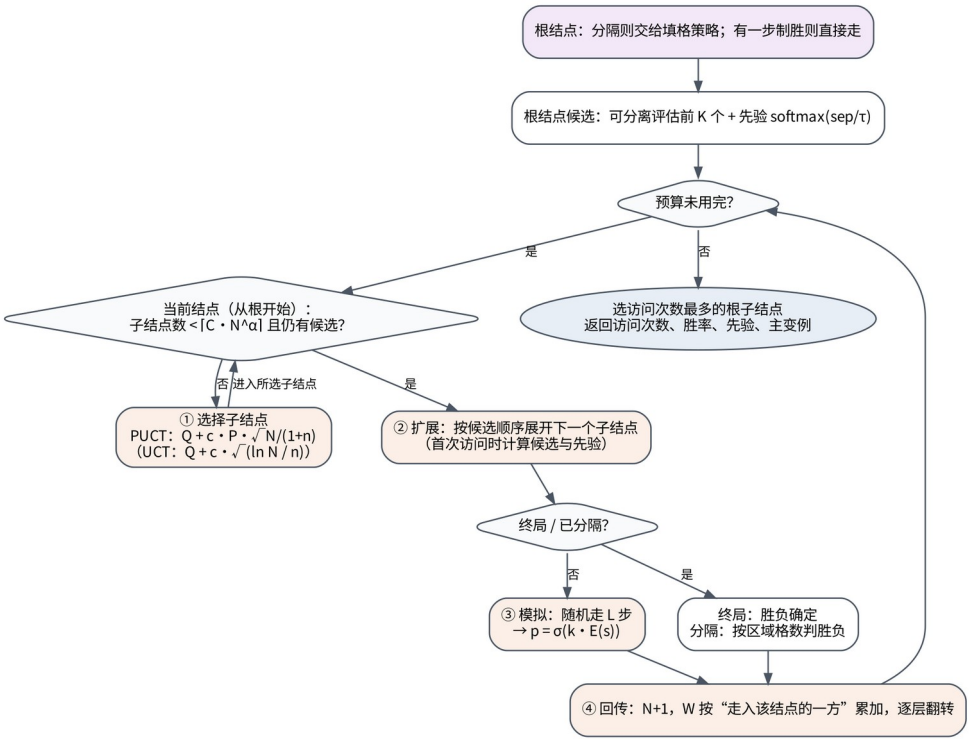


图 4-3 MCTS 单次模拟的流程

注：作者绘制（Graphviz）。

代码 4-8 MCTS 主循环（engine/mcts.js）

```
  search(b0) {
    const o = this.o;
    const b = b0.clone();
    const t0 = Date.now(), deadline = t0 + o.timeMs;
    const root = new Node(null, -1, 1);
    this.initCandidates(root, b);
    // 一步制胜检查：走完后对方无子可动
    if (o.winCheck) for (const m of b.genMoves()) { b.play(m); const win = !
b.canMove(b.side); b.undo(); if (win) return { move: m, reason: 'win',
simulations: 0, winrate: 1, timeMs: Date.now() - t0 }; }
    let sims = 0, maxDepth = 0, depthSum = 0;
    while (true) {
      if (o.maxSims ? sims >= o.maxSims : ((sims & 15) === 0 && Date.now() >=
deadline)) break;
      let node = root, d = 0;
      // 1. 选择/ 2. 扩展
      while (true) {
        if (node.terminal) break;
        if (!node.cand) this.initCandidates(node, b);
        if (this.canExpand(node)) {
          const m = node.cand[node.next];
          const p = node.candP ? node.candP[node.next] : 1 / node.cand.length;
          node.next++;
          const c = new Node(node, m, p);
          node.children.push(c);
          b.play(m); d++;
          if (!b.canMove(b.side)) c.terminal = 1;
          node = c;
          break;
        }
        if (!node.children.length) break;
        node = this.select(node);
        b.play(node.move); d++;
      }
      // 3. 模拟：value 为“走入 node 的一方”的收益
      let value;
      if (node.terminal) value = 1;
      else if (o.sepCheck && E.separated(b)) {
        const v = E.evaluate(b, this.w); value = v > 0 ? 0 : 1;
      } else value = 1 - this.leafValue(b);
      // 4. 回传
      for (let x = node; x; x = x.parent) { x.n++; x.w += value; value = 1 -
value; }
      for (let i = 0; i < d; i++) b.undo();
      sims++; depthSum += d; if (d > maxDepth) maxDepth = d;
      if (o.onProgress && (sims & 63) === 0) {
        const now = Date.now();
        if (!this._lastProg || now - this._lastProg >= 150) {
          this._lastProg = now;
          let bc = null; for (const c of root.children) if (!bc || c.n > bc.n) bc
= c;
          o.onProgress({ simulations: sims, ms: now - t0, best:
C.moveToStr(bc.move), winrate: bc.w / Math.max(1, bc.n), maxDepth });
```

```
        }
      }
    }
    let best = null;
    for (const c of root.children) if (!best || c.n > best.n || (c.n === best.n &&
c.w > best.w)) best = c;
    if (!best) { const ms = M.topMoves(b, 1).moves; return { move: ms[0], reason:
'fallback', simulations: sims, winrate: 0.5, timeMs: Date.now() - t0 }; }
    const kids = root.children.slice().sort((x, y) => y.n - x.n);
    // 主变例：沿访问次数最多的子结点下降
    const pv = []; for (let x = best; x; ) { pv.push(C.moveToStr(x.move)); let nb
= null; for (const c of x.children) if (!nb || c.n > nb.n) nb = c; if (!nb || nb.n
< 2) break; x = nb; if (pv.length >= 8) break; }
    return {
      move: best.move, reason: 'mcts', simulations: sims, winrate: best.w /
Math.max(1, best.n),
      selDepth: maxDepth, avgDepth: sims ? depthSum / sims : 0, timeMs: Date.now()
- t0, pv,
      rootChildren: root.children.length,
      rootMoves: kids.slice(0, 12).map(c => ({ move: c.move, visits: c.n, winrate:
c.n ? c.w / c.n : 0, prior: c.prior })),
    };
  }
}
```

4.4.2 候选、展开与选择

每个结点在第一次被访问时才计算候选列表（代码 4-9 的 `initCandidates`），并按配置使用随机顺序、廉价分数、可分离评估或学习策略排序；对后三者，同时计算先验概率。`canExpand` 实现了式 2-11 的渐进展开条件；`select` 实现了式 2-10 与式 2-12，未访问结点的平均收益取 0.5。

代码 4-9 候选初始化、渐进展开与选择（engine/mcts.js）

```
  initCandidates(node, b) {
    const o = this.o;
    if (o.order === 'random') {
      const ms = b.genMoves();
      for (let i = ms.length - 1; i > 0; i--) { const j = (this.rng() * (i + 1)) |
0; const t = ms[i]; ms[i] = ms[j]; ms[j] = t; }
      node.cand = ms; node.candP = null;
    } else if (o.order === 'learned') {
      const r = o.policyModel.rank(b, o.maxCand);
      node.cand = r.moves; node.candP = r.probs;
    } else {
      const r = o.order === 'sep' ? M.sepTopMoves(b, o.maxCand, this.w) :
M.topMoves(b, o.maxCand);
      node.cand = r.moves;
      // 启发式先验：对排序分数做 softmax（仅在 PUCT 下使用；温度 tau）
      const mx = r.scores[0] || 0; let z = 0;
      const p = r.scores.map(s => { const e = Math.exp((s - mx) / o.tau); z += e;
return e; });
      node.candP = p.map(x => x / z);
```

```
    }
  }

  canExpand(node) {
    if (node.next >= node.cand.length) return false;
    if (!this.o.pw) return true;
    return node.children.length < Math.ceil(this.o.pwC * Math.pow(node.n + 1,
this.o.pwA));
  }

  select(node) {
    const o = this.o;
    let best = null, bv = -Infinity;
    const lnN = Math.log(node.n + 1), sq = Math.sqrt(node.n + 1);
    for (const c of node.children) {
      if (c.terminal) return c;                              // 一步制胜
      const q = c.n ? c.w / c.n : 0.5;
      const u = o.selection === 'puct'
        ? q + o.cPuct * c.prior * sq / (1 + c.n)
        : q + o.cUct * Math.sqrt(lnN / (c.n + 1e-9));
      if (u > bv) { bv = u; best = c; }
    }
    return best;
  }
```

4.4.3 模拟策略

代码 4-10 是三种模拟策略：随机下到终局（`random`）、随机走 L 步后用评估函数截断（`cutoff`）、直接评估（`eval`）。评估值通过 logistic 函数换算为胜率（式 2-13）。注意随机模拟走完后要逐步撤销，恢复树中的局面。

代码 4-10 叶结点价值：模拟与截断评估（engine/mcts.js）

```
  leafValue(b) {
    const o = this.o;
    if (!b.canMove(b.side)) return 0;
    if (o.rollout === 'random') return this.randomPlayout(b, 400);
    if (o.rollout === 'eval') return this.evalToProb(b);
    // cutoff：随机走 cutoff 步后评估
    const s0 = b.side; let k = 0;
    for (; k < o.cutoff; k++) {
      if (!b.canMove(b.side)) break;
      b.play(b.randomMove(this.rng));
    }
    let v;
    if (!b.canMove(b.side)) v = b.side === s0 ? 0 : 1;
    else { const p = this.evalToProb(b); v = b.side === s0 ? p : 1 - p; }
    for (let i = 0; i < k; i++) b.undo();
    return v;
  }
  evalToProb(b) {
    const v = E.evaluate(b, this.w);
    return 1 / (1 + Math.exp(-this.o.k * v));
```

```
  }
  randomPlayout(b, cap) {
    const s0 = b.side; let k = 0;
    while (k < cap && b.canMove(b.side)) { b.play(b.randomMove(this.rng)); k++; }
    const v = b.canMove(b.side) ? 0.5 : (b.side === s0 ? 0 : 1);
    for (let i = 0; i < k; i++) b.undo();
    return v;
  }
```

4.5 填格策略与智能体

4.5.1 共用的填格策略

分隔之后，除随机智能体与“纯 MCTS”之外，所有智能体都调用同一个填格策略（代码 4-11）：枚举全部合法着法，选择走完后己方王距离可达空格数最多的一个，同分时优先把箭射在周围空格最少的位置。

代码 4-11 填格策略（engine/agents.js）

```
function reachable(b, side) {
  E.kingDist(b.cells, b.queens[side], kbuf);
  let n = 0; for (let i = 0; i < C.NN; i++) if (b.cells[i] === C.EMPTY && kbuf[i]
< C.INF) n++;
  return n;
}
function fillerMove(b) {
  const s = b.side, ms = b.genMoves();
  let best = ms[0], bv = -Infinity;
  for (const m of ms) {
    const a = C.mArrow(m);
    b.play(m);
    let free = 0; for (const x of C.NEIGH[a]) if (b.cells[x] === C.EMPTY) free++;
    const v = reachable(b, s) * 10 - free;
    b.undo();
    if (v > bv) { bv = v; best = m; }
  }
  return best;
}

function maybeFiller(b, o, t0) {
  if (o.filler === false) return null;
  if (!E.separated(b)) return null;
  return { move: fillerMove(b), reason: 'filler', score: E.evaluate(b, o.weights
|| undefined), timeMs: Date.now() - t0 };
}
```

4.5.2 智能体预设

所有参与实验的智能体都以“预设”的形式定义在 PRESETS 中（代码 4-12），实验与网页通过同一个名字创建同一个智能体；消融实验在预设的基础上用 overrides 修改个别参数。

代码 4-12 智能体预设（engine/agents.js）

```
const PRESETS = {
  'random': { kind: 'random' },
  'greedy': { kind: 'greedy' },
  // α-β 完整版与消融
  'ab': { kind: 'ab' },
  'ab-noTT': { kind: 'ab', useTT: false },
  'ab-noKiller': { kind: 'ab', useKillers: false, useHistory: false },
  'ab-noPVS': { kind: 'ab', usePVS: false },
  'ab-cheapOrder': { kind: 'ab', order: 'cheap' },
  'ab-sepLeaf': { kind: 'ab', leafOrder: 'sep' },
  'ab-wide': { kind: 'ab', width: 16, preWidth: 48, rootPre: 120, rootWidth: 60 },
  'ab-narrow': { kind: 'ab', width: 4, preWidth: 12, rootPre: 30, rootWidth: 15 },
  'ab-simpleEval': { kind: 'ab', weights: [1, 0, 0, 0, 0, 1, 0, 0, 0, 0] },
  'ab-tuned': { kind: 'ab', weights: 'tuned' },
  'ab-plain': { kind: 'ab', useTT: false, useKillers: false, useHistory: false,
usePVS: false },
  // MCTS 系列
  'mcts-pure': { kind: 'mcts', selection: 'uct', order: 'random', pw: false,
rollout: 'random', sepCheck: false, winCheck: false, filler: false },
  'mcts-uct': { kind: 'mcts', selection: 'uct', order: 'random', pw: false,
rollout: 'cutoff' },
  'mcts-pw': { kind: 'mcts', selection: 'uct', order: 'heuristic', pw: true,
rollout: 'random' },
  'mcts-cutoff': { kind: 'mcts', selection: 'uct', order: 'heuristic', pw: true,
rollout: 'cutoff' },
  'mcts-sep': { kind: 'mcts', selection: 'uct', order: 'sep', pw: true, rollout:
'cutoff', maxCand: 30 },
  'mcts-puct': { kind: 'mcts', selection: 'puct', order: 'sep', pw: true, rollout:
'cutoff', maxCand: 30 },
  'mcts-learned': { kind: 'mcts', selection: 'puct', order: 'learned', pw: true,
rollout: 'cutoff', maxCand: 30 },
};

let POLICY_THETA = null;
let TUNED_WEIGHTS = null;
```

4.6 网页界面的实现

4.6.1 Web Worker 架构

浏览器主线程负责绘制与交互，引擎在 Web Worker 中运行（图 4-4）。主线程向 Worker 发送 `{type:'think', id, moves, agent, opts}` 消息；Worker 按着法序列重建棋盘，创建智能体，在搜索过程中通过 `progress` 消息推送迭代加深的每一层结果或 MCTS 的阶段性统计，最后以 `result` 消息返回着法与完整诊断信息。若用户在 AI 思考期间悔棋或开新局，主线程直接终止 Worker 并新建一个，保证界面状态与引擎状态一致。

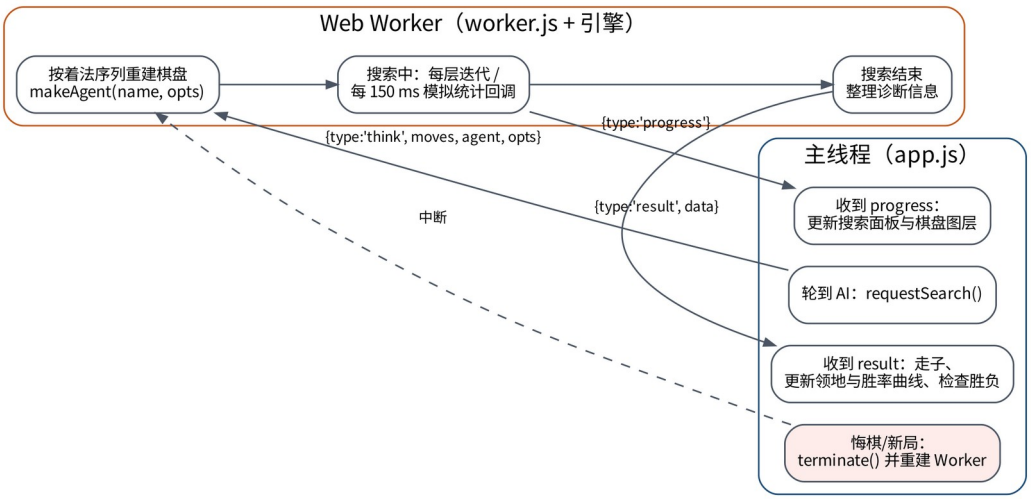


图 4-4 界面主线程与引擎 Worker 的消息时序

注：作者绘制（Graphviz）。

代码 4-13 Web Worker 消息处理（game/src/worker.js）

```
/*
 * worker.js — 在Web Worker 中运行AI 搜索，避免阻塞界面。
 * 打包时build.js 会把引擎代码拼接在本文件之前，全局对象AmazonsAI 可直接使用。
 * 消息协议:
 *   主线程→worker: { type: 'think', id, moves: [着法字符串...], agent, opts }
 *   worker → 主线程: { type: 'progress', id, data } / { type: 'result', id, data }
/ { type: 'error', id, message }
 */
/* global AmazonsAI, POLICY_THETA, TUNED_WEIGHTS */
(function () {
  const G = AmazonsAI;
  if (typeof POLICY_THETA !== 'undefined' && POLICY_THETA)
G.setPolicyTheta(POLICY_THETA);
  if (typeof TUNED_WEIGHTS !== 'undefined' && TUNED_WEIGHTS)
G.setTunedWeights(TUNED_WEIGHTS);

  self.onmessage = function (ev) {
    const msg = ev.data;
    if (msg.type !== 'think') return;
    const id = msg.id;
    try {
      const b = G.Board.fromMoves(msg.moves);
      const opts = Object.assign({}, msg.opts || {});
      if (msg.agent.startsWith('ab')) opts.onIteration = it => self.postMessage({
type: 'progress', id, data: { kind: 'ab', iteration: it } });
      else if (msg.agent.startsWith('mcts')) opts.onProgress = rep =>
self.postMessage({ type: 'progress', id, data: Object.assign({ kind: 'mcts' },
rep) });
      const agent = G.makeAgent(msg.agent, opts);
      const t0 = Date.now();
      const r = agent.think(b);
      const out = {
        move: G.moveToStr(r.move), reason: r.reason, score: r.score, depth:
```

```
r.depth, selDepth: r.selDepth, nodes: r.nodes,
        timeMs: r.timeMs, wallMs: Date.now() - t0, pv: r.pv, iterations:
r.iterations, partialUsed: r.partialUsed,
        simulations: r.simulations, winrate: r.winrate, avgDepth: r.avgDepth,
rootChildren: r.rootChildren,
        rootMoves: (r.rootMoves || []).map(x => Object.assign({}, x, { move:
G.moveToStr(x.move) })),
      };
      self.postMessage({ type: 'result', id, data: out });
    } catch (e) {
      self.postMessage({ type: 'error', id, message: String(e && e.stack || e) });
    }
  };
})();
```

为了满足“单个 HTML 文件、双击即可运行”的要求，打包脚本（代码 4-14）把引擎模块包装在一个极简的模块加载器中，并把 Worker 的代码放在 `type="text/plain"` 的脚本标签里，运行时用 `Blob` 和 `URL.createObjectURL` 创建 Worker。这样在 `file://` 协议下也能使用 Web Worker，而不需要单独的 `.js` 文件。

代码 4-14 单文件打包脚本（game/build.js）

```
'use strict';
/*
 * 打包脚本：把引擎（CommonJS 模块）、模型参数、Worker 与界面代码内联为单个HTML 文件。
 *   node game/build.js   →   game/dist/amazons.html
 * 生成的文件双击即可在浏览器中打开（file:// 协议），无需服务器与网络。
 */
const fs = require('fs');
const path = require('path');
const root = path.join(__dirname, '..');
const rd = p => fs.readFileSync(path.join(root, p), 'utf8');

const files = ['core.js', 'eval.js', 'movegen.js', 'search.js', 'mcts.js',
'policy.js', 'agents.js', 'index.js'];
let engine = '/* AmazonsAI engine bundle — generated by game/build.js */\
n(function (global) {\n"use strict";\nconst __mods = {};\nconst __cache = {};\n' +
  'function require(name) { const k = name.replace(/^\\.\\//,
"").replace(/^\\.\\.\\/engine\\//, ""); if (__cache[k]) return __cache[k].exports;
const m = { exports: {} }; __cache[k] = m; __mods[k](m, m.exports, require);
return m.exports; }\n';
for (const f of files) engine += `__mods[${JSON.stringify(f)}] = function (module,
exports, require) {\n${rd('engine/' + f)}\n};\n`;
engine += 'global.AmazonsAI = require("index.js");\n})(typeof self !== "undefined"
? self : this);\n';

const readJSON = p => fs.existsSync(path.join(root, p)) ? JSON.parse(rd(p)) :
null;
const pw = readJSON('models/policy_weights.json'), tw =
readJSON('models/eval_weights_tuned.json'), ui =
readJSON('models/ui_config.json');
let models = '';
models += `var POLICY_THETA = ${pw ? JSON.stringify(pw.theta) : 'null'};\n`;
```

```
models += `var TUNED_WEIGHTS = ${tw ? JSON.stringify(tw.weights) : 'null'};\n`;
models += `var WIN_K = ${tw && (tw.K_hand || tw.K) ? (tw.K_hand || tw.K) :
"null"};\n`;
models += `var AI_LIST_CFG = ${ui ? JSON.stringify(ui.ai_list) : 'undefined'};\n`;
const op = JSON.parse(rd('data/openings.json')).openings.map(o => o.moves);
models += `var OPENINGS = ${JSON.stringify(op)};\n`;

const worker = engine + '\n' + models + '\n' + rd('game/src/worker.js');
let html = rd('game/src/index.html');
const put = (tag, s) => { if (!html.includes(tag)) throw new Error(tag); html =
html.split(tag).join(s.replace(/<\/script/gi, '<\\/script')); };
put('/*STYLE*/', rd('game/src/style.css'));
put('/*ENGINE*/', engine);
put('/*MODELS*/', models);
put('/*WORKER*/', worker);
put('/*APP*/', rd('game/src/app.js'));
fs.mkdirSync(path.join(__dirname, 'dist'), { recursive: true });
fs.writeFileSync(path.join(__dirname, 'dist/amazons.html'), html);
fs.writeFileSync(path.join(__dirname, 'dist/amazons-engine.js'), engine);
console.log('built game/dist/amazons.html', (html.length / 1024).toFixed(1) + '
KB');
```

4.6.2 人机交互

亚马逊棋的一步由三次点击完成：先点击己方的一个后，界面以绿色圆点标出它能到达的全部格子；再点击目的地，界面以红色圆点标出从目的地能射箭的全部格子（包括刚刚空出的出发格）；最后点击射箭位置。任何时候右键或按 Esc 都可以取消当前选择。这种“分步高亮”的交互把 2000 多个着法拆成了三个各只有十几到几十个选项的小选择，与 3.2.3 节的“动作分解”思想一致。

4.6.3 决策过程的可视化

界面的绘制在一个 Canvas 上分层完成（表 4-2），每一层都可以单独开关。

表 4-2 棋盘的可视化图层

图层	数据来源	呈现方式
领地	双方后距离的比较（territoryMap）	蓝色/红色底纹表示白方/黑方能更快到达的空格，均势格为浅灰
搜索候选	MCTS：根结点各子结点的访问次数与胜率；α-β：根结点各候选的搜索分数	从出发格到目的格、再到射箭格的折线箭头，粗细与颜色深浅表示访问次数或排名
主变例	搜索返回的 PV	虚线箭头与序号，显示 AI“预想”的后续着法
距离数字	双方的后距离	每个空格上显示“白距离/黑距离”
箭的手数	对局历史	箭上的数字
走法提示	当前选择的后或目的地	绿色（目的地）与红色（射箭位置）圆点

右侧面板有四个标签页：“搜索过程”显示着法理由、深度、结点数/模拟次数、用时、迭代加深表格或 MCTS 候选表；“局势”显示双方的领地格数、争夺格数、阶

段系数与每一手后白方的估计胜率曲线（α-β 的分数用式 2-13 换算）；“棋谱”可导出、导入、复制对局记录；“算法说明”用简短文字解释当前 AI 的原理。界面截图见 5.2 节。

4.7 实验系统的实现

4.7.1 对局器与任务执行

对局器 `playGame` 从给定开局开始，交替调用双方智能体直到终局，记录每一步的着法、用时（高精度计时）、着法理由、深度、结点数、模拟次数等。任务清单由 `make_jobs.js` 生成：每个任务指定白黑双方的智能体与参数、开局和随机种子。执行器 `runner.js` 用 Node.js 子进程并行执行任务，每完成一局就把结果追加到 JSONL 文件；若程序中断，重新运行时会自动跳过已完成的任务（实验过程中容器曾经重启，自对弈数据正是依靠这一机制续跑完成的）。整个流程如图 4-5 所示。

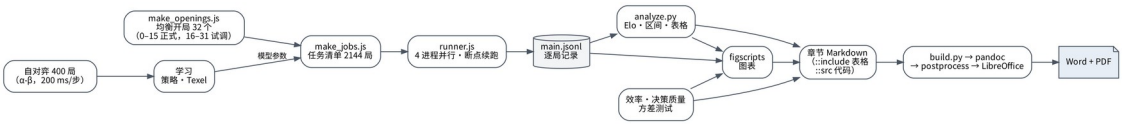


图 4-5 实验与报告生成流水线

注：作者绘制（Graphviz）。

代码 4-15 多进程任务执行器（experiments/runner.js）

```
'use strict';
/*
 * 并行对局调度器
 *   node experiments/runner.js jobs.json out.jsonl [workers]
 * jobs.json: [{ id, tag, white: {agent, opts}, black: {agent, opts}, opening:
[...着法字符串], seed }]（白方先走）
 * 每局结果写一行JSON；已完成的id 会被跳过（支持断点续跑）。
 */
const fs = require('fs');
const path = require('path');
const { fork } = require('child_process');

if (process.argv[2] === '--worker') {
  const G = require('../engine');
  const { playGame } = require('./arena.js');
  const root = path.join(__dirname, '..');
  const pw = path.join(root, 'models', 'policy_weights.json');
  if (fs.existsSync(pw)) G.setPolicyTheta(JSON.parse(fs.readFileSync(pw)).theta);
  const tw = path.join(root, 'models', 'eval_weights_tuned.json');
  if (fs.existsSync(tw))
G.setTunedWeights(JSON.parse(fs.readFileSync(tw)).weights);
  process.on('message', job => {
    const mk = s => G.makeAgent(s.agent, Object.assign({ seed: (job.seed || 1) *
7919 + (s === job.white ? 1 : 2) }, s.opts || {}));
```

```
    const t0 = Date.now();
    const g = playGame(mk(job.white), mk(job.black),
job.opening.map(G.parseMove));
    const out = { id: job.id, tag: job.tag, white: job.white, black: job.black,
opening: job.opening, winner: g.winner, length: g.length,
      moves: g.moves.map(G.moveToStr), wallMs: Date.now() - t0 };
    out.log = g.log.map(x => ({ s: x.side, m: G.moveToStr(x.move), ms:
+x.ms.toFixed(1), r: x.reason, sc: x.score, d: x.depth, n: x.nodes, sims: x.sims,
wr: x.winrate != null ? +(+x.winrate).toFixed(4) : undefined, sd: x.selDepth }));
    process.send(out);
  });
  process.send({ ready: true });
  return;
}

const [jobsFile, outFile, nw] = process.argv.slice(2);
const jobs = JSON.parse(fs.readFileSync(jobsFile));
const done = new Set();
if (fs.existsSync(outFile)) for (const line of fs.readFileSync(outFile,
'utf8').split('\n')) if (line.trim()) done.add(JSON.parse(line).id);
const queue = jobs.filter(j => !done.has(j.id));
const W = +(nw || 3);
const TOTAL = queue.length;
console.log(`${jobs.length} jobs, ${done.size} done, ${queue.length} to run, ${W}
workers`);
const out = fs.openSync(outFile, 'a');
let finished = 0, active = 0;
const t0 = Date.now();
for (let i = 0; i < Math.min(W, queue.length); i++) {
  const w = fork(__filename, ['--worker']);
  active++;
  w.on('message', msg => {
    if (!msg.ready) {
      fs.writeSync(out, JSON.stringify(msg) + '\n');
      finished++;
      if (finished % 20 === 0 || finished === TOTAL) {
        const el = (Date.now() - t0) / 1000;
        console.log(`${finished}/${TOTAL}  ${el.toFixed(0)}s  eta ${(el / finished
* (TOTAL - finished)).toFixed(0)}s`);
      }
    }
    const j = queue.shift();
    if (j) w.send(j); else { w.kill(); if (--active === 0) fs.closeSync(out); }
  });
}
```

4.7.2 公平性措施

为保证比较公平，实验系统采取了以下措施：

1. **相同时间**：除缩放实验外，双方每步思考时间都是 200 ms；
2. **相同开局**：32 个开局由程序生成：从标准初始局面出发，白、黑各走一步（共 2 手），每一手都从静态评估排序的前 30 名中随机选取；再用 1 秒的 α-β 搜索

评估开局后的局面，评估绝对值不小于 4 格的视为失衡而丢弃（共生成 40 个，保留 32 个）。其中第 0—15 号用于正式实验，第 16—31 号只用于试调（5.1.3 节）；

3. **交换先后手**：每个开局下两局，双方各执白一次；
4. **相同知识**：所有智能体共享同一个评估函数、着法排序与填格策略（学习版本除外，它们是被比较的对象）；
5. **固定种子**：MCTS 与随机智能体的随机数种子由任务编号决定；
6. **并行干扰**：4 个工作进程同时运行时，各进程会竞争 CPU。时间预算对双方相同，因此影响是对称的，但绝对搜索量会比单进程略低。效率测试（5.9 节）在单进程下单独进行。

4.8 测试

4.8.1 单元测试

`tests/run_tests.js` 包含 27 项自动化测试，分为六组（表 4-3）。其中多项测试采用“与独立实现对比”的差分测试思路：`isLegal` 与 `genMoves` 对比、快速 BFS 与参考 BFS 对比、分隔判定与独立的连通分量判定对比、α-β 与穷举极小极大对比。

表 4-3 单元测试概览

组别	测试数	主要内容
规则与走法生成	6	开局 2176 个合法着法；500 个随机局面中走法无重复且全部合法；随机编码 20 万次时 `isLegal` 与 `genMoves` 完全一致；`canMove` 与“存在合法走法”等价；悔棋完全恢复棋盘与哈希；着法字符串互逆
评估与排序	5	快速距离计算与参考实现一致；评估在“翻转并交换颜色”下对称；终局与分隔时的评估值；评估是 10 维特征的线性函数；部分排序与完整排序的前 k 个一致
α-β 搜索	3	12 个小局面上，全宽深度 2 的 α-β（带与不带全部增强）与穷举极小极大根值完全相同；能找到一步制胜；不修改输入棋盘且遵守时间预算
智能体	2	全部预设智能体给出合法着法；填格策略选出的着法使己方可达格数最大
MCTS	4	访问次数之和等于模拟次数；相同种子可复现；渐进展开的子结点数上界；先验是概率分布
知识层、学习模型与对局	7	分隔判定与独立的连通分量判定一致（含敌对的后相邻的情形）；一旦分隔之后始终保持分隔（回归测试，5.10.1 节）；领地图与后距离比较一致；可分离排序的输出合法且单调；线性策略输出的概率分布；随机对局在 92 手内结束；

组别	测试数	主要内容
		α-β 的主变例合法

注：运行 `node tests/run_tests.js` 的输出见附录 C，结果同时写入 `data/unit_tests.json`。

代码 4-16 展示其中一个差分测试：在 12 个合法着法较少的随机局面上，用不带任何剪枝的穷举极小极大计算深度 2 的根结点值，与全宽 α-β（分别开启与关闭置换表、杀手、历史和 PVS）的结果逐一比较。这一测试在开发中发现过一个错误：最初的参考实现没有在“已分隔”的局面停止，与 α-β 的终止条件不一致（3.7.3 节）。

代码 4-16 差分测试示例：α-β 与穷举极小极大（tests/run_tests.js）

```
function bruteNegamax(b, d, w, ply = 0) {
  if (!b.canMove(b.side)) return -E.WIN + ply;          // 与α-β 相同：越早获胜分数越高
  if (d === 0 || E.separated(b)) return E.evaluate(b, w);      // 与α-β 相同：分隔后不再向下搜索
  let best = -Infinity;
  for (const m of b.genMoves()) { b.play(m); const v = -bruteNegamax(b, d - 1, w, ply + 1); b.undo(); if (v > best) best = v; }
  return best;
}
```

4.8.2 界面自动化测试

`tests/ui_test.js` 使用 Playwright 驱动无头 Chromium，以 `file://` 方式打开打包后的 `amazons.html`，通过真实的鼠标点击、键盘与表单操作执行一组用例：三步点击走子与 AI 应着、非法点击与思考期间的输入屏蔽、右键取消、悔棋、提示、图层开关、棋谱导入导出、暂停继续、搜索中途切换模式、每种 AI 与贪心基线完整对局一局、手机竖屏布局等，同时监听控制台错误。每个用例记录“通过/未通过”与观测值，写入 `data/ui_test.json`，结果见 5.2.5 节。5.2 节的界面截图由另一个脚本 `tests/screenshots.js` 以同样方式自动截取。

4.9 本章小结

本章给出了系统的架构、模块、流程图与关键代码。整个系统的一个核心约束是“同一份引擎代码同时服务于网页与实验”，这保证了报告中的测量结果与用户的实际体验一致。代码中的关键部分（走法生成、距离计算、分隔判定、α-β、MCTS）都配有差分测试或性质测试，界面配有自动化测试。下一章报告界面呈现与全部实验结果。

五、算法结果和界面呈现

本章先介绍实验设置与评价方法，然后展示游戏界面与 AI 的决策过程，再依次报告循环赛、α-β 消融、MCTS 消融、算力缩放、学习组件、单步决策质量与搜索效率的实验结果，最后分析失败案例、局限与有效性威胁。本章所有数字均来自本项目程序的实际运行，表格由分析脚本直接从对局记录生成。

5.1 实验设置

5.1.1 硬件与软件环境

全部实验在同一台云端 Linux 容器中完成：4 个 CPU 核心，Node.js 22.23 运行引擎，Python 3.13（NumPy、SciPy、Matplotlib）做统计与绘图。对局实验用 4 个工作进程并行；搜索效率、决策质量与随机性实验用单进程顺序运行，以避免进程间竞争对计时的影响。

5.1.2 参赛智能体

表 5-1 列出参加主实验循环赛的 8 个智能体。它们共享同一套棋盘表示、评估函数与着法排序，除随机智能体与纯随机 MCTS 外也共享同一个填格策略，区别只在搜索/决策框架与组件。

表 5-1 循环赛中的 8 个智能体

名称	类型	说明
随机	基线	在全部合法着法中均匀随机选择
一步贪心	基线	对全部合法着法各走一步做静态评估，选最好的一个，不向前搜索
α-β（完整）	主方法	2.3 节的全部增强：迭代加深、PVS、置换表、杀手与历史启发、可分离排序、束宽 8
α-β（Texel 权重）	学习	同完整 α-β，评估权重换为 Texel 调参结果（2.5.2 节）
MCTS（纯随机）	基线	UCT，随机顺序全部展开，随机模拟到终局，不使用分隔判定与填格策略
MCTS（UCT+评估截断）	变体	UCT，廉价分数排序，渐进展开，随机走 4 步后评估截断
MCTS（PUCT+可分离先验）	主方法	PUCT，可分离评估排序与先验（候选上限 30，温度 4），渐进展开，评估截断
MCTS（学习先验）	学习	同上，先验与排序换为学习得到的线性策略（2.5.1 节）

5.1.3 开局、配对设计与参数试调

为消除先手优势和确定性带来的偏差（1.3.5 节），所有对局都从 16 个均衡开局之一开始。开局由程序生成（4.7.2 节）：白、黑各走一步，每一手从静态评估排序的前 30 名中随机选取，再用 1 秒的 α-β 搜索过滤掉评估绝对值不小于 4 格的开局。每个开局下两局，双方交换先后手，因此每组对阵共 32 局。图 5-1 展示了这 16 个开局，标题中的数字是 1 秒 α-β 对开局局面（轮白方走）的评估。可以看到，多数开局的评估略偏向白方，这与亚马逊棋先手略占优势的一般认识一致；交换先后手的配对设计使这一偏差在每组对阵中相互抵消。

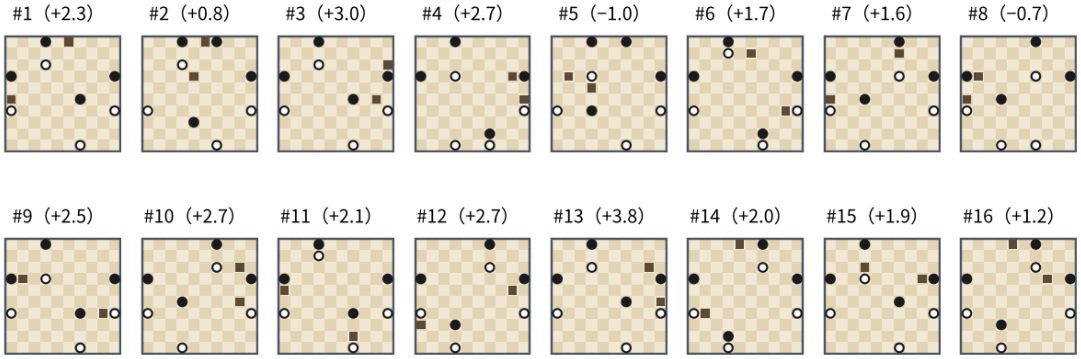


图 5-1 实验使用的 16 个均衡开局

注：程序生成（见 4.7.2 节）。每个开局两手，白先，下一手由白方走；空心圆为白后，实心圆为黑后，方块为箭。标题中的数字为 1 秒 α-β 的评估（白方视角，单位：格）。

程序另外生成了 16 个开局（第 17—32 号），**只用于参数试调，从不用于正式实验**。试调的内容与结果如下（均为 200 ms/步、每组 32 局或更少）：

- 着法排序：α-β 用廉价分数排序时对一步贪心只赢 7/16，改用可分离评估排序后赢 15/16，这促成了可分离评估的设计（3.2.3 节）；
- PUCT 先验的温度：$\tau=4$ 与 $\tau=16$ 对 $\tau=1$ 都是 22/32，$\tau=16$ 对 $\tau=4$ 为 11/32，因此取 $\tau=4$；
- PUCT 的探索系数：$c_{\text{puct}}=3$ 对 $c_{\text{puct}}=1.5$ 为 10/32，因此保留 1.5。

需要说明，上述试调是在修复分隔判定（5.10.1 节）之前的第一版程序上进行的，修复后没有重新试调。由于试调只涉及少量参数、且与正式实验的开局严格分开，它不会造成“在测试集上调参”的问题，但所选参数对修复后的程序未必最优——5.5 节的消融实验恰好提供了这些参数的事后检验。

5.1.4 实验清单

主实验共 2144 局，分为五组（表 5-2）。此外还有 400 局自对弈（用于学习与测试局面抽取）、384 局重复实验（5.6.3 节），以及搜索效率、决策质量与随机性实验。

表 5-2　主实验清单

组别	内容	对阵数	局数	每步时间	章节
T	8 个智能体单循环赛	28	896	200 ms	5.3
A	α-β 的 11 个变体对完整 α-β	11	352	200 ms	5.4
M	MCTS 的 15 个变体对 MCTS-PUCT	15	480	200 ms	5.5
S	不同思考时间的 α-β、MCTS-PUCT 对自身 200 ms	8	256	50—800 ms	5.6
X	相同思考时间下 α-β 对 MCTS-PUCT	5	160	50—800 ms	5.6
合计		67	2144		

5.1.5　评价指标

主要指标是得分率（亚马逊棋没有和棋，得分率即胜率）及换算的 Elo 差（式 2-18）。对每组对阵报告 Wilson 95% 区间、按开局配对的自助 95% 区间与配对符号检验的 p 值（2.7.2 节）。循环赛用 Bradley–Terry 模型拟合 Elo，以“一步贪心”为 0，区间由 300 次分层自助重采样得到。需要强调的是，**每组 32 局的样本量只能可靠地分辨约 ±150 Elo 以上的差异**（32 局的 Wilson 区间半宽约为 17 个百分点）；对于更小的差异，本章只报告“没有观察到显著差别”，不下“相同”或“更强”的结论。

5.2　游戏界面与决策过程展示

5.2.1　界面总览

图 5-2 是游戏打开时的界面。左侧是棋盘与图层开关，右侧上方是对局设置（模式、双方 AI、思考时间、开局、按钮），下方是五个标签页（搜索过程、领地、局势曲线、棋谱、算法说明）。棋盘下方有一行操作说明，提示玩家用三次点击完成一步。界面在桌面浏览器与手机竖屏下都能正常使用（图 5-13）。

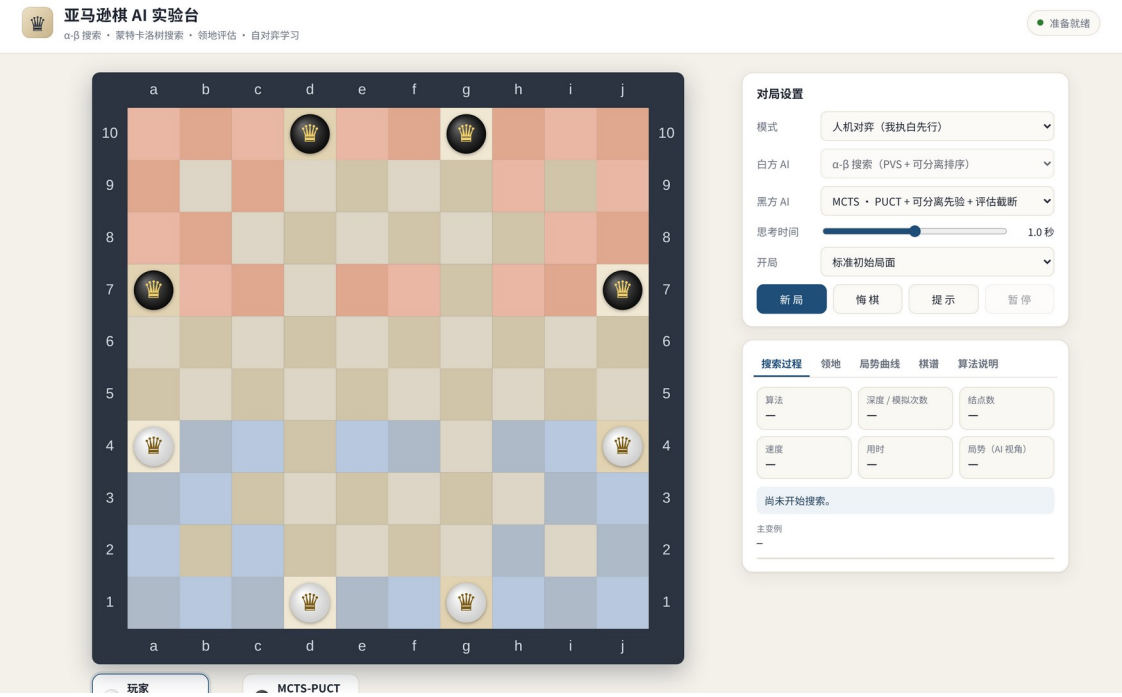


图 5-2　游戏初始界面

注：由 Playwright 在 Chromium 中以 file:// 方式打开 amazons.html 自动截取。

5.2.2　AI 决策过程的可视化

图 5-3 与图 5-4 是 AI 对 AI 演示中的两个瞬间（白方完整 α-β，黑方 MCTS-PUCT，各 1 秒）。棋盘上的蓝色/红色底纹是双方的“领地”（后距离更近的空格），它直接对应评估函数中的 t_1 项；折线箭头是根结点的候选着法（出发格 → 目的格 → 射箭格），线越粗表示访问次数越多（MCTS）或排名越靠前（α-β）；虚线箭头与序号是搜索给出的主变例。右侧“搜索过程”标签页列出算法、完成深度或模拟次数、结点数、速度、用时、评估与着法理由，下方表格对 MCTS 是每个候选的访问次数 N、平均胜率 Q 与先验 P，对 α-β 是迭代加深每一层的最佳着法、评估、累计结点数与用时。



图 5-3　MCTS 的决策过程展示

注：黑方（MCTS-PUCT）刚走完第 10 手。折线箭头为根结点访问次数最多的候选，右侧表格列出访问次数 N、胜率 Q 与先验 P。



图 5-4　α-β 的决策过程展示

注：白方（完整 α-β）刚走完第 11 手。折线箭头为根结点各候选，虚线与序号为主变例；右侧表格为迭代加深每一层的结果。

“领地”标签页（图 5-5）给出双方的领地格数、争夺格数、阶段系数 w 与是否已分隔；打开“距离数字”图层（图 5-6）后，每个空格上显示“白方后距离 / 黑方后距离”，读者可以逐格核对领地的划分，这是理解评估函数最直接的方式。



图 5-5　领地标签页

注：同一局面；右侧显示领地格数、争夺格数与阶段系数。



图 5-6　距离数字图层

注：每个空格上的两个数字分别为白方与黑方的后距离；底纹为领地归属。

5.2.3 人机对弈与提示

人类走子分三步（图 5-7）：点击己方的后后，界面用绿色圆点标出它能到达的全部格子；点击目的地后，再用红色圆点标出能射箭的全部格子。这种逐步高亮的交互把两千多个着法拆成了三个小选择，也让初学者直观地理解“后的走法”。“提示”按钮让 AI 为当前行棋方搜索一次，并把推荐着法、候选与主变例画在棋盘上（图 5-8）。

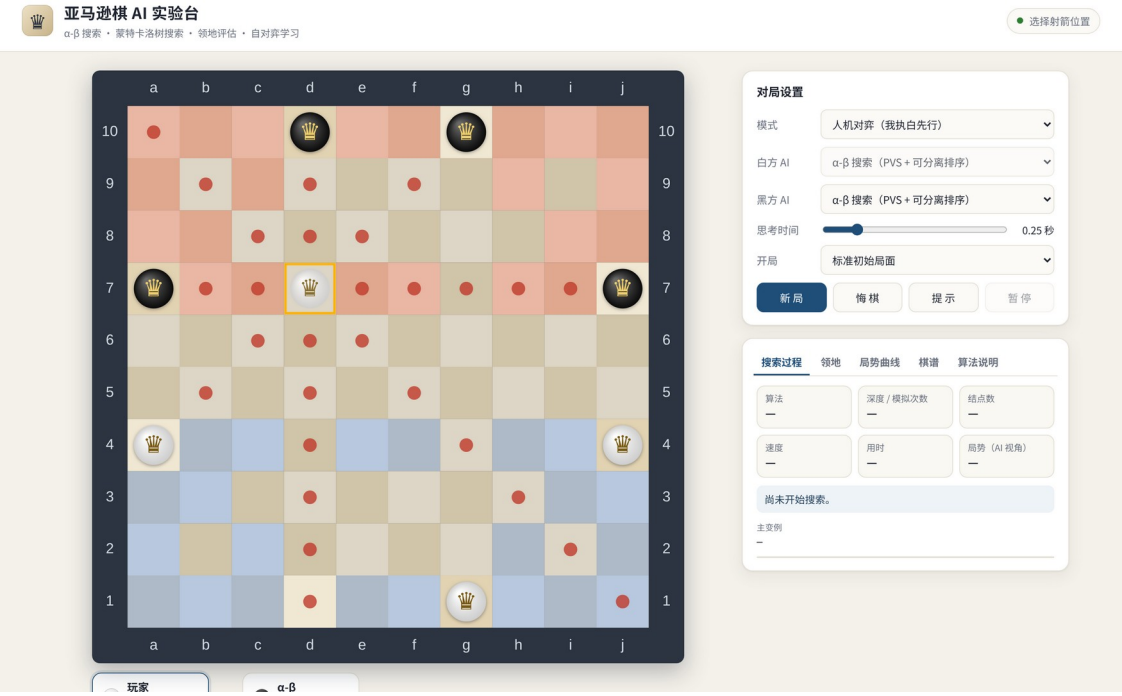


图 5-7 人类走子：选择目的地之后显示可射箭位置

注：玩家执白，已选择 d1 的后并点击了 d7，红色圆点为可射箭的格子。



图 5-8　“提示”功能：AI 为玩家推荐着法

注：推荐着法、候选与主变例同时显示。

5.2.4　局势曲线、棋谱与结果

对局结束时弹出结果对话框（图 5-9），说明哪一方无子可动。“局势曲线”标签页画出每一手后白方的估计胜率（图 5-10），α-β 的评估值按式 2-13 换算；“棋谱”标签页可以复制、导出（JSON）与导入对局记录（图 5-11）；“算法说明”标签页用简短的文字解释当前 AI 的原理（图 5-12）。

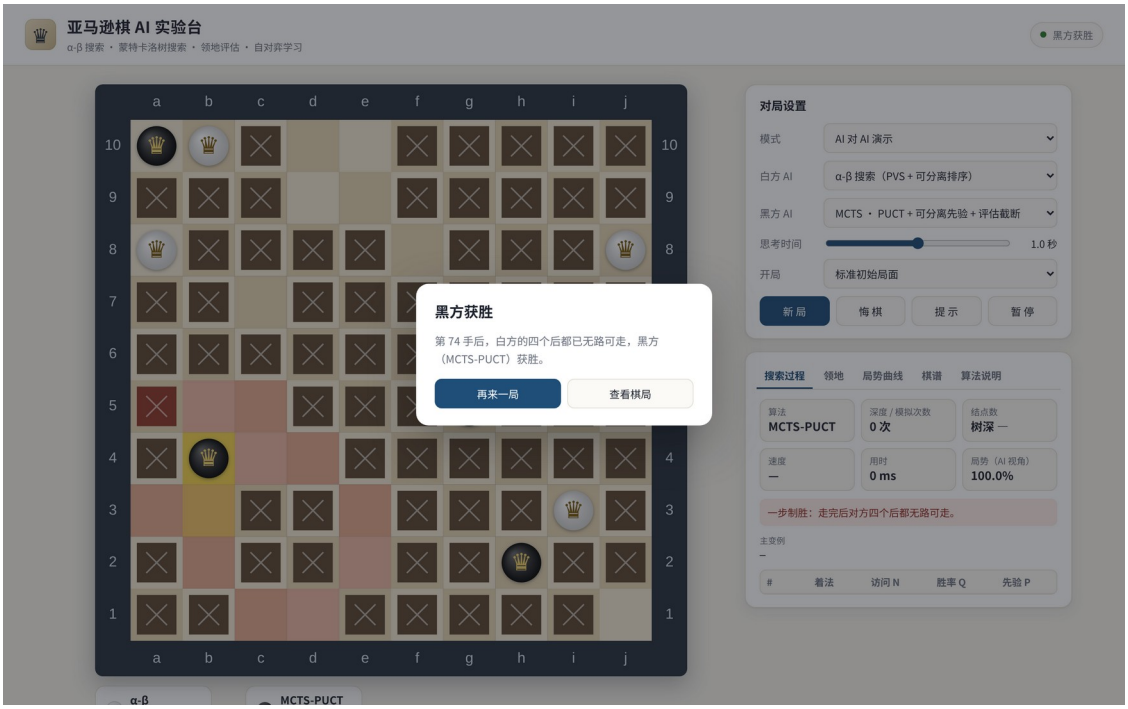


图 5-9　对局结束时的结果对话框

注：AI 对 AI 演示：白方 α-β，黑方 MCTS-PUCT，各 1 秒；本局 74 手，黑方获胜。



图 5-10　局势曲线

注：截取自右侧面板，同一局对局；纵轴为白方估计胜率，横轴为手数。前 40 手的锯齿来自双方 AI 对同一局势的估计不同（奇数手为 α-β 的估计，偶数手为 MCTS 的估计）。



图 5-11　棋谱标签页

注：截取自右侧面板。支持复制、导出（JSON）与导入。



图 5-12　算法说明标签页

注：截取自右侧面板（只显示前半部分）。



图 5-13 手机竖屏布局

注：视口宽度 390 像素，棋盘与面板上下排列。

5.2.5 稳定性测试

表 5-3 是界面自动化测试（4.8.2 节）的结果。27 个用例全部通过，总用时约 4.5 分钟。用例覆盖了三次点击走子、非法点击、AI 思考期间的输入屏蔽、悔棋与快捷键、提示、图层切换、棋谱的导入导出与错误提示、暂停与继续、对局中途切换模式，以及 8 个 AI 各自在浏览器的 Web Worker 中执白对一步贪心完成一整局对弈（每步思考 0.1 秒，30—91 手）。全程控制台没有出现任何错误。手机竖屏（390×844）下页面没有横向滚动，棋盘宽 358 像素。

表 5-3 界面测试结果

编号	测试项	结果	观测
U01	页面加载与初始状态	通过	初始局面 8 个后，8 个可选 AI，画布宽 760px，加载 1658 ms

编号	测试项	结果	观测
U02	三次点击完成一步，AI 随后应着	通过	白 d1-d7/g7 → AI 应 a7-e3/c1，用时 408 ms（含 250 ms 最短展示）
U03	非法点击被忽略或取消选择	通过	点空格→未选中；点黑后 e3→未选中；a4 到 j10（不可达）→取消；手数保持 2
U04	AI 思考期间点击棋盘无效	通过	思考中=true，期间点击未产生选择，AI 落子后 4 手
U05	搜索面板显示决策信息	通过	算法=MCTS-PUCT，深度/模拟=2.4k 次，结点/树深=树深 5，用时=2006 ms，候选表 12 行
U06	悔棋退回到人类行棋	通过	手数 4 → 2，轮到白（人）
U07	Ctrl+Z 快捷键悔棋	通过	手数 2 → 0
U08	提示功能只推荐不落子	通过	状态栏「提示：d1-d7/g7」，手数保持 0
U09	图层开关切换无错误	通过	5 个图层各切换 2 次
U10	导入完整棋谱并判定胜负	通过	导入 70 手随机对局，弹窗「黑方获胜」
U11	非法棋谱给出提示	通过	状态栏「棋谱格式有误：illegal move d1-d2/d3」，原棋局保持 70 手
U12	导出棋谱为 JSON	通过	文件 amazons_1794043912988.json，70 手，结果 black
U13	人执黑时 AI 先走	通过	AI 执白首着 d1-d7/g7
U14	随机均衡开局	通过	开局 d1-d7/f9 g10-g5/g2
U15	领地面板统计	通过	显示双方领地格数与 8 个后的机动性：白方领地（后距离更近） 26 格 黑方领地 26 格 均势 / 无人可达 38 / 0 剩余空格 90 阶段系数 p 1…
U16	暂停与继续	通过	暂停时 3 → 1.5 s 后 3；继续后 5
U17	对局中途切换模式与新局	通过	搜索中切换到双人模式并新局：手数 0，搜索已取消=true
U18	完整对局：α-β 搜索（PVS + 可分离排序）	通过	83 手，白胜，38.0 s
U19	完整对局：MCTS · PUCT + 可分离先验 + 评估截断	通过	91 手，白胜，41.6 s
U20	完整对局：MCTS · 学习先验	通过	89 手，白胜，40.7 s
U21	完整对局：α-β · Texel 学习权重	通过	72 手，黑胜，33.0 s
U22	完整对局：MCTS · UCT + 评估截断	通过	71 手，白胜，32.6 s
U23	完整对局：MCTS · 纯随机模拟	通过	50 手，黑胜，23.1 s
U24	完整对局：贪心 · 一步评估	通过	73 手，白胜，33.5 s
U25	完整对局：随机走子	通过	30 手，黑胜，14.1 s
U26	手机竖屏布局（390×844）	通过	页面宽 390px（无横向滚动），棋盘宽 358px
U27	全程无控制台错误	通过	共 26 个用例，0 个错误，总用时 271 s

5.3 循环赛：各算法的总体棋力

5.3.1 Elo 排名

表 5-4 与图 5-14 是 8 个智能体单循环赛（T 组，896 局，200 ms/步）的 Bradley–Terry 拟合结果。排名可以分为四档：

- **第一档**：完整 α-β（+427）与 MCTS-PUCT（+362）。二者的自助区间有较大重叠，循环赛中直接对局 α-β 得分 69%（22 胜 10 负，表 5-5）；把同一对阵的全部 160 局（含重复实验）合在一起，α-β 得 60.0%，区间 [52.3%, 67.3%]（5.6.3 节）。
- **第二档**：学习先验 MCTS（+298）与 Texel 权重 α-β（+141）。两个“学习”版本都没有超过对应的手工版本，5.7 节专门讨论这一点。
- **第三档**：UCT+评估截断 MCTS（+28）与一步贪心（0）。二者没有显著差别。也就是说，在 200 ms 内，不使用先验的 MCTS 比“只看一步”强不了多少。
- **第四档**：纯随机 MCTS（−771）与随机（≤−1200）。纯随机 MCTS 对随机智能体全胜，但对其余所有智能体全负。

表 5-4 循环赛 Elo 排名（200 ms/步）

排名	智能体	Elo（相对一步贪心）	95% 自助区间	平均得分率	平均对局手数
1	α-β（完整）	+427	[+341, +546]	85.3%	70.3
2	MCTS（PUCT+可分离先验）	+362	[+275, +445]	78.6%	72.3
3	MCTS（学习先验）	+298	[+224, +386]	73.2%	72.2
4	α-β（Texel 权重）	+141	[+69, +208]	57.1%	73.0
5	MCTS（UCT+评估截断）	+28	[−48, +109]	46.9%	68.8
6	一步贪心	+0	[−0, +0]	44.6%	68.5
7	MCTS（纯随机）	−771	[−820, −717]	14.3%	59.3
8	随机	≤−1200	[≤−1200, ≤−1200]	0.0%	44.8

注：随机智能体全部告负，其等级分在弱先验下趋于负无穷，表中只记为“≤− 1200”。一步贪心为参照点，因此其区间退化为 0。“平均对局手数”包含 2 手开局。

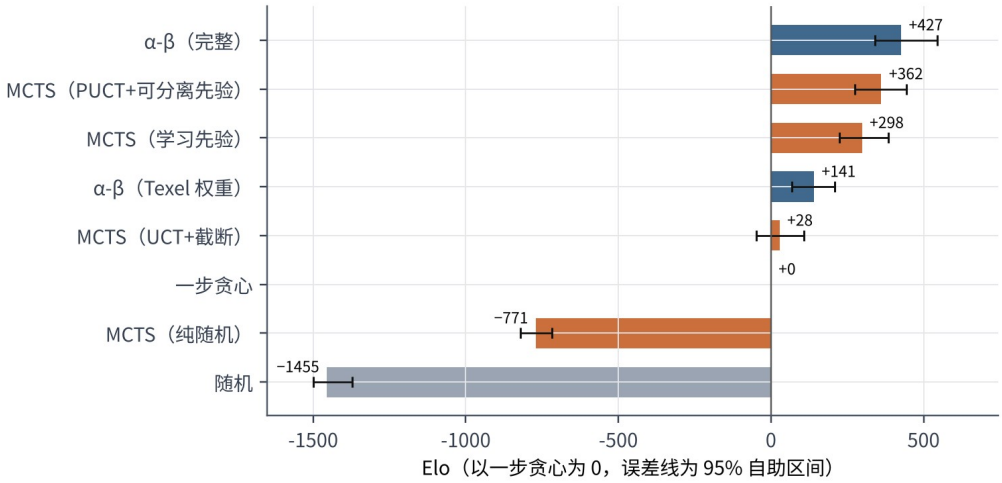


图 5-14 循环赛 Elo 等级分及 95% 自助区间

注：蓝色为 α-β 系列，橙色为 MCTS 系列，灰色为基线。

这一排名首先回答了 RQ1（3.6.2 节）：**在 200 ms/步、10×10 棋盘上，加入领域知识之后的 α-β 与 MCTS 处于同一水平**，α-β 略占上风。两种方法与一步贪心之间都有 350—430 Elo 的差距，换算成得分率约为 88%—92%。

两类“没有领域知识”的方法则非常弱。纯随机 MCTS 每步能做约一万二千次模拟（表 5-6），比 MCTS-PUCT 多十倍以上，结果却比一步贪心低 771 Elo。原因在于：亚马逊棋的随机对局与真实对局相差太远，随机对局的胜负几乎不包含局面信息。Lorentz[24]在把 MCTS 引入亚马逊棋时就指出了这一点，并因此采用评估截断；此后亚马逊棋上的 MCTS 研究也大多围绕模拟与选择策略的改进展开[48,57,58]。

5.3.2 交叉胜率

表 5-5 与图 5-15 给出每一对智能体之间的得分率。整张表几乎是“传递”的：按 Elo 排序后，绝大多数格子在对角线以上大于 50%、以下小于 50%。

只有两处与排名不完全一致：

- Texel 权重 α-β 对 UCT+截断 MCTS 只有 50%，对一步贪心只有 59%，明显低于由 Elo 差推算的约 67%—69%；
- 一步贪心对 UCT+截断 MCTS 为 53%，与 Elo 顺序相反，但差别在统计误差之内。

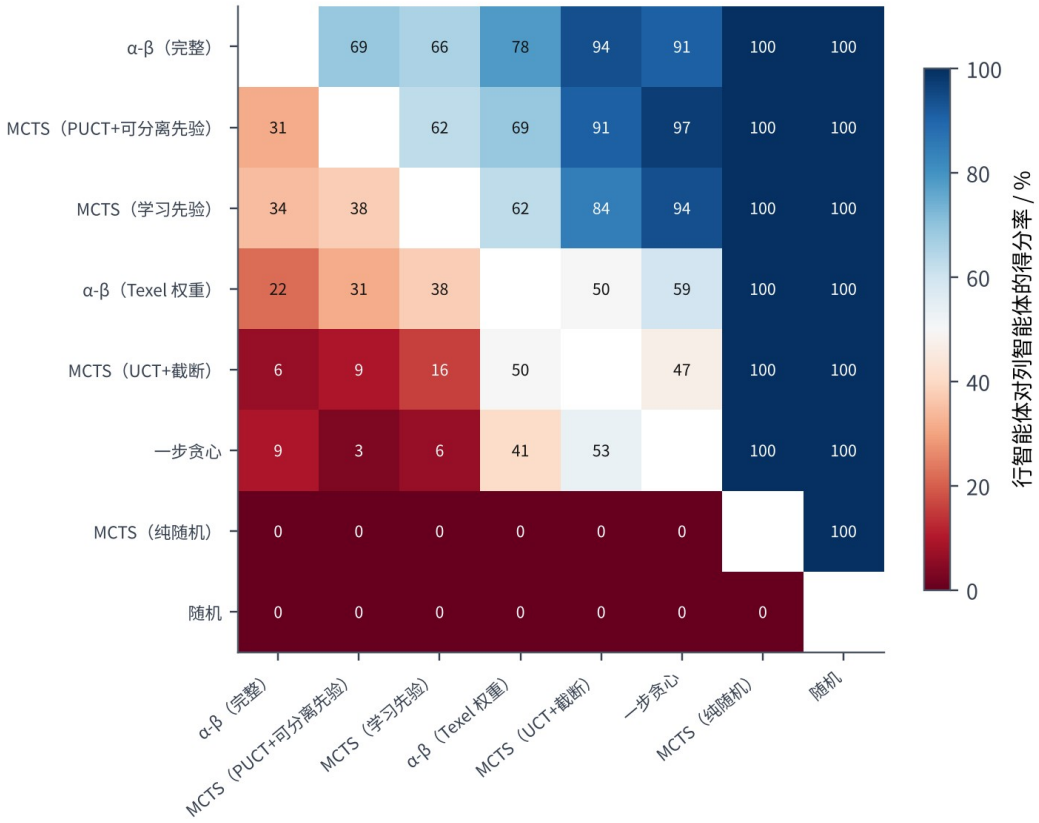


图 5-15　循环赛交叉得分率

注：每格为行智能体对列智能体的得分率（%），按 Elo 排序。

表 5-5　循环赛交叉得分率（%，行对列）

行 列	1	2	3	4	5	6	7	8
1. α-β（完整）	—	69	66	78	94	91	100	100
2. MCTS（PUCT+可分离先验）	31	—	62	69	91	97	100	100
3. MCTS（学习先验）	34	38	—	62	84	94	100	100
4. α-β（Texel 权重）	22	31	38	—	50	59	100	100
5. MCTS（UCT+评估截断）	6	9	16	50	—	47	100	100
6. 一步贪心	9	3	6	41	53	—	100	100
7. MCTS（纯随机）	0	0	0	0	0	0	—	100
8. 随机	0	0	0	0	0	0	0	—

注：列号与行号对应，按 Elo 从高到低排序。每格 32 局。

主实验中共有 45 局“冷门”，即强智能体（两种 α-β、MCTS-PUCT、学习先验 MCTS）输给弱智能体（一步贪心、UCT+截断 MCTS）。其中 29 局是 Texel 权重 α-β 输掉的：13 局负于一步贪心，16 局负于 UCT+截断 MCTS。完整 α-β 只输了 5 局，MCTS-PUCT 只输了 4 局。5.10.3 节复盘其中一局。

5.3.3 对局行为统计

表 5-6 统计每个智能体在“常规搜索”着法上的用时与搜索量，并给出“填格着法”的比例。

表 5-6 各智能体每一步的搜索统计（循环赛，只统计常规搜索着法）

智能体（200 ms/步）	搜索着法数	平均用时/ms	最长用时/ms	平均完成深度	结点数中位数	模拟次数中位数	平均最大树深	填格着法比例
α-β（完整）	22891	154.9	209	4.70	3224（12543）	—	—	19.0%
α-β（Texel 权重）	6714	153.6	269	4.69	3017（11248）	—	—	15.6%
MCTS（纯随机）	6375	200.1	214	—	—	11696（43360）	2.8	0.0%
MCTS（UCT+评估截断）	6172	200.1	210	—	—	3712（15725）	4.2	17.2%
MCTS（PUCT+可分离先验）	28205	202.0	233	—	—	1040（7632）	4.9	15.2%
MCTS（学习先验）	7652	202.4	226	—	—	800（5328）	4.8	16.1%

注：括号内为第 90 百分位数。模拟次数与结点数的分布高度右偏：接近分隔时着法很少，每次模拟也很短，次数会增加一个数量级，因此报告中位数而不是平均数。“填格着法比例”指分隔之后由填格策略（2.6 节）直接给出的着法占该智能体全部着法的比例。

表中有几点值得说明。

第一，**两类方法的时间使用方式不同**。α-β 平均只用掉 155 ms（预算的 77%），因为它在用掉一半预算后不再开始新一轮迭代（4.3.4 节）；MCTS 则用满预算。除 Texel 权重 α-β 有一步用时 269 ms 外，各智能体的最长用时都不超过预算的 1.17 倍；那一步超时的原因没有查明，可能与并行进程的资源竞争或 JavaScript 的垃圾回收停顿有关。在全部 2144 局、121115 步常规搜索中，最长的一步为 819.7 ms（预算 800 ms），第 99.9 百分位为 807 ms。除上面那一步（1.35 倍）之外，全部着法都满足 1.4 节“用时不超过设定时间 1.2 倍”的要求。

第二，**α-β 在 200 ms 内平均完成 4.7 层**。亚马逊棋开局分支因子超过两千，如果不剪枝，4 层需要 $2000^4 \approx 1.6 \times 10^{13}$ 个结点；束宽 8 把它压缩到几千个结点。代价是：每一层只看可分离评估排名前 8 的着法，搜索的“深”是用“窄”换来的。5.4 节的消融实验表明，这个束宽在 200 ms 下偏窄。

第三，**MCTS-PUCT 的模拟次数中位数约为一千次**，平均树深 4.9 层。相比之下，纯随机 MCTS 的模拟次数多十倍，但树深只有 2.8 层：它对根结点的两千多个着法逐一展开，大部分时间都花在了“每个着法试一次”上。

5.3.4 先后手、对局长度与分隔

图 5-16 左侧是全部 2144 局主实验对局的总手数分布。循环赛（蓝色）呈双峰：含随机智能体的对局在 30—50 手结束，较强智能体之间的对局集中在 75—92 手。消融与缩放实验的双方实力接近，对局长度集中在 80—90 手。

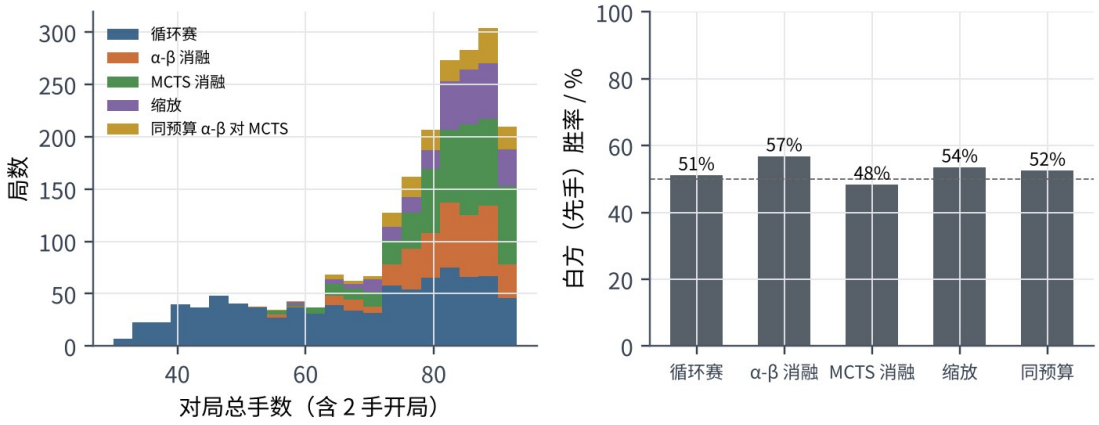


图 5-16 对局长度分布（左）与各实验组的白方胜率（右）

注：数据：主实验 2144 局。

在不含随机智能体与纯随机 MCTS 的 1408 局中，用修复后的判定统计双方首次完全分隔的手数：

- 平均在第 67 手分隔，中位数为第 65 手；10%—90% 分位为第 53—86 手；
- 分隔之后的填格阶段平均占全局的 17%。

这一时间略晚于 α-β 自对弈中的第 61 手（1.2.3 节），因为强弱悬殊的对局中，弱方常常更早被围死，而棋力接近的双方会争夺更久。

右图是各组的白方（先手）胜率。全部对局中白方胜 1111 局（51.8%）；在双方为同一算法不同变体的 A、M、S 组中，白方胜 569/1088（52.3%）。按二项检验，二者与 50% 的差别都不显著（$p=0.10$ 与 $p=0.14$）。单看 A 组，白方胜率为 56.8%（$p=0.012$），偏高；但 A 组每个开局都交换先后手，先手优势对“变体”与“完整 α-β”的影响相同，不会改变变体的得分率。也就是说，经过均衡开局筛选后，先手优势很小。即便存在这样的小优势，交换先后手的配对设计（5.1.3 节）也已经把它从每组对阵的结果中抵消。

5.4 α-β 搜索的消融实验

5.4.1 棋力

表 5-7 与图 5-17 是 11 个 α-β 变体各自与完整 α-β 对弈 32 局的结果（A 组）。每个变体只改动一处，其余设置与完整 α-β 相同。

表 5-7 α-β 消融实验结果（200 ms/步，对完整 α-β）

变体（对 完整 α-β）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
无置换表	13–19	40.6%	[25.5, 57.7]	[21.9, 59.4]	−66	0.508	81.4
无杀手/历史启发	16–16	50.0%	[33.6, 66.4]	[34.4, 65.6]	−0	1.000	83.8
无 PVS（普通 α-β 窗口）	17–15	53.1%	[36.4, 69.1]	[40.6, 65.6]	+22	1.000	81.6
朴素 α-β（以上增强全部关闭）	16–16	50.0%	[33.6, 66.4]	[34.4, 65.6]	−0	1.000	81.6
内部结点用廉价分数排序	4–28	12.5%	[5.0, 28.1]	[3.1, 25.0]	−338	0.000	80.7
叶前结点也用可分离评估排序	13–19	40.6%	[25.5, 57.7]	[21.9, 59.4]	−66	0.508	82.8
束宽加倍（16/48，根 120/60）	23–9	71.9%	[54.6, 84.4]	[56.2, 87.5]	+163	0.039	81.1
束宽减半（4/12，根 30/15）	7–25	21.9%	[11.0, 38.8]	[6.2, 40.6]	−221	0.022	83.2
只用 t1 领地项的评估	3–29	9.4%	[3.2, 24.2]	[0.0, 18.8]	−394	0.000	75.2
评估去掉移动性	17–15	53.1%	[36.4, 69.1]	[34.4, 71.9]	+22	1.000	82.8
评估去掉阶段插值（两组权重取平均）	11–21	34.4%	[20.4, 51.7]	[18.8, 50.0]	−112	0.180	83.7

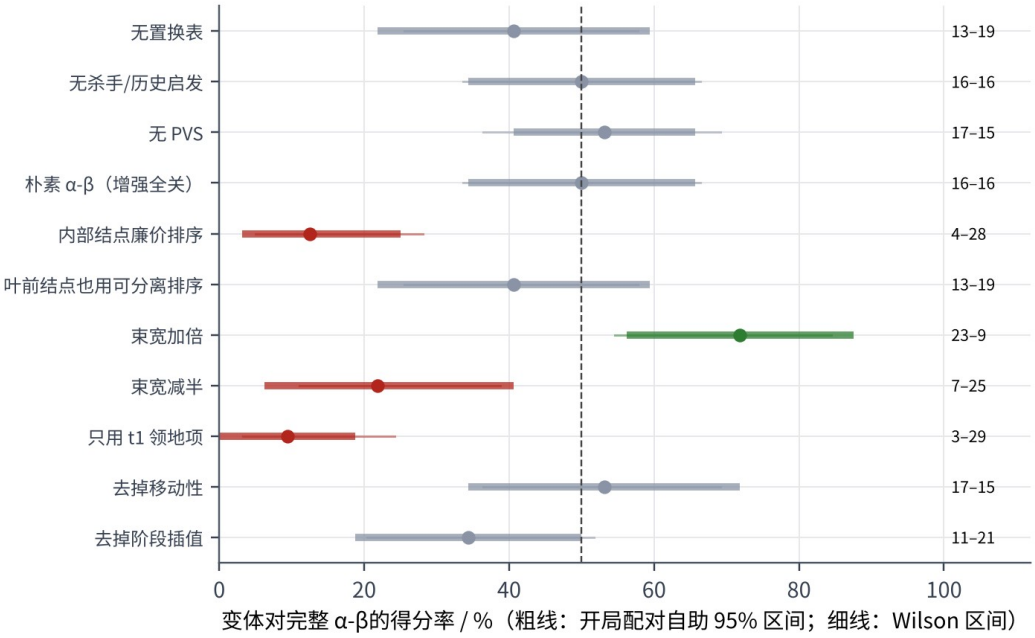


图 5-17 α-β 消融实验的得分率与置信区间

注：绿色/红色表示自助区间完全位于 50% 之上/之下；灰色表示不显著。

结果可以分成三组。

影响最大的是着法排序与评估函数。 内部结点改用廉价分数排序后，变体只得 12.5%（−338 Elo，$p<0.001$）。评估只保留 t_1 领地项时只得 9.4%（−394 Elo）。前者说明，在束宽剪枝之下，“排在前 8 名的是哪些着法”直接决定了搜索能看到什么；廉价分数与真实评估的相关性不高（5.9.1 节），好的着法常被排在束宽之外。后者说明，亚马逊棋的评估不能只数领地：开局与中局里，“距离近”并不等于“能占住”，c_1、c_2 与移动性项承担了相当一部分判断。

束宽的影响显著，而且方向出人意料。 束宽加倍（内部 16、叶前 48、根 120/60）得 71.9%（+163 Elo，$p=0.039$），束宽减半得 21.9%（$p=0.022$）。也就是说，在 200 ms 下，默认的束宽 8 偏窄：宁可少搜一层，也应该在每层多看几个着法。束宽 8 是在第一版程序上凭经验定下的，修复分隔判定后没有重新调整（5.1.3 节）。这个结果提示，默认设置还有改进空间。但本文没有据此修改默认值，否则所有主实验都需要重跑，而且“在测试开局上调参”会让后续结论失去独立性。

经典增强都没有显著影响。 去掉置换表、杀手/历史启发或 PVS 的变体得分在 40.6%—53.1% 之间；把它们全部关闭的“朴素 α-β”得分恰好 50%。评估中去掉移动性项（53.1%）或去掉阶段插值（34.4%，$p=0.18$）也都不显著。叶前结点改用可分离评估排序（40.6%）同样不显著。

5.4.2 为什么经典增强没有帮助

置换表、杀手启发与 PVS 在国际象棋程序中都能带来大幅度的效率提升[10,71,103]。它们在本文中没有体现出棋力差别，原因可能有三。

第一，**排序已经很好**。所有变体都用可分离评估排序，它对“最好的着法”已经给出了相当可靠的位置（5.9.1 节）。杀手与历史启发的作用正是改进排序，因此可提升的空间很小。

第二，**树很小**。束宽 8 之下，200 ms 内的树只有几千个结点（表 5-6）。置换表能节省的重复结点数量有限。5.9.2 节在固定深度 3 下测量了这些增强对结点数的影响：它们只减少了 0—13% 的结点，这样的效率差别换不来多一层的深度。

第三，**样本量有限**。32 局只能分辨约 ±150 Elo 以上的差别。几十 Elo 量级的真实效果，在本实验中会被判为“不显著”。因此，正确的表述是“在本实验的样本量下，没有观察到这些增强带来的棋力差别”，而不是“这些增强无效”。

5.5 MCTS 的消融实验

5.5.1 棋力

表 5-8 与图 5-18 是 15 个 MCTS 变体各自与 MCTS-PUCT 对弈 32 局的结果（M 组）。

表 5-8 MCTS 消融实验结果（200 ms/步，对 MCTS-PUCT）

变体（对 MCTS-PUCT）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
UCT 选择（无先验，同样排序）	3–29	9.4%	[3.2, 24.2]	[0.0, 18.8]	−394	0.000	79.4
廉价分数排序与先验（上限 60）	13–19	40.6%	[25.5, 57.7]	[28.1, 53.1]	−66	0.375	84.5
随机顺序、全部展开、UCT	2–30	6.2%	[1.7, 20.1]	[0.0, 15.6]	−470	0.000	74.9
无渐进展开	18–14	56.2%	[39.3, 71.8]	[40.6, 68.8]	+44	0.688	84.2
候选上限 10	12–20	37.5%	[22.9, 54.7]	[18.8, 59.4]	−89	0.388	82.3
候选上限 60	18–14	56.2%	[39.3, 71.8]	[34.4, 78.1]	+44	0.791	83.6
无模拟（直接评估）	16–16	50.0%	[33.6, 66.4]	[34.4, 65.6]	−0	1.000	84.1

变体（对 MCTS-PUCT）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
模拟截断 2 步	23–9	71.9%	[54.6, 84.4]	[53.1, 87.5]	+163	0.065	85.2
模拟截断 8 步	7–25	21.9%	[11.0, 38.8]	[9.4, 34.4]	−221	0.004	83.8
随机模拟到终局	1–31	3.1%	[0.6, 15.7]	[0.0, 9.4]	−597	0.000	75.0
c_puct = 0.5	17–15	53.1%	[36.4, 69.1]	[34.4, 71.9]	+22	1.000	84.1
c_puct = 4	7–25	21.9%	[11.0, 38.8]	[9.4, 37.5]	−221	0.012	79.8
先验温度 τ = 1	5–27	15.6%	[6.9, 31.8]	[6.2, 28.1]	−293	0.001	81.0
先验温度 τ = 16	19–13	59.4%	[42.3, 74.5]	[43.8, 75.0]	+66	0.453	84.2
学习先验	21–11	65.6%	[48.3, 79.6]	[46.9, 81.2]	+112	0.180	84.8

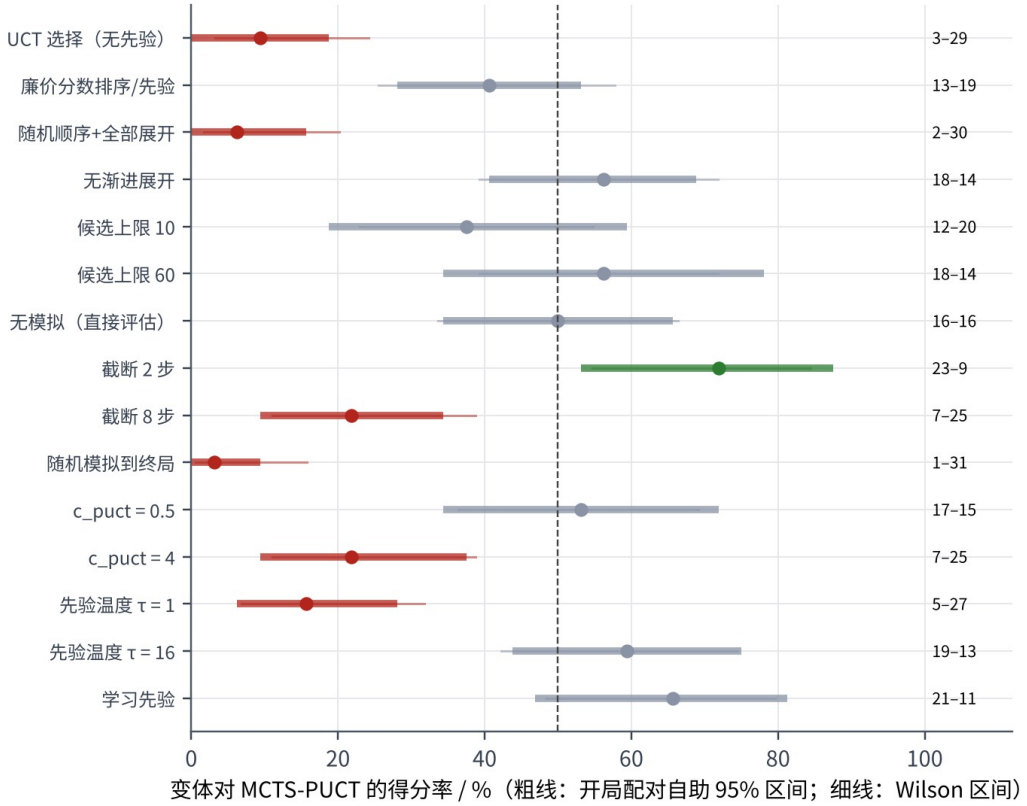


图 5-18 MCTS 消融实验的得分率与置信区间

注：颜色含义同 α-β 消融实验的图。

5.5.2 主要发现

先验是 MCTS-PUCT 最重要的组件。 保留同样的排序与渐进展开、但把 PUCT 换成 UCT 选择，变体只得 9.4%（−394 Elo）；连排序也去掉、随机顺序全部展开时

只得 6.2%。先验的形状也很重要：温度 $\tau=1$ 的先验过于尖锐，只得 15.6%（$p=0.001$）；$\tau=16$ 更平坦，得 59.4%，与 $\tau=4$ 没有显著差别。这与 5.7.2 节的离线结果一致：可分离评估的 softmax 在 $\tau=1$ 时的负对数似然（3.80）比均匀分布（3.55）还差，而学习得到的等效温度约为 7.8。也就是说，可分离评估的“名次”有用，但它的分数差距被过分放大了，必须把温度调高才能做先验。

模拟越短越好。 随机模拟到终局只得 3.1%。随机走 8 步再截断得 21.9%（$p=0.004$），2 步得 71.9%（$p=0.065$，接近显著），完全不模拟、直接评估叶结点得 50%。随机着法在亚马逊棋中几乎总是坏着，模拟每多走一步，叶结点评估中的噪声就多一分。本文默认的 4 步参照了文献中的做法[24]，在本文的评估函数下，它看来偏长。

探索系数不宜过大。 $c_{\text{puct}}=4$ 只得 21.9%（$p=0.012$）；$c_{\text{puct}}=0.5$ 得 53.1%，与 1.5 无显著差别。先验已经承担了主要的“探索”功能，过大的探索项会把模拟浪费在先验很低的着法上。

候选集合与渐进展开的影响都不显著。 候选上限 10、60 与无渐进展开的得分在 37.5%—56.2% 之间，置信区间都跨过 50%。廉价分数排序与先验（候选上限 60）得 40.6%，也不显著。最后一点与 α-β 的情况不同：MCTS 会在模拟中反复修正着法的价值，对初始排序的依赖比固定束宽的 α-β 小。

学习先验的结果前后不一致。 M 组中，学习先验 MCTS 对 MCTS-PUCT 得 65.6%（$p=0.18$）。但在循环赛中，同一对阵 MCTS-PUCT 得 62%，即学习先验只得 37.5%。两组各 32 局、开局相同，只有随机种子不同，结果却分别落在 50% 的两侧。5.6.3 节的重复实验表明，合计 160 局时学习先验得 46.2%，与 MCTS-PUCT 没有显著差别。

5.5.3 两类方法的对照

把两组消融放在一起看，可以总结出亚马逊棋中“知识”与“搜索”的关系（RQ2、RQ3）：

1. **两种方法都依赖同一份领域知识。** α-β 依赖可分离评估排序来决定束宽内的着法，MCTS 依赖它来给出先验与展开顺序。去掉这份知识后，两种方法都会损失 300 Elo 以上。
2. **搜索框架上的“经典增强”贡献较小。** α-β 的置换表、杀手、PVS，MCTS 的渐进展开与候选上限，都没有显著影响。

3. **有几处参数明显不是最优。** α-β 的束宽偏窄，MCTS 的模拟偏长。这两点都在消融中被发现，但本文没有据此修改主实验，5.11 节会讨论它们对结论的影响。

5.6 算力缩放

5.6.1 对自身的缩放

S 组让 50、100、400、800 ms 的 α-β 与 MCTS-PUCT 各自与 200 ms 的自身对弈（表 5-9、5-10，图 5-19 左）。

表 5-9 α-β 的算力缩放（对 α-β 200 ms）

变体（对 α-β 200 ms）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
α-β 50 ms	5–27	15.6%	[6.9, 31.8]	[6.2, 28.1]	−293	0.001	81.2
α-β 100 ms	16–16	50.0%	[33.6, 66.4]	[31.2, 68.8]	−0	1.000	84.7
α-β 400 ms	18–14	56.2%	[39.3, 71.8]	[40.6, 71.9]	+44	0.688	86.1
α-β 800 ms	21–11	65.6%	[48.3, 79.6]	[53.0, 81.2]	+112	0.125	83.9

表 5-10 MCTS 的算力缩放（对 MCTS-PUCT 200 ms）

变体（对 MCTS 200 ms）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
MCTS 50 ms	3–29	9.4%	[3.2, 24.2]	[0.0, 18.8]	−394	0.000	80.0
MCTS 100 ms	7–25	21.9%	[11.0, 38.8]	[9.4, 37.5]	−221	0.012	82.1
MCTS 400 ms	27–5	84.4%	[68.2, 93.1]	[68.8, 96.9]	+293	0.003	83.8
MCTS 800 ms	26–6	81.2%	[64.7, 91.1]	[65.6, 93.8]	+255	0.006	79.5

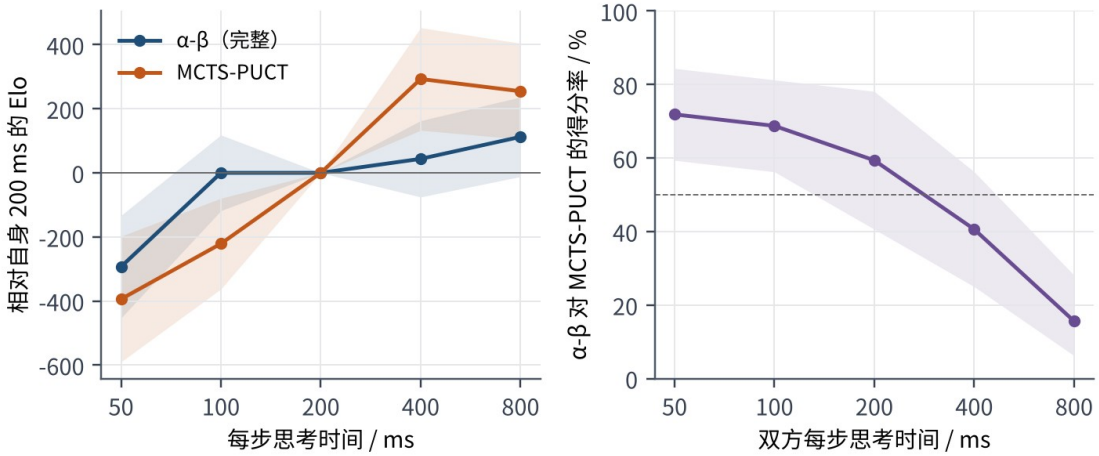


图 5-19 算力缩放

注：左：相对自身 200 ms 的 Elo（阴影为 Wilson 区间换算）；右：相同思考时间下 α-β 对 MCTS-PUCT 的得分率（阴影为开局配对自助区间）。

两种方法的缩放曲线形状很不相同：

- **α-β 在 100 ms 以上几乎饱和。** 从 50 ms 到 100 ms 提升约 300 Elo；此后 100、400、800 ms 对 200 ms 的得分分别为 50%、56.2%、65.6%，都不显著。图 5-20 显示，平均完成深度从 50 ms 的 3.4 层增加到 800 ms 的 6.6 层，深度在增加，棋力却没有相应提高。合理的解释是：束宽 8 把每层的着法限制得太死，更深的搜索只是在一棵“错过了好着法”的窄树里看得更远。这与 5.4 节“束宽加倍更强”的结果相互印证。
- **MCTS 的缩放陡峭且持续。** 从 50 ms 到 400 ms，MCTS 对 200 ms 自身的得分从 9.4% 上升到 84.4%（+293 Elo，$p=0.003$）；800 ms 为 81.2%，与 400 ms 相当。模拟次数中位数随预算近似线性增长（200 ms 约 1050 次，800 ms 约 4100 次）。

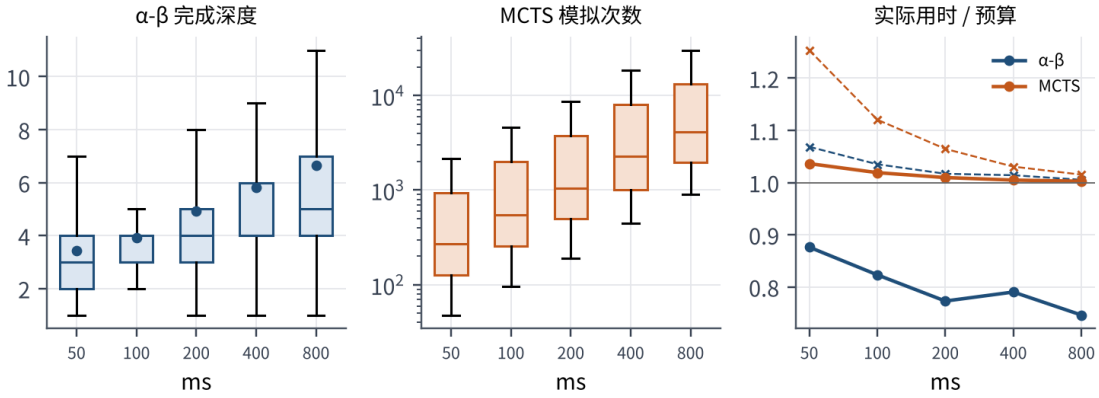


图 5-20 不同时间预算下的搜索量与时间控制

注：左：α-β 完成深度；中：MCTS 模拟次数（对数坐标）；右：实际用时与预算之比（实线为平均，虚线为第 99 百分位）。数据：S、X 两组对局。

5.6.2 相同预算下的 α-β 对 MCTS

X 组让 α-β 与 MCTS-PUCT 在相同的思考时间下对弈（表 5-11，图 5-19 右）。结果呈现出清楚的**交叉**：

- 50 ms 与 100 ms 时，α-β 分别得 71.9%（$p=0.016$）与 68.8%（$p=0.031$），显著占优；
- 200 ms 时 α-β 得 59.4%，不显著；
- 400 ms 时 α-β 得 40.6%，不显著；
- 800 ms 时 α-β 只得 15.6%（$p=0.001$），MCTS 显著占优。

表 5-11 相同思考时间下 α-β 对 MCTS-PUCT

变体（对 同预算 MCTS-PUCT）	胜–负	得分率	Wilson 95% 区间	开局配对自助 95% 区间	Elo 差	配对符号检验 p	平均对局手数
α-β 对 MCTS-PUCT，均 50 ms	23–9	71.9%	[54.6, 84.4]	[59.4, 84.4]	+163	0.016	79.0
α-β 对 MCTS-PUCT，均 100 ms	22–10	68.8%	[51.4, 82.0]	[56.2, 81.2]	+137	0.031	80.6
α-β 对 MCTS-PUCT，均 200 ms	19–13	59.4%	[42.3, 74.5]	[40.6, 78.1]	+66	0.549	82.7
α-β 对 MCTS-PUCT，均 400 ms	13–19	40.6%	[25.5, 57.7]	[25.0, 56.2]	−66	0.453	83.8
α-β 对 MCTS-PUCT，均 800 ms	5–27	15.6%	[6.9, 31.8]	[6.2, 28.1]	−293	0.001	83.1

交叉点大约在 200—400 ms 之间。这一结果回答了 RQ4，也说明 5.3 节“α-β 略强于 MCTS”的排名**只在 200 ms 这个预算下成立**。在网页中，玩家可以把 AI 的思考时间调到 1 秒以上；按本节的结果，在这样的时间下 MCTS-PUCT 很可能是更强的选择（本文没有在 1 秒以上做对局实验）。

需要强调，这一交叉与具体实现密切相关。α-β 早早饱和，在很大程度上是束宽 8 造成的；如果按 5.4 节的结果放宽束宽，交叉点很可能右移。因此，本文的结论是“**在本文的实现中**，α-β 在短时间内占优，MCTS 在长时间内占优”，而不是关于两类算法的一般性论断。文献中的亚马逊棋程序既有以 α-β/PVS 为主的，也有以 MCTS 为主或二者混合的[24,29,42,46]，这与本文的观察相符：两类方法的相对强弱取决于实现细节与时间预算。

在第一版程序（分隔判定有漏洞，5.10.1 节）的数据中，也观察到了同样的交叉。这说明，该现象并不依赖于那处漏洞。

5.6.3 重复实验：结论有多稳定

32 局的对阵结果波动有多大？为此，本文挑出四组关键对比，换用新的随机种子（同样的 16 个开局、交换先后手）各重复 3 次，每次 32 局，共 384 局（表 5-12）。表中同时列出主实验中对应对阵的结果，“合计”行把同一对比的全部批次合在一起。

表 5-12 重复实验结果（200 ms/步）

对比（A 对 B）	批次	局数	A 胜–负	A 得分率	Wilson 95% 区间	开局配对自助 95% 区间
α-β（完整）对 MCTS-PUCT	主实验 X 组	32	19–13	59.4%	[42.3, 74.5]	[40.6, 78.1]
	主实验 T 组	32	22–10	68.8%	[51.4, 82.0]	[50.0, 84.4]

对比（A 对 B）	批次	局数	A 胜–负	A 得分率	Wilson 95% 区间	开局配对自助 95% 区间
	重复 1	32	14–18	43.8%	[28.2, 60.7]	[25.0, 62.5]
	重复 2	32	21–11	65.6%	[48.3, 79.6]	[46.9, 81.2]
	重复 3	32	20–12	62.5%	[45.3, 77.1]	[46.9, 78.1]
	合计	160	96–64	60.0%	[52.3, 67.3]	[51.9, 68.1]
学习先验 MCTS 对 MCTS-PUCT	主实验 M 组	32	21–11	65.6%	[48.3, 79.6]	[50.0, 81.2]
	主实验 T 组	32	12–20	37.5%	[22.9, 54.7]	[18.8, 56.2]
	重复 1	32	15–17	46.9%	[30.9, 63.6]	[31.2, 62.5]
	重复 2	32	11–21	34.4%	[20.4, 51.7]	[18.8, 50.0]
	重复 3	32	15–17	46.9%	[30.9, 63.6]	[34.4, 59.4]
	合计	160	74–86	46.2%	[38.7, 54.0]	[38.8, 53.8]
Texel 权重 α-β 对 完整 α-β	主实验 T 组	32	7–25	21.9%	[11.0, 38.8]	[6.2, 40.6]
	重复 1	32	7–25	21.9%	[11.0, 38.8]	[9.4, 37.5]
	重复 2	32	6–26	18.8%	[8.9, 35.3]	[6.2, 34.4]
	重复 3	32	4–28	12.5%	[5.0, 28.1]	[3.1, 25.0]
	合计	128	24–104	18.8%	[12.9, 26.4]	[11.7, 26.6]
MCTS-PUCT 对 MCTS（UCT+截断）	主实验 T 组	32	29–3	90.6%	[75.8, 96.8]	[81.2, 100.0]
	重复 1	32	27–5	84.4%	[68.2, 93.1]	[71.9, 93.8]
	重复 2	32	28–4	87.5%	[71.9, 95.0]	[71.9, 100.0]
	重复 3	32	27–5	84.4%	[68.2, 93.1]	[71.9, 93.8]
	合计	128	111–17	86.7%	[79.8, 91.5]	[80.5, 92.2]

四组对比的结论稳定性差别很大。

α-β 对 MCTS-PUCT。 5 个批次的得分在 43.8%—68.8% 之间，有一个批次低于 50%。合计 160 局，α-β 得 60.0%，Wilson 区间 [52.3%, 67.3%]，开局配对自助区间 [51.9%, 68.1%]，都位于 50% 之上，约相当于 +70 Elo。也就是说，在 200 ms 下，α-β 确实略强于 MCTS-PUCT。但单看任何一个 32 局的批次，都不足以得出这一结论。

学习先验 MCTS 对 MCTS-PUCT。 5 个批次的得分在 34.4%—65.6% 之间。主实验 M 组的 65.6% 是唯一高于 50% 的批次，其余 4 个批次都在 50% 以下。合计 160 局，学习先验得 46.2%，区间 [38.7%, 54.0%] 包含 50%。因此，**没有证据表明学习先验优于可分离评估先验**；M 组的 65.6% 很可能只是一次偏高的波动。如果只做了 M 组那一次实验，本文会错误地报告“学习先验提升了约 110 Elo”。

Texel 权重 α-β 对完整 α-β。 4 个批次都在 12.5%—21.9% 之间，合计 128 局只得 18.8%（区间 [12.9%, 26.4%]），约 −250 Elo。这一结论非常稳定。

MCTS-PUCT 对 UCT+截断 MCTS。 4 个批次都在 84.4%—90.6% 之间，合计 86.7%，同样非常稳定。

这组实验说明了两件事。第一，**差距大的结论很稳定，差距小的结论则需要更多对局**：真实得分率在 60% 左右时，32 局的结果可以从 44% 波动到 69%。第二，**相同设置的两次 32 局实验，可以落在 50% 的两侧**（学习先验在 M 组与 T 组中的结果就是如此）。因此，本章对所有“只有一次 32 局”的中等差距结论（例如 5.4、5.5 节中 p 值在 0.01—0.2 之间的消融结果）都只作为初步证据看待。

5.7 学习组件的评估

5.7.1 训练数据

学习用的数据是 400 局 α-β 自对弈（200 ms/步，与主实验的完整 α-β 相同）。每局从随机的 2—4 手开局开始：每一手从可分离评估排名前 40 的着法中均匀随机选取，以增加局面的多样性。这批开局与主实验的 32 个开局独立生成。对局平均 83.7 手，平均在第 61 手分隔。

数据按**对局**划分：前 320 局为训练集，后 80 局为测试集。这样同一盘棋的局面不会同时出现在训练集与测试集中，避免了相邻局面高度相关造成的泄漏。

- **走子策略**：使用分隔之前、由 α-β 常规搜索给出的着法，共 24814 个样本。其中 92.4%（22940 个）落在可分离评估的前 40 名候选内，只保留这些样本（训练 18327 个，测试 4613 个）。
- **Texel 调参**：每局按固定间隔抽取局面，以最终胜负为标签，训练 20820 个，测试 5205 个。测试集中白方胜率为 51.7%。

5.7.2 走子策略

表 5-13 与图 5-21 比较学习得到的线性走子策略与几个基线。

表 5-13 走子策略在测试集上的表现（4613 个局面）

模型	测试集 NLL / nat	Top-1	Top-3	Top-5	Top-10
均匀分布（基线）	3.552	3.6%	—	—	—
可分离评估排序（按名次）	—	15.8%	28.5%	36.7%	53.5%
可分离评估 softmax（τ = 1）	3.799	16.5%	29.2%	36.9%	53.8%
学习策略（12 维，完整）	**3.276**	11.6%	26.0%	37.0%	57.8%
消融：只用可分离评估分数（相当于学习温度）	3.478	16.5%	29.2%	36.9%	53.8%
消融：去掉可分离评估分数（11 维）	3.297	10.9%	25.2%	36.1%	56.9%

模型	测试集 NLL / nat	Top-1	Top-3	Top-5	Top-10
（参考）学习策略在训练集上	3.283	11.5%	26.5%	38.1%	57.7%

注：负对数似然在前 40 名候选上归一化计算；“可分离评估排序（按名次）”没有概率输出，只报告命中率。Top-k 指 α-β 实际走的着法落在模型前 k 名中的比例。

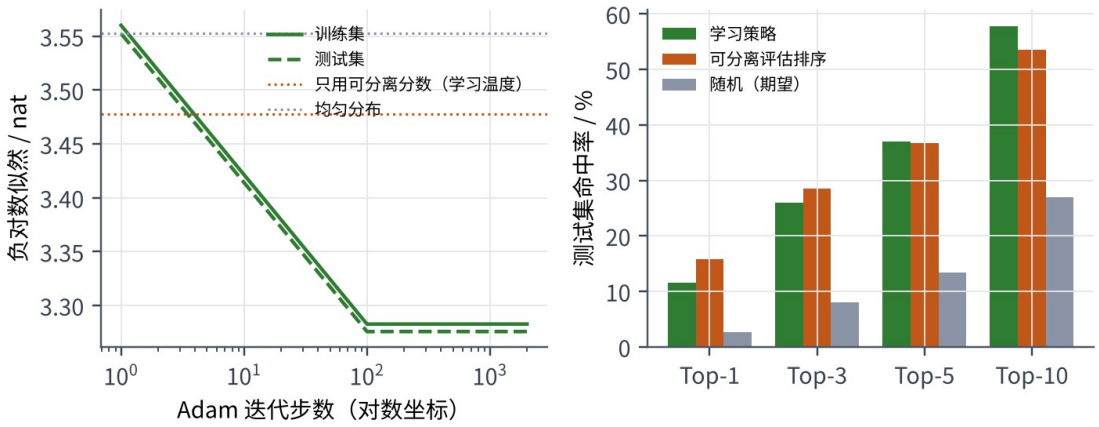


图 5-21 走子策略的训练曲线（左）与 Top-k 命中率（右）

注：测试集为后 80 局棋谱。“随机（期望）”为在 40 个候选中均匀随机时的期望命中率。

有三点值得注意。

学习策略的概率更好，但第一名的命中率更低。 学习策略的负对数似然为 3.276，比均匀分布（3.552）与 $\tau=1$ 的 softmax（3.799）都低，Top-10 命中率也最高（57.8%）。但它的 Top-1 命中率只有 11.6%，低于可分离评估排序的 15.8%。原因在于优化目标：负对数似然奖励“把概率合理地分散到多个可能的着法上”，而不奖励“把第一名押对”。对 PUCT 先验来说，前者正是需要的性质。

可分离评估分数本身需要更高的温度。 只用可分离评估分数、只学一个温度系数的模型，负对数似然为 3.478，对应的等效温度约为 7.8。这与 MCTS 消融中 $\tau=1$ 显著变差、$\tau=16$ 不比 $\tau=4$ 差的结果一致（5.5 节）。

其余 11 个特征提供了大部分增益。 去掉可分离评估分数后，负对数似然只从 3.276 上升到 3.297。各特征的系数（表 5-14）大多符合直觉：

- 让对方移动性下降的着法（`opMobLoss`，+1.49）、靠近对方领地的着法（`dOpT1`、`dOpT2`）更可能被选中；
- 射箭距离较远（`arrowDist`，+1.04）的着法更常见，这与人类“远箭封路”的经验相符；
- `mobT`（−2.0）为负，表示 α-β 不倾向于把后移到移动性很高的空旷位置。

表 5-14 学习得到的走子策略系数（特征已标准化）

特征	学习到的系数 θ
mobDelta	+0.630
mobT	−2.005
oppAdj	+0.748
ownAdj	−0.230
terr	+0.004
dOpT1	+1.193
dOpT2	+0.798
arrowNearOpp	+0.062
arrowDist	+1.037
qDist	−0.795
opMobLoss	+1.492
sepScore	+0.151

然而，这一离线优势并没有转化为棋力。循环赛中，学习先验 MCTS 比 MCTS-PUCT 低 64 Elo，M 组中则高 112 Elo；把这两组与 3 次重复实验合在一起，160 局中学习先验只得 46.2%，与 50% 没有显著差别（5.6.3 节）。一个可能的原因是：策略模仿的是 α-β 的着法，而 α-β 的选择本身受束宽与评估的限制；先验与搜索之间的配合，也不能只用离线命中率来衡量。

5.7.3 评估函数的 Texel 调参

表 5-15 与图 5-22 是 Texel 调参的结果。

表 5-15 评估函数学习结果（测试集 5205 个局面）

模型	参数数	训练集交叉熵	测试集交叉熵	测试集预测准确率
常数模型（只预测白方胜率 51.7%）	1	—	0.694	—
手工权重，只拟合尺度 K（K = 0.260）	1	0.516	0.486	75.3%
Texel 调参（λ = 0.0001，K = 0.261）	10	0.467	0.448	74.0%

表 5-16 按手数分段的测试集交叉熵

手数区间	测试局面数	手工权重交叉熵	Texel 权重交叉熵
0-20	1600	0.722	0.689
20-40	1600	0.534	0.528
40-60	1404	0.262	0.213
60-100	601	0.249	0.143

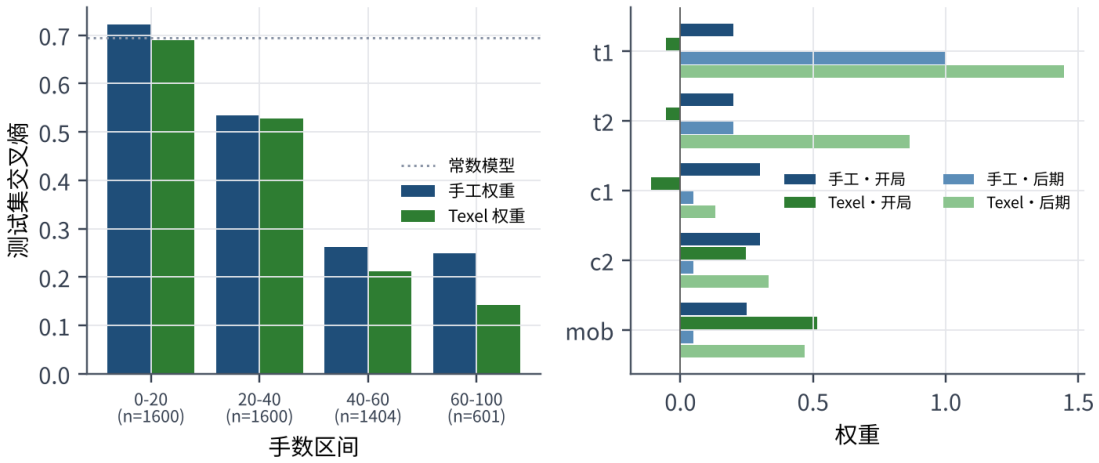


图 5-22 Texel 调参：按手数分段的测试集交叉熵（左）与手工/学习权重对比（右）

注：左图虚线为只预测平均胜率的常数模型。

拟合结果。 手工权重只拟合尺度 K 时，$K=0.260$，测试集交叉熵为 0.486，远好于常数模型的 0.694。Texel 调参把交叉熵进一步降到 0.448，但预测准确率反而从 75.3% 略降为 74.0%。按手数分段看，调参的改进几乎全部来自 40 手以后（60 手以后从 0.249 降到 0.143）。前 20 手两组权重的交叉熵都接近常数模型，也就是说，在开局阶段，静态评估几乎无法预测胜负。

学到的权重（表 5-17）。 后期的 t_1 权重从 1.0 增大到 1.45，t_2 从 0.2 增大到 0.86；开局的 t_1、t_2、c_1 权重都接近 0 甚至为负，而开局移动性权重从 0.25 增大到 0.52。

表 5-17 手工权重与 Texel 调参权重

特征	开局权重（手工）	开局权重（Texel）	后期权重（手工）	后期权重（Texel）
t1	0.20	−0.055	1.00	+1.445
t2	0.20	−0.055	0.20	+0.864
c1	0.30	−0.109	0.05	+0.133
c2	0.30	+0.246	0.05	+0.331
mob	0.25	+0.515	0.05	+0.469

预测更准，下棋却更弱。 用 Texel 权重的 α-β 在循环赛中只排第四（+141），比完整 α-β 低 286 Elo，并在 45 局冷门中占了 29 局。这一结果与 Texel 方法在国际象棋程序中的普遍经验相反，原因可以从三方面理解：

1. **优化目标与使用方式不一致。** Texel 调参优化的是“局面 → 最终胜负”的预测。但开局局面的胜负几乎无法预测（交叉熵约等于常数模型），拟合器于是把开局的领地权重压到接近 0。然而搜索需要的是“区分同一局面下不同着法的好

坏”，开局的领地差别虽然不能预测最终胜负，却能区分着法。权重压到 0 之后，α-β 在开局几乎只看移动性，着法选择因此变差。

2. **数据分布偏移。** 训练局面来自手工权重 α-β 的自对弈。换上新权重后，程序会走到训练数据中少见的局面。
3. **评估同时影响排序。** 可分离评估排序（3.2.3 节）使用同一组权重。权重变化后，束宽内保留哪些着法也随之改变，而 5.4 节已经表明排序对棋力影响极大。

在第一版程序的数据上，Texel 权重 α-β 的 Elo 名义上排第一（+259），但前四名（两种 α-β 与两种 PUCT 类 MCTS）都在 +234—+259 之间，区间大幅重叠（附录 B 表 B-4）。修复分隔判定、重新生成训练数据并重新调参之后，Texel 权重 α-β 显著落到了完整 α-β 之后。这一事实本身说明：**由少量自对弈数据学到的评估权重，其效果对数据与程序细节都很敏感**。本文不对“Texel 调参是否有效”下一般性结论，只报告在本文设置下观察到的结果。

5.8 单步决策质量

对局胜负是最终指标，但一局棋有几十步，单步的好坏被平均掉了。本节换一个角度：在 60 个固定的测试局面上，比较各智能体在 200 ms 内选出的着法，与一次更长时间的“参考搜索”选出的着法相差多少。

测试局面用固定随机种子从自对弈测试集（后 80 局，不参与学习模型的训练）中抽取，共 60 个尚未分隔的局面，手数在 5—59 之间。参考搜索为 3 s 的完整 α-β。每个着法 m 的价值 $v(m)$ 定义为：走 m 之后，用 1.5 s 的 α-β 搜索对方局面，取负值（单位：格）。遗憾值 $\mathrm{regret}(m) = v(m^{*}) - v(m)$，其中 m^{*} 为参考着法。

表 5-18 单步决策质量（60 个局面，200 ms/步）

智能体（200 ms）	与参考着法相同	平均遗憾值 / 格（截断到 ±10）	遗憾值中位数	遗憾 ≥ 2 格的局面	遗憾 ≥ 5 格的局面
α-β（完整）	42%	0.14	0.00	10%	0%
MCTS（PUCT+可分离先验）	18%	0.66	0.40	12%	2%
α-β（Texel 权重）	32%	0.67	0.00	18%	2%
MCTS（学习先验）	15%	0.76	0.63	18%	2%
α-β（廉价排序）	22%	0.78	0.58	23%	10%
MCTS（UCT+截断）	13%	1.19	1.16	28%	7%
一步贪心	10%	1.48	1.30	37%	8%
MCTS（纯随机）	5%	2.73	2.39	57%	17%

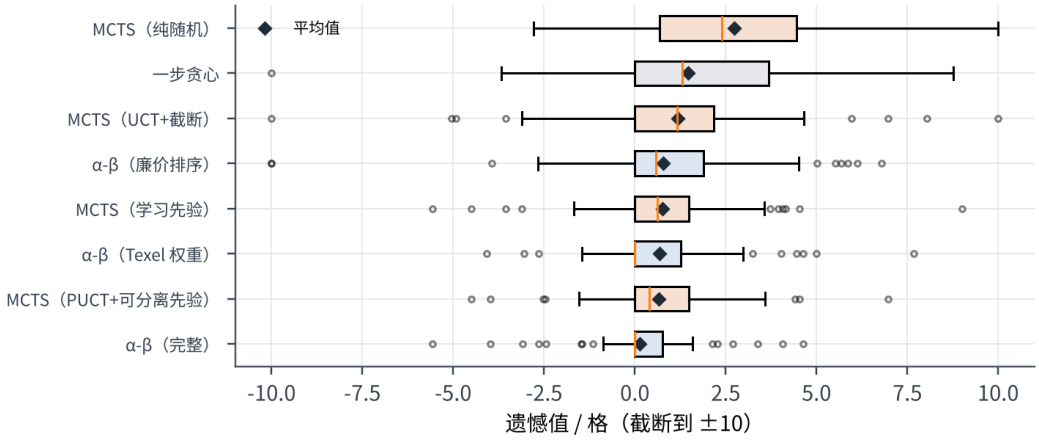


图 5-23　单步决策质量

注：各智能体在 60 个测试局面上的遗憾值分布（截断到 ±10 格）；菱形为平均值，橙色竖线为中位数。蓝色为 α-β 系列，橙色为 MCTS 系列，灰色为基线。

参考搜索在 60 个局面上平均完成 7.1 层。表 5-18 与图 5-23 的结果如下。

- **完整 α-β 的单步质量最好。** 它在 42% 的局面上与参考着法完全相同，遗憾值中位数为 0，只有 10% 的局面遗憾达到 2 格以上，没有一个局面达到 5 格。Texel 权重 α-β 的平均遗憾（0.67 格）比完整 α-β（0.14 格）大，与 5.7.3 节它在对局中明显较弱的结果方向一致。
- **MCTS 系列的着法很少与参考相同（13%—18%），但大多数遗憾不大。** MCTS-PUCT 的遗憾中位数为 0.40 格，遗憾达到 2 格以上的局面只有 12%、5 格以上的只有 2%。学习先验与 MCTS-PUCT 几乎没有差别，这与重复实验的结论一致（5.6.3 节）。
- **基线与弱变体的遗憾明显更大。** 一步贪心、UCT+截断 MCTS 与廉价排序 α-β 有 23%—37% 的局面遗憾达到 2 格以上；纯随机 MCTS 的遗憾中位数达到 2.39 格，57% 的局面遗憾在 2 格以上。
- **按单步遗憾的排序与 Elo 排序大体一致**，但有一处例外：廉价排序的 α-β 平均遗憾只有 0.78 格，与 MCTS-PUCT 接近，对局中却只得 12.5%。原因可能有二：一是它在 10% 的局面中犯下 5 格以上的大错，而一局棋中只要犯一次这样的错误就可能输掉；二是单步测试只看测试局面，不反映错误在对局中的累积。

需要说明，遗憾值本身也有噪声。表中出现了负的遗憾，即某些局面中被测着法在 1.5 s 的价值搜索下比参考着法还好；个别局面中，价值搜索恰好越过分隔边界，得到 ±200 量级的分隔计分（分别出现在 UCT+截断 MCTS、廉价排序 α-β 与一步贪心的 1—2 个局面中）。为避免这些局面主导均值，表中的平均遗憾把每个局面的遗憾截断到 ±10 格；中位数与比例不受截断影响。

这一测试对 α-β 偏有利。 参考搜索与价值计算都使用 α-β 与同一个评估函数，因此 α-β 智能体的着法天然更容易与参考一致。MCTS 选出的着法即使真的更好，也可能被参考搜索判为“有遗憾”。所以，本节的结果只用于比较**同一类方法内部**的差别（例如 α-β 与 Texel 权重 α-β、MCTS-PUCT 与学习先验 MCTS），不用于比较 α-β 与 MCTS 孰优孰劣。

5.9 搜索效率

本节的测试都在单进程中顺序运行，使用与 5.8 节相同的 60 个测试局面。

5.9.1 着法排序的质量

表 5-19 着法排序方法的质量与耗时（60 个局面）

局面组	局面数	平均合法着法数	静态最佳在廉价排序前 8 / 24 / 60	静态最佳在可分离排序前 1 / 8 / 30	逐一评估全部着法 / ms	廉价排序 / ms	可分离评估 / ms
开局（< 20 手）	14	879	21% / 29% / 50%	36% / 79% / 100%	14.5	0.43	2.21
中局（20—39 手）	23	278	4% / 35% / 52%	22% / 70% / 87%	3.3	0.09	1.10
后期（≥ 40 手）	23	76	65% / 87% / 100%	30% / 70% / 91%	0.7	0.05	0.50
全部	60	341	32% / 53% / 70%	28% / 72% / 92%	4.9	0.15	1.13

表 5-19 衡量各排序方法能否把“静态评估最好的着法”（对全部合法着法逐一走一步、做一次完整评估后的第一名）排在前面，以及它们的耗时。

- **廉价分数的排序质量很差。** 全部局面中，静态最佳着法只有 32% 落在廉价排序的前 8 名、53% 落在前 24 名；中局尤其糟糕，前 8 名只有 4%。这正是 2.2.5 节“约一半”这一说法的来源，也解释了 5.4 节中“内部结点用廉价分数排序”为何损失 338 Elo：束宽只有 8 时，好着法多半根本没有进入搜索。
- **可分离评估接近完整评估，代价却低得多。** 静态最佳着法有 72% 落在可分离排序的前 8 名、92% 落在前 30 名（开局为 100%）。它平均每次只需 1.1 ms，而逐一评估全部着法需要 4.9 ms；开局时分别为 2.2 ms 与 14.5 ms，相差约 6.6 倍。合法着法越多，节省越大。
- **可分离排序的第一名只有 28% 恰好是静态最佳。** 这说明“移后与射箭可加”的近似确实有误差（3.2.3 节）；但对束宽剪枝来说，重要的是好着法进入前几名，而不是恰好排第一。

5.9.2 固定深度下的搜索量

表 5-20 固定深度 3 的 α-β：各增强组合的搜索量

变体	结点数（几何均值）	相对完整版	平均用时 / ms	首着截断率	置换表命中 / 内部结点	与完整版选择相同着法
完整（PVS+置换表+杀手/历史）	1,197	1.00×	52	66.8%	11.2%	100%
无置换表	1,201	1.00×	52	74.8%	—	95%
无杀手/历史	1,356	1.13×	55	43.9%	11.1%	97%
无 PVS	1,323	1.11×	54	69.2%	0.1%	100%
朴素 α-β（增强全关）	1,338	1.12×	52	53.7%	—	95%
内部结点廉价排序	1,094	0.91×	29	70.6%	10.8%	32%
叶前结点也用可分离排序	500	0.42×	116	85.8%	11.7%	45%

表 5-20 在 60 个局面上做固定深度 3 的 α-β 搜索，比较各增强组合的搜索量。由于只搜 3 层，这一测试衡量的是“同样的深度需要多少工作”，而不是棋力。

- **经典增强只节省了 0—13% 的结点。** 去掉杀手/历史启发，结点数增加 13%，首着截断率从 66.8% 降到 43.9%；去掉 PVS 增加 11%；全部关闭增加 12%；去掉置换表几乎没有变化（置换表命中只占内部结点的 11%）。这与 5.4 节“经典增强没有显著影响棋力”的结果一致：在束宽 8、深度 3—5 的小树上，10% 左右的效率差别远不足以多搜一层。
- **排序方式改变的是“搜什么”，而不只是“搜多少”。** 内部结点改用廉价排序时，结点数反而少 9%、用时少 44%，但只有 32% 的局面选出了与完整版相同的着法；叶前结点也用可分离排序时，结点数降到 42%，但排序本身更贵，用时增加到 2.2 倍。可见，对束宽剪枝的 α-β 而言，排序决定了搜索看到哪些着法，比结点数更重要。

5.9.3 固定预算下的搜索量

表 5-21 200 ms 预算下的搜索深度与模拟次数（按对局阶段分组）

智能体	开局（< 20 手）	中局（20—39 手）	后期（≥ 40 手）	平均用时 / ms
α-β（完整）	深度 3.00，2,038 结点	深度 3.70，3,471 结点	深度 5.09，8,366 结点	163
α-β（廉价排序）	深度 3.86，3,209 结点	深度 4.04，5,970 结点	深度 6.65，11,753 结点	161
MCTS（纯随机）	5,697 次模拟，树深 2.0，根子结点 879	10,681 次模拟，树深 2.3，根子结点 278	22,019 次模拟，树深 4.1，根子结点 76	200
MCTS（UCT+截断，随机顺序）	3,240 次模拟，树深 2.0，根子结点 879	5,549 次模拟，树深 2.2，根子结点 278	13,826 次模拟，树深 3.7，根子结点 76	200

智能体	开局（< 20 手）	中局（20—39 手）	后期（≥ 40 手）	平均用时 / ms
MCTS（UCT+截断）	2,647 次模拟，树深 4.0，根子结点 60	3,995 次模拟，树深 4.1，根子结点 60	10,914 次模拟，树深 4.7，根子结点 52	200
MCTS（UCT+可分离排序）	323 次模拟，树深 3.0，根子结点 30	650 次模拟，树深 3.3，根子结点 30	2,825 次模拟，树深 4.0，根子结点 29	203
MCTS（PUCT+可分离先验）	437 次模拟，树深 4.2，根子结点 30	883 次模拟，树深 4.8，根子结点 30	3,051 次模拟，树深 5.9，根子结点 29	202
MCTS（学习先验）	423 次模拟，树深 4.2，根子结点 30	839 次模拟，树深 4.9，根子结点 30	2,632 次模拟，树深 5.9，根子结点 29	202

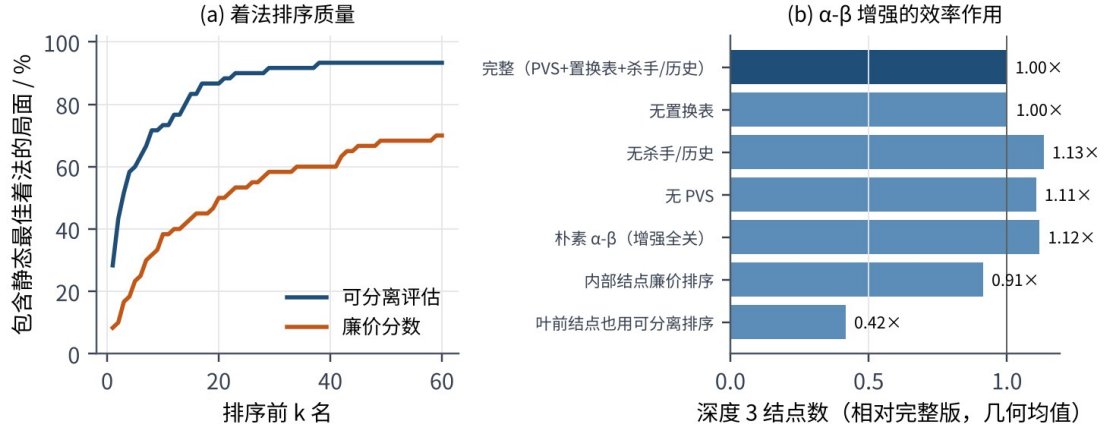


图 5-24 搜索效率

注：(a) 静态最佳着法落在各排序方法前 k 名中的局面比例；(b) 固定深度 3 时各 α-β 变体的结点数（相对完整版）。数据：60 个测试局面，单进程。

表 5-21 给出单进程、200 ms 预算下的搜索量。与对局中（4 进程并行）的统计相比，数值略高，但规律相同。

- **α-β 在开局只能搜 3 层**，中局 3.7 层，后期 5 层。开局合法着法平均 879 个，即使束宽为 8，每一层的排序本身也要花不少时间。
- **廉价排序的 α-β 搜得更深，却更弱。** 它在开局能搜 3.9 层、后期 6.7 层，都比完整 α-β 深，但对局中只得 12.5%（5.4 节）。“更深”并不等于“更好”，前提是搜索的着法是对的。
- **MCTS 的模拟次数相差两个数量级。** 纯随机 MCTS 在开局每步做约 5700 次模拟，PUCT 只做约 440 次。但纯随机 MCTS 的树深只有 2，根结点的 879 个子结点平均每个只被访问几次；PUCT 只展开 30 个候选，树深达到 4—6。在亚马逊棋上，把有限的模拟集中在少数好着法上，比做更多的模拟重要得多。

5.9.4 浏览器中的速度

表 5-22 浏览器（Chromium）与 Node.js 中的搜索速度对比

环境	α-β 每秒结点（深度 3，20 个局面）	MCTS-PUCT 每秒模拟（1 s，10 个局面）
Node.js v22.23.4	14.3 k	1,898
Chromium 153.0.8010.12（页面主线程）	14.3 k	1,924
比值（浏览器 / Node）	1.00	1.01

网页中的 AI 与实验中的 AI 是同一份引擎代码（4.6 节），但运行在浏览器里。表 5-22 在同一台机器上，用相同的局面比较 Node.js 与 Chromium 中的搜索速度：表中这次测量里，α-β 与 MCTS 在浏览器中的速度分别是 Node.js 的 1.00 倍与 1.01 倍；此前一次测量（当时机器上还运行着其他实验）的比值为 0.97 与 0.92。可见两者的速度基本相同，差别在测量波动范围之内。也就是说，网页中的 AI 与本章实验中的 AI 在相同思考时间下的搜索量基本一致，本章的棋力结论可以直接用来理解网页游戏中各 AI 的强弱。

5.9.5 决策的随机性

表 5-23 同一局面重复决策 5 次的一致性（30 个局面）

智能体（200 ms）	5 次决策完全相同的局面	众数着法占比	每个局面的不同着法数（平均）
α-β（完整）	93%	97%	1.10
MCTS（PUCT+可分离先验）	17%	56%	3.00
MCTS（UCT+截断）	30%	70%	2.30
MCTS（学习先验）	20%	59%	2.80

在 30 个测试局面上，让每个智能体用不同的随机种子各决策 5 次（表 5-23）。

- **α-β 几乎是确定性的**：93% 的局面 5 次选择完全相同，平均每个局面只出现 1.10 种着法。剩下的差异来自计时：迭代加深在时间边界附近可能多完成或少完成一层。
- **MCTS 的随机性很大**：MCTS-PUCT 只有 17% 的局面 5 次选择完全相同，平均每个局面出现 3 种不同的着法，众数着法只占 56%。结合 5.8 节的结果，这些不同的着法大多质量相近（遗憾中位数只有 0.40 格），即 MCTS 在 200 ms 内常常无法在几个差不多好的候选之间分出高下。

- 这也解释了为什么本文必须交换先后手、使用多个开局并重复实验：MCTS 参与的每一局都有相当的偶然性，单局甚至单个 32 局批次的结果都可能偏离真实水平。

5.10 失败案例分析

5.10.1 案例一：分隔判定的漏洞

本文最重要的一次失败，不是某个 AI 的一步坏棋，而是程序本身的一个逻辑错误。第一版程序在完成全部主实验之后，在复盘失败对局时才发现这个错误。

错误的内容。 第一版的分隔判定只检查一件事：棋盘上是否还有“双方的后都能到达的空格”（争夺格）。若没有，就认为双方已经分隔，交给填格策略。这一判定漏掉了一种情形：**敌对的两个后紧挨在一起，中间没有空格**。此时确实不存在争夺格，但只要其中一个后走开，它腾出的格子就成了双方都能到达的空格，双方重新接触。图 5-25 是一个实例（第一版主实验，T:ab~mcts-pure|14|BA）：

- (a) 第 40 手前，f10 的白后与 g10 的黑后相邻，g1 的白后与 h2 的黑后相邻。旧判定认为已经分隔，α-β（黑方）从这一手起交给填格策略，搜索评估为 +249.5（“已分隔，黑方领先 49.5 格”）；
- 第 40 手，黑方的填格策略把 h2 的黑后走到 h9（h9-h10 射箭），腾出了 h2；第 41 手，白方立即把 g1 的后走到 h2（射箭 h7），进入原本属于黑方的区域；
- (b) 第 42 手前，双方重新接触，原本属于黑方的大片区域重新变成争夺区。黑方恢复搜索时，评估已降到 +31。此后这局棋又两次被旧判定误认为“已分隔”，黑方最终仍然落败。

(a) 第 40 手前：旧判定认为已分隔　　(b) 第 42 手前：双方重新接触

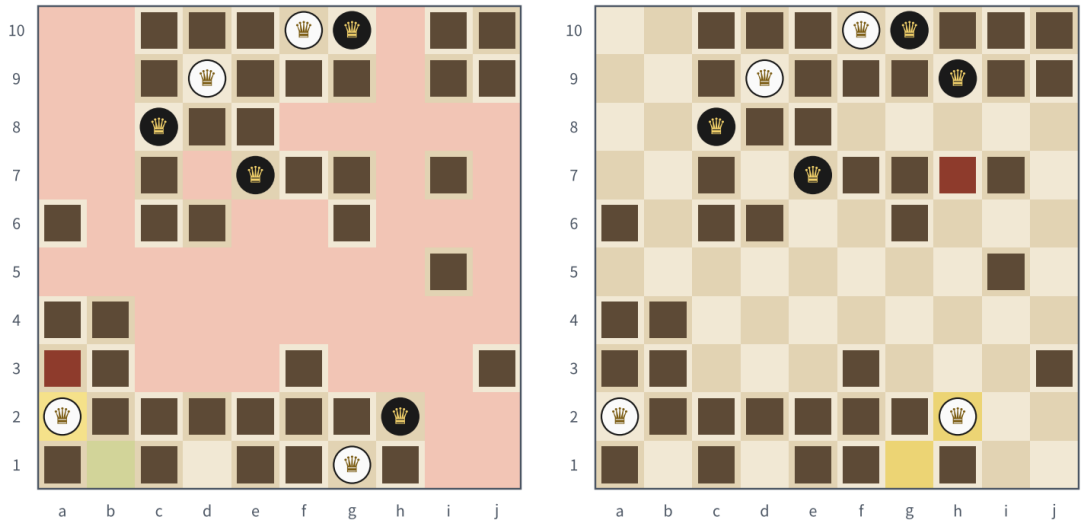


图 5-25　案例一：旧分隔判定的错误

注：第一版主实验对局 T:ab~mcts-pure|14|BA。(a) 底纹为旧判定下的区域，红色为黑方；(b) 两手之后双方重新接触，底纹消失。

影响有多大。 用 `analysis/failure.js` 回放第一版的 1728 局（不含随机与纯随机 MCTS）：

- 有 1723 局曾被旧判定认为“已分隔”，其中 488 局（28%）在此之后又恢复了接触；
- 在旧判定认为已分隔的 43694 个局面中，有 20275 个（46%）按正确定义其实并未分隔；
- 有 71 局的最终胜负与“首次分隔时的区域格数”所预测的结果相反，其中 67 局发生过恢复接触。

这些数字说明，漏洞并不罕见：它系统性地让程序在接触状态下停止搜索，而对手（同样受此影响，或者根本不使用分隔判定的纯随机 MCTS）可能趁机反击。

修复与验证。 正确的定义是：把箭以外的格子按 8 邻域连通，没有任何一个连通分量同时含有双方的后（1.2.3 节）。修复后，在 `engine/eval.js` 与 `engine/core.js` 中都加入了这一检查，并增加了两项单元测试（4.8.1 节）：一项与暴力连通分量算法逐局面比对，另一项检查“一旦分隔，之后的局面始终保持分隔”。修复之后，本文重新运行了自对弈、学习与全部主实验。第一版数据保存在 `data_v1/` 目录中，只用于本节的对比。

修复后，在主实验的 1225 局中，按“首次分隔时的区域格数”预测胜负只错了 5 局（0.4%）。这 5 局在分隔时双方区域格数的差别都不超过半格（考虑行棋方之后），

属于“谁多走一步谁赢”的临界情形。图 5-26 是其中一局的分隔时刻：棋盘几乎填满，双方各剩一两个格子，区域格数并不等于实际能走的步数。

分隔时刻的区域（蓝：白方，红：黑方）

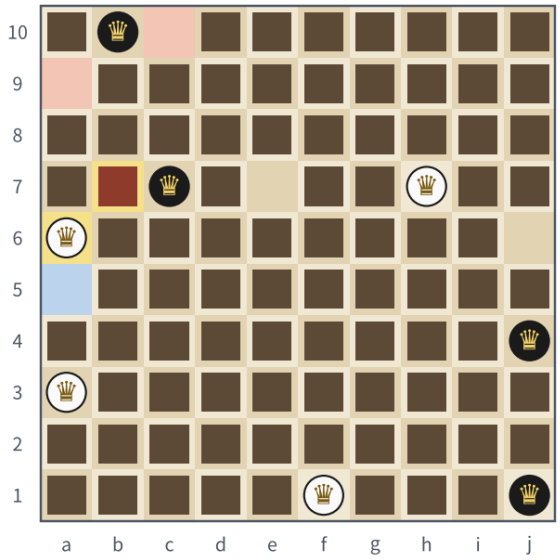


图 5-26 案例二：修复后区域计数预测失败的对局

注：消融对局 A:noTT|12|AB 第 87 手前，双方各只剩一两个可走的格子。

教训。 这个漏洞不会让程序崩溃，也不会让任何单元测试失败（第一版的测试只覆盖了“确实分隔”的局面），所有实验都能顺利完成并给出看似合理的结果。它之所以被发现，是因为失败案例分析要求逐步复盘：评估从 +249.5 突然跌到 +31，这在“已分隔”的局面中是不可能的。这段经历在 3.7 节的 AI 工具使用记录与第六章中都有记载。

两版数据的比较也很有意义（附录 B 表 B-4）：第一版中前四名都在 +234—+259 之间、难分高下；修复之后拉开为 +427、+362、+298、+141，Texel 权重 α-β 从名义上的第一落到第四。而“α-β 短时间占优、MCTS 长时间占优”的交叉在两版数据中都存在。也就是说，一部分结论对实现细节很敏感，另一部分则相当稳健。

5.10.2 案例二：分隔之后的近似

分隔之后，所有智能体都交给同一个贪心填格策略（2.6 节）。这一策略并不最优：在形状复杂的区域中，贪心地选“移动后可达格最多”的着法，可能把自己的后困在小块区域里，浪费格子。

上面的统计给出了这一近似的代价：修复后，“首次分隔时区域格数多的一方获胜”的预测在 1225 局中只错了 5 局，而且都是临界情形。因此，在本文的对局中，

填格策略带来的误差可以忽略，胜负几乎完全在分隔之前决定。这也支持了 2.6 节的设计：把实验的比较集中在“分隔之前的争夺”上。

5.10.3 案例三：强智能体输给弱智能体

本节复盘一局完整 α-β 输给一步贪心的对局（T:greedy~ab|3|BA，α-β 执白，86 手负）。方法是：取 α-β 每一步的搜索评估，找出分隔之前评估下降最多的一步，再用 5 s 的 α-β 重新搜索那一步之前的局面。

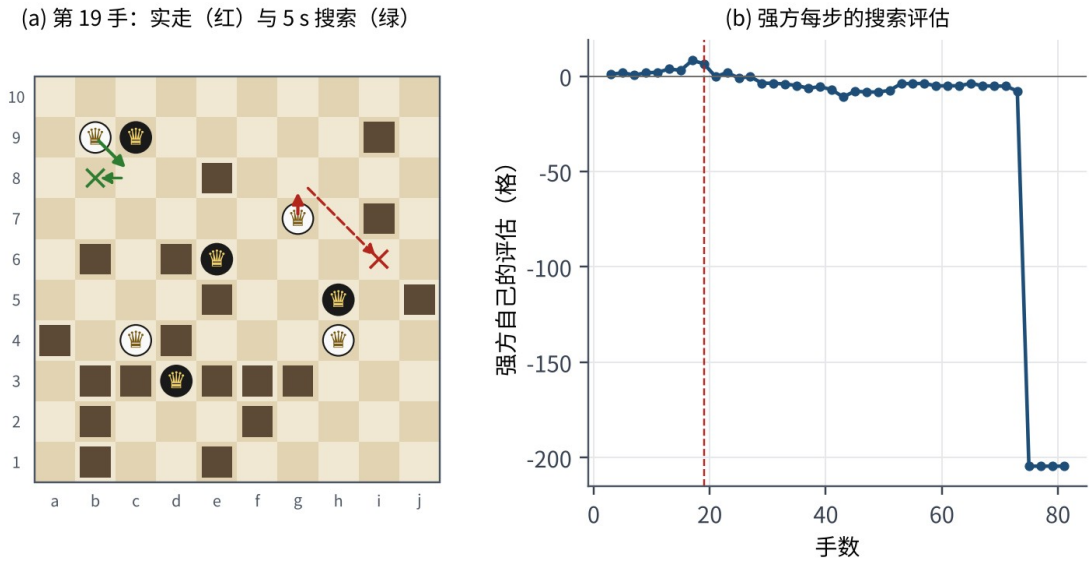


图 5-27 案例三：α-β 输给一步贪心

注：(a) 第 19 手前的局面，红色箭头为 α-β 实际所走，绿色箭头为 5 s 搜索的推荐；(b) α-β 每一步的搜索评估（白方视角）。

如图 5-27 所示：

- 第 19 手，α-β 在 200 ms 内只完成了 3 层搜索（低于 4.7 层的平均值，此时白方有 700 多个合法着法），评估为 +6.5，走了 g7-g8/i6；
- 一步贪心应以 e6-h3/i3。α-β 下一步对新局面的评估降到 −0.2，下降了 6.7 格，此后再也没有恢复优势，最终在分隔时落后约 5 格；
- 5 s 的搜索（6 层）推荐 b9-c8/b8，评估 +5.6；而对实际所走的 g7-g8/i6，5 s 搜索给出的价值只有 −2.3。

也就是说，200 ms 的搜索把这一步高估了约 9 格。进一步检查发现，问题不在束宽：黑方的应着 e6-h3/i3 在可分离评估中排第 2 名，推荐着法 b9-c8/b8 在根结点的静态排序中排第 27 名，都在束宽之内，200 ms 的搜索都“看到”了它们。问题更可能在深度：3 层搜索只看到“白走—黑应—白再走”，g7-g8/i6 的不利后果要在更深处

才显现，静态评估在 3 层的叶结点上没有识别出来。这是典型的**地平线效应**，属于 3.5.1 节所说的“评估误差”。5 s 的搜索多看了 3 层，就纠正了这一判断。

这类失败并不罕见。完整 α-β 在循环赛中输给弱智能体的 5 局里，分隔前评估下降最多的一步分别出现在第 19、21、32、44、45 手，降幅都在 6—10 格之间，都发生在中局着法最多、搜索最浅的阶段。这与 5.6 节“α-β 在 50 ms 时明显变弱”、5.4 节“束宽加倍更强”的结果一起说明：在中局，α-β 的搜索量还远远不够。

5.11 局限与有效性威胁

5.11.1 结论的适用范围

本文的所有结论都限定在以下条件之下：10×10 标准亚马逊棋，单线程 JavaScript 引擎，每步 50—800 ms，16 个两手均衡开局。亚马逊棋顶尖程序通常使用多核并行、每步数十秒，并有大量手工调整的知识[24,28]；在那样的条件下，两类方法的相对强弱可能完全不同。本文的智能体也远没有达到顶尖程序的水平，本文没有条件与它们对弈。

5.6.2 节的交叉现象尤其依赖实现：α-β 的束宽、MCTS 的模拟长度都不是最优的（5.4、5.5 节）。如果调好这两处，交叉点可能移动，甚至消失。

5.11.2 统计效力

每组对阵 32 局，只能可靠分辨约 ±150 Elo 以上的差别。本章多处结论是“没有观察到显著差别”，不能解读为“没有差别”。表中报告的配对符号检验 p 值没有做多重比较校正；A、M 两组共 26 个对比，按 $p<0.05$ 期望约有 1.3 个假阳性。因此，p 值在 0.01—0.05 之间的个别结论（例如束宽加倍的 $p=0.039$）应当谨慎看待。5.6.3 节的重复实验说明了 32 局结果的实际波动范围。

5.11.3 参数选择

α-β 与 MCTS 的大部分参数（束宽、模拟长度、k 等）是在第一版程序上凭经验或少量试调确定的，修复后没有重新调整（5.1.3 节）。MCTS 中评估到胜率的换算系数 $k=0.18$ 来自第一版的拟合；修复后重新拟合得到 0.26（5.7.3 节），但本文没有修改 k，以免主实验的设置与开始时不一致。两种方法得到的“调参关注”也不完全对等：MCTS 的温度与探索系数经过了试调，α-β 的束宽则没有。

5.11.4 计时与并行的影响

对局实验用 4 个工作进程并行，各进程会竞争 CPU 与内存带宽。时间预算对双方相同，因此影响是对称的；但它会降低绝对搜索量，而 α-β 与 MCTS 对搜索量的敏

感程度不同（5.6 节），因此可能对两者的比较产生轻微的不对称影响。单进程的效率测试（5.9 节）可以作为参照。

5.11.5 开局的代表性

16 个开局都只有两手，由程序在“静态评估前 30 名”中随机生成，并用 1 秒 α-β 过滤。这些开局不一定代表人类或强程序实际会走的开局。用同一个 α-β 来判断“均衡”，也可能让开局集合对 α-β 略有偏向。

5.11.6 实现层面的近似

- **分隔之后的填格策略是贪心的**，不保证最优。5.10.2 节表明它对胜负的影响很小，但这一结论只适用于本文对局中出现的区域形状。
- **评估函数是手工设计的**，没有使用神经网络。本文的结论不能推广到 AlphaZero 式的“神经网络 + MCTS”[19]。
- **决策质量测试的参考搜索与 α-β 同源**（5.8 节），对 MCTS 不公平。
- **第一版程序的分隔判定有错误**（5.10.1 节）。虽然已修复并重跑了全部实验，但这一经历说明，程序中仍可能存在没有被发现的错误。本文尽量通过单元测试、与暴力算法的对比以及失败案例复盘来降低这一风险，但不能完全排除。

5.12 本章小结

本章用 2144 局主实验、384 局重复实验以及一系列单进程测试，回答了 3.6.2 节提出的研究问题：

1. **总体棋力（RQ1）**：在 200 ms/步下，完整 α-β（+427 Elo）与 MCTS-PUCT（+362）处于同一档，α-β 略强（160 局合计 60.0%）；两者都比一步贪心强 350 Elo 以上，而不使用领域知识的纯随机 MCTS 比一步贪心还弱 771 Elo。
2. **α-β 的组件（RQ2）**：着法排序与评估函数是决定性因素（各约 −340 至 −390 Elo）；束宽 8 偏窄，加倍后显著更强；置换表、杀手、PVS 在本实验中没有显著影响。
3. **MCTS 的组件（RQ3）**：先验是最重要的组件（UCT 选择 −394 Elo），先验温度需要足够高；模拟越短越好，随机模拟到终局几乎全负。
4. **算力缩放（RQ4）**：α-β 在 100 ms 以上基本饱和，MCTS 持续提升；相同预算下，二者在 200—400 ms 之间交叉。

5. **学习（RQ5）**：线性走子策略的离线指标优于基线，但作为 MCTS 先验时，160 局合计得 46.2%，没有显著提升；Texel 调参改善了胜负预测，却让 α-β 明显变弱（128 局合计只得 18.8%）。
6. **质量、效率与失败（RQ6）**：单步决策中，完整 α-β 与参考搜索最一致 MCTS-PUCT 的着法很少与参考相同但遗憾同样很小；可分离评估以约四分之一的代价给出了接近完整评估的排序，而经典 α-β 增强只节省 0—13% 的结点；MCTS 的决策随机性远大于 α-β。最重要的失败来自程序本身的分隔判定漏洞，修复后重跑了全部实验；强智能体输给弱智能体的对局，典型原因是束宽之外的对方应着。

六、个人学习和实践心得

6.1 对博弈搜索算法的理解

课堂上学 α-β 剪枝时，我对它的印象主要是“极小极大加上一个上下界，能剪掉一些分支”，以及“最好情况下能把 b^d 降到大约 $b^{d/2}$”这个结论。换成亚马逊棋之后，我才第一次真正体会到这个公式里的 b 有多可怕：开局一步就有 2176 种走法，中局也常有几百种（1.3.2 节）。就算把 b^d 开平方，4 层也还是天文数字。所以本项目的 α-β 能在 200 ms 内搜到 4—5 层，靠的不是 α-β 剪枝本身，而是束宽剪枝：每一层只看排名前 8 的着法。这样一来，“排名”就成了整个搜索的命门。消融实验把这一点表现得非常直接：内部结点的排序从可分离评估换成廉价分数，棋力就掉了 338 Elo；而教材里篇幅不小的置换表、杀手启发和 PVS，关掉之后看不出任何差别（5.4 节）。

我原来以为“增强越多越强”，实验告诉我，一个组件的价值取决于它所在的系统。在排序已经很好、树又很小的程序里，置换表能省下的结点有限；而一旦束宽太窄，再深的搜索也只是在一棵“漏掉了好着法”的窄树里看得更远。5.6 节中 α-β 在 100 ms 以上几乎不再变强，5.10.3 节的失败对局中 α-β 只搜了 3 层就走出坏棋，这两件事让我对“深度”与“宽度”的权衡有了切身的感受。

MCTS 给我的冲击更大。课本上的 MCTS 是一个很“通用”的算法：不需要评估函数，靠随机模拟就能估计胜率。可是在亚马逊棋里，纯随机模拟的 MCTS 每步能做上万次模拟，Elo 却比只看一步的贪心算法低 771 分（5.3 节）。原因很简单：随机走出来的亚马逊棋和真实对局毫无相似之处，随机对局的胜负几乎不包含局面信息。加上可分离评估给出的先验、把模拟截短到几步再用评估函数打分之后，MCTS 才和 α-β 站到了同一档。消融实验还显示，模拟越短越好，完全不模拟、直接评估也不比默认设置差（5.5 节）。这让我意识到，MCTS 的“通用”是有代价的，它的强弱主要取决于注入了多少领域知识。AlphaZero 之所以显得“通用”，是因为它用神经网络把知识学了出来，而不是因为它不需要知识。

最让我觉得有意思的是“可分离评估”这个想法。一步亚马逊棋由“移动后”和“射箭”两部分组成，如果假设这两部分对局面价值的贡献近似可以相加，就只需要评估“后的走法数 + 箭的位置数”个局面，而不是二者的乘积（2.2.5 节）。这个近似显然不严格，但它把排序的代价降低了一个数量级，又比廉价分数准确得多。课堂上讲过“问题分解”和“近似”，这是我第一次在自己的项目里看到一个近似假设带来如此大的收益。

6.2 对实验结果的思考

这个项目里最让我意外的，是几个“反直觉”的结果。

第一，结论会随时间预算翻转。 200 ms 时 α-β 略强于 MCTS，50 ms 时 α-β 明显更强，800 ms 时却只剩 15.6% 的得分率（5.6.2 节）。如果只做 200 ms 一组实验，我可能会写出“α-β 比 MCTS 更适合亚马逊棋”这样的结论，而它只在一个很窄的条件下成立。这让我明白，比较两个算法时，“在什么条件下比较”和“比较的结果”同样重要。

第二，离线指标好不等于实际效果好。 Texel 调参把评估函数的测试集交叉熵从 0.486 降到了 0.448，看起来是明显的改进；可是换上新权重的 α-β 在循环赛中比原版低了将近 300 Elo（5.7.3 节）。仔细分析后我才明白，Texel 优化的是“预测最终胜负”，而开局局面的胜负几乎无法预测，于是拟合器把开局的领地权重压到接近零；可搜索需要的是“区分同一局面下哪一步更好”，开局的领地差别虽然预测不了胜负，却能区分着法。这提醒我，在机器学习中选择什么指标来衡量“好”，本身就是需要思考的问题。

第三，同一个实验重复做，结果可能差很多。 学习先验 MCTS 对 MCTS-PUCT，在循环赛中得分 38%，在消融实验中得分 65.6%，两组设置只差随机种子。如果只做一次，我可能会得出两个完全相反的结论。后来用新的随机种子又重复了三次，5 批共 160 局合计得分 46.2%，与 50% 没有显著差别：消融实验中那个 65.6% 只是一次偏高的波动（5.6.3 节）。反过来，Texel 权重 α-β 输给原版的结论在 4 个批次中都稳定地出现。这让我真正理解了置信区间和重复实验的意义：一个漂亮的数字本身说明不了什么，要看它的不确定性有多大、能不能重复出来。

6.3 一个程序漏洞带来的教训

这个项目最重要的一课，来自一个程序错误。第一版程序判断“双方已分隔”时，只检查“有没有双方都能到达的空格”，漏掉了“敌对的两个后紧挨着”的情况（5.10.1 节）。这个错误不会让程序崩溃，所有单元测试都能通过，全部主实验也顺利跑完，Elo 表、消融表、缩放曲线看上去都很合理。直到做失败案例分析、逐步复盘一局棋时，才看到一个说不通的现象：程序认为“已经分隔、我领先 49 格”，两手之后评估却跌到了 31。顺着这个线索查下去，才发现在旧判定认为“已分隔”的局面中，将近一半其实并没有分隔。

修复之后，我们重新运行了自对弈、学习和全部 2144 局主实验，结果确实发生了变化：Texel 权重 α-β 从名义上的第一名落到第四名，而“α-β 短时间占优、MCTS 长时间占优”的结论则在两版数据中都成立。这件事让我深刻体会到三点：

1. “程序能跑、结果看起来合理”不等于程序是对的。真正发现错误的，是对具体对局的仔细复盘，而不是汇总后的统计表；
2. 写测试时，不仅要测“应该成立的情况”，还要想清楚定义的边界在哪里。修复后增加的测试，把分隔判定和一个暴力连通分量算法逐局面比对，这才真正覆盖了边界情况；
3. 发现错误之后，最省事的做法是悄悄修好、只报告新结果。但旧结果也是真实运行出来的，而且两版数据的对比本身就说明了哪些结论稳健、哪些结论脆弱。所以本文把第一版数据保留下来，在 5.10.1 节和附录 B 中如实报告了这个错误及其影响。

6.4 关于使用 AI 工具的体会

这个项目大量使用了 AI 工具，包括本报告的撰写，3.7 节已经如实说明了它做了哪些工作、犯过哪些错误。本章的文字也是应我的要求由 AI 起草的，内容基于本项目的实际过程与结果。回头看，我最大的体会是：AI 可以很快地生成大量代码和文字，但“生成”和“正确”之间隔着一整套验证工作。上面说的分隔判定漏洞，正是 AI 写的代码中的错误，而且是 AI 自己在失败案例分析中发现的；但如果不是一开始就要求“每个结论都要有真实数据支撑、失败案例要逐步复盘”，这个错误很可能就留在了最终报告里。参考文献也是一样：AI 凭记忆写出的文献中有一部分无法核验，只有逐条在线检索才能筛掉。

另一个体会是，AI 并不会替我理解。报告写得再完整，如果我讲不清楚 α-β 为什么能剪枝、PUCT 公式的每一项是什么意思、为什么 Texel 权重预测更准却下得更差，那这次作业对我的学习就没有意义。因此，我认为使用 AI 工具的正确方式，是把它当作一个能力很强但会犯错的助手：由人来提出问题、设定标准、检查结果、对结论负责。

在这个项目中，我的角色更像是“需求方”和“审核者”：提出作业的全部要求与约束（选题、六个章节、篇幅、参考文献必须真实可核验、数据必须来自真实运行、如实说明 AI 的使用），在老师认为五子棋选题过于常见之后决定换成亚马逊棋，并在 AI 完成后审阅最终成果、决定提交。这让我体会到，提出清楚、可检验的要求本身就是一种重要能力。

6.5 与自动化专业的联系

作为自动化专业的学生，我注意到博弈搜索和控制理论之间有不少相通之处。

迭代加深的 α-β 每一步都向前“预演”若干步，选出当前最优的一手，走完之后再根据新的局面重新搜索。这和模型预测控制（MPC）的“滚动时域优化”几乎是同一个思想：用模型预测未来，只执行第一步，然后在新的状态上重新优化。两者也面临相似的问题：预测时域越长越好，但计算量随时域急剧增长，必须在实时性和最优性之间权衡。本项目中“200 ms 能搜几层”“时间不够时怎样取舍”，与 MPC 中“采样周期内能否解完优化问题”是同一类问题。束宽剪枝则有点像控制中的“降阶”：为了实时性，主动舍弃一部分可能性，代价是可能错过真正的最优解。

评估函数像控制中的“终端代价”：搜索不可能看到终局，只能在有限深度处用一个估计值代替真实回报，估计得好不好直接决定决策质量。亚马逊棋的“分隔”还让我想到系统的“解耦”：一旦双方的区域互不连通，整个博弈就分解成了两个独立的单人问题，复杂度骤降。而 MCTS 用随机采样估计期望回报、在“利用”和“探索”之间平衡，又和随机控制、强化学习中的思想一脉相承。

这些联系让我觉得，课程中学到的搜索、学习与决策方法，并不只是用来下棋的，它们是“在不确定环境中做序贯决策”的一般工具，在机器人路径规划、调度和智能控制中都有用武之地。

6.6 不足与今后的打算

回顾这个项目，还有不少不足：

1. **参数没有在修复后重新调整。** 消融实验已经表明 α-β 的束宽偏窄、MCTS 的模拟偏长，但为了保持实验设置的一致，本文没有修改默认值（5.11.3 节）；
2. **没有与外部强程序比较**，所有 Elo 都只是项目内部的相对强弱；
3. **没有尝试深度学习。** 线性策略的离线指标已经优于手工排序，如果用卷积网络和自对弈训练，结论可能完全不同；
4. **统计效力有限。** 大多数对比每组只有 32 局，只能分辨较大的差别。

如果今后有机会继续，我想先按消融实验的结果调好束宽和模拟长度，看看 α-β 与 MCTS 的交叉点会不会移动；再尝试用小型神经网络代替线性策略，看看“学习”能否真正改变两类方法之间的强弱关系。

参考文献

[1] KNUTH D E, MOORE R W. An analysis of alpha-beta pruning[J]. Artificial Intelligence, 1975, 6(4): 293-326. DOI: 10.1016/0004-3702(75)90019-3.

[2] KOCSIS L, SZEPESVÁRI C. Bandit Based Monte-Carlo Planning[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2006: 282-293. DOI: 10.1007/11871842_29.

[3] BROWNE C B, POWLEY E, WHITEHOUSE D, et al. A Survey of Monte Carlo Tree Search Methods[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(1): 1-43. DOI: 10.1109/tciaig.2012.2186810.

[4] SHANNON C E. XXII. Programming a computer for playing chess[J]. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, 1950, 41(314): 256-275. DOI: 10.1080/14786445008521796.

[5] TURING A M. Chess[M]//Computer Chess Compendium. Springer New York, 1988: 14-17. DOI: 10.1007/978-1-4757-1968-0_2.

[6] NEWELL A, SHAW J C, SIMON H A. Chess-Playing Programs and the Problem of Complexity[J]. IBM Journal of Research and Development, 1958, 2(4): 320-335. DOI: 10.1147/rd.24.0320.

[7] SAMUEL A L. Some Studies in Machine Learning Using the Game of Checkers[J]. IBM Journal of Research and Development, 1959, 3(3): 210-229. DOI: 10.1147/rd.33.0210.

[8] SAMUEL A L. Some Studies in Machine Learning Using the Game of Checkers. II—Recent Progress[J]. IBM Journal of Research and Development, 1967, 11(6): 601-617. DOI: 10.1147/rd.116.0601.

[9] ZOBRIST A L. A New Hashing Method with Application for Game Playing1[J]. ICGA Journal, 1990, 13(2): 69-73. DOI: 10.3233/icg-1990-13203.

[10] SCHAEFFER J. The history heuristic and alpha-beta search enhancements in practice[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1989, 11(11): 1203-1212. DOI: 10.1109/34.42858.

[11] CAMPBELL M, HOANE A J, HSU F H. Deep Blue[J]. Artificial Intelligence, 2002, 134(1-2): 57-83. DOI: 10.1016/s0004-3702(01)00129-1.

[12] FENG-HSIUNG HSU. IBM’s Deep Blue Chess grandmaster chips[J]. IEEE Micro, 1999, 19(2): 70-81. DOI: 10.1109/40.755469.

[13] SCHAEFFER J, CULBERSON J, TRELOAR N, et al. A world championship caliber checkers program[J]. Artificial Intelligence, 1992, 53(2-3): 273-289. DOI: 10.1016/0004-3702(92)90074-8.

[14] SCHAEFFER J, BURCH N, BJÖRNSSON Y, et al. Checkers Is Solved[J]. Science, 2007, 317(5844): 1518-1522. DOI: 10.1126/science.1144079.

[15] TESAURO G. TD-Gammon, a Self-Teaching Backgammon Program, Achieves Master-Level Play[J]. Neural Computation, 1994, 6(2): 215-219. DOI: 10.1162/neco.1994.6.2.215.

[16] TESAURO G. Temporal Difference Learning and TD-Gammon[J]. ICGA Journal, 1995, 18(2): 88-88. DOI: 10.3233/icg-1995-18207.

[17] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529(7587): 484-489. DOI: 10.1038/nature16961.

[18] SILVER D, SCHRITTWIESER J, SIMONYAN K, et al. Mastering the game of Go without human knowledge[J]. Nature, 2017, 550(7676): 354-359. DOI: 10.1038/nature24270.

[19] SILVER D, HUBERT T, SCHRITTWIESER J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. Science, 2018, 362(6419): 1140-1144. DOI: 10.1126/science.aar6404.

[20] RUSSELL S J, NORVIG P. Artificial intelligence: A modern approach[M]. Hoboken: Pearson, 2020.

[21] YANNAKAKIS G N, TOGELIUS J. Artificial Intelligence and Games[M]. Cham: Springer, 2018. DOI: 10.1007/978-3-319-63519-4.

[22] VAN DEN HERIK H J, UITERWIJK J W H M, VAN RIJSWIJCK J. Games solved: Now and in the future[J]. Artificial Intelligence, 2002, 134(1-2): 277-311. DOI: 10.1016/s0004-3702(01)00152-7.

[23] MÜLLER M, TEGOS T. Experiments in Computer Amazons[J]. More Games of No Chance, 2002: 243-260. DOI: 10.1017/9781316135167.018.

[24] LORENTZ R J. Amazons Discover Monte-Carlo[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 13-24. DOI: 10.1007/978-3-540-87608-3_2.

[25] KLOETZER J. Monte-Carlo Opening Books for Amazons[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2011: 124-135. DOI: 10.1007/978-3-642-17928-0_12.

[26] KLOETZER J, IIDA H, BOUZY B. A comparative study of solvers in Amazons endgames[C]//2008 IEEE Symposium On Computational Intelligence and Games. IEEE, 2008: 378-384. DOI: 10.1109/cig.2008.5035665.

[27] WANG H, QIU H. The parallel optimization based on the PVS algorithm and research on the evaluation function in the Game of the Amazons[C]//2021 33rd Chinese Control and Decision Conference (CCDC). IEEE, 2021: 2201-2206. DOI: 10.1109/ccdc52312.2021.9602698.

[28] JU J, QIU H, WANG F, et al. Research on Thread Optimization and Opening Library Based on Parallel PVS Algorithm in Amazon Chess[C]//2021 33rd Chinese Control and Decision Conference (CCDC). IEEE, 2021: 2207-2212. DOI: 10.1109/ccdc52312.2021.9602204.

[29] LI Z, NING C, CAO J, et al. Amazon Chess Based on UCT-PVS Hybrid Algorithm[C]//2021 33rd Chinese Control and Decision Conference (CCDC). IEEE, 2021: 2179-2183. DOI: 10.1109/ccdc52312.2021.9602418.

[30] LIEBERUM J. An evaluation function for the game of amazons[J]. Theoretical Computer Science, 2005, 349(2): 230-244. DOI: 10.1016/j.tcs.2005.09.048.

[31] LIEBERUM J. An Evaluation Function for the Game of Amazons[M]//Advances in Computer Games. Springer US, 2004: 299-308. DOI: 10.1007/978-0-387-35706-5_19.

[32] BERLEKAMP E R. Sums of $N \times 2$ Amazons[M]//Institute of Mathematical Statistics Lecture Notes - Monograph Series. Institute of Mathematical Statistics, 2000: 1-34. DOI: 10.1214/lnms/1215089741.

[33] BURO M. Simple Amazons Endgames and Their Connection to Hamilton Circuits in Cubic Subgrid Graphs[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2001: 250-261. DOI: 10.1007/3-540-45579-5_17.

[34] SONG J, MULLER M. An Enhanced Solver for the Game of Amazons[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2015, 7(1): 16-27. DOI: 10.1109/tciaig.2014.2309077.

[35] SNATZKE R G. New results of exhaustive search in the game Amazons[J]. Theoretical Computer Science, 2004, 313(3): 499-509. DOI: 10.1016/j.tcs.2002.10.005.

[36] HEARN R A. Amazons, Konane, and Cross Purposes are PSPACE-complete[J]. Games of No Chance 3, 2009: 287-306. DOI: 10.1017/cbo9780511807251.015.

[37] HEARN R A, DEMAINE E D. Games, Puzzles, and Computation[M]. Wellesley, MA: A K Peters, 2009. DOI: 10.1201/b10581.

[38] REISCH S. Gobang ist PSPACE-vollständig[J]. Acta Informatica, 1980, 13(1): 59-66. DOI: 10.1007/bf00288536.

[39] LICHTENSTEIN D, SIPSER M. GO Is Polynomial-Space Hard[J]. Journal of the ACM, 1980, 27(2): 393-401. DOI: 10.1145/322186.322201.

[40] STOCKMEYER L J, CHANDRA A K. Provably Difficult Combinatorial Games[J]. SIAM Journal on Computing, 1979, 8(2): 151-174. DOI: 10.1137/0208013.

[41] IIDA H, MÜLLER M. Report on the second open computer-amazons championship[J]. ICGA Journal, 2000, 23(1): 51-54. DOI: 10.3233/icg-2000-23111.

[42] AVETISYAN H, LORENTZ R J. Selective Search in an Amazons Program[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2003: 123-141. DOI: 10.1007/978-3-540-40031-8_9.

[43] GUO Q, LI S, BAO H. The research of searching algorithm in Amazons game[C]//2011 Chinese Control and Decision Conference (CCDC). IEEE, 2011: 1859-1862. DOI: 10.1109/ccdc.2011.5968503.

[44] HONGKUN Q, PENG Z, YAJIE W, et al. Analysis of search algorithm in computer game of Amazons[C]//The 26th Chinese Control and Decision Conference (2014 CCDC). IEEE, 2014: 3947-3950. DOI: 10.1109/ccdc.2014.6852870.

[45] YUAN W, ZHOU K. Improvement on Amazons Search Algorithm[C]//2014 Seventh International Symposium on Computational Intelligence and Design. IEEE, 2014: 311-314. DOI: 10.1109/iscid.2014.55.

[46] GUO T, QIU H, TONG B, et al. Optimization and Comparison of Multiple Game Algorithms in Amazon Chess[C]//2019 Chinese Control And Decision Conference (CCDC). IEEE, 2019: 6299-6304. DOI: 10.1109/ccdc.2019.8833089.

[47] HONGXIA X, YUAN X, GUOYU Z. A region-divided search algorithms for game Amazons[C]//2013 25th Chinese Control and Decision Conference (CCDC). IEEE, 2013: 933-938. DOI: 10.1109/ccdc.2013.6561057.

[48] LI X, HOU L, WU L. UCT algorithm in Amazons human-computer games[C]//The 26th Chinese Control and Decision Conference (2014 CCDC). IEEE, 2014: 3358-3361. DOI: 10.1109/ccdc.2014.6852755.

[49] ZHONG R, ZHOU K. An Optimization for Amazons Search and Evaluation[C]//2014 Seventh International Symposium on Computational Intelligence and Design. IEEE, 2014: 315-318. DOI: 10.1109/iscid.2014.63.

[50] CHAI Z, FANG Z, ZHU J. Amazons Evaluation Optimization Strategy Based on PSO Algorithm[C]//2019 Chinese Control And Decision Conference (CCDC). IEEE, 2019: 6334-6336. DOI: 10.1109/ccdc.2019.8832447.

[51] QIU S, MA Y, XU Y, et al. Optimizing the evaluation parameters of Amazon chess with parallel genetic algorithm[C]//2023 35th Chinese Control and Decision Conference (CCDC). IEEE, 2023: 2298-2302. DOI: 10.1109/ccdc58219.2023.10326586.

[52] CHANG Y, QIU H. Solve game of the amazons with neural networks and distributed computing[C]//2018 Chinese Control And Decision Conference (CCDC). IEEE, 2018: 6627-6632. DOI: 10.1109/ccdc.2018.8408297.

[53] MENG D, JIANBO B, YIZHONG Q, et al. Design of Amazon Chess Game System Based on Reinforcement Learning[C]//2019 Chinese Control And Decision Conference (CCDC). IEEE, 2019: 6337-6342. DOI: 10.1109/ccdc.2019.8832999.

[54] JIANBO B, MENG D, YIZHONG Q, et al. Design of Amazon Chess Evaluation Function Based on Reinforcement Learning[C]//2019 Chinese Control And Decision Conference (CCDC). IEEE, 2019: 6330-6333. DOI: 10.1109/ccdc.2019.8832463.

[55] HUANG H, LI S. The Application of Reinforcement Learning in Amazons[C]//2019 International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI). IEEE, 2019: 369-372. DOI: 10.1109/mlbdbi48998.2019.00083.

[56] SUN Y, YUAN D, GAO M, et al. GPU Acceleration of Monte Carlo Tree Search Algorithm for Amazon chess and Its Evaluation Function[C]//2022 International Conference on Artificial Intelligence, Information Processing and Cloud Computing (AIIPCC). IEEE, 2022: 434-440. DOI: 10.1109/aiipcc57291.2022.00098.

[57] ZHANG L, ZOU H, ZHU Y. An Efficient Optimization of the Monte Carlo Tree Search Algorithm for Amazons[J]. Algorithms, 2024, 17(8): 334. DOI: 10.3390/a17080334.

[58] TIAN C. Monte-Carlo tree search with Epsilon-Greedy for game of amazons[J]. Applied and Computational Engineering, 2023, 6(1): 775-780. DOI: 10.54254/2755-2721/6/20230956.

[59] SUN Y, GAO M. Overview of Amazon chess' game playing algorithms[C]//3rd International Conference on Applied Mathematics, Modelling, and Intelligent Computing (CAMMIC 2023). SPIE, 2023: 170. DOI: 10.1117/12.2686190.

[60] N. E, VON NEUMANN J, MORGENSTERN O. Theory of Games and Economic Behavior[J]. The Journal of Philosophy, 1945, 42(20): 550. DOI: 10.2307/2019327.

[61] V. NEUMANN J. Zur Theorie der Gesellschaftsspiele[J]. Mathematische Annalen, 1928, 100(1): 295-320. DOI: 10.1007/bf01448847.

[62] SCHAEFER T J. On the complexity of some two-person perfect-information games[J]. Journal of Computer and System Sciences, 1978, 16(2): 185-225. DOI: 10.1016/0022-0000(78)90045-4.

[63] SADIKOV A, BRATKO I, KONONENKO I. Search versus Knowledge: An Empirical Study of Minimax on KRK[M]//Advances in Computer Games. Springer US, 2004: 33-44. DOI: 10.1007/978-0-387-35706-5_3.

[64] SADIKOV A, BRATKO I, KONONENKO I. Bias and pathology in minimax search[J]. Theoretical Computer Science, 2005, 349(2): 268-281. DOI: 10.1016/j.tcs.2005.09.073.

[65] MICHIE D. Information and complexity in chess[M]//Advances in Computer Chess. Elsevier, 1982: 139-143. DOI: 10.1016/b978-0-08-026898-9.50014-0.

[66] COHEN P R. Empirical methods for artificial intelligence[M]. Cambridge, MA: MIT Press, 1995.

[67] HENDERSON P, ISLAM R, BACHMAN P, et al. Deep Reinforcement Learning That Matters[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2018, 32(1). DOI: 10.1609/aaai.v32i1.11694.

[68] KORF R E. Depth-first iterative-deepening[J]. Artificial Intelligence, 1985, 27(1): 97-109. DOI: 10.1016/0004-3702(85)90084-0.

[69] SLATE D J, ATKIN L R. CHESS 4.5—The Northwestern University chess program[M]//Chess Skill in Man and Machine. Springer Berlin Heidelberg, 1977: 82-118. DOI: 10.1007/978-3-662-06239-5_4.

[70] REINEFELD A. An Improvement to the Scout Tree Search Algorithm[J]. ICGA Journal, 1983, 6(4): 4-14. DOI: 10.3233/icg-1983-6402.

[71] MARSLAND T A. A Review of Game-Tree Pruning1[J]. ICGA Journal, 1986, 9(1): 3-19. DOI: 10.3233/icg-1986-9102.

[72] BREUKER D M, UITERWIJK J W H M, VAN DEN HERIK H J. Replacement Schemes for Transposition Tables[J]. ICGA Journal, 1994, 17(4): 183-193. DOI: 10.3233/icg-1994-17402.

[73] AKL S G, NEWBORN M M. The principal continuation and the killer heuristic[C]//Proceedings of the 1977 annual conference on - ACM '77. ACM Press, 1977: 466-473. DOI: 10.1145/800179.810240.

[74] BAIER H, WINANDS M H M. Beam Monte-Carlo Tree Search[C]//2012 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 2012: 227-233. DOI: 10.1109/cig.2012.6374160.

[75] COULOM R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2007: 72-83. DOI: 10.1007/978-3-540-75538-8_7.

[76] CHASLOT G M J B, WINANDS M H M, UITERWIJK J W H M, et al. Progressive strategies for monte-carlo tree search[M]//Information Sciences 2007. World scientific, 2007: 655-661. DOI: 10.1142/9789812709677_0246.

[77] COULOM R. Computing "elo ratings" of move patterns in the game of go1[J]. ICGA Journal, 2007, 30(4): 198-208. DOI: 10.3233/icg-2007-30403.

[78] LORENTZ R. Early Playout Termination in MCTS[M]//Lecture Notes in Computer Science. Springer International Publishing, 2015: 12-19. DOI: 10.1007/978-3-319-27992-3_2.

[79] LORENTZ R. Using evaluation functions in Monte-Carlo Tree Search[J]. Theoretical Computer Science, 2016, 644: 106-113. DOI: 10.1016/j.tcs.2016.06.026.

[80] ROSIN C D. Multi-armed bandits with episode context[J]. Annals of Mathematics and Artificial Intelligence, 2011, 61(3): 203-230. DOI: 10.1007/s10472-011-9258-6.

[81] SUPPES P, LUCE R D. Individual Choice Behavior A Theoretical Analysis.[J]. Journal of the American Statistical Association, 1961, 56(293): 172. DOI: 10.2307/2282347.

[82] ARAKI N, YOSHIDA K, TSURUOKA Y, et al. Move Prediction in Go with the Maximum Entropy Method[C]//2007 IEEE Symposium on Computational Intelligence and Games. IEEE, 2007. DOI: 10.1109/cig.2007.368097.

[83] BURO M. Statistical Feature Combination for the Evaluation of Game Positions[J]. Journal of Artificial Intelligence Research, 1995, 3: 373-382. DOI: 10.1613/jair.179.

[84] HOKI K, KANEKO T. Large-Scale Optimization for Evaluation Functions with Minimax Search[J]. Journal of Artificial Intelligence Research, 2014, 49: 527-568. DOI: 10.1613/jair.4217.

[85] BRADLEY R A, TERRY M E. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons[J]. Biometrika, 1952, 39(3/4): 324. DOI: 10.2307/2334029.

[86] ELO A E. The rating of chessplayers, past and present[M]. New York: Arco, 1978.

[87] EFRON B. An introduction to the bootstrap[M]. New York: Chapman & Hall, 1993.

[88] GELLY S, KOCSIS L, SCHOENAUER M, et al. The grand challenge of computer Go: Monte Carlo tree search and extensions[J]. Communications of the ACM, 2012, 55(3): 106-113. DOI: 10.1145/2093548.2093574.

[89] RAMANUJAN R, SABHARWAL A, SELMAN B. On Adversarial Search Spaces and Sampling-Based Planning[J]. Proceedings of the International Conference on Automated Planning and Scheduling, 2010, 20: 242-245. DOI: 10.1609/icaps.v20i1.13437.

[90] RAMANUJAN R, SELMAN B. Trade-Offs in Sampling-Based Adversarial Planning[J]. Proceedings of the International Conference on Automated Planning and Scheduling, 2011, 21: 202-209. DOI: 10.1609/icaps.v21i1.13472.

[91] BAIER H, WINANDS M H M. Monte-Carlo Tree Search and minimax hybrids[C]//2013 IEEE Conference on Computational Inteligence in Games (CIG). IEEE, 2013: 1-8. DOI: 10.1109/cig.2013.6633630.

[92] BAIER H, WINANDS M H M. MCTS-Minimax Hybrids[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2015, 7(2): 167-179. DOI: 10.1109/tciaig.2014.2366555.

[93] LANCTOT M, WINANDS M H M, PEPELS T, et al. Monte Carlo Tree Search with heuristic evaluations using implicit minimax backups[C]//2014 IEEE Conference on Computational Intelligence and Games. IEEE, 2014: 1-8. DOI: 10.1109/cig.2014.6932903.

[94] WINANDS M H M, BJORNSSON Y. αβ-based play-outs in Monte-Carlo Tree Search[C]//2011 IEEE Conference on Computational Intelligence and Games (CIG'11). IEEE, 2011: 110-117. DOI: 10.1109/cig.2011.6031996.

[95] KNUTH D. The art of computer programming[M]. Reading, MA: Addison-Wesley, 1998.

[96] CARTER J L, WEGMAN M N. Universal classes of hash functions[J]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.

[97] MARSAGLIA G. Xorshift RNGs[J]. Journal of Statistical Software, 2003, 8(14). DOI: 10.18637/jss.v008.i14.

[98] MATSUMOTO M, NISHIMURA T. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator[J]. ACM Transactions on Modeling and Computer Simulation, 1998, 8(1): 3-30. DOI: 10.1145/272991.272995.

[99] PEARL J. Asymptotic properties of minimax trees and game-searching procedures[J]. Artificial Intelligence, 1980, 14(2): 113-138. DOI: 10.1016/0004-3702(80)90037-5.

[100] PEARL J. The solution for the branching factor of the alpha-beta pruning algorithm and its optimality[J]. Communications of the ACM, 1982, 25(8): 559-564. DOI: 10.1145/358589.358616.

[101] CAMPBELL M S, MARSLAND T A. A comparison of minimax tree search algorithms[J]. Artificial Intelligence, 1983, 20(4): 347-367. DOI: 10.1016/0004-3702(83)90001-2.

[102] STOCKMAN G C. A minimax algorithm better than alpha-beta?[J]. Artificial Intelligence, 1979, 12(2): 179-196. DOI: 10.1016/0004-3702(79)90016-x.

[103] PLAAT A, SCHAEFFER J, PIJLS W, et al. Best-first fixed-depth minimax algorithms[J]. Artificial Intelligence, 1996, 87(1-2): 255-293. DOI: 10.1016/0004-3702(95)00126-3.

[104] DE BRUIN A, PIJLS W, PLAAT A. Solution Trees as a Basis for Game-Tree Search[J]. ICGA Journal, 1994, 17(4): 207-219. DOI: 10.3233/icg-1994-17404.

[105] BEAL D F, SMITH M C. Multiple Probes of Transposition Tables[J]. ICGA Journal, 1996, 19(4): 227-233. DOI: 10.3233/icg-1996-19404.

[106] DONNINGER C. Null Move and Deep Search[J]. ICGA Journal, 1993, 16(3): 137-143. DOI: 10.3233/icg-1993-16304.

[107] HEINZ E A. Adaptive Null-Move Pruning[J]. ICGA Journal, 1999, 22(3): 123-132. DOI: 10.3233/icg-1999-22302.

[108] TABIBI O D, NETANYAHU N S. Verified null-move pruning[J]. ICGA Journal, 2002, 25(3): 153-161. DOI: 10.3233/icg-2002-25305.

[109] HOKI K, MURAMATSU M. Efficiency of three forward-pruning techniques in shogi: Futility pruning, null-move pruning, and Late Move Reduction (LMR)[J]. Entertainment Computing, 2012, 3(3): 51-57. DOI: 10.1016/j.entcom.2011.11.003.

[110] ABRAMSON B. Expected-outcome: a general model of static evaluation[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1990, 12(2): 182-193. DOI: 10.1109/34.44404.

[111] TESAURO G, GALPERIN G R. On-line Policy Improvement using Monte-Carlo Search[C]//Advances in Neural Information Processing Systems. 1996. https://proceedings.neurips.cc/paper_files/paper/1996/hash/996009f2374006606f4c0b0fda878af1-Abstract.html.

[112] BOUZY B, HELMSTETTER B. Monte-Carlo Go Developments[M]//Advances in Computer Games. Springer US, 2004: 159-174. DOI: 10.1007/978-0-387-35706-5_11.

[113] AUER P, CESA-BIANCHI N, FISCHER P. Finite-time Analysis of the Multiarmed Bandit Problem[J]. Machine Learning, 2002, 47(2-3): 235-256. DOI: 10.1023/a:1013689704352.

[114] LAI T L, ROBBINS H. Asymptotically efficient adaptive allocation rules[J]. Advances in Applied Mathematics, 1985, 6(1): 4-22. DOI: 10.1016/0196-8858(85)90002-8.

[115] BUBECK S. Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems[M]. now Publishers Inc, 2012. DOI: 10.1561/9781601986276.

[116] LATTIMORE T, SZEPESVÁRI C. Bandit Algorithms[M]. Cambridge: Cambridge University Press, 2020. DOI: 10.1017/9781108571401.

[117] GELLY S, SILVER D. Combining online and offline knowledge in UCT[C]//Proceedings of the 24th international conference on Machine learning. ACM, 2007: 273-280. DOI: 10.1145/1273496.1273531.

[118] GELLY S, SILVER D. Monte-Carlo tree search and rapid action value estimation in computer Go[J]. Artificial Intelligence, 2011, 175(11): 1856-1875. DOI: 10.1016/j.artint.2011.03.007.

[119] ENZENBERGER M, MULLER M, ARNESON B, et al. Fuego—An Open-Source Framework for Board Games and Go Engine Based on Monte Carlo Tree Search[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2010, 2(4): 259-270. DOI: 10.1109/tciaig.2010.2083662.

[120] CHEN K H, DU D, ZHANG P. Monte-Carlo Tree Search and Computer Go[M]//Studies in Computational Intelligence. Springer Berlin Heidelberg, 2009: 201-225. DOI: 10.1007/978-3-642-04141-9_10.

[121] ARNESON B, HAYWARD R B, HENDERSON P. Monte Carlo Tree Search in Hex[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2010, 2(4): 251-258. DOI: 10.1109/tciaig.2010.2067212.

[122] WINANDS M H M, BJORNSSON Y, SAITO J T. Monte Carlo Tree Search in Lines of Action[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2010, 2(4): 239-250. DOI: 10.1109/tciaig.2010.2061050.

[123] FINNSSON H. Generalized Monte-Carlo Tree Search Extensions for General Game Playing[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2021, 26(1): 1550-1556. DOI: 10.1609/aaai.v26i1.8330.

[124] MEHAT J, CAZENAVE T. Combining UCT and Nested Monte Carlo Search for Single-Player General Game Playing[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2010, 2(4): 271-277. DOI: 10.1109/tciaig.2010.2088123.

[125] COWLING P I, POWLEY E J, WHITEHOUSE D. Information Set Monte Carlo Tree Search[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(2): 120-143. DOI: 10.1109/tciaig.2012.2200894.

[126] URIARTE A, ONTAÑÓN S. High-Level Representations for Game-Tree Search in RTS Games[J]. Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 2014, 10(2): 14-18. DOI: 10.1609/aiide.v10i2.12734.

[127] ŚWIECHOWSKI M, GODLEWSKI K, SAWICKI B, et al. Monte Carlo Tree Search: a review of recent modifications and applications[J]. Artificial Intelligence Review, 2023, 56(3): 2497-2562. DOI: 10.1007/s10462-022-10228-y.

[128] WINANDS M H M, BJÖRNSSON Y. Evaluation Function Based Monte-Carlo LOA[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2010: 33-44. DOI: 10.1007/978-3-642-12993-3_4.

[129] MATSUZAKI K, KITAMURA N. Do evaluation functions really improve Monte-Carlo tree search?[J]. ICGA Journal, 2019, 40(3): 294-304. DOI: 10.3233/icg-180060.

[130] PLACKETT R L. The Analysis of Permutations[J]. Applied Statistics, 1975, 24(2): 193. DOI: 10.2307/2346567.

[131] HUNTER D R. MM algorithms for generalized Bradley-Terry models[J]. The Annals of Statistics, 2004, 32(1): 384-406. DOI: 10.1214/aos/1079120141.

[132] STERN D, HERBRICH R, GRAEPEL T. Bayesian pattern ranking for move prediction in the game of Go[C]//Proceedings of the 23rd international conference on Machine learning - ICML '06. ACM Press, 2006: 873-880. DOI: 10.1145/1143844.1143954.

[133] MADDISON C J, HUANG A, SUTSKEVER I, et al. Move Evaluation in Go Using Deep Convolutional Neural Networks[EB/OL]. arXiv: 1412.6564, 2014. https://arxiv.org/abs/1412.6564.

[134] CLARK C, STORKEY A. Teaching Deep Convolutional Neural Networks to Play Go[EB/OL]. arXiv: 1412.3409, 2014. https://arxiv.org/abs/1412.3409.

[135] BISHOP C M. Pattern Recognition and Machine Learning (Information Science and Statistics)[M]. New York: Springer, 2008.

[136] HASTIE T, TIBSHIRANI R, FRIEDMAN J. The Elements of Statistical Learning[M]. New York: Springer, 2009. DOI: 10.1007/978-0-387-84858-7.

[137] GOODFELLOW I, BENGIO Y, COURVILLE A, et al. Deep Learning[M]. Cambridge, MA: MIT Press, 2016.

[138] KINGMA D P, BA J. Adam: A Method for Stochastic Optimization[EB/OL]. arXiv: 1412.6980, 2014. https://arxiv.org/abs/1412.6980.

[139] BURO M. From Simple Features to Sophisticated Evaluation Functions[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 1999: 126-145. DOI: 10.1007/3-540-48957-6_8.

[140] BURO M. Improving heuristic mini-max search by supervised learning[J]. Artificial Intelligence, 2002, 134(1-2): 85-99. DOI: 10.1016/s0004-3702(01)00093-5.

[141] TESAURO G. Connectionist Learning of Expert Preferences by Comparison Training[C]//Advances in Neural Information Processing Systems. 1988. https://proceedings.neurips.cc/paper_files/paper/1988/hash/a8baa56554f96369ab93e4f3bb068c22-Abstract.html.

[142] SUTTON R S. Learning to Predict by the Methods of Temporal Differences[J]. Machine Learning, 1988, 3(1): 9-44. DOI: 10.1023/a:1022633531479.

[143] SCHRAUDOLPH N N, DAYAN P, SEJNOWSKI T J. Temporal Difference Learning of Position Evaluation in the Game of Go[C]//Advances in Neural Information Processing Systems. 1993. https://proceedings.neurips.cc/paper_files/paper/1993/hash/e2a2dcc36a08a345332c751b2f2e476c-Abstract.html.

[144] THRUN S. Learning to Play the Game of Chess[C]//Advances in Neural Information Processing Systems. 1994. https://proceedings.neurips.cc/paper_files/paper/1994/hash/d7322ed717dedf1eb4e6e52a37ea7bcd-Abstract.html.

[145] LUCAS S, RUNARSSON T. Temporal Difference Learning Versus Co-Evolution for Acquiring Othello Position Evaluation[C]//2006 IEEE Symposium on Computational Intelligence and Games. IEEE, 2006: 52-59. DOI: 10.1109/cig.2006.311681.

[146] VENESS J, SILVER D, BLAIR A, et al. Bootstrapping from Game Tree Search[C]//Advances in Neural Information Processing Systems. 2009. https://proceedings.neurips.cc/paper_files/paper/2009/hash/389bc7bb1e1c2a5e7e147703232a88f6-Abstract.html.

[147] BOSKOVIC B, GREINER S, BREST J, et al. A Differential Evolution for the Tuning of a Chess Evaluation Function[C]//2006 IEEE International Conference on Evolutionary Computation. IEEE, 2006: 1851-1856. DOI: 10.1109/cec.2006.1688532.

[148] KENDALL G, WHITWELL G. An evolutionary approach for the tuning of a chess evaluation function using population dynamics[C]//Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546). IEEE, 2001: 995-1002. DOI: 10.1109/cec.2001.934299.

[149] VAZQUEZ-FERNANDEZ E, COELLO C A C, TRONCOSO F D S. An evolutionary algorithm for tuning a chess evaluation function[C]//2011 IEEE Congress of Evolutionary Computation (CEC). IEEE, 2011: 842-848. DOI: 10.1109/cec.2011.5949706.

[150] NOCEDAL J, WRIGHT S J. Numerical Optimization[M]. New York: Springer, 2006. DOI: 10.1007/978-0-387-40065-5.

[151] ANTHONY T, TIAN Z, BARBER D. Thinking Fast and Slow with Deep Learning and Tree Search[EB/OL]. arXiv: 1705.08439, 2017. https://arxiv.org/abs/1705.08439.

[152] WU D J. Accelerating Self-Play Learning in Go[EB/OL]. arXiv: 1902.10565, 2019. https://arxiv.org/abs/1902.10565.

[153] COULOM R. Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 113-124. DOI: 10.1007/978-3-540-87608-3_11.

[154] GLICKMAN M E. Parameter Estimation in Large Dynamic Paired Comparison Experiments[J]. Journal of the Royal Statistical Society Series C: Applied Statistics, 1999, 48(3): 377-394. DOI: 10.1111/1467-9876.00159.

[155] HERBRICH R, MINKA T, GRAEPEL T. TrueSkill™: A Bayesian Skill Rating System[M]//Advances in Neural Information Processing Systems 19. The MIT Press, 2007: 569-576. DOI: 10.7551/mitpress/7503.003.0076.

[156] BERG A. Statistical Analysis of the Elo Rating System in Chess[J]. CHANCE, 2020, 33(3): 31-38. DOI: 10.1080/09332480.2020.1820249.

[157] CHAN V. MSE-optimal K-factor of the Elo rating system for round-robin tournament[J]. Journal of Quantitative Analysis in Sports, 2022, 18(1): 59-72. DOI: 10.1515/jqas-2021-0079.

[158] OLESKER-TAYLOR S, ZANETTI L. An Analysis of Elo Rating Systems via Markov Chains[C]//Advances in Neural Information Processing Systems 37. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024: 138289-138323. DOI: 10.52202/079017-4389.

[159] HAMILTON A H, KALENKOVA A, ROUGHAN M. The impact of intransitivity on the Elo rating system[J]. PLOS One, 2025, 20(12): e0338261. DOI: 10.1371/journal.pone.0338261.

[160] SZCZECINSKI L, DJEBBI A. Understanding draws in Elo rating algorithm[J]. Journal of Quantitative Analysis in Sports, 2020, 16(3): 211-220. DOI: 10.1515/jqas-2019-0102.

[161] WILSON E B. Probable Inference, the Law of Succession, and Statistical Inference[J]. Journal of the American Statistical Association, 1927, 22(158): 209-212. DOI: 10.1080/01621459.1927.10502953.

[162] AGRESTI A, COULL B A. Approximate Is Better than “Exact” for Interval Estimation of Binomial Proportions[J]. The American Statistician, 1998, 52(2): 119. DOI: 10.2307/2685469.

[163] CLOPPER C J, PEARSON E S. The use of confidence or fiducial limits illustrated in the case of the binomial[J]. Biometrika, 1934, 26(4): 404-413. DOI: 10.1093/biomet/26.4.404.

[164] EFRON B. Bootstrap Methods: Another Look at the Jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26. DOI: 10.1214/aos/1176344552.

[165] DEMŠAR J. Statistical Comparisons of Classifiers over Multiple Data Sets[J]. Journal of Machine Learning Research, 2006, 7: 1-30. https://jmlr.org/papers/v7/demsar06a.html.

[166] AGARWAL R, SCHWARZER M, CASTRO P S, et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice[EB/OL]. arXiv: 2108.13264, 2021. https://arxiv.org/abs/2108.13264.

[167] BRIER G W. Verification of forecasts expressed in terms of probability[J]. Monthly Weather Review, 1950, 78(1): 1-3. DOI: 10.1175/1520-0493(1950)078<0001:vofeit>2.0.co;2.

[168] DEGROOT M H, FIENBERG S E. The Comparison and Evaluation of Forecasters[J]. The Statistician, 1983, 32(1/2): 12. DOI: 10.2307/2987588.

[169] NICULESCU-MIZIL A, CARUANA R. Predicting good probabilities with supervised learning[C]//Proceedings of the 22nd international conference on Machine learning - ICML ’05. ACM Press, 2005: 625-632. DOI: 10.1145/1102351.1102430.

[170] GUO C, PLEISS G, SUN Y, et al. On Calibration of Modern Neural Networks[EB/OL]. arXiv: 1706.04599, 2017. https://arxiv.org/abs/1706.04599.

[171] GNEITING T, RAFTERY A E. Strictly Proper Scoring Rules, Prediction, and Estimation[J]. , 2005. DOI: 10.21236/ada454828.

[172] J. N N. Principles of artificial intelligence[M]. Palo Alto: Tioga, 1980.

[173] MILLINGTON I. AI for Games[M]. Boca Raton: CRC Press, 2019. DOI: 10.1201/9781351053303.

[174] YANNAKAKIS G N, TOGELIUS J. A Panorama of Artificial and Computational Intelligence in Games[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2015, 7(4): 317-335. DOI: 10.1109/tciaig.2014.2339221.

[175] MARSLAND T A. Single-Agent and Game-Tree Search[J]. ICGA Journal, 1991, 14(3): 149-149. DOI: 10.3233/icg-1991-14311.

[176] IIDA H. Heuristic Theories on Game-Tree Search[J]. ICGA Journal, 1994, 17(4): 220-220. DOI: 10.3233/icg-1994-17407.

[177] MARSLAND T A. Relative Performance of the Alpha-Beta Algorithm[J]. ICGA Journal, 1982, 5(2): 21-24. DOI: 10.3233/icg-1982-5213.

[178] BEAL D F. Benefits of minimax search[M]//Advances in Computer Chess. Elsevier, 1982: 17-24. DOI: 10.1016/b978-0-08-026898-9.50005-x.

[179] MUTCHLER D. The multi-player version of minimax displays game-tree pathology[J]. Artificial Intelligence, 1993, 64(2): 323-336. DOI: 10.1016/0004-3702(93)90108-n.

[180] ALTHÖFER I, BALKENHOL B. A game tree with distinct leaf values which is easy for the alpha-beta algorithm[J]. Artificial Intelligence, 1991, 52(2): 183-190. DOI: 10.1016/0004-3702(91)90042-i.

[181] THOMPSON K. Computer chess strength[M]//Advances in Computer Chess. Elsevier, 1982: 55-56. DOI: 10.1016/b978-0-08-026898-9.50008-5.

[182] FINK W. An Enhancement to the Iterative, Alpha-Beta, Minimax Search Procedure[J]. ICGA Journal, 1982, 5(1): 34-35. DOI: 10.3233/icg-1982-5107.

[183] GREER K R C, OJHA P C, BELL D A. A Pattern-Oriented Approach to Move Ordering: The Chessmaps Heuristic[J]. ICGA Journal, 1999, 22(1): 13-21. DOI: 10.3233/icg-1999-22103.

[184] GREER K. Computer chess move-ordering schemes using move influence[J]. Artificial Intelligence, 2000, 120(2): 235-250. DOI: 10.1016/s0004-3702(00)00026-6.

[185] KE Y F, PARNG T M. The Guard Heuristic: Legal Move Ordering with Forward Game-Tree Pruning[J]. ICGA Journal, 1993, 16(2): 76-85. DOI: 10.3233/icg-1993-16203.

[186] SLATE D J. A Chess Program that Uses its Transposition Table to Learn from Experience[J]. ICGA Journal, 1987, 10(2): 59-71. DOI: 10.3233/icg-1987-10202.

[187] VENESS J, BLAIR A. Effective Use of Transposition Tables in Stochastic Game Tree Search[C]//2007 IEEE Symposium on Computational Intelligence and Games. IEEE, 2007: 112-116. DOI: 10.1109/cig.2007.368086.

[188] HYATT R M, MANN T. A lockless transposition-table implementation for parallel search[J]. ICGA Journal, 2002, 25(1): 36-39. DOI: 10.3233/icg-2002-25104.

[189] BEAL D F. A generalised quiescence search algorithm[J]. Artificial Intelligence, 1990, 43(1): 85-98. DOI: 10.1016/0004-3702(90)90072-8.

[190] SCHRÜFER G. A Strategic Quiescence Search[J]. ICGA Journal, 1989, 12(1): 3-9. DOI: 10.3233/icg-1989-12102.

[191] BEAL D F. Mating Sequences in the Quiescence Search[J]. ICGA Journal, 1984, 7(3): 133-137. DOI: 10.3233/icg-1984-7302.

[192] ANANTHARAMAN T, CAMPBELL M, HSU F H. Singular Extensions: Adding Selectivity to Brute-Force Searching1[J]. ICGA Journal, 1988, 11(4): 135-143. DOI: 10.3233/icg-1988-11402.

[193] GOETSCH G, CAMPBELL M S. Experiments with the Null-Move Heuristic[M]//Computers, Chess, and Cognition. Springer New York, 1990: 159-168. DOI: 10.1007/978-1-4613-9080-0_9.

[194] TSURUOKA Y, YOKOYAMA D, CHIKAYAMA T. Game-tree search algorithm based on realization probability[J]. ICGA Journal, 2002, 25(3): 145-152. DOI: 10.3233/icg-2002-25304.

[195] HART P, NILSSON N, RAPHAEL B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107. DOI: 10.1109/tssc.1968.300136.

[196] BERLINER H. The B∗ tree search algorithm: A best-first proof procedure[J]. Artificial Intelligence, 1979, 12(1): 23-40. DOI: 10.1016/0004-3702(79)90003-1.

[197] MCALLESTER D A. Conspiracy numbers for min-max search[J]. Artificial Intelligence, 1988, 35(3): 287-310. DOI: 10.1016/0004-3702(88)90019-7.

[198] MCALLESTER D, YURET D. Alpha-beta-conspiracy search1[J]. ICGA Journal, 2002, 25(1): 16-35. DOI: 10.3233/icg-2002-25103.

[199] RIVEST R L. Game tree searching by min/max approximation[J]. Artificial Intelligence, 1987, 34(1): 77-96. DOI: 10.1016/0004-3702(87)90004-x.

[200] KORF R E, CHICKERING D M. Best-first minimax search[J]. Artificial Intelligence, 1996, 84(1-2): 299-337. DOI: 10.1016/0004-3702(95)00096-8.

[201] MÜLLER M. Partial order bounding: A new approach to evaluation in game tree search[J]. Artificial Intelligence, 2001, 129(1-2): 279-311. DOI: 10.1016/s0004-3702(01)00085-6.

[202] JUNGHANNS A. Are there Practical Alternatives to Alpha-Beta?[J]. ICGA Journal, 1998, 21(1): 14-32. DOI: 10.3233/icg-1998-21103.

[203] MAŃDZIUK J, OSMAN D. Alpha-beta search enhancements with a real-value game-state evaluation function[J]. ICGA Journal, 2004, 27(1): 38-43. DOI: 10.3233/icg-2004-27104.

[204] ADELSON-VELSKIY G M, ARLAZAROV V L, DONSKOY M V. Some methods of controlling the tree search in chess programs[J]. Artificial Intelligence, 1975, 6(4): 361-371. DOI: 10.1016/0004-3702(75)90021-1.

[205] BALLARD B W. The *-minimax search procedure for trees containing chance nodes[J]. Artificial Intelligence, 1983, 21(3): 327-350. DOI: 10.1016/s0004-3702(83)80015-0.

[206] MARSLAND T A, POPOWICH F. Parallel Game-Tree Search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1985, PAMI-7(4): 442-452. DOI: 10.1109/tpami.1985.4767683.

[207] HYATT R M. A High-Performance Parallel Algorithm to Search Depth-First Game Trees[J]. ICGA Journal, 1988, 11(4): 165-166. DOI: 10.3233/icg-1988-11411.

[208] NEWBORN M. Unsynchronized iteratively deepening parallel alpha-beta search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1988, 10(5): 687-694. DOI: 10.1109/34.6777.

[209] BROCKINGTON M G. A Taxonomy of Parallel Game-Tree Search Algorithms[J]. ICGA Journal, 1996, 19(3): 162-174. DOI: 10.3233/icg-1996-19303.

[210] THOMPSON K. Retrograde Analysis of Certain Endgames[J]. ICGA Journal, 1986, 9(3): 131-139. DOI: 10.3233/icg-1986-9302.

[211] NEWBORN M. The History and Basic Ideas of Computer Chess[M]//Computer Chess. Elsevier, 1975: 5-29. DOI: 10.1016/b978-0-12-517250-9.50007-2.

[212] HEINZ E A. Computer-Chess Primer[M]//Scalable Search in Computer Chess. Vieweg+Teubner Verlag, 2000: 7-26. DOI: 10.1007/978-3-322-90178-1_2.

[213] MARSLAND T A. New Directions in Game-Tree Search[J]. ICGA Journal, 1989, 12(2): 83-83. DOI: 10.3233/icg-1989-12207.

[214] FERREIRA D R. The Impact of the Search Depth on Chess Playing Strength[J]. ICGA Journal, 2013, 36(2): 67-80. DOI: 10.3233/icg-2013-36202.

[215] HEINZ E A. Self-play, deep search and diminishing returns[J]. ICGA Journal, 2001, 24(2): 75-79. DOI: 10.3233/icg-2001-24205.

[216] ALLIS L V, VAN DER MEULEN M, VAN DEN HERIK H J. Proof-number search[J]. Artificial Intelligence, 1994, 66(1): 91-124. DOI: 10.1016/0004-3702(94)90004-3.

[217] ALUS L V, VAN DEN HERIK, H J, HUNTJENS M P H. Go-moku solved by new search techniques[J]. Computational Intelligence, 1996, 12(1): 7-23. DOI: 10.1111/j.1467-8640.1996.tb00250.x.

[218] KISHIMOTO A, WINANDS M H M, MÜLLER M, et al. Game-Tree Search Using Proof Numbers: The First Twenty Years[J]. ICGA Journal, 2012, 35(3): 131-156. DOI: 10.3233/icg-2012-35302.

[219] VAN DEN HERIK H J, WINANDS M H M. Proof-Number Search and Its Variants[M]//Studies in Computational Intelligence. Springer Berlin Heidelberg, 2008: 91-118. DOI: 10.1007/978-3-540-70829-2_6.

[220] WINANDS M H M, UITERWIJK J W H M, VAN DEN HERIK H J. An effective two-level proof-number search algorithm[J]. Theoretical Computer Science, 2004, 313(3): 511-525. DOI: 10.1016/j.tcs.2002.10.006.

[221] SEO M, IIDA H, UITERWIJK J W H M. The PN∗-search algorithm: Application to tsume-shogi[J]. Artificial Intelligence, 2001, 129(1-2): 253-277. DOI: 10.1016/s0004-3702(01)00084-4.

[222] SCHIJF M, ALLIS L V, UITERWIJK J W H M. Proof-Number Search and Transpositions[J]. ICGA Journal, 1994, 17(2): 63-74. DOI: 10.3233/icg-1994-17203.

[223] KISHIMOTO A, MÜLLER M. A solution to the GHI problem for depth-first proof-number search[J]. Information Sciences, 2005, 175(4): 296-314. DOI: 10.1016/j.ins.2004.04.012.

[224] KISHIMOTO A, MÜLLER M. About the Completeness of Depth-First Proof-Number Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 146-156. DOI: 10.1007/978-3-540-87608-3_14.

[225] KISHIMOTO A. Dealing with Infinite Loops, Underestimation, and Overestimation of Depth-First Proof-Number Search[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2010, 24(1): 108-113. DOI: 10.1609/aaai.v24i1.7534.

[226] YOSHIZOE K. A New Proof-Number Calculation Technique for Proof-Number Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 135-145. DOI: 10.1007/978-3-540-87608-3_13.

[227] ISHITOBI T, PLAAT A, IIDA H, et al. Reducing the Seesaw Effect with Deep Proof-Number Search[M]//Lecture Notes in Computer Science. Springer International Publishing, 2015: 185-197. DOI: 10.1007/978-3-319-27992-3_17.

[228] KANEKO T. Parallel Depth First Proof Number Search[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2010, 24(1): 95-100. DOI: 10.1609/aaai.v24i1.7551.

[229] PAWLEWICZ J, HAYWARD R B. Scalable Parallel DFPN Search[M]//Lecture Notes in Computer Science. Springer International Publishing, 2014: 138-150. DOI: 10.1007/978-3-319-09165-5_12.

[230] PRIMANITA A, KHALID M N A, IIDA H. Characterizing the Nature of Probability-Based Proof Number Search: A Case Study in the Othello and Connect Four Games[J]. Information, 2020, 11(5): 264. DOI: 10.3390/info11050264.

[231] GAO C, MÜLLER M, HAYWARD R. Focused Depth-first Proof Number Search using Convolutional Neural Networks for the Game of Hex[C]//Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence. International Joint Conferences on Artificial Intelligence Organization, 2017: 3668-3674. DOI: 10.24963/ijcai.2017/513.

[232] SAFFIDINE A, JOUANDEAU N, CAZENAVE T. Solving breakthrough with Race Patterns and Job-Level Proof Number Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2012: 196-207. DOI: 10.1007/978-3-642-31866-5_17.

[233] SAITO J T, CHASLOT G, UITERWIJK J W H M, et al. Monte-Carlo Proof-Number Search for Computer Go[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2007: 50-61. DOI: 10.1007/978-3-540-75538-8_5.

[234] DOE E, WINANDS M H M, SOEMERS D J N J, et al. Combining Monte-Carlo Tree Search with Proof-Number Search[C]//2022 IEEE Conference on Games (CoG). IEEE, 2022: 206-212. DOI: 10.1109/cog51982.2022.9893635.

[235] KOWALSKI J, DOE E, WINANDS M H M, et al. Proof Number-Based Monte Carlo Tree Search[J]. IEEE Transactions on Games, 2025, 17(1): 148-157. DOI: 10.1109/tg.2024.3403750.

[236] THOMSEN T. Lambda-search in game trees – with application to go[J]. ICGA Journal, 2000, 23(4): 203-217. DOI: 10.3233/icg-2000-23402.

[237] CAZENAVE T. Abstract Proof Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2001: 39-54. DOI: 10.1007/3-540-45579-5_3.

[238] BERLEKAMP E R, CONWAY J H, GUY R K. Winning Ways for Your Mathematical Plays, 2nd Edition, Volume 1[M]. Natick, MA: A K Peters, 2001.

[239] KEARNS M, MANSOUR Y, NG A Y. A Sparse Sampling Algorithm for Near-Optimal Planning in Large Markov Decision Processes[J]. Machine Learning, 2002, 49(2-3): 193-208. DOI: 10.1023/a:1017932429737.

[240] ROBBINS H. Some aspects of the sequential design of experiments[J]. Bulletin of the American Mathematical Society, 1952, 58(5): 527-535. DOI: 10.1090/s0002-9904-1952-09620-8.

[241] SILVER D, VENESS J. Monte-Carlo Planning in Large POMDPs[C]//Advances in Neural Information Processing Systems. 2010. https://proceedings.neurips.cc/paper_files/paper/2010/hash/edfbe1afcf9246bb0d40eb4d8027d90f-Abstract.html.

[242] VODOPIVEC T, SAMOTHRAKIS S, STER B. On Monte Carlo Tree Search and Reinforcement Learning[J]. Journal of Artificial Intelligence Research, 2017, 60: 881-936. DOI: 10.1613/jair.5507.

[243] GRILL J B, ALTCHÉ F, TANG Y, et al. Monte-Carlo Tree Search as Regularized Policy Optimization[EB/OL]. arXiv: 2007.12509, 2020. https://arxiv.org/abs/2007.12509.

[244] FU M C. Monte carlo tree search: a tutorial[C]//2018 Winter Simulation Conference (WSC). IEEE, 2018: 222-236. DOI: 10.1109/wsc.2018.8632344.

[245] MCGUINNESS C. Classification of Monte Carlo tree search variants[C]//2016 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2016: 357-363. DOI: 10.1109/cec.2016.7743816.

[246] CHASLOT G, FITER C, HOOCK J B, et al. Adding Expert Knowledge and Exploration in Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2010: 1-13. DOI: 10.1007/978-3-642-12993-3_1.

[247] CHASLOT G M J B, WINANDS M H M, SZITA I, et al. Cross-entropy for monte-carlo tree search[J]. ICGA Journal, 2008, 31(3): 145-156. DOI: 10.3233/icg-2008-31303.

[248] RIMMEL A, TEYTAUD F. Multiple Overlapping Tiles for Contextual Monte Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2010: 201-210. DOI: 10.1007/978-3-642-12239-2_21.

[249] GRAF T, PLATZNER M. Common fate graph patterns in Monte Carlo Tree Search for computer go[C]//2014 IEEE Conference on Computational Intelligence and Games. IEEE, 2014: 1-8. DOI: 10.1109/cig.2014.6932863.

[250] BAUDIŠ P. Balancing MCTS by Dynamically Adjusting the Komi Value[J]. ICGA Journal, 2011, 34(3): 131-139. DOI: 10.3233/icg-2011-34302.

[251] BAIER H, WINANDS M H M. Time Management for Monte-Carlo Tree Search in Go[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2012: 39-51. DOI: 10.1007/978-3-642-31866-5_4.

[252] BAIER H, WINANDS M H M. Time Management for Monte Carlo Tree Search[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2016, 8(3): 301-314. DOI: 10.1109/tciaig.2015.2443123.

[253] FERNANDO S, MÜLLER M. Analyzing Simulations in Monte-Carlo Tree Search for the Game of Go[M]//Lecture Notes in Computer Science. Springer International Publishing, 2014: 72-83. DOI: 10.1007/978-3-319-09165-5_7.

[254] HUANG S C, MÜLLER M. Investigating the Limits of Monte-Carlo Tree Search Methods in Computer Go[M]//Lecture Notes in Computer Science. Springer International Publishing, 2014: 39-48. DOI: 10.1007/978-3-319-09165-5_4.

[255] IKEDA K, VIENNOT S. Efficiency of Static Knowledge Bias in Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer International Publishing, 2014: 26-38. DOI: 10.1007/978-3-319-09165-5_3.

[256] KAO K Y, WU I C, YEN S J, et al. Incentive Learning in Monte Carlo Tree Search[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2013, 5(4): 346-352. DOI: 10.1109/tciaig.2013.2248086.

[257] TAKEUCHI S, KANEKO T, YAMAGUCHI K. Evaluation of Monte Carlo tree search and the application to Go[C]//2008 IEEE Symposium On Computational Intelligence and Games. IEEE, 2008: 191-198. DOI: 10.1109/cig.2008.5035639.

[258] GRAF T, PLATZNER M. Using Deep Convolutional Neural Networks in Monte Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer International Publishing, 2016: 11-21. DOI: 10.1007/978-3-319-50935-8_2.

[259] TIAN Y, ZHU Y. Better Computer Go Player with Neural Network and Long-term Prediction[EB/OL]. arXiv: 1511.06410, 2015. https://arxiv.org/abs/1511.06410.

[260] LIU Y C, TSURUOKA Y. Adapting Improved Upper Confidence Bounds for Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer International Publishing, 2015: 53-64. DOI: 10.1007/978-3-319-27992-3_6.

[261] LIU Y C, TSURUOKA Y. Modification of improved upper confidence bounds for regulating exploration in Monte-Carlo tree search[J]. Theoretical Computer Science, 2016, 644: 92-105. DOI: 10.1016/j.tcs.2016.06.034.

[262] LIU Y C, TSURUOKA Y. Regulation of exploration for simple regret minimization in Monte-Carlo tree search[C]//2015 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 2015: 35-42. DOI: 10.1109/cig.2015.7317923.

[263] GRAY R C, ZHU J, ONTAÑÓN S. Beyond UCT: MAB Exploration Improvements for Monte Carlo Tree Search[C]//2023 IEEE Conference on Games (CoG). IEEE, 2023: 1-8. DOI: 10.1109/cog57401.2023.10333233.

[264] BAIER H, KAISERS M. Novelty in Monte Carlo Tree Search[J]. IEEE Transactions on Games, 2025, 17(4): 1013-1023. DOI: 10.1109/tg.2025.3589115.

[265] GALVÁN E, SIMPSON G, AMENEYRO F V. Evolving the MCTS Upper Confidence Bounds for Trees Using a Semantic-Inspired Evolutionary Algorithm in the Game of Carcassonne[J]. IEEE Transactions on Games, 2023, 15(3): 420-429. DOI: 10.1109/tg.2022.3203232.

[266] CAZENAVE T. Monte Carlo Search Algorithms Discovering Monte Carlo Tree Search Exploration Terms[M]//Lecture Notes in Computer Science. Springer Nature Switzerland, 2025: 103-115. DOI: 10.1007/978-3-031-86585-5_9.

[267] CAZENAVE T. Generation and Evaluation of Exploration Terms for Monte Carlo Tree Search[J]. ICGA Journal: The Journal of the Computer Games Community, 2026, 48(2): 62-76. DOI: 10.1177/13896911261463788.

[268] SIRONI C F, WINANDS M H M. Comparing Randomization Strategies for Search-Control Parameters in Monte-Carlo Tree Search[C]//2019 IEEE Conference on Games (CoG). IEEE, 2019: 1-8. DOI: 10.1109/cig.2019.8848056.

[269] SIRONI C F, LIU J, WINANDS M H M. Self-Adaptive Monte Carlo Tree Search in General Game Playing[J]. IEEE Transactions on Games, 2020, 12(2): 132-144. DOI: 10.1109/tg.2018.2884768.

[270] SIRONI C F, WINANDS M H M. Analysis of the Impact of Randomization of Search-Control Parameters in Monte-Carlo Tree Search[J]. Journal of Artificial Intelligence Research, 2021, 72: 717-757. DOI: 10.1613/jair.1.12065.

[271] SIRONI C F, WINANDS M H M. Comparison of rapid action value estimation variants for general game playing[C]//2016 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 2016: 1-8. DOI: 10.1109/cig.2016.7860429.

[272] SILVER D, SUTTON R S, MÜLLER M. Temporal-difference search in computer Go[J]. Machine Learning, 2012, 87(2): 183-219. DOI: 10.1007/s10994-012-5280-0.

[273] GRAF T, PLATZNER M. Adaptive Playouts in Monte-Carlo Tree Search with Policy-Gradient Reinforcement Learning[M]//Lecture Notes in Computer Science. Springer International Publishing, 2015: 1-11. DOI: 10.1007/978-3-319-27992-3_1.

[274] GRAF T, PLATZNER M. Adaptive playouts for online learning of policies during Monte Carlo Tree Search[J]. Theoretical Computer Science, 2016, 644: 53-62. DOI: 10.1016/j.tcs.2016.06.029.

[275] ROBLES D, ROHLFSHAGEN P, LUCAS S M. Learning non-random moves for playing Othello: Improving Monte Carlo Tree Search[C]//2011 IEEE Conference on Computational Intelligence and Games (CIG'11). IEEE, 2011: 305-312. DOI: 10.1109/cig.2011.6032021.

[276] SIRONI C F, CAZENAVE T, WINANDS M H M. Enhancing Playout Policy Adaptation for General Game Playing[M]//Communications in Computer and Information Science. Springer International Publishing, 2021: 116-139. DOI: 10.1007/978-3-030-89453-5_9.

[277] BUZER L, CAZENAVE T. Playout Optimization for Monte-Carlo Search Algorithms. Application to Morpion Solitaire[C]//2021 IEEE Conference on Games (CoG). IEEE, 2021: 01-08. DOI: 10.1109/cog52621.2021.9618896.

[278] TEYTAUD F, TEYTAUD O. On the huge benefit of decisive moves in Monte-Carlo Tree Search algorithms[C]//Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games. IEEE, 2010: 359-364. DOI: 10.1109/itw.2010.5593334.

[279] CHILDS B E, BRODEUR J H, KOCSIS L. Transpositions and move groups in Monte Carlo tree search[C]//2008 IEEE Symposium On Computational Intelligence and Games. IEEE, 2008: 389-395. DOI: 10.1109/cig.2008.5035667.

[280] VAN EYCK G, MÜLLER M. Revisiting Move Groups in Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2012: 13-23. DOI: 10.1007/978-3-642-31866-5_2.

[281] CAZENAVE T, SAFFIDINE A. Score Bounded Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2011: 93-104. DOI: 10.1007/978-3-642-17928-0_9.

[282] PEPELS T, WINANDS M H M. Ancestor-Based α-β Bounds for Monte-Carlo Tree Search[C]//2024 IEEE Conference on Games (CoG). IEEE, 2024: 1-4. DOI: 10.1109/cog60054.2024.10645672.

[283] CAZENAVE T. Batch Monte Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Nature Switzerland, 2023: 146-162. DOI: 10.1007/978-3-031-34017-8_13.

[284] CHASLOT G M J B, WINANDS M H M, VAN DEN HERIK H J. Parallel Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 60-71. DOI: 10.1007/978-3-540-87608-3_6.

[285] CAZENAVE T, JOUANDEAU N. A Parallel Monte-Carlo Tree Search Algorithm[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 72-80. DOI: 10.1007/978-3-540-87608-3_7.

[286] ENZENBERGER M, MÜLLER M. A Lock-Free Multithreaded Monte-Carlo Tree Search Algorithm[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2010: 14-20. DOI: 10.1007/978-3-642-12993-3_2.

[287] MIRSOLEIMANI S A, PLAAT A, VAN DEN HERIK J, et al. An Analysis of Virtual Loss in Parallel MCTS[C]//Proceedings of the 9th International Conference on Agents and Artificial Intelligence. SCITEPRESS - Science and Technology Publications, 2017: 648-652. DOI: 10.5220/0006205806480652.

[288] STEINMETZ E, GINI M. More Trees or Larger Trees: Parallelizing Monte Carlo Tree Search[J]. IEEE Transactions on Games, 2021, 13(3): 315-320. DOI: 10.1109/tg.2020.3048331.

[289] SCHAEFERS L, PLATZNER M. Distributed Monte Carlo Tree Search: A Novel Technique and its Application to Computer Go[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2015, 7(4): 361-374. DOI: 10.1109/tciaig.2014.2346997.

[290] CHEN C H, WU W L, CHEN Y H, et al. Some improvements in Monte Carlo tree search algorithms for sudden death games[J]. ICGA Journal, 2019, 40(4): 460-470. DOI: 10.3233/icg-180065.

[291] LONG J, STURTEVANT N, BURO M, et al. Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2010, 24(1): 134-140. DOI: 10.1609/aaai.v24i1.7562.

[292] LANZI P L. Evaluating the Complexity of Players' Strategies using MCTS Iterations[C]//2019 IEEE Conference on Games (CoG). IEEE, 2019: 1-8. DOI: 10.1109/cig.2019.8848122.

[293] BERTHIER V, DOGHMEN H, TEYTAUD O. Consistency Modifications for Automatically Tuned Monte-Carlo Tree Search[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2010: 111-124. DOI: 10.1007/978-3-642-13800-3_9.

[294] WINANDS M H M, BJÖRNSSON Y, SAITO J T. Monte-Carlo Tree Search Solver[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2008: 25-36. DOI: 10.1007/978-3-540-87608-3_3.

[295] FÜRNKRANZ J. Machine Learning in Computer Chess: The Next Generation[J]. ICGA Journal, 1996, 19(3): 147-161. DOI: 10.3233/icg-1996-19302.

[296] MAŃDZIUK J. Knowledge-Free and Learning-Based Methods in Intelligent Game Playing[M]. Berlin: Springer, 2010. DOI: 10.1007/978-3-642-11678-0.

[297] SZUBERT M, JASKOWSKI W. Temporal difference learning of N-tuple networks for the game 2048[C]//2014 IEEE Conference on Computational Intelligence and Games. IEEE, 2014: 1-8. DOI: 10.1109/cig.2014.6932907.

[298] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533. DOI: 10.1038/nature14236.

[299] BELLEMARE M G, NADDAF Y, VENESS J, et al. The Arcade Learning Environment: An Evaluation Platform for General Agents[J]. Journal of Artificial Intelligence Research, 2013, 47: 253-279. DOI: 10.1613/jair.3912.

[300] MACHADO M C, BELLEMARE M G, TALVITIE E, et al. Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents[J]. Journal of Artificial Intelligence Research, 2018, 61: 523-562. DOI: 10.1613/jair.5699.

[301] BROCKMAN G, CHEUNG V, PETTERSSON L, et al. OpenAI Gym[EB/OL]. arXiv: 1606.01540, 2016. https://arxiv.org/abs/1606.01540.

[302] LANCTOT M, LOCKHART E, LESPIAU J B, et al. OpenSpiel: A Framework for Reinforcement Learning in Games[EB/OL]. arXiv: 1908.09453, 2019. https://arxiv.org/abs/1908.09453.

[303] PEREZ-LIEBANA D, SAMOTHRAKIS S, TOGELIUS J, et al. General Video Game AI: Competition, Challenges and Opportunities[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2016, 30(1). DOI: 10.1609/aaai.v30i1.9869.

[304] SILVER D, HUBERT T, SCHRITTWIESER J, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm[EB/OL]. arXiv: 1712.01815, 2017. https://arxiv.org/abs/1712.01815.

[305] SCHRITTWIESER J, ANTONOGLOU I, HUBERT T, et al. Mastering Atari, Go, chess and shogi by planning with a learned model[J]. Nature, 2020, 588(7839): 604-609. DOI: 10.1038/s41586-020-03051-4.

[306] HUBERT T, SCHRITTWIESER J, ANTONOGLOU I, et al. Learning and Planning in Complex Action Spaces[EB/OL]. arXiv: 2104.06303, 2021. https://arxiv.org/abs/2104.06303.

[307] TIAN Y, MA J, GONG Q, et al. ELF OpenGo: An Analysis and Open Reimplementation of AlphaZero[EB/OL]. arXiv: 1902.04522, 2019. https://arxiv.org/abs/1902.04522.

[308] DAVID O E, NETANYAHU N S, WOLF L. DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess[M]//Lecture Notes in Computer Science. Springer International Publishing, 2016: 88-96. DOI: 10.1007/978-3-319-44781-0_11.

[309] LAI M. Giraffe: Using Deep Reinforcement Learning to Play Chess[EB/OL]. arXiv: 1509.01549, 2015. https://arxiv.org/abs/1509.01549.

[310] SUTTON R S. Reinforcement Learning: An Introduction[M]. Cambridge, MA: MIT Press, 2018.

[311] WATKINS C J C H, DAYAN P. Q-learning[J]. Machine Learning, 1992, 8(3-4): 279-292. DOI: 10.1007/bf00992698.

[312] WILLIAMS R J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning[M]//Reinforcement Learning. Springer US, 1992: 5-32. DOI: 10.1007/978-1-4615-3618-5_2.

[313] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal Policy Optimization Algorithms[EB/OL]. arXiv: 1707.06347, 2017. https://arxiv.org/abs/1707.06347.

[314] SILVER D, SINGH S, PRECUP D, et al. Reward is enough[J]. Artificial Intelligence, 2021, 299: 103535. DOI: 10.1016/j.artint.2021.103535.

[315] MORAVČÍK M, SCHMID M, BURCH N, et al. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker[J]. Science, 2017, 356(6337): 508-513. DOI: 10.1126/science.aam6960.

[316] BROWN N, SANDHOLM T. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals[J]. Science, 2018, 359(6374): 418-424. DOI: 10.1126/science.aao1733.

[317] BROWN N, SANDHOLM T. Superhuman AI for multiplayer poker[J]. Science, 2019, 365(6456): 885-890. DOI: 10.1126/science.aay2400.

[318] VINYALS O, BABUSCHKIN I, CZARNECKI W M, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning[J]. Nature, 2019, 575(7782): 350-354. DOI: 10.1038/s41586-019-1724-z.

[319] OPENAI, :, BERNER C, et al. Dota 2 with Large Scale Deep Reinforcement Learning[EB/OL]. arXiv: 1912.06680, 2019. https://arxiv.org/abs/1912.06680.

[320] META FUNDAMENTAL AI RESEARCH DIPLOMACY TEAM (FAIR)†, BAKHTIN A, BROWN N, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning[J]. Science, 2022, 378(6624): 1067-1074. DOI: 10.1126/science.ade9097.

[321] PEROLAT J, DE VYLDER B, HENNES D, et al. Mastering the game of Stratego with model-free multiagent reinforcement learning[J]. Science, 2022, 378(6623): 990-996. DOI: 10.1126/science.add4679.

[322] SCHMID M, MORAVČÍK M, BURCH N, et al. Student of Games: A unified learning algorithm for both perfect and imperfect information games[J]. Science Advances, 2023, 9(46). DOI: 10.1126/sciadv.adg3256.

[323] FAWZI A, BALOG M, HUANG A, et al. Discovering faster matrix multiplication algorithms with reinforcement learning[J]. Nature, 2022, 610(7930): 47-53. DOI: 10.1038/s41586-022-05172-4.

[324] MANKOWITZ D J, MICHI A, ZHERNOV A, et al. Faster sorting algorithms discovered using deep reinforcement learning[J]. Nature, 2023, 618(7964): 257-263. DOI: 10.1038/s41586-023-06004-9.

[325] CLAUSEN C, REICHHUBER S, THOMSEN I, et al. Improvements to Increase the Efficiency of the AlphaZero Algorithm: A Case Study in the Game 'Connect 4'[C]//Proceedings of the 13th International Conference on Agents and Artificial Intelligence. SCITEPRESS - Science and Technology Publications, 2021: 803-811. DOI: 10.5220/0010245908030811.

[326] SCHEIERMANN J, KONEN W. AlphaZero-Inspired Game Learning: Faster Training by Using MCTS Only at Test Time[J]. IEEE Transactions on Games, 2023, 15(4): 637-647. DOI: 10.1109/tg.2022.3206733.

[327] DEMURA Y, KANEKO T. Initial state diversification for efficient AlphaZero-style training[J]. ICGA Journal, 2024, 46(2): 40-66. DOI: 10.3233/icg-240255.

[328] CHENG S, TSAI M Y, HONG D Y, et al. Uncertainty-Guided Exploration for Efficient AlphaZero Training[C]//Advances in Neural Information Processing Systems 38. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025: 45623-45649. DOI: 10.52202/085713-1361.

[329] WANG H, EMMERICH M, PREUSS M, et al. Alternative Loss Functions in AlphaZero-like Self-play[C]//2019 IEEE Symposium Series on Computational Intelligence (SSCI). IEEE, 2019: 155-162. DOI: 10.1109/ssci44817.2019.9002814.

[330] WEINER E M, MONTANEZ G D, TRUJILLO A, et al. Hyperparameter Choice as Search Bias in AlphaZero[C]//2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC). IEEE, 2021: 2389-2394. DOI: 10.1109/smc52423.2021.9659008.

[331] HSUEH C H, WU I C, CHEN J C, et al. AlphaZero for a Non-Deterministic Game[C]//2018 Conference on Technologies and Applications of Artificial Intelligence (TAAI). IEEE, 2018: 116-121. DOI: 10.1109/taai.2018.00034.

[332] GUO Y. Implementing the AlphaZero algorithm for Connect Four: A deep reinforcement learning approach[J]. Applied and Computational Engineering, 2024, 33(1): 34-41. DOI: 10.54254/2755-2721/33/20230228.

[333] CHEN Y C, CHEN C H, LIN S S. Exact-Win Strategy for Overcoming AlphaZero[C]//Proceedings of the 2018 International Conference on Computational Intelligence and Intelligent Systems. ACM, 2018: 26-31. DOI: 10.1145/3293475.3293486.

[334] NEUMANN O, GROS C. AlphaZero Neural Scaling and Zipf's Law: a Tale of Board Games and Power Laws[C]//Advances in Neural Information Processing Systems 38. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025: 65070-65100. DOI: 10.52202/085713-1957.

[335] PLAAT A. Self-Play[M]//Learning to Play. Springer International Publishing, 2020: 195-232. DOI: 10.1007/978-3-030-59238-7_7.

[336] PLAAT A. Two-Agent Self-Play[M]//Deep Reinforcement Learning. Springer Nature Singapore, 2022: 169-218. DOI: 10.1007/978-981-19-0638-1_6.

[337] VAN DER REE M, WIERING M. Reinforcement learning in the game of Othello: Learning against a fixed opponent and learning from self-play[C]//2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL). IEEE, 2013: 108-115. DOI: 10.1109/adprl.2013.6614996.

[338] NEUMANN O, GROS C. Size Scaling in Self-Play Reinforcement Learning[C]//ESANN 2022 proceedings. Ciaco - i6doc.com, 2022: 521-526. DOI: 10.14428/esann/2022.es2022-53.

[339] MCGRATH T, KAPISHNIKOV A, TOMAŠEV N, et al. Acquisition of chess knowledge in AlphaZero[J]. Proceedings of the National Academy of Sciences, 2022, 119(47). DOI: 10.1073/pnas.2206625119.

[340] LOVERING C, FORDE J, KONIDARIS G, et al. Evaluation Beyond Task Performance: Analyzing Concepts in AlphaZero in Hex[C]//Advances in Neural Information Processing Systems 35. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022: 25992-26006. DOI: 10.52202/068431-1885.

[341] TOMAŠEV N, PAQUET U, HASSABIS D, et al. Reimagining chess with AlphaZero[J]. Communications of the ACM, 2022, 65(2): 60-66. DOI: 10.1145/3460349.

[342] WANG T T, GLEAVE A, TSENG T, et al. Adversarial Policies Beat Superhuman Go AIs[EB/OL]. arXiv: 2211.00241, 2022. https://arxiv.org/abs/2211.00241.

[343] LAN L C, ZHANG H, WU T R, et al. Are AlphaZero-Like Agents Robust to Adversarial Perturbations?[C]//Advances in Neural Information Processing Systems 35. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022: 11229-11240. DOI: 10.52202/068431-0816.

[344] JONES A L. Scaling Scaling Laws with Board Games[EB/OL]. arXiv: 2104.03113, 2021. https://arxiv.org/abs/2104.03113.

[345] NEUMANN O, GROS C. Scaling Laws for a Multi-Agent Reinforcement Learning Model[EB/OL]. arXiv: 2210.00849, 2022. https://arxiv.org/abs/2210.00849.

[346] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling Laws for Neural Language Models[EB/OL]. arXiv: 2001.08361, 2020. https://arxiv.org/abs/2001.08361.

[347] SNELL C, LEE J, XU K, et al. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters[EB/OL]. arXiv: 2408.03314, 2024. https://arxiv.org/abs/2408.03314.

[348] BROWN B, JURAVSKY J, EHRLICH R, et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling[EB/OL]. arXiv: 2407.21787, 2024. https://arxiv.org/abs/2407.21787.

[349] RUOSS A, DELÉTANG G, MEDAPATI S, et al. Amortized Planning with Large-Scale Transformers: A Case Study on Chess[EB/OL]. arXiv: 2402.04494, 2024. https://arxiv.org/abs/2402.04494.

[350] SCHULTZ J, ADAMEK J, JUSUP M, et al. Mastering Board Games by External and Internal Planning with Language Models[EB/OL]. arXiv: 2412.12119, 2024. https://arxiv.org/abs/2412.12119.

[351] IWATA S, KASAI T. The Othello game on an n × n board is PSPACE-complete[J]. Theoretical Computer Science, 1994, 123(2): 329-340. DOI: 10.1016/0304-3975(94)90131-7.

[352] FRAENKEL A S, LICHTENSTEIN D. Computing a perfect strategy for n×n chess requires time exponential in n[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 1981: 278-293. DOI: 10.1007/3-540-10843-2_23.

[353] UITERWIJK J. Solving narrow Konane boards[J]. ICGA Journal, 2022, 43(3): 162-183. DOI: 10.3233/icg-220198.

[354] ALTHÖFER I. Clobber - a new game with very simple rules[J]. ICGA Journal, 2002, 25(2): 123-125. DOI: 10.3233/icg-2002-25214.

[355] DEMAINE E D, DEMAINE M L, FLEISCHER R. Solitaire Clobber[J]. Theoretical Computer Science, 2004, 313(3): 325-338. DOI: 10.1016/j.tcs.2003.02.001.

[356] UITERWIJK J W H M, GRIEBEL J. Combining Combinatorial Game Theory with an

α

-

β

Solver for Clobber: Theory and Experiments[M]//Communications in Computer and Information Science. Springer International Publishing, 2017: 78-92. DOI: 10.1007/978-3-319-67468-1_6.

[357] KATO H, TAKAYA M, YAMAMURA A. Analysis of a Monte Carlo Tree Search in Knight-Amazons[J]. Procedia Computer Science, 2015, 62: 31-38. DOI: 10.1016/j.procs.2015.08.406.

[358] ABDELBAR A M, TAGLIARINI G A. Using neural network learning in an Othello evaluation function[J]. Journal of Experimental & Theoretical Artificial Intelligence, 1998, 10(2): 217-229. DOI: 10.1080/095281398146833.

[359] MANNING E P. Temporal Difference Learning of an Othello Evaluation Function for a Small Neural Network with Shared Weights[C]//2007 IEEE Symposium on Computational Intelligence and Games. IEEE, 2007: 216-223. DOI: 10.1109/cig.2007.368101.

[360] OSAKI Y, SHIBAHARA K, TAJIMA Y, et al. An Othello evaluation function based on Temporal Difference Learning using probability of winning[C]//2008 IEEE Symposium On Computational Intelligence and Games. IEEE, 2008: 205-211. DOI: 10.1109/cig.2008.5035641.

[361] RUNARSSON T P, JONSSON E O. Effect of look-ahead search depth in learning position evaluation functions for Othello using -greedy exploration[C]//2007 IEEE Symposium on Computational Intelligence and Games. IEEE, 2007: 210-215. DOI: 10.1109/cig.2007.368100.

[362] MANNING E P. Using Resource-Limited Nash Memory to Improve anOthelloEvaluation Function[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2010, 2(1): 40-53. DOI: 10.1109/tciaig.2010.2042598.

[363] RUNARSSON T P, LUCAS S M. Preference Learning for Move Prediction and Evaluation Function Approximation in Othello[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2014, 6(3): 300-313. DOI: 10.1109/tciaig.2014.2307272.

[364] TANIMOTO S, NAKANO R. Learning an Evaluation Function for Shogi from Data of Games[M]//Lecture Notes in Computer Science. Springer Berlin Heidelberg, 2004: 609-615. DOI: 10.1007/978-3-540-30133-2_80.

[365] YAMAMOTO M, SUZUKI K, OHUCHI A. An Acquisition of Evaluation Function for Shogi by Learning Self-Play[J]. International Transactions in Operational Research, 2001, 8(3): 305-315. DOI: 10.1111/1475-3995.00267.

[366] WAN S, KANEKO T. Building Evaluation Functions for Chess and Shogi with Uniformity Regularization Networks[C]//2018 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 2018: 1-8. DOI: 10.1109/cig.2018.8490455.

[367] WAN S, KANEKO T. Heterogeneous Multi-task Learning of Evaluation Functions for Chess and Shogi[M]//Lecture Notes in Computer Science. Springer International Publishing, 2018: 347-358. DOI: 10.1007/978-3-030-04182-3_31.

[368] YAMANA T, HOSHINO J. Compressing the Evaluation Function With Small-Scale Deep Learning on Othello[J]. IEEE Transactions on Games, 2026, 18(1): 128-138. DOI: 10.1109/tg.2025.3624825.

[369] IIDA H, SAKUTA M, ROLLASON J. Computer shogi[J]. Artificial Intelligence, 2002, 134(1-2): 121-144. DOI: 10.1016/s0004-3702(01)00157-6.

[370] TAKEUCHI S, KANEKO T. Estimating Ratings of Computer Players by the Evaluation Scores and Principal Variations in Shogi[C]//2015 3rd International Conference on Applied Computing and Information Technology/2nd International Conference on Computational Science and Intelligence. IEEE, 2015: 85-90. DOI: 10.1109/acit-csi.2015.23.

[371] TAKEUCHI S. Comparison of Search Behaviors in Chess, Shogi, and the game of Go[C]//2022 International Conference on Technologies and Applications of Artificial Intelligence (TAAI). IEEE, 2022: 183-188. DOI: 10.1109/taai57707.2022.00041.

[372] GOMBOC D, MARSLAND T A, BURO M. Evaluation Function Tuning via Ordinal Correlation[M]//Advances in Computer Games. Springer US, 2004: 1-18. DOI: 10.1007/978-0-387-35706-5_1.

[373] TINGTING FU, HONGFENG YIN. Designing a hybrid position evaluation function for Chinese-chess computer game[C]//2012 IEEE International Conference on Computer Science and Automation Engineering. IEEE, 2012: 75-78. DOI: 10.1109/icsess.2012.6269409.

[374] YEN S J, CHEN J C, YANG T N, et al. Computer chinese chess[J]. ICGA Journal, 2004, 27(1): 3-18. DOI: 10.3233/icg-2004-27102.

[375] WINANDS M H M, HERIK H J, UITERWIJK J W H M. An Evaluation Function for Lines of Action[M]//Advances in Computer Games. Springer US, 2004: 249-260. DOI: 10.1007/978-0-387-35706-5_16.

[376] RUNARSSON T P, LUCAS S M. On imitating Connect-4 game trajectories using an approximate n-tuple evaluation function[C]//2015 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, 2015: 208-213. DOI: 10.1109/cig.2015.7317961.

[377] TOURÉ A W. Evaluation of the Use of Minimax Search in Connect-4 —How Does the Minimax Search Algorithm Perform in Connect-4 with Increasing Grid Sizes?[J]. Applied Mathematics, 2023, 14(06): 419-427. DOI: 10.4236/am.2023.146025.

[378] CHANG H J, FAN G Y, CHEN J C, et al. Validating and Fine-Tuning of Game Evaluation Functions Using Endgame Databases[M]//Communications in Computer and Information Science. Springer International Publishing, 2018: 137-150. DOI: 10.1007/978-3-319-75931-9_10.

[379] BJÖRNSSON Y, HAYWARD R, JOHANSON M, et al. Dead Cell Analysis in Hex and the Shannon Game[M]//Graph Theory in Paris. Birkhäuser Basel, 2006: 45-59. DOI: 10.1007/978-3-7643-7400-6_5.

[380] HAYWARD R, BJÖRNSSON Y, JOHANSON M, et al. Solving 7×7 Hex: Virtual Connections and Game-State Reduction[M]//Advances in Computer Games. Springer US, 2004: 261-278. DOI: 10.1007/978-0-387-35706-5_17.

[381] LIANG X, WEI T H, WU I C. Solving Hex Openings Using Job-Level UCT Search[J]. ICGA Journal, 2015, 38(3): 149-164. DOI: 10.3233/icg-2015-38303.

[382] GALE D. The Game of Hex and the Brouwer Fixed-Point Theorem[J]. The American Mathematical Monthly, 1979, 86(10): 818-827. DOI: 10.1080/00029890.1979.11994922.

[383] CLUNE J E. Heuristic Evaluation Functions for General Game Playing[J]. KI - Künstliche Intelligenz, 2011, 25(1): 73-74. DOI: 10.1007/s13218-010-0074-7.

[384] WALEDZIK K, MANDZIUK J. An Automatically Generated Evaluation Function in General Game Playing[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2014, 6(3): 258-270. DOI: 10.1109/tciaig.2013.2286825.

[385] SWIECHOWSKI M, MANDZIUK J. Self-Adaptation of Playing Strategies in General Game Playing[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2014, 6(4): 367-381. DOI: 10.1109/tciaig.2013.2275163.

[386] GENESERETH M, BJÖRNSSON Y. The International General Game Playing Competition[J]. AI Magazine, 2013, 34(2): 107-111. DOI: 10.1609/aimag.v34i2.2475.

[387] LI W, LIU Z. Elo rating of local contextual patterns[C]//2011 Chinese Control and Decision Conference (CCDC). IEEE, 2011: 2522-2526. DOI: 10.1109/ccdc.2011.5968634.

[388] LIPTON Z C, STEINHARDT J. Troubling Trends in Machine Learning Scholarship: Some ML papers suffer from flaws that could mislead the public and stymie future research.[J]. Queue, 2019, 17(1): 45-77. DOI: 10.1145/3317287.3328534.

[389] GUNDERSEN O E, KJENSMO S. State of the Art: Reproducibility in Artificial Intelligence[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2018, 32(1). DOI: 10.1609/aaai.v32i1.11503.

[390] PINEAU J, VINCENT-LAMARRE P, SINHA K, et al. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)[EB/OL]. arXiv: 2003.12206, 2020. https://arxiv.org/abs/2003.12206.

[391] PENG R D. Reproducible Research in Computational Science[J]. Science, 2011, 334(6060): 1226-1227. DOI: 10.1126/science.1213847.

[392] SANDVE G K, NEKRUTENKO A, TAYLOR J, et al. Ten Simple Rules for Reproducible Computational Research[J]. PLoS Computational Biology, 2013, 9(10): e1003285. DOI: 10.1371/journal.pcbi.1003285.

[393] WILSON G, ARULIAH D A, BROWN C T, et al. Best Practices for Scientific Computing[J]. PLoS Biology, 2014, 12(1): e1001745. DOI: 10.1371/journal.pbio.1001745.

[394] HOHMAN F, KAHNG M, PIENTA R, et al. Visual Analytics in Deep Learning: An Interrogative Survey for the Next Frontiers[J]. IEEE Transactions on Visualization and Computer Graphics, 2019, 25(8): 2674-2693. DOI: 10.1109/tvcg.2018.2843369.

[395] PURI N, VERMA S, GUPTA P, et al. Explain Your Move: Understanding Agent Actions Using Specific and Relevant Feature Attribution[EB/OL]. arXiv: 1912.12191, 2019. https://arxiv.org/abs/1912.12191.

[396] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for Human-AI Interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. ACM, 2019: 1-13. DOI: 10.1145/3290605.3300233.

[397] SHNEIDERMAN B. Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy[J]. International Journal of Human–Computer Interaction, 2020, 36(6): 495-504. DOI: 10.1080/10447318.2020.1741118.

[398] MCILROY-YOUNG R, SEN S, KLEINBERG J, et al. Aligning Superhuman AI with Human Behavior: Chess as a Model System[C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. ACM, 2020: 1677-1687. DOI: 10.1145/3394486.3403219.

[399] SHIN M, KIM J, VAN OPHEUSDEN B, et al. Superhuman Artificial Intelligence Can Improve Human Decision Making by Increasing Novelty[J]. , 2023. DOI: 10.31234/osf.io/rn3vx.

[400] CLEVELAND W S, MCGILL R. Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods[J]. Journal of the American Statistical Association, 1984, 79(387): 531-554. DOI: 10.1080/01621459.1984.10478080.

[401] HEER J, BOSTOCK M. Crowdsourcing graphical perception: using mechanical turk to assess visualization design[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. ACM, 2010: 203-212. DOI: 10.1145/1753326.1753357.

[402] MUNZNER T. Visualization Analysis and Design[M]. Boca Raton: CRC Press, 2014. DOI: 10.1201/b17511.

[403] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code[EB/OL]. arXiv: 2107.03374, 2021. https://arxiv.org/abs/2107.03374.

[404] VAITHILINGAM P, ZHANG T, GLASSMAN E L. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models[C]//CHI Conference on Human Factors in Computing Systems Extended Abstracts. ACM, 2022: 1-7. DOI: 10.1145/3491101.3519665.

[405] BARKE S, JAMES M B, POLIKARPOVA N. Grounded Copilot: How Programmers Interact with Code-Generating Models[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85-111. DOI: 10.1145/3586030.

[406] WALTERS W H, WILDER E I. Fabrication and errors in the bibliographic citations generated by ChatGPT[J]. Scientific Reports, 2023, 13(1). DOI: 10.1038/s41598-023-41032-5.

[407] ALKAISSI H, MCFARLANE S I. Artificial Hallucinations in ChatGPT: Implications in Scientific Writing[J]. Cureus, 2023. DOI: 10.7759/cureus.35179.

[408] JI Z, LEE N, FRIESKE R, et al. Survey of Hallucination in Natural Language Generation[J]. ACM Computing Surveys, 2023, 55(12): 1-38. DOI: 10.1145/3571730.

[409] LIU N F, ZHANG T, LIANG P. Evaluating Verifiability in Generative Search Engines[EB/OL]. arXiv: 2304.09848, 2023. https://arxiv.org/abs/2304.09848.

[410] CHEN T Y, KUO F C, LIU H, et al. Metamorphic Testing: A Review of Challenges and Opportunities[J]. ACM Computing Surveys, 2019, 51(1): 1-27. DOI: 10.1145/3143561.

[411] SEGURA S, FRASER G, SANCHEZ A B, et al. A Survey on Metamorphic Testing[J]. IEEE Transactions on Software Engineering, 2016, 42(9): 805-824. DOI: 10.1109/tse.2016.2532875.

[412] CLAESSEN K, HUGHES J. QuickCheck: a lightweight tool for random testing of Haskell programs[J]. ACM SIGPLAN Notices, 2000, 35(9): 268-279. DOI: 10.1145/357766.351266.

[413] PENG S, KALLIAMVAKOU E, CIHON P, et al. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot[EB/OL]. arXiv: 2302.06590, 2023. https://arxiv.org/abs/2302.06590.

[414] ZIEGLER A, KALLIAMVAKOU E, LI X A, et al. Measuring GitHub Copilot’s Impact on Productivity[J]. Communications of the ACM, 2024, 67(3): 54-63. DOI: 10.1145/3633453.

[415] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?[EB/OL]. arXiv: 2310.06770, 2023. https://arxiv.org/abs/2310.06770.

[416] KASNECI E, SESSLER K, KÜCHEMANN S, et al. ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education[J]. , 2023. DOI: 10.35542/osf.io/5er8f.

[417] BECKER B A, DENNY P, FINNIE-ANSLEY J, et al. Programming Is Hard - Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation[C]//Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1. ACM, 2023: 500-506. DOI: 10.1145/3545945.3569759.

[418] PRATHER J, DENNY P, LEINONEN J, et al. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education[C]//Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education. ACM, 2023: 108-159. DOI: 10.1145/3623762.3633499.

[419] DENNY P, PRATHER J, BECKER B A, et al. Computing Education in the Era of Generative AI[J]. Communications of the ACM, 2024, 67(2): 56-67. DOI: 10.1145/3624720.

[420] OPENAI, ACHIAM J, ADLER S, et al. GPT-4 Technical Report[EB/OL]. arXiv: 2303.08774, 2023. https://arxiv.org/abs/2303.08774.

[421] BUBECK S, CHANDRASEKARAN V, ELDAN R, et al. Sparks of Artificial General Intelligence: Early experiments with GPT-4[EB/OL]. arXiv: 2303.12712, 2023. https://arxiv.org/abs/2303.12712.

附录 A　符号与术语

表 A-1　主要符号

符号	含义	首次出现
s、S	局面、局面集合	1.2.2
$\rho(s)$、$\acute{\rho}$	行棋方、对手	1.2.2
$A(s)$、$T(s,a)$	合法着法集合、转移函数	1.2.2
$V(s)$	博弈论值（行棋方视角）	式 1-1
(f,t,a)	一个着法：后从 f 走到 t，再向 a 射箭	2.2.1
$D_p^Q(x)$、$D_p^K(x)$	玩家 p 的后到达格子 x 的后距离、王距离	2.2.2
t_1、t_2	按后距离、王距离统计的领地差	式 2-1
c_1、c_2	按后距离、王距离计算的位置项	式 2-2
κ	双方同时到达的格子给行棋方的分数（0.2）	2.2.3
w、p、W_0	争夺程度、阶段系数 $p=min(1,w/W_0)$、$W_0=60$	式 2-3
mob、m_q	移动性特征、后 q 的可走步数	式 2-4
a_k^{E}、a_k^{L}	评估函数中特征 k 的开局、后期权重	式 2-5
$E(s)$	静态评估函数（行棋方视角，单位约为“格”）	式 2-5
R_p、δ	分隔后玩家 p 可达的空格数、区域格数差	式 2-6
$Q(f,t)$、$A(a)$	可分离评估的“移后”项与“射箭”项	式 2-8
D、b	搜索深度、分支因子	式 2-9
α、β	α-β 搜索窗口的下界与上界	2.3.1
W、P	内部结点束宽（8）、叶前结点候选数（24）	2.3.5
$N(s)$、	MCTS 访问次数	式 2-10

符号	含义	首次出现
$N(s,a)$		
$\acute{Q}(s,a)$	MCTS 平均收益	式 2-10
c、c_{puct}	UCT（0.5）、PUCT（1.5）探索常数	式 2-10、2-12
C_{pw}、α	渐进展开的系数（2）与指数（0.5）	式 2-11
$P(s,a)$、τ	先验概率、先验的 softmax 温度（4）	式 2-12
L、k	截断模拟的随机步数（4）、评估值到胜率的尺度（0.18）	式 2-13
$\phi(x,m)$、θ	走子策略的 12 维着法特征与参数	式 2-14
K、a	Texel 调参中的尺度与 10 维评估权重	式 2-16
R_i、ΔR	Elo 等级分、等级分差	式 2-17、2-18

表 A-2　主要术语与缩写

术语/缩写	全称或英文	含义
后、箭	amazon (queen), arrow	亚马逊棋的棋子与障碍物
分隔	separation	双方的后不再共享任何可达空格，对局分解为互不影响的区域
填格	filling	分隔后在己方区域内逐步走完空格的阶段
后距离 / 王距离	queen / king distance	按后的走法 / 王的走法到达某格的最少步数
可分离评估	separable evaluation	把“移后”与“射箭”分别评估再相加的着法排序方法（本文设计）
PV	Principal Variation	主变例，搜索认为双方最佳的着法序列
PVS	Principal Variation Search	主变例搜索（NegaScout）
TT	Transposition Table	置换表
MCTS	Monte Carlo Tree Search	蒙特卡洛树搜索
UCT	Upper Confidence bounds applied to Trees	基于 UCB1 的树搜索
PUCT	Predictor + UCT	带先验的 UCT 变体
PW	Progressive Widening	渐进展开
EBF	Effective Branching Factor	有效分支因子
Elo	—	等级分体系
NLL	Negative Log-Likelihood	负对数似然（交叉熵）

附录 B　实验数据明细

本附录列出正文统计所依据的原始数据概况与逐组明细。所有数值都由 `analysis/appendix_tables.py` 等脚本直接从 `data/` 下的原始记录计算，未做任何手工修改。

各实验组概况

表 B-1　主实验各组概况

实验组	对阵组数	对局数	白方胜率	平均手数	总着法数	单局平均墙钟时间/s
T（循环赛）	28	896	51.1%	66.1	57,466	8.2
A（α-β 消融）	11	352	56.8%	81.6	28,026	9.9
M（MCTS 消融）	15	480	48.3%	82.1	38,432	13.6
S（算力缩放）	8	256	53.5%	82.7	20,647	15.4
X（同预算 α-β 对 MCTS）	5	160	52.5%	81.8	12,772	18.2
合计	67	2144	51.8%	75.4	157,343	11.3

注：“单局平均墙钟时间”是单个对局从开始到结束的时间。实验以 4 个进程并行，2144 局的单局时间累计约 6.7 小时，实际墙钟时间约为其四分之一。

循环赛逐对结果

表 B-2 是循环赛 28 对智能体的逐对结果，是表 5-5 与 Elo 拟合（表 5-4）的原始数据。“A 执白时得分”指 A 执白的 16 局中 A 的得分。

表 B-2　循环赛逐对结果（每对 32 局）

序号	A	B	A 胜–负	A 得分率	A 执白时得分	A 执黑时得分	平均手数
1	随机	一步贪心	0–32	0.0%	0/16	0/16	35.9
2	随机	α-β（完整）	0–32	0.0%	0/16	0/16	41.8
3	随机	α-β（Texel 权重）	0–32	0.0%	0/16	0/16	44.4
4	随机	MCTS（纯随机）	0–32	0.0%	0/16	0/16	62.8
5	随机	MCTS（UCT+截断）	0–32	0.0%	0/16	0/16	43.1
6	随机	MCTS-PUCT	0–32	0.0%	0/16	0/16	42.6
7	随机	MCTS（学习先验）	0–32	0.0%	0/16	0/16	42.6

序号	A	B	A 胜–负	A 得分率	A 执白时得分	A 执黑时得分	平均手数
8	一步贪心	α-β（完整）	3–29	9.4%	1/16	2/16	75.8
9	一步贪心	α-β（Texel 权重）	13–19	40.6%	7/16	6/16	80.3
10	一步贪心	MCTS（纯随机）	32–0	100.0%	16/16	16/16	55.2
11	一步贪心	MCTS（UCT+截断）	17–15	53.1%	11/16	6/16	74.9
12	一步贪心	MCTS-PUCT	1–31	3.1%	1/16	0/16	77.3
13	一步贪心	MCTS（学习先验）	2–30	6.2%	2/16	0/16	80.0
14	α-β（完整）	α-β（Texel 权重）	25–7	78.1%	14/16	11/16	81.0
15	α-β（完整）	MCTS（纯随机）	32–0	100.0%	16/16	16/16	53.8
16	α-β（完整）	MCTS（UCT+截断）	30–2	93.8%	15/16	15/16	74.2
17	α-β（完整）	MCTS-PUCT	22–10	68.8%	11/16	11/16	81.0
18	α-β（完整）	MCTS（学习先验）	21–11	65.6%	10/16	11/16	84.7
19	α-β（Texel 权重）	MCTS（纯随机）	32–0	100.0%	16/16	16/16	61.1
20	α-β（Texel 权重）	MCTS（UCT+截断）	16–16	50.0%	7/16	9/16	81.7
21	α-β（Texel 权重）	MCTS-PUCT	10–22	31.2%	4/16	6/16	82.1
22	α-β（Texel 权重）	MCTS（学习先验）	12–20	37.5%	7/16	5/16	80.3
23	MCTS（纯随机）	MCTS（UCT+截断）	0–32	0.0%	0/16	0/16	58.9
24	MCTS（纯随机）	MCTS-PUCT	0–32	0.0%	0/16	0/16	63.6
25	MCTS（纯随机）	MCTS（学习先验）	0–32	0.0%	0/16	0/16	59.6
26	MCTS（UCT+截断）	MCTS-PUCT	3–29	9.4%	2/16	1/16	75.2
27	MCTS（UCT+截断）	MCTS（学习先验）	5–27	15.6%	2/16	3/16	73.7
28	MCTS-PUCT	MCTS（学习先验）	20–12	62.5%	11/16	9/16	84.2

各开局的统计

表 B-3 16 个实验开局在主实验中的结果（全部 2144 局）

开局	前两手	1 秒 α-β 评估（白方视角）	对局数	白方胜率	平均手数
#1	d1-d8/f10 g10-g5/a5	+2.34	134	52.2%	74.1
#2	d1-d8/f10 a7-e3/e7	+0.76	134	52.2%	75.9
#3	d1-d8/j8 g10-g5/i5	+3.05	134	54.5%	74.9
#4	a4-d7/i7 g10-g2/j5	+2.67	134	53.0%	75.6
#5	d1-d7/b7 a7-d4/d6	−0.98	134	50.7%	76.5
#6	d1-d9/f9 g10-g2/i4	+1.74	134	50.0%	76.2
#7	g1-g7/g9 d10-d5/a5	+1.56	134	56.7%	74.4
#8	j4-g7/b7 d10-d5/a5	−0.71	134	53.0%	75.2
#9	d1-d7/b7 g10-g4/i4	+2.51	134	49.3%	74.9
#10	g1-g8/i8 d10-d5/i5	+2.71	134	52.2%	74.9
#11	d1-d9/a6 g10-g4/g2	+2.11	134	53.7%	75.8
#12	g1-g8/i6 d10-d3/a3	+2.66	134	50.7%	76.5
#13	d1-d8/i8 g10-g5/j5	+3.83	134	53.7%	75.1
#14	g1-g8/e10 d10-d2/b4	+2.02	134	50.0%	75.3
#15	d1-d7/i7 g10-g5/d8	+1.86	134	50.7%	74.9
#16	g1-g8/e10 d10-d3/h7	+1.22	134	46.3%	76.0

注：每个开局在每组对阵中都使用两次（双方各执白一次），因此各开局的对局数相同。白方胜率在开局之间的差异，反映了开局本身的倾向与 1 秒 α-β“均衡”判断的误差。

修复分隔判定前后的循环赛 Elo

表 B-4 对比第一版程序（分隔判定有漏洞，见 5.10.1 节）与修复后程序的循环赛 Elo。两版使用相同的智能体、开局与对局设计。第一版的数据只用于这一对比与 5.10.1 节的失败案例分析，正文其余结论全部基于修复后的数据。

表 B-4 第一版与修复后程序的循环赛 Elo（200 ms/步，相对一步贪心）

智能体	第一版 Elo	第一版 95% 区间	修复后 Elo	修复后 95% 区间
α-β（完整）	+234	[+155, +295]	+427	[+341, +546]
MCTS-PUCT	+247	[+162, +317]	+362	[+275, +445]
MCTS（学习先验）	+251	[+168, +313]	+298	[+224, +386]
α-β（Texel 权重）	+259	[+169, +326]	+141	[+69, +208]
MCTS（UCT+截断）	+12	[−78, +91]	+28	[−48, +109]
一步贪心	+0	[−0, +0]	+0	[−0, +0]

智能体	第一版 Elo	第一版 95% 区间	修复后 Elo	修复后 95% 区间
MCTS（纯随机）	−419	[−604, −337]	−771	[−820, −717]
随机	−1177	[≤−1200, −1094]	≤−1200	[≤−1200, ≤−1200]

自对弈数据

表 B-5 学习数据（自对弈）的概况

项目	数值
对局数	400（前 320 局训练，后 80 局测试）
白胜 / 黑胜	207 / 193
平均手数（含开局）	83.7
开局	随机 2—4 手，每手从可分离评估前 40 名中均匀选取
双方配置	完整 α-β，200 ms/步

数据文件清单

表 B-6 提交的原始数据文件

文件	大小	内容
`data/raw/main.jsonl`	13.9 MB	主实验 2144 局逐局记录（每步着法、用时、理由、深度、结点数、模拟次数等）
`data/jobs/main.json`	435 KB	主实验任务清单（双方配置、开局、种子）
`data/raw/selfplay.jsonl`	2.9 MB	自对弈 400 局（学习数据）
`data/raw/replicate.jsonl`	1.6 MB	重复实验 384 局
`data/raw/pilot_tau.jsonl`	419 KB	参数试调对局（第 17—32 号开局，第一版程序）
`data/raw/pilot_tau2.jsonl`	424 KB	参数试调对局（续）
`data/openings.json`	3 KB	开局生成记录（含 1 秒 α-β 评估）
`data/positions.json`	38 KB	60 个测试局面
`data/branching.json`	458 KB	自对弈局面的合法着法数与分隔手数
`data/failure.json`	22 KB	失败案例复盘数据
`data/analysis.json`	39 KB	主实验统计分析结果
`data/learn/policy_meta.json`	322 KB	策略学习样本元数据
`data/learn/texel.json`	2.6 MB	Texel 调参报告
`models/policy_weights.json`	4 KB	线性策略参数与训练报告
`models/eval_weights_tuned.json`	2 KB	Texel 权重
`data_v1/raw/main.jsonl`	13.3 MB	第一版程序（分隔判定修复前）的主实验记录，仅用于 5.10.1 节

附录 C　运行说明

代码仓库与在线试玩

本项目的全部源代码、原始实验数据与报告源文件公开在 GitHub 仓库 https://github.com/NEUMelon/amazon-ai 中；网页游戏同时通过 GitHub Pages 发布，可在 https://neumelon.github.io/amazon-ai/ 直接在线试玩，页面上也提供报告 PDF 的下载。仓库中只省略了一个可由脚本重新生成的学习中间文件（`data/learn/policy_X.f32`，约 40 MB）。

直接运行游戏

提交材料中的 `amazons.html` 是一个完全独立的单文件网页（约 160 KB），包含界面、引擎与模型参数，不需要安装任何软件，也不需要网络连接：

1. 用 Chrome、Edge、Firefox 或 Safari 的较新版本**双击打开** `amazons.html`；
2. 在右侧“对局设置”中选择模式（人机对弈我执白 / 我执黑、AI 对 AI 演示、双人对弈）、双方的 AI 算法和每步思考时间，点击“新局”；
3. 人类走子分三次点击：先点击己方的一个后，再点击绿色圆点（目的地），最后点击红色圆点（射箭位置）；右键或 Esc 取消选择。“悔棋”撤回到上一次轮到人类走棋时；“提示”让 AI 为当前行棋方推荐一手；AI 对 AI 时可“暂停/继续”；
4. 棋盘上方的复选框控制各可视化图层（领地、搜索候选、主变例、距离数字、箭的手数），右侧标签页查看搜索过程、局势曲线、棋谱与算法说明。

开发环境

本项目在以下环境中开发与运行：Linux（x86-64，4 个 CPU 核心），Node.js 22.23，Python 3.13（NumPy 2.5、SciPy 1.18、Matplotlib 3.11），Graphviz 2.43，pandoc 3.1，LibreOffice 24.2。引擎与网页只依赖 Node.js（运行实验）或浏览器（运行游戏）；Python 仅用于离线训练、统计与绘图，Playwright 仅用于界面测试与截图。

复现步骤

源代码包解压后，在项目根目录依次执行下列命令即可复现全部结果。括号中为在上述环境中的大致耗时。

```
# 1. 单元测试（约1 分钟）
node tests/run_tests.js
# 2. 生成均衡开局与任务清单
node experiments/make_openings.js
node experiments/make_jobs.js selfplay
# 3. 自对弈（400 局，4 进程）
node experiments/runner.js data/jobs/selfplay.json data/raw/selfplay.jsonl 4
# 4. 测试局面、学习数据抽取、策略训练与Texel 调参
node experiments/make_positions.js data/raw/selfplay.jsonl data/positions.json
node learning/extract.js data/raw/selfplay.jsonl data/learn
python3 learning/train_policy.py data/learn models
python3 learning/texel.py data/learn models
# 5. 打包网页
node game/build.js
# 6. 主实验（中断后重新执行同一命令会自动续跑）
node experiments/make_jobs.js main
node experiments/runner.js data/jobs/main.json data/raw/main.jsonl 4
# 6b. 重复实验
node experiments/make_jobs.js replicate
node experiments/runner.js data/jobs/replicate.json data/raw/replicate.jsonl 4
# 7. 专项实验（单进程运行，避免计时干扰）
node experiments/efficiency.js data/positions.json data/efficiency.json
node experiments/quality.js ref data/positions.json data/quality_ref.json
node experiments/quality.js eval data/positions.json data/quality_ref.json
data/quality.json ab,mcts-puct,mcts-cutoff,mcts-learned,ab-tuned,ab-
cheapOrder,mcts-pure,greedy
node experiments/variance.js data/positions.json data/variance.json ab,mcts-
puct,mcts-cutoff,mcts-learned
node analysis/branching.js data/raw/selfplay.jsonl
node analysis/failure.js
# 8. 统计分析与绘图
python3 analysis/analyze.py
python3 analysis/learn_tables.py
python3 analysis/efficiency_report.py
python3 analysis/replicate_report.py
python3 analysis/appendix_tables.py
python3 report/figscripts/figs_basic.py
python3 report/figscripts/figs_results.py
python3 report/figscripts/figs_misc.py
# 9. 界面测试与截图
node tests/ui_test.js
node tests/screenshots.js
python3 report/figscripts/crop_ui.py
node tests/browser_bench.js
python3 analysis/ui_table.py
# 10. 生成报告（Word + PDF）
python3 report/gen_appendix_e.py
python3 report/build.py
```

关于可重复性的说明

- 所有随机过程（Zobrist 随机数、开局生成、自对弈开局、MCTS 与随机智能体）都使用固定种子，种子写在任务清单中；

- 以**时间**为预算的搜索（α-β 的迭代加深何时停止、MCTS 做多少次模拟）受机器速度与负载影响，因此在不同机器上重跑时，具体对局的着法可能不同，统计结论应在置信区间范围内一致；以**模拟次数**为预算时，MCTS 的结果完全确定（单元测试“相同种子可复现”）；
- 学习脚本使用全批量优化，结果确定。

附录 D　参考文献核验记录

核验流程

作业要求参考文献“真实、可核验、与正文相关”。由于本项目使用生成式 AI 辅助写作，而生成式模型编造或错误著录文献的问题已有大量报道[406,407]，本项目对参考文献采用了“先检索核验、后引用”的流程：

1. **候选生成**：先由 AI 根据记忆列出候选文献（作者、年份、题名、主题）。本项目最初以五子棋为题，列出了 244 条候选（`refs/candidates.py`）；换题为亚马逊棋后，又补充了 37 条与亚马逊棋、选择性搜索和 MCTS 改进相关的候选（`refs/candidates_amz.py`），合计 281 条；
2. **逐条核验**：脚本 `refs/verify.py` 依次在 Crossref、arXiv、NeurIPS 论文集目录、OpenLibrary 等数据源检索每条候选。只有同时满足“题名相似度（归一化后的序列匹配率）不低于 0.88、第一作者姓氏出现在检索结果的作者列表中、出版年份相差不超过 2 年”的条目才记为“已核验”；
3. **以检索结果著录**：已核验条目的作者、题名、刊名、卷期页、DOI 等著录信息一律取自检索结果，而不是候选中的记忆信息；
4. **主题检索补充**：脚本 `refs/discover.py` 以多组主题关键词（如 “alpha-beta search”“proof-number search”“Monte Carlo tree search”“game of Amazons”“evaluation function tuning”“Elo rating”等）在 Crossref 上检索，得到 1544 条结果；换题后，`refs/discover_amz.py` 又以 56 组关键词（如 “Amazons evaluation function”“Monte-Carlo Amazons”“progressive widening”“MCTS early playout termination”“beam search game playing”）补充检索，得到 732 条结果（其中 165 条与第一轮重复）。从中人工筛选出与本文主题直接相关的 226 条（键名以 `cr_` 开头），其著录信息同样直接来自 Crossref；
5. **引用检查**：排版脚本在生成报告时检查正文中的每一个引用键，若引用了未核验的条目则直接报错终止；参考文献表按正文中首次引用的顺序自动编号，因此不存在“列出但未引用”或“引用但未列出”的条目。

核验结果

表 D-1　参考文献核验结果统计

来源	核验渠道	核验通过	其中被正文引用
主题检索补充	Crossref（DOI 元数据）	226	169

来源	核验渠道	核验通过	其中被正文引用
记忆候选	arXiv API	37	33
记忆候选	Crossref（DOI 元数据）	209	202
记忆候选	NeurIPS 论文集目录	6	6
记忆候选	OpenLibrary（图书）	10	10
记忆候选	官方页面核对	1	1
合计	—	489	421

注：“核验通过”指满足上述三项匹配条件；“被正文引用”为最终出现在参考文献表中的条目数。

在第一阶段的 244 条记忆候选中，首轮有 23 条未能核验；换题后补充的 37 条全部核验通过。逐条检查后发现，其中一部分是检索方式的问题（例如 arXiv 接口的速率限制、标题中的停用词、图书没有 DOI），改进检索方式后得以核验；另一部分是候选信息本身有误，例如：一篇论文的正式发表版题名与预印本不同（按检索结果更正）；一本教材被 Crossref 错误地匹配到一篇同名短文（改用 OpenLibrary 核验）；一篇论文的 DOI 需要人工查找后确认。最终仍有 18 条无法核验，全部弃用，列于表 D-2。其中 4 条在主题检索中找到了对应的正式出版版本（著录信息不同，如作者姓名的拼写或出版年份），改为引用主题检索得到的条目。

表 D-2 未通过核验而弃用的候选文献

候选键	候选信息（作者，年份，题名）	处理
`turing1953`	Turing，1953，Digital computers applied to games	弃用；改引主题检索中找到的对应条目（cr_turing1988_359）
`wagner2001`	Wagner，2001，Solving renju	弃用；改引主题检索中找到的对应条目（cr_wgner2001_726）
`erdos1973`	Erdős，1973，On a combinatorial game	弃用；改引主题检索中找到的对应条目（cr_erds1973_392）
`cazenave2009`	Cazenave，2009，Nested Monte-Carlo search	弃用，正文不引用
`finnsson2008`	Finnsson，2008，Simulation-based approach to general game playing	弃用，正文不引用
`danihelka2022`	Danihelka，2022，Policy improvement by planning with Gumbel	弃用，正文不引用
`antonoglou2022`	Antonoglou，2022，Planning in Stochastic Environments with a Learned Model	弃用，正文不引用
`baxter2000`	Baxter，2000，Learning to Play Chess Using Temporal Differences	弃用，正文不引用
`freisleben1995`	Freisleben，1995，A neural network that learns to play Five-in-a-Row	弃用，正文不引用
`tesauro2001`	Tesauro，2001，Comparison training of chess	弃用，正文不引用

候选键	候选信息（作者，年份，题名）	处理
	evaluation functions	
platt1999	Platt，1999，Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods	弃用，正文不引用
genesereth2005	Genesereth，2005，General Game Playing: Overview of the AAAI Competition	弃用，正文不引用
junghanns1997	Junghanns，1997，Diminishing Returns for Additional Search in Chess	弃用，正文不引用
mckeeman1998	McKeeman，1998，Differential testing for software	弃用，正文不引用
yoshizoe2007	Yoshizoe，2007，Lambda Depth-First Proof Number Search and Its Application to Go	弃用，正文不引用
kishimoto2005	Kishimoto，2005，A General Solution to the Graph History Interaction Problem	弃用；改引主题检索中找到的对应条目（cr_kishimoto2005_680）
pearl1984	Pearl，1984，Heuristics: Intelligent Search Strategies for Computer Problem Solving	弃用，正文不引用
henderson2009	Henderson，2009，Solving 8x8 Hex	弃用，正文不引用

注：“未通过核验”并不意味着文献一定不存在，也可能是题名、年份等记忆信息有误，或者该文献未被上述数据源收录。按照“无法核实则不引用”的原则，本文一律不引用。

局限说明

自动核验只能保证“这条文献存在，且著录信息与数据库一致”，不能保证“文献内容确实支持正文中引用它的那句话”。对于经典文献与核心文献（如 α-β、MCTS、UCT、AlphaZero、Lieberum 的亚马逊棋评估函数、Lorentz 的亚马逊棋 MCTS 程序等），正文中的描述与本人（及 AI）对其内容的了解一致；对于主题检索补充的文献，正文的引用多数停留在“题名所示的研究主题”层面，本人没有逐篇阅读全文。读者若需要引用其中的具体结论，应查阅原文。

附录 E　源代码

本附录收录引擎、网页逻辑与对局实验框架的全部源代码（由排版脚本直接从源文件读取）。网页的 HTML/CSS、其余实验脚本、学习脚本、分析脚本与测试代码见提交的源代码包。代码中的注释为中文，便于阅读。

engine/core.js （246 行）

```
'use strict';
/*
 * core.js —— 10×10 亚马逊棋的棋盘表示、走法生成与距离计算
 *
 * 规则要点：
 *   1. 双方各有4 个“亚马逊”（后），白方先行；
 *   2. 一步棋= 移动一个己方的后（像国际象棋的后一样沿8 个方向直线移动任意格，不能越过任何棋子或障碍）
 *      + 从落点再像后一样射出一支“箭”，箭落的格子永久成为障碍；
 *   3. 轮到某方时若无合法走法，该方判负（不存在和棋）。
 *
 * 表示：格点编号idx = r * 10 + c，r = 0 为最上一行（第10 行），c = 0 为a 列。
 *       cells[idx]：0 空，1 白后，2 黑后，3 箭（障碍）。
 *       一步棋编码为整数from * 10000 + to * 100 + arrow，字符串形如"d1-d7/g7"。
 */
const N = 10, NN = 100;
const EMPTY = 0, WHITE = 1, BLACK = 2, ARROW = 3;
const DIRS = [[-1, -1], [-1, 0], [-1, 1], [0, -1], [0, 1], [1, -1], [1, 0], [1, 1]];
const INF = 99;

// 射线表：RAYS[idx][d] = 从idx 出发沿方向d 依次经过的格点（不含idx）
const RAYS = [];
const NEIGH = [];          // 8 邻域
for (let r = 0; r < N; r++) for (let c = 0; c < N; c++) {
  const rays = [], nb = [];
  for (const [dr, dc] of DIRS) {
    const ray = [];
    let rr = r + dr, cc = c + dc;
    while (rr >= 0 && rr < N && cc >= 0 && cc < N) { ray.push(rr * N + cc); rr += dr; cc +=
dc; }
    rays.push(ray);
    if (ray.length) nb.push(ray[0]);
  }
  RAYS.push(rays); NEIGH.push(nb);
}

// Zobrist 随机数（两段32 位，避免JS 位运算的53 位限制）
function mulberry32(a) {
  return function () {
    a |= 0; a = a + 0x6D2B79F5 | 0;
    let t = Math.imul(a ^ a >>> 15, 1 | a);
    t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t;
    return ((t ^ t >>> 14) >>> 0) / 4294967296;
  };
}
const zr = mulberry32(20241107);
const ZLO = [], ZHI = [];
for (let i = 0; i < NN * 4; i++) { ZLO.push((zr() * 4294967296) >>> 0); ZHI.push((zr() *
4294967296) >>> 0); }
const ZSIDE_LO = (zr() * 4294967296) >>> 0, ZSIDE_HI = (zr() * 4294967296) >>> 0;

const COLS = 'abcdefghij';
const sqToStr = i => COLS[i % N] + (N - ((i / N) | 0));
function strToSq(s) {
```

```
  const c = COLS.indexOf(s[0].toLowerCase()), row = parseInt(s.slice(1), 10);
  if (c < 0 || !(row >= 1 && row <= N)) throw new Error('bad square ' + s);
  return (N - row) * N + c;
}
const mFrom = m => (m / 10000) | 0, mTo = m => ((m / 100) | 0) % 100, mArrow = m => m % 100;
const mk = (f, t, a) => f * 10000 + t * 100 + a;
const moveToStr = m => `${sqToStr(mFrom(m))}-${sqToStr(mTo(m))}/${sqToStr(mArrow(m))}`;
function parseMove(s) {
  const m = /^([a-j]\d{1,2})-([a-j]\d{1,2})\/([a-j]\d{1,2})$/i.exec(s.trim());
  if (!m) throw new Error('bad move ' + s);
  return mk(strToSq(m[1]), strToSq(m[2]), strToSq(m[3]));
}

const START_WHITE = ['a4', 'd1', 'g1', 'j4'].map(strToSq);
const START_BLACK = ['a7', 'd10', 'g10', 'j7'].map(strToSq);

class Board {
  constructor() {
    this.cells = new Int8Array(NN);
    this.queens = [null, START_WHITE.slice(), START_BLACK.slice()];
    for (const q of this.queens[WHITE]) this.cells[q] = WHITE;
    for (const q of this.queens[BLACK]) this.cells[q] = BLACK;
    this.side = WHITE;
    this.history = [];
    this.hashLo = 0; this.hashHi = 0;
    for (let i = 0; i < NN; i++) if (this.cells[i]) { this.hashLo ^= ZLO[i * 4 +
this.cells[i]]; this.hashHi ^= ZHI[i * 4 + this.cells[i]]; }
  }
  static fromMoves(list) {
    const b = new Board();
    for (const s of list) {
      const m = typeof s === 'number' ? s : parseMove(s);
      if (!b.isLegal(m)) throw new Error('illegal move ' + (typeof s === 'number' ?
moveToStr(s) : s));
      b.play(m);
    }
    return b;
  }
  clone() {
    const b = Object.create(Board.prototype);
    b.cells = this.cells.slice(); b.queens = [null, this.queens[1].slice(),
this.queens[2].slice()];
    b.side = this.side; b.history = this.history.slice(); b.hashLo = this.hashLo; b.hashHi =
this.hashHi;
    return b;
  }
  get arrows() { return this.history.length; }
  _set(i, v) {
    const old = this.cells[i];
    if (old) { this.hashLo ^= ZLO[i * 4 + old]; this.hashHi ^= ZHI[i * 4 + old]; }
    this.cells[i] = v;
    if (v) { this.hashLo ^= ZLO[i * 4 + v]; this.hashHi ^= ZHI[i * 4 + v]; }
  }
  play(m) {
    const f = mFrom(m), t = mTo(m), a = mArrow(m), s = this.side;
    const qs = this.queens[s]; qs[qs.indexOf(f)] = t;
    this._set(f, EMPTY); this._set(t, s); this._set(a, ARROW);
    this.history.push(m);
    this.side = 3 - s; this.hashLo ^= ZSIDE_LO; this.hashHi ^= ZSIDE_HI;
  }
  undo() {
    const m = this.history.pop();
    const f = mFrom(m), t = mTo(m), a = mArrow(m), s = 3 - this.side;
    this.side = s; this.hashLo ^= ZSIDE_LO; this.hashHi ^= ZSIDE_HI;
    this._set(a, EMPTY); this._set(t, EMPTY); this._set(f, s);
    const qs = this.queens[s]; qs[qs.indexOf(t)] = f;
  }
  lastMove() { return this.history.length ? this.history[this.history.length - 1] : -1; }
  // 某方是否还有合法走法：只要某个后旁边有空格，就可以走一步再把箭射回原处
```

```
  canMove(side = this.side) {
    for (const q of this.queens[side]) for (const n of NEIGH[q]) if (this.cells[n] ===
EMPTY) return true;
    return false;
  }
  isOver() { return !this.canMove(this.side); }
  get winner() { return this.isOver() ? 3 - this.side : 0; }
  // 一个后从 q 出发一步能到达的空格（目的地）
  queenTargets(q, out = []) {
    const rays = RAYS[q], cells = this.cells;
    for (let d = 0; d < 8; d++) { const ray = rays[d]; for (let k = 0; k < ray.length; k++) {
const x = ray[k]; if (cells[x] !== EMPTY) break; out.push(x); } }
    return out;
  }
  // 全部合法走法（整数编码）。开局时约2176 步
  genMoves(side = this.side) {
    const out = [], cells = this.cells;
    for (const f of this.queens[side]) {
      cells[f] = EMPTY;                                    // 射箭时原位置已空出
      const rays = RAYS[f];
      for (let d = 0; d < 8; d++) {
        const ray = rays[d];
        for (let k = 0; k < ray.length; k++) {
          const t = ray[k]; if (cells[t] !== EMPTY) break;
          const base = f * 10000 + t * 100, rt = RAYS[t];
          for (let e = 0; e < 8; e++) { const r2 = rt[e]; for (let j = 0; j < r2.length; j++)
{ const a = r2[j]; if (cells[a] !== EMPTY) break; out.push(base + a); } }
        }
      }
      cells[f] = side;
    }
    return out;
  }
  countMoves(side = this.side) {
    let n = 0; const cells = this.cells;
    for (const f of this.queens[side]) {
      cells[f] = EMPTY;
      for (const ray of RAYS[f]) for (const t of ray) { if (cells[t] !== EMPTY) break; for
(const r2 of RAYS[t]) for (const a of r2) { if (cells[a] !== EMPTY) break; n++; } }
      cells[f] = side;
    }
    return n;
  }
  isLegal(m) {
    if (!(m >= 0)) return false;
    const f = mFrom(m), t = mTo(m), a = mArrow(m);
    if (f >= NN || t >= NN || a >= NN || this.cells[f] !== this.side || this.isOver()) return
false;
    if (!this._reach(f, t)) return false;
    this.cells[f] = EMPTY; const ok = this._reach(t, a); this.cells[f] = this.side;
    return ok;
  }
  _reach(f, t) {
    for (const ray of RAYS[f]) for (const x of ray) { if (this.cells[x] !== EMPTY) break; if
(x === t) return true; }
    return false;
  }
  // 随机走法（不生成全部走法）：随机选后、方向、距离，再随机选箭。用于快速随机模拟
  randomMove(rng, side = this.side) {
    const cells = this.cells, qs = this.queens[side];
    for (let tries = 0; tries < 64; tries++) {
      const f = qs[(rng() * 4) | 0];
      const tg = this.queenTargets(f);
      if (!tg.length) continue;
      const t = tg[(rng() * tg.length) | 0];
      cells[f] = EMPTY;
      const ag = this.queenTargets(t);
      cells[f] = side;
      if (!ag.length) continue;                            // 理论上不会发生（至少能射回原处）
```

```
      return mk(f, t, ag[(rng() * ag.length) | 0]);
    }
    const all = this.genMoves(side);
    return all.length ? all[(rng() * all.length) | 0] : -1;
  }
  toString() {
    let s = '';
    for (let r = 0; r < N; r++) {
      s += String(N - r).padStart(2) + ' ';
      for (let c = 0; c < N; c++) s += '.WBx'[this.cells[r * N + c]] + ' ';
      s += '\n';
    }
    return s + '   ' + COLS.split('').join(' ');
  }
}

// ---------------------------------------------------------------- 距离计算
// 后距离（queen distance）：从某方任一后出发，最少需要几步“后的移动”能到达该格（不考虑对方阻挡的动态变化）
function queenDist(b, side, out = new Uint8Array(NN)) {
  out.fill(INF);
  const cells = b.cells, q = [];
  for (const s of b.queens[side]) { out[s] = 0; q.push(s); }
  let h = 0;
  while (h < q.length) {
    const x = q[h++], d = out[x] + 1, rays = RAYS[x];
    for (let k = 0; k < 8; k++) {
      const ray = rays[k];
      for (let j = 0; j < ray.length; j++) {
        const y = ray[j]; if (cells[y] !== EMPTY) break;
        if (out[y] > d) { out[y] = d; q.push(y); }
        else if (out[y] < d) continue;
      }
    }
  }
  return out;
}
// 王距离（king distance）：每步只走一格（8 邻域）
function kingDist(b, side, out = new Uint8Array(NN)) {
  out.fill(INF);
  const cells = b.cells, q = [];
  for (const s of b.queens[side]) { out[s] = 0; q.push(s); }
  let h = 0;
  while (h < q.length) {
    const x = q[h++], d = out[x] + 1;
    for (const y of NEIGH[x]) if (cells[y] === EMPTY && out[y] > d) { out[y] = d; q.push(y);
}
  }
  return out;
}
// 双方的后距离都有限的空格数；为0 表示双方已被完全分隔（进入“填格”阶段）
function contestedCount(b, dW = queenDist(b, WHITE), dB = queenDist(b, BLACK)) {
  let n = 0;
  for (let i = 0; i < NN; i++) if (b.cells[i] === EMPTY && dW[i] < INF && dB[i] < INF) n++;
  return n;
}
// 已分隔：没有争夺格，且双方的后不在同一个“非箭格子”的8 邻域连通分量中（敌对的后紧挨着时，移开一方即会重新连通）
function isSeparated(b) {
  if (contestedCount(b) !== 0) return false;
  const seen = new Uint8Array(NN), st = b.queens[WHITE].slice();
  for (const q of st) seen[q] = 1;
  while (st.length) { const x = st.pop(); for (const y of NEIGH[x]) { if (seen[y] ||
b.cells[y] === ARROW) continue; if (b.cells[y] === BLACK) return false; seen[y] = 1;
st.push(y); } }
  return true;
}

module.exports = {
```

```
  N, NN, EMPTY, WHITE, BLACK, ARROW, INF, DIRS, RAYS, NEIGH, Board, mulberry32,
  sqToStr, strToSq, moveToStr, parseMove, mFrom, mTo, mArrow, mk, queenDist, kingDist,
contestedCount, isSeparated,
};
```

engine/eval.js （173 行）

```
/*
 * eval.js — 亚马逊棋静态评估
 *
 * 采用Lieberum (2005) 提出、后被多数亚马逊棋程序沿用的“领地+ 位置+ 机动性”评估框架:
 *   t1: 按后距离 (queen distance) 划分领地，谁先到达某格谁得1 分，同时到达给行棋方κ 分;
 *   t2: 同样的划分，改用王距离 (king distance) ;
 *   c1: Σ 2·(2^-dW − 2^-dB)，后距离意义下的“位置”项;
 *   c2: Σ clip((kB − kW)/6, −1, 1)，王距离意义下的“位置”项;
 *   mob: 双方各后的机动性 (一步可到达的格数，截断到12) 之差;
 *   w : Σ 2^-|dW − dB| (双方都能到达的格子) ，衡量局面“还剩多少争夺”，用来确定阶段。
 * 评估值= Σ_k [ p·aE_k + (1−p)·aL_k ] · f_k，其中p = min(1, w / W0) 是阶段系数。
 * 这样的写法对权重是线性的，可以直接用Texel 方法拟合 (learning/texel.py) 。
 *
 * 双方被完全分隔后 (没有任何空格同时可被双方到达) ，改用“区域格数”精确地比较剩余步数的上界。
 * 分数以“格”为单位，正值对行棋方有利。
 */
'use strict';
const C = require('./core.js');
const { NN, EMPTY, WHITE, BLACK, ARROW, INF, RAYS, NEIGH } = C;

const FEATURES = ['t1', 't2', 'c1', 'c2', 'mob'];
const NF = FEATURES.length;
// 手工设定的默认权重: 前5 个是开局 (p=1) 权重，后5 个是终局 (p=0) 权重
const DEFAULT_WEIGHTS = [0.20, 0.20, 0.30, 0.30, 0.25, 1.00, 0.20, 0.05, 0.05, 0.05];
const W0 = 60;              // 阶段归一化常数 (开局时w 约为70，此后逐渐减小)
const KAPPA = 0.2;          // 同时到达的格子给行棋方的分数
const WIN = 10000;
const SEP_BONUS = 200;

const qbuf = new Int32Array(NN * 8);
const POW2 = new Float64Array(INF + 1);
for (let i = 0; i <= INF; i++) POW2[i] = Math.pow(2, -i);

// 快速后距离BFS (结果写入out)
function queenDist(cells, queens, out) {
  out.fill(INF);
  let h = 0, t = 0;
  for (let i = 0; i < 4; i++) { const s = queens[i]; out[s] = 0; qbuf[t++] = s; }
  while (h < t) {
    const x = qbuf[h++], d = out[x] + 1, rays = RAYS[x];
    for (let k = 0; k < 8; k++) {
      const ray = rays[k];
      for (let j = 0; j < ray.length; j++) {
        const y = ray[j]; if (cells[y] !== EMPTY) break;
        if (out[y] > d) { out[y] = d; qbuf[t++] = y; }
      }
    }
  }
  return out;
}
function kingDist(cells, queens, out) {
  out.fill(INF);
  let h = 0, t = 0;
  for (let i = 0; i < 4; i++) { const s = queens[i]; out[s] = 0; qbuf[t++] = s; }
  while (h < t) {
    const x = qbuf[h++], d = out[x] + 1, nb = NEIGH[x];
    for (let j = 0; j < nb.length; j++) { const y = nb[j]; if (cells[y] === EMPTY && out[y] >
d) { out[y] = d; qbuf[t++] = y; } }
  }
  return out;
```

```
}
function queenMobility(cells, q) {
  let n = 0; const rays = RAYS[q];
  for (let k = 0; k < 8; k++) { const ray = rays[k]; for (let j = 0; j < ray.length; j++) {
if (cells[ray[j]] !== EMPTY) break; n++; } }
  return n;
}

const dW = new Uint8Array(NN), dB = new Uint8Array(NN), kW = new Uint8Array(NN), kB = new
Uint8Array(NN);

/**
 * 计算特征（白方视角）。返回{ f: Float64Array(5), p, w, contested, regionW, regionB }
 * 行棋方side 决定κ 的符号。
 */
function features(b, out = new Float64Array(NF)) {
  const cells = b.cells, qW = b.queens[WHITE], qB = b.queens[BLACK];
  queenDist(cells, qW, dW); queenDist(cells, qB, dB);
  kingDist(cells, qW, kW); kingDist(cells, qB, kB);
  const kap = b.side === WHITE ? KAPPA : -KAPPA;
  let t1 = 0, t2 = 0, c1 = 0, c2 = 0, w = 0, contested = 0, rW = 0, rB = 0;
  for (let i = 0; i < NN; i++) {
    if (cells[i] !== EMPTY) continue;
    const a = dW[i], c = dB[i];
    if (a < INF) rW++;
    if (c < INF) rB++;
    if (a === INF && c === INF) continue;
    if (a < c) t1 += 1; else if (a > c) t1 -= 1; else t1 += kap;
    const ka = kW[i], kc = kB[i];
    if (ka < kc) t2 += 1; else if (ka > kc) t2 -= 1; else if (ka < INF) t2 += kap;
    c1 += 2 * (POW2[a] - POW2[c]);
    let v = (kc - ka) / 6; if (v > 1) v = 1; else if (v < -1) v = -1;
    c2 += v;
    if (a < INF && c < INF) { contested++; const diff = a > c ? a - c : c - a; w +=
POW2[diff]; }
  }
  let mob = 0;
  for (let i = 0; i < 4; i++) { const m = queenMobility(cells, qW[i]); mob += m < 12 ? m :
12; }
  for (let i = 0; i < 4; i++) { const m = queenMobility(cells, qB[i]); mob -= m < 12 ? m :
12; }
  out[0] = t1; out[1] = t2; out[2] = c1; out[3] = c2; out[4] = mob / 4;
  const p = w / W0 > 1 ? 1 : w / W0;
  return { f: out, p, w, contested, regionW: rW, regionB: rB };
}

// 分隔后的区域格数：用王距离可达的空格数（剩余步数的上界）
function separatedScore(b, info) {
  // info.regionW/B 用后距离可达性统计；分隔后与王距离可达性一致（同一个连通区域）
  const own = b.side === WHITE ? info.regionW : info.regionB;
  const opp = b.side === WHITE ? info.regionB : info.regionW;
  const diff = own - opp - 0.5;            // 步数相同时行棋方先用完，所以减0.5
  return (diff > 0 ? SEP_BONUS : -SEP_BONUS) + diff;
}

const fbuf = new Float64Array(NF);
/** 行棋方视角的评估值*/
function evaluate(b, weights = DEFAULT_WEIGHTS) {
  if (!b.canMove(b.side)) return -WIN;
  const info = features(b, fbuf);
  if (info.contested === 0 && !queensTouch(b.cells, b.queens[WHITE])) return
separatedScore(b, info);
  const p = info.p, f = info.f;
  let s = 0;
  for (let k = 0; k < NF; k++) s += (p * weights[k] + (1 - p) * weights[NF + k]) * f[k];
  return b.side === WHITE ? s : -s;
}

/**
```

```
 * 判断是否已分隔：在“空格+ 后所在格”构成的8 邻域连通分量中，没有任何一个分量同时含有双方的后。
 * 注意：只检查“没有空格同时可被双方到达”是不够的——两个敌对的后紧挨着、彼此之间没有空格时，
 * 任何一方把后移开，空出的格子就会把两块区域连通（第一版实现正是这样判断的，见5.10 节）。
 */
const stk = new Int8Array(NN), seen = new Uint8Array(NN);
function separated(b) {
  const cells = b.cells;
  queenDist(cells, b.queens[WHITE], dW); queenDist(cells, b.queens[BLACK], dB);
  for (let i = 0; i < NN; i++) if (cells[i] === EMPTY && dW[i] < INF && dB[i] < INF) return
false;
  return !queensTouch(cells, b.queens[WHITE]);
}
// 从白方的后出发，在非箭的格子上做8 邻域洪泛，若能碰到黑方的后则返回true
function queensTouch(cells, qW) {
  seen.fill(0); let top = 0;
  for (const q of qW) { seen[q] = 1; stk[top++] = q; }
  while (top) {
    const x = stk[--top];
    for (const y of C.NEIGH[x]) {
      if (seen[y]) continue;
      const v = cells[y];
      if (v === ARROW) continue;
      if (v === BLACK) return true;
      seen[y] = 1; stk[top++] = y;
    }
  }
  return false;
}

/** Texel 用：返回10 维线性特征（白方视角）与是否分隔*/
function texelFeatures(b) {
  const info = features(b, new Float64Array(NF));
  const x = new Array(2 * NF);
  for (let k = 0; k < NF; k++) { x[k] = info.p * info.f[k]; x[NF + k] = (1 - info.p) *
info.f[k]; }
  return { x, separated: info.contested === 0 && !queensTouch(b.cells, b.queens[WHITE]), p:
info.p, w: info.w, regionW: info.regionW, regionB: info.regionB };
}

/** 供界面使用：每个空格的归属（1 白/ -1 黑/ 0 均势/ null 无人可达）与双方后距离*/
function territoryMap(b) {
  const cells = b.cells;
  const a = queenDist(cells, b.queens[WHITE], new Uint8Array(NN));
  const c = queenDist(cells, b.queens[BLACK], new Uint8Array(NN));
  const own = new Array(NN).fill(null);
  for (let i = 0; i < NN; i++) {
    if (cells[i] !== EMPTY) continue;
    if (a[i] === INF && c[i] === INF) own[i] = null;
    else own[i] = a[i] < c[i] ? 1 : a[i] > c[i] ? -1 : 0;
  }
  return { owner: own, dW: Array.from(a), dB: Array.from(c) };
}

module.exports = { FEATURES, NF, DEFAULT_WEIGHTS, W0, KAPPA, WIN, SEP_BONUS, evaluate,
separated, features, texelFeatures, territoryMap, queenDist, kingDist, queenMobility };
```

engine/movegen.js （181 行）

```
/*
 * movegen.js — 着法的廉价打分与筛选
 *
 * 亚马逊棋开局时约有2176 个合法走法，中局也常有数百个，无法全宽搜索。
 * 这里给每个走法一个只需常数次查表的“廉价分数”，用于：α-β 的束宽筛选与排序、
 * MCTS 的渐进展开顺序与先验、贪心与模拟策略。分数由两部分组成：
 *   后的移动：0.5 ×（到达点的机动性 − 出发点的机动性），机动性截断到12；
 *   箭的位置：2 ×（箭旁相邻的对方后数）−1 ×（箭旁相邻的己方后数）
 *             + 1 ×（该格对方后距离 ≤ 己方后距离，即对方的领地或均势格）
```

```
 *              + 2^(1-d)（d 为对方后到该格的距离，越靠近对方的后越好）。
 * 第一项鼓励把后走到开阔处，第二项鼓励用箭封堵对方的后、争夺对方领地而不堵自己。
 * 这些规则是本文作者根据亚马逊棋的常见经验设定的（Lieberum 2005；Kloetzer 等2007），
 * 线性策略（policy.js）用同一组特征从对局数据中学习权重。
 */
'use strict';
const C = require('./core.js');
const E = require('./eval.js');
const { NN, EMPTY, INF, RAYS, NEIGH } = C;

const ROW = new Int8Array(NN), COL = new Int8Array(NN);
for (let i = 0; i < NN; i++) { ROW[i] = (i / 10) | 0; COL[i] = i % 10; }
function adj(a, b) { const dr = ROW[a] - ROW[b], dc = COL[a] - COL[b]; return a !== b && dr
<= 1 && dr >= -1 && dc <= 1 && dc >= -1; }

const dMe = new Uint8Array(NN), dOp = new Uint8Array(NN);
const oppAdj = new Int8Array(NN), ownAdj = new Int8Array(NN);
const POW2 = new Float64Array(INF + 1);
for (let i = 0; i <= INF; i++) POW2[i] = i === 0 ? 0 : Math.pow(2, 1 - i);   // 1, 0.5, 0.25
…（距离1 记1）
const W_DEFAULT = { mob: 0.5, oppAdj: 2, ownAdj: 1, terr: 1 };

// 所有候选走法及分数的缓冲区（容量足够开局局面）
const MAXM = 4096;
const bufM = new Int32Array(MAXM), bufS = new Float64Array(MAXM);

/** 预计算结点信息：双方后距离与相邻关系*/
function prepare(b) {
  const s = b.side, o = 3 - s, cells = b.cells;
  E.queenDist(cells, b.queens[s], dMe); E.queenDist(cells, b.queens[o], dOp);
  oppAdj.fill(0); ownAdj.fill(0);
  for (const q of b.queens[o]) for (const n of NEIGH[q]) oppAdj[n]++;
  for (const q of b.queens[s]) for (const n of NEIGH[q]) ownAdj[n]++;
}

/**
 * 生成全部走法与廉价分数；返回走法数n，结果在bufM/bufS 中。
 * w 为权重对象（默认W_DEFAULT）。
 */
function scoreAll(b, w = W_DEFAULT) {
  prepare(b);
  const s = b.side, cells = b.cells;
  let n = 0;
  for (const f of b.queens[s]) {
    const mobF = Math.min(12, E.queenMobility(cells, f));
    cells[f] = EMPTY;
    const rf = RAYS[f];
    for (let d = 0; d < 8; d++) {
      const ray = rf[d];
      for (let k = 0; k < ray.length; k++) {
        const t = ray[k]; if (cells[t] !== EMPTY) break;
        const mobT = Math.min(12, E.queenMobility(cells, t));
        const qs = w.mob * (mobT - mobF);
        const base = f * 10000 + t * 100, rt = RAYS[t];
        for (let e = 0; e < 8; e++) {
          const r2 = rt[e];
          for (let j = 0; j < r2.length; j++) {
            const a = r2[j]; if (cells[a] !== EMPTY) break;
            const own = ownAdj[a] - (adj(a, f) ? 1 : 0) + (adj(a, t) ? 1 : 0);
            const terr = (dOp[a] <= dMe[a] && dOp[a] < INF ? 1 : 0) + POW2[dOp[a]];
            bufM[n] = base + a; bufS[n] = qs + w.oppAdj * oppAdj[a] - w.ownAdj * own + w.terr
* terr; n++;
          }
        }
      }
    }
    cells[f] = s;
  }
  return n;
```

```
}

/** 廉价分数最高的k 个走法（k<=0 表示全部），按分数降序（同分按编码升序，保证确定性）；返回{moves,
scores, total} */
function topMoves(b, k, w) { return selectTop(scoreAll(b, w), k); }

/*
 * 可分离评估排序（本文采用的近似，第2.5 节）：
 *   score(f→t, a) ≈ Q(f,t) + A(a),
 *   Q(f,t)：只移动后、不射箭时的静态评估（行棋方视角）；
 *   A(a)  ：不移动后、只在a 处放一支箭时评估值的变化量。
 * 只需约（后的走法数+ 空格数）次静态评估（中局约150 次，≈1 ms），就能给全部走法打分，
 * 比逐个评估全部走法（数百至两千次）便宜一个数量级；在随机中局局面中，
 * 真正的静态最佳走法有约90% 落在近似排序的前8 名（第5.9 节）。
 */
const Qtab = new Float64Array(NN * NN), Atab = new Float64Array(NN);
function sepScoreAll(b, weights) {
  const s = b.side, o = 3 - s, cells = b.cells, qs = b.queens[s];
  // A：只放箭（视角仍为行棋方，评估时轮到对方走，故取负）
  b.side = o;
  const e0 = -E.evaluate(b, weights);
  for (let a = 0; a < NN; a++) {
    if (cells[a] !== EMPTY) { Atab[a] = 0; continue; }
    cells[a] = C.ARROW; Atab[a] = -E.evaluate(b, weights) - e0; cells[a] = EMPTY;
  }
  b.side = s;
  let n = 0;
  for (let qi = 0; qi < 4; qi++) {
    const f = qs[qi];
    cells[f] = EMPTY;
    const rf = RAYS[f];
    for (let d = 0; d < 8; d++) {
      const ray = rf[d];
      for (let k = 0; k < ray.length; k++) {
        const t = ray[k]; if (cells[t] !== EMPTY) break;
        cells[t] = s; qs[qi] = t; b.side = o;
        const q = -E.evaluate(b, weights);
        b.side = s; qs[qi] = f; cells[t] = EMPTY;
        const base = f * 10000 + t * 100, rt = RAYS[t];
        for (let e = 0; e < 8; e++) {
          const r2 = rt[e];
          for (let j = 0; j < r2.length; j++) {
            const a = r2[j]; if (cells[a] !== EMPTY) break;
            bufM[n] = base + a; bufS[n] = q + (a === f ? 0 : Atab[a]); n++;
          }
        }
      }
    }
    cells[f] = s;
  }
  return n;
}

function selectTop(n, k) {
  if (k <= 0 || k >= n / 4) {
    const arr = new Array(n); for (let i = 0; i < n; i++) arr[i] = i;
    arr.sort((x, y) => bufS[y] - bufS[x] || bufM[x] - bufM[y]);
    const m = k > 0 ? Math.min(k, n) : n;
    const moves = new Array(m), scores = new Array(m);
    for (let i = 0; i < m; i++) { moves[i] = bufM[arr[i]]; scores[i] = bufS[arr[i]]; }
    return { moves, scores, total: n };
  }
  const tm = new Int32Array(k), ts = new Float64Array(k); let c = 0;
  for (let i = 0; i < n; i++) {
    const v = bufS[i], mv = bufM[i];
    if (c === k && (v < ts[k - 1] || (v === ts[k - 1] && mv > tm[k - 1]))) continue;
    let j = c < k ? c++ : k - 1;
    while (j > 0 && (ts[j - 1] < v || (ts[j - 1] === v && tm[j - 1] > mv))) { ts[j] = ts[j -
1]; tm[j] = tm[j - 1]; j--; }
```

```
    ts[j] = v; tm[j] = mv;
  }
  return { moves: Array.from(tm.subarray(0, c)), scores: Array.from(ts.subarray(0, c)),
total: n };
}
/** 可分离评估排序的前k 名*/
function sepTopMoves(b, k, weights) { return selectTop(sepScoreAll(b, weights), k); }

/** 学习策略使用的走法特征（与廉价分数同源，另加若干距离特征）。在prepare(b) 之后调用*/
const POLICY_FEATURES = ['mobDelta', 'mobT', 'oppAdj', 'ownAdj', 'terr', 'dOpT1', 'dOpT2',
'arrowNearOpp', 'arrowDist', 'qDist', 'opMobLoss'];
function moveFeatures(b, m, out = new Float64Array(POLICY_FEATURES.length)) {
  const s = b.side, cells = b.cells, f = C.mFrom(m), t = C.mTo(m), a = C.mArrow(m);
  const mobF = Math.min(12, E.queenMobility(cells, f));
  cells[f] = EMPTY; cells[t] = s;
  const mobT0 = Math.min(12, E.queenMobility(cells, t));
  // 箭对对方后机动性的削减量
  let loss = 0;
  for (const q of b.queens[3 - s]) {
    const before = E.queenMobility(cells, q);
    cells[a] = C.ARROW; const after = E.queenMobility(cells, q); cells[a] = EMPTY;
    loss += before - after;
  }
  cells[t] = EMPTY; cells[f] = s;
  const own = ownAdj[a] - (adj(a, f) ? 1 : 0) + (adj(a, t) ? 1 : 0);
  const cheb = (x, y) => Math.max(Math.abs(ROW[x] - ROW[y]), Math.abs(COL[x] - COL[y]));
  out[0] = (mobT0 - mobF) / 4;
  out[1] = mobT0 / 12;
  out[2] = oppAdj[a];
  out[3] = own;
  out[4] = dOp[a] <= dMe[a] && dOp[a] < INF ? 1 : 0;
  out[5] = dOp[t] === 1 ? 1 : 0;
  out[6] = dOp[t] === 2 ? 1 : 0;
  out[7] = dOp[a] === 1 ? 1 : 0;
  out[8] = cheb(a, t) / 9;
  out[9] = cheb(f, t) / 9;
  out[10] = loss / 8;
  return out;
}

module.exports = { prepare, scoreAll, topMoves, sepScoreAll, sepTopMoves, selectTop,
moveFeatures, POLICY_FEATURES, W_DEFAULT, bufM, bufS, adj };
```

engine/search.js （234 行）

```
/*
 * search.js — α-β 博弈树搜索
 *
 * 负极大值（negamax）形式的α-β 剪枝+ 迭代加深，辅以：
 *   置换表（Zobrist 哈希，存储深度、分数类型与最佳着法）；
 *   杀手着法与历史启发；
 *   主变例搜索（PVS，零窗口试探+ 失败重搜）；
 *   束宽剪枝：每个结点只搜索排序后的前width 个走法（排序方式见candidates）；
 *   根结点用更宽的候选集合（rootPre 个候选，全部按静态评估排序，搜索前rootWidth 个）。
 * 每一项都可以通过选项关闭，用于消融实验（第5.4 节）。
 */
'use strict';
const C = require('./core.js');
const E = require('./eval.js');
const M = require('./movegen.js');

const WIN = E.WIN, WIN_BOUND = WIN - 500;
const INF = 1e9;
const EXACT = 0, LOWER = 1, UPPER = 2;

class TimeUp extends Error {}
```

```
const DEFAULTS = {
  timeMs: 200, maxDepth: 30,
  useTT: true, useKillers: true, useHistory: true, usePVS: true,
  order: 'sep',                // 内部结点的排序方式：'sep' 可分离评估/ 'cheap' 廉价分数+ 静态评估
  leafOrder: 'cheap',          // 剩余深度为1 的结点：'cheap' 取廉价分数前preWidth 个全部评估/
'sep'
  width: 8, preWidth: 24,      // 内部结点束宽；廉价候选数
  rootPre: 60, rootWidth: 30,  // 根结点：候选数 → 静态评估排序 → 搜索前rootWidth 个
  weights: null,               // 评估权重（null 表示默认手工权重）
  ttBits: 18,
};

class AlphaBeta {
  constructor(opts = {}) {
    this.o = Object.assign({}, DEFAULTS, opts);
    this.w = this.o.weights || E.DEFAULT_WEIGHTS;
    const size = 1 << this.o.ttBits;
    this.ttMask = size - 1;
    if (this.o.useTT) {
      this.ttKey = new Int32Array(size); this.ttKey2 = new Int32Array(size);
      this.ttDepth = new Int8Array(size).fill(-1); this.ttFlag = new Int8Array(size);
      this.ttVal = new Float64Array(size); this.ttMove = new Int32Array(size);
    }
    this.history = new Float64Array(100 * 100 * 2);
  }

  evaluate(b) { return E.evaluate(b, this.w); }

  /**
   * 候选生成：返回按优先级排序的走法数组。
   *  order='sep'  ：内部结点用可分离评估排序取前width 个（movegen.js）；
   *  order='cheap'：内部结点先按廉价分数取preWidth 个，再逐个静态评估、取前width 个；
   *  剩余深度为1 的结点（子结点即叶结点）若leafOrder='cheap'，直接取廉价分数前preWidth 个，全部评估。
   *  根结点：取rootPre 个候选，逐个静态评估排序，搜索前rootWidth 个。
   */
  candidates(b, depth, ply, ttMove) {
    const o = this.o;
    let list;
    if (ply === 0) {
      const top = (o.order === 'sep' ? M.sepTopMoves(b, o.rootPre, this.w) : M.topMoves(b,
o.rootPre)).moves;
      const sc = new Array(top.length);
      for (let i = 0; i < top.length; i++) { b.play(top[i]); sc[i] = -this.evaluate(b);
b.undo(); }
      const idx = top.map((_, i) => i).sort((x, y) => sc[y] - sc[x] || x - y);
      list = idx.slice(0, o.rootWidth).map(i => top[i]);
      this.rootStatic = idx.map(i => ({ move: top[i], score: sc[i] }));
    } else if (depth === 1 && o.leafOrder === 'cheap') {
      list = M.topMoves(b, o.preWidth).moves;
    } else if (o.order === 'sep') {
      list = M.sepTopMoves(b, o.width, this.w).moves;
    } else {
      const top = M.topMoves(b, o.preWidth).moves;
      if (depth >= 2) {
        const sc = new Array(top.length);
        for (let i = 0; i < top.length; i++) { b.play(top[i]); sc[i] = -this.evaluate(b);
b.undo(); }
        list = top.map((_, i) => i).sort((x, y) => sc[y] - sc[x] || x - y).slice(0,
o.width).map(i => top[i]);
      } else list = top;
    }
    // 杀手着法与历史启发：稳定地把它们提前
    if (ply !== 0 && (o.useKillers || o.useHistory)) {
      const k = this.killers[ply] || [];
      const pri = new Map();
      list.forEach((m, i) => {
        let p = -i;                                          // 原顺序
        if (o.useHistory) p += Math.min(4, this.history[(C.mFrom(m) * 100 + C.mTo(m)) * 2 +
(b.side - 1)] / 1e3);
```

```
        if (o.useKillers && (m === k[0] || m === k[1])) p += 1000;
        pri.set(m, p);
      });
      list.sort((x, y) => pri.get(y) - pri.get(x));
    }
    if (ttMove > 0) {
      const j = list.indexOf(ttMove);
      if (j > 0) { list.splice(j, 1); list.unshift(ttMove); }
      else if (j < 0 && b.isLegal(ttMove)) list.unshift(ttMove);
    }
    return list;
  }

  search(b0) {
    const o = this.o;
    const b = b0.clone();
    this.t0 = Date.now(); this.deadline = this.t0 + o.timeMs;
    this.nodes = 0; this.cutoffs = 0; this.firstCutoffs = 0; this.ttHits = 0; this.interior =
0; this.killers = []; this.history.fill(0); this.lastBest = 0; this.rootStatic = null;
    if (o.useTT) this.ttDepth.fill(-1);
    const iters = [];
    let best = -1, bestScore = 0, depthDone = 0, pv = [], rootScores = [], partialUsed =
false;
    for (let d = 1; d <= o.maxDepth; d++) {
      this.rootDepth = d;
      try {
        const r = this.root(b, d);
        best = r.move; bestScore = r.score; depthDone = d; rootScores = r.scores;
        pv = this.extractPV(b, best, d);
        iters.push({ depth: d, move: C.moveToStr(best), score: +bestScore.toFixed(2), nodes:
this.nodes, ms: Date.now() - this.t0 });
        if (o.onIteration) o.onIteration(Object.assign({ pv: pv.map(C.moveToStr) },
iters[iters.length - 1]));
        if (Math.abs(bestScore) > WIN_BOUND) break;          // 已找到胜负
        const el = Date.now() - this.t0;
        if (el > o.timeMs * 0.5) break;                      // 剩余时间不足一半时不再开始新的一层
      } catch (e) {
        if (!(e instanceof TimeUp)) throw e;
        // 未完成的一层：若上一层的最佳着法已在本层搜完，则可采用本层已搜完部分中的最佳着法
        const p = this.partial;
        if (p && p.count >= 1 && p.move !== best) {
          best = p.move; bestScore = p.score; partialUsed = true;
          iters.push({ depth: d, move: C.moveToStr(best), score: +bestScore.toFixed(2),
nodes: this.nodes, ms: Date.now() - this.t0, partial: p.count });
        }
        break;
      }
    }
    if (best < 0) best = this.rootStatic && this.rootStatic.length ? this.rootStatic[0].move
: b.genMoves()[0];
    return {
      move: best, score: bestScore, depth: depthDone, nodes: this.nodes, timeMs: Date.now() -
this.t0,
      reason: 'search', pv: pv.map(C.moveToStr), iterations: iters, partialUsed,
      stats: { cutoffs: this.cutoffs, firstCutoffs: this.firstCutoffs, ttHits: this.ttHits,
interior: this.interior },
      rootMoves: rootScores.slice(0, 12).map(x => ({ move: x.move, score: x.score })),
    };
  }

  root(b, depth) {
    const o = this.o;
    const moves = this.candidates(b, depth, 0, depth > 1 ? this.lastBest : 0);
    let alpha = -INF, beta = INF, best = moves[0], bestV = -INF;
    const scores = [];
    this.partial = null;
    for (let i = 0; i < moves.length; i++) {
      const m = moves[i];
      b.play(m);
```

```
      let v;
      if (i === 0 || !o.usePVS) v = -this.negamax(b, depth - 1, -beta, -alpha, 1);
      else {
        v = -this.negamax(b, depth - 1, -alpha - 1e-6, -alpha, 1);
        if (v > alpha) v = -this.negamax(b, depth - 1, -beta, -alpha, 1);
      }
      b.undo();
      scores.push({ move: m, score: v });
      if (v > bestV) { bestV = v; best = m; }
      if (v > alpha) alpha = v;
      this.partial = { move: best, score: bestV, count: i + 1 };
    }
    this.lastBest = best;
    scores.sort((x, y) => y.score - x.score);
    return { move: best, score: bestV, scores };
  }

  negamax(b, depth, alpha, beta, ply) {
    const o = this.o;
    if ((++this.nodes & 7) === 0 && Date.now() > this.deadline) throw new TimeUp();
    if (!b.canMove(b.side)) return -WIN + ply;
    if (depth <= 0) return this.evaluate(b);
    const a0 = alpha;
    let ttMove = 0, slot = -1;
    if (o.useTT) {
      slot = b.hashLo & this.ttMask;
      if (this.ttKey[slot] === b.hashLo && this.ttKey2[slot] === b.hashHi &&
this.ttDepth[slot] >= 0) {
        ttMove = this.ttMove[slot];
        if (this.ttDepth[slot] >= depth) {
          const v = this.ttVal[slot], f = this.ttFlag[slot];
          this.ttHits++;
          if (f === EXACT) return v;
          if (f === LOWER && v >= beta) return v;
          if (f === UPPER && v <= alpha) return v;
        }
      }
    }
    const sv = this.evaluate(b);
    if (Math.abs(sv) >= E.SEP_BONUS - 60 && Math.abs(sv) < WIN_BOUND) return sv;   // 已分隔:
区域计数即为估计
    const moves = this.candidates(b, depth, ply, ttMove);
    this.interior++;
    let best = -INF, bestM = moves[0];
    for (let i = 0; i < moves.length; i++) {
      const m = moves[i];
      b.play(m);
      let v;
      if (i === 0 || !o.usePVS) v = -this.negamax(b, depth - 1, -beta, -alpha, ply + 1);
      else {
        v = -this.negamax(b, depth - 1, -alpha - 1e-6, -alpha, ply + 1);
        if (v > alpha && v < beta) v = -this.negamax(b, depth - 1, -beta, -alpha, ply + 1);
      }
      b.undo();
      if (v > best) { best = v; bestM = m; }
      if (v > alpha) alpha = v;
      if (alpha >= beta) {
        this.cutoffs++; if (i === 0) this.firstCutoffs++;
        if (o.useKillers) {
          const k = this.killers[ply] || (this.killers[ply] = [0, 0]);
          if (k[0] !== m) { k[1] = k[0]; k[0] = m; }
        }
        if (o.useHistory) this.history[(C.mFrom(m) * 100 + C.mTo(m)) * 2 + (b.side - 1)] +=
depth * depth;
        break;
      }
    }
    if (o.useTT) {
      this.ttKey[slot] = b.hashLo; this.ttKey2[slot] = b.hashHi; this.ttDepth[slot] = depth;
```

```
      this.ttVal[slot] = best; this.ttMove[slot] = bestM;
      this.ttFlag[slot] = best <= a0 ? UPPER : best >= beta ? LOWER : EXACT;
    }
    return best;
  }

  extractPV(b, first, depth) {
    const pv = [first];
    if (!this.o.useTT) return pv;
    const c = b.clone(); c.play(first);
    for (let i = 1; i < depth; i++) {
      const slot = c.hashLo & this.ttMask;
      if (!(this.ttKey[slot] === c.hashLo && this.ttKey2[slot] === c.hashHi &&
this.ttDepth[slot] >= 0)) break;
      const m = this.ttMove[slot];
      if (!c.isLegal(m)) break;
      pv.push(m); c.play(m);
    }
    return pv;
  }
}

module.exports = { AlphaBeta, DEFAULTS, WIN, WIN_BOUND, TimeUp };
```

engine/mcts.js （179 行）

```
/*
 * mcts.js — 蒙特卡洛树搜索
 *
 * 四个步骤：选择（UCT 或PUCT）、扩展、模拟、回传。可配置的部分：
 *   order    ：子结点的展开顺序。'random' 为随机打乱全部合法走法；'heuristic' 为按廉价分数
 *              取前maxCand 个；'sep' 为按可分离评估排序取前maxCand 个；'learned' 为按学习策略的概率排
序（同时提供PUCT 先验）。
 *   pw       ：渐进展开（progressive widening），结点访问n 次时最多展开⌈pwC·n^pwA⌉ 个子结点；
 *              关闭时每次访问展开一个新子结点，直到全部展开（标准UCT）。
 *   rollout  ：'random' 随机下到终局；'cutoff' 随机走cutoff 步后用评估函数估计胜率（Lorentz 2008）；
 *              'eval' 不模拟，直接评估叶结点。
 *   sepCheck：双方已被分隔时，直接用区域格数判定胜负，不再模拟。
 * 胜率估计p = 1 / (1 + exp(-k·v))，v 为评估值（行棋方视角）。
 */
'use strict';
const C = require('./core.js');
const E = require('./eval.js');
const M = require('./movegen.js');

const DEFAULTS = {
  timeMs: 200, maxSims: 0, seed: 1,
  selection: 'uct', cUct: 0.5, cPuct: 1.5,
  order: 'heuristic', maxCand: 60, tau: 4,
  pw: true, pwC: 2, pwA: 0.5,
  rollout: 'cutoff', cutoff: 4, k: 0.18,
  sepCheck: true, winCheck: true, weights: null, policyModel: null,
};

class Node {
  constructor(parent, move, prior) {
    this.parent = parent; this.move = move; this.prior = prior;
    this.children = []; this.n = 0; this.w = 0;            // w：从“走入本结点的一方”看的累计收益
    this.cand = null; this.candP = null; this.next = 0;    // 待展开的候选走法
    this.terminal = 0;                                     // 1：走入本结点的一方已获胜（对方无子可
动）
  }
}

class MCTS {
  constructor(opts = {}) {
    this.o = Object.assign({}, DEFAULTS, opts);
```

```
    this.rng = C.mulberry32(this.o.seed || 1);
    this.w = this.o.weights || E.DEFAULT_WEIGHTS;
  }

  initCandidates(node, b) {
    const o = this.o;
    if (o.order === 'random') {
      const ms = b.genMoves();
      for (let i = ms.length - 1; i > 0; i--) { const j = (this.rng() * (i + 1)) | 0; const t
= ms[i]; ms[i] = ms[j]; ms[j] = t; }
      node.cand = ms; node.candP = null;
    } else if (o.order === 'learned') {
      const r = o.policyModel.rank(b, o.maxCand);
      node.cand = r.moves; node.candP = r.probs;
    } else {
      const r = o.order === 'sep' ? M.sepTopMoves(b, o.maxCand, this.w) : M.topMoves(b,
o.maxCand);
      node.cand = r.moves;
      // 启发式先验：对排序分数做 softmax（仅在 PUCT 下使用；温度 tau）
      const mx = r.scores[0] || 0; let z = 0;
      const p = r.scores.map(s => { const e = Math.exp((s - mx) / o.tau); z += e; return
e; });
      node.candP = p.map(x => x / z);
    }
  }

  canExpand(node) {
    if (node.next >= node.cand.length) return false;
    if (!this.o.pw) return true;
    return node.children.length < Math.ceil(this.o.pwC * Math.pow(node.n + 1, this.o.pwA));
  }

  select(node) {
    const o = this.o;
    let best = null, bv = -Infinity;
    const lnN = Math.log(node.n + 1), sq = Math.sqrt(node.n + 1);
    for (const c of node.children) {
      if (c.terminal) return c;                              // 一步制胜
      const q = c.n ? c.w / c.n : 0.5;
      const u = o.selection === 'puct'
        ? q + o.cPuct * c.prior * sq / (1 + c.n)
        : q + o.cUct * Math.sqrt(lnN / (c.n + 1e-9));
      if (u > bv) { bv = u; best = c; }
    }
    return best;
  }

  // 叶结点价值：返回“行棋方”的胜率估计
  leafValue(b) {
    const o = this.o;
    if (!b.canMove(b.side)) return 0;
    if (o.rollout === 'random') return this.randomPlayout(b, 400);
    if (o.rollout === 'eval') return this.evalToProb(b);
    // cutoff：随机走 cutoff 步后评估
    const s0 = b.side; let k = 0;
    for (; k < o.cutoff; k++) {
      if (!b.canMove(b.side)) break;
      b.play(b.randomMove(this.rng));
    }
    let v;
    if (!b.canMove(b.side)) v = b.side === s0 ? 0 : 1;
    else { const p = this.evalToProb(b); v = b.side === s0 ? p : 1 - p; }
    for (let i = 0; i < k; i++) b.undo();
    return v;
  }
  evalToProb(b) {
    const v = E.evaluate(b, this.w);
    return 1 / (1 + Math.exp(-this.o.k * v));
  }
```

```
  randomPlayout(b, cap) {
    const s0 = b.side; let k = 0;
    while (k < cap && b.canMove(b.side)) { b.play(b.randomMove(this.rng)); k++; }
    const v = b.canMove(b.side) ? 0.5 : (b.side === s0 ? 0 : 1);
    for (let i = 0; i < k; i++) b.undo();
    return v;
  }

  search(b0) {
    const o = this.o;
    const b = b0.clone();
    const t0 = Date.now(), deadline = t0 + o.timeMs;
    const root = new Node(null, -1, 1);
    this.initCandidates(root, b);
    // 一步制胜检查：走完后对方无子可动
    if (o.winCheck) for (const m of b.genMoves()) { b.play(m); const win = !
b.canMove(b.side); b.undo(); if (win) return { move: m, reason: 'win', simulations: 0,
winrate: 1, timeMs: Date.now() - t0 }; }
    let sims = 0, maxDepth = 0, depthSum = 0;
    while (true) {
      if (o.maxSims ? sims >= o.maxSims : ((sims & 15) === 0 && Date.now() >= deadline))
break;
      let node = root, d = 0;
      // 1. 选择/ 2. 扩展
      while (true) {
        if (node.terminal) break;
        if (!node.cand) this.initCandidates(node, b);
        if (this.canExpand(node)) {
          const m = node.cand[node.next];
          const p = node.candP ? node.candP[node.next] : 1 / node.cand.length;
          node.next++;
          const c = new Node(node, m, p);
          node.children.push(c);
          b.play(m); d++;
          if (!b.canMove(b.side)) c.terminal = 1;
          node = c;
          break;
        }
        if (!node.children.length) break;
        node = this.select(node);
        b.play(node.move); d++;
      }
      // 3. 模拟：value 为“走入 node 的一方”的收益
      let value;
      if (node.terminal) value = 1;
      else if (o.sepCheck && E.separated(b)) {
        const v = E.evaluate(b, this.w); value = v > 0 ? 0 : 1;
      } else value = 1 - this.leafValue(b);
      // 4. 回传
      for (let x = node; x; x = x.parent) { x.n++; x.w += value; value = 1 - value; }
      for (let i = 0; i < d; i++) b.undo();
      sims++; depthSum += d; if (d > maxDepth) maxDepth = d;
      if (o.onProgress && (sims & 63) === 0) {
        const now = Date.now();
        if (!this._lastProg || now - this._lastProg >= 150) {
          this._lastProg = now;
          let bc = null; for (const c of root.children) if (!bc || c.n > bc.n) bc = c;
          o.onProgress({ simulations: sims, ms: now - t0, best: C.moveToStr(bc.move),
winrate: bc.w / Math.max(1, bc.n), maxDepth });
        }
      }
    }
    let best = null;
    for (const c of root.children) if (!best || c.n > best.n || (c.n === best.n && c.w >
best.w)) best = c;
    if (!best) { const ms = M.topMoves(b, 1).moves; return { move: ms[0], reason: 'fallback',
simulations: sims, winrate: 0.5, timeMs: Date.now() - t0 }; }
    const kids = root.children.slice().sort((x, y) => y.n - x.n);
    // 主变例：沿访问次数最多的子结点下降
```

```
    const pv = []; for (let x = best; x; ) { pv.push(C.moveToStr(x.move)); let nb = null; for
(const c of x.children) if (!nb || c.n > nb.n) nb = c; if (!nb || nb.n < 2) break; x = nb; if
(pv.length >= 8) break; }
    return {
      move: best.move, reason: 'mcts', simulations: sims, winrate: best.w / Math.max(1,
best.n),
      selDepth: maxDepth, avgDepth: sims ? depthSum / sims : 0, timeMs: Date.now() - t0, pv,
      rootChildren: root.children.length,
      rootMoves: kids.slice(0, 12).map(c => ({ move: c.move, visits: c.n, winrate: c.n ? c.w
/ c.n : 0, prior: c.prior })),
    };
  }
}

module.exports = { MCTS, DEFAULTS };
```

engine/policy.js （40 行）

```
/*
 * policy.js — 线性softmax 走子策略
 *
 * π(m | s) ∝ exp(θ · φ(s, m))，φ 为movegen.js 中定义的11 维走法特征，外加该走法的
 * 可分离评估分数（相对于候选中最高分的差值，单位：格），共12 维。
 * 由于合法走法太多，先用可分离评估取前pre 个候选，再在候选集合内计算softmax。
 * θ 由learning/train_policy.py 从α-β 自对弈棋谱中用最大似然拟合。
 */
'use strict';
const M = require('./movegen.js');

const NF = M.POLICY_FEATURES.length + 1;
/** 候选集合与特征矩阵（训练数据提取与推断共用，保证一致）*/
function candidateFeatures(b, pre) {
  const r = M.sepTopMoves(b, pre);
  M.prepare(b);
  const feats = r.moves.map((m, i) => {
    const phi = new Float64Array(NF);
    M.moveFeatures(b, m, phi);
    phi[NF - 1] = (r.scores[i] - r.scores[0]) / 4;
    return phi;
  });
  return { moves: r.moves, sepScores: r.scores, feats };
}

class LinearPolicy {
  constructor(theta, pre = 40) { this.theta = theta; this.pre = pre; }
  /** 返回概率最高的k 个走法及其（在候选集合内归一化的）概率*/
  rank(b, k) {
    const c = candidateFeatures(b, this.pre), th = this.theta;
    const cand = c.moves;
    const lg = c.feats.map(phi => { let z = 0; for (let i = 0; i < th.length; i++) z += th[i]
* phi[i]; return z; });
    const mx = Math.max(...lg); let z = 0;
    const p = lg.map(x => { const e = Math.exp(x - mx); z += e; return e / 1; });
    const idx = cand.map((_, i) => i).sort((x, y) => p[y] - p[x] || x - y).slice(0, k);
    return { moves: idx.map(i => cand[i]), probs: idx.map(i => p[i] / z) };
  }
}

module.exports = { LinearPolicy, candidateFeatures, NF, FEATURE_NAMES:
M.POLICY_FEATURES.concat(['sepScore']) };
```

engine/agents.js （132 行）

```
/*
 * agents.js — 统一的智能体接口与预设配置
```

```
 *
 * 每个智能体实现 think(board) → { move, ...诊断信息}。
 * 实验脚本、网页界面与单元测试都通过 makeAgent(name, overrides) 创建智能体，
 * 保证“游戏里运行的算法”与“报告中测量的算法”是同一份代码。
 *
 * 双方被完全分隔后（填格阶段），除随机智能体外，所有智能体都改用同一个填格策略 fillerMove，
 * 使实验比较集中在双方仍有接触的阶段（第 2.6 节）。
 */
'use strict';
const C = require('./core.js');
const E = require('./eval.js');
const M = require('./movegen.js');
const { AlphaBeta } = require('./search.js');
const { MCTS } = require('./mcts.js');
const { LinearPolicy } = require('./policy.js');

/**
 * 填格策略：在已分隔的局面中，选择走完后“己方可达空格数”最大的走法；
 * 同分时优先把箭射在周围空格最少的位置（先填死角，减少对自身区域的切割）。
 */
const kbuf = new Uint8Array(C.NN);
function reachable(b, side) {
  E.kingDist(b.cells, b.queens[side], kbuf);
  let n = 0; for (let i = 0; i < C.NN; i++) if (b.cells[i] === C.EMPTY && kbuf[i] < C.INF) n+
+;
  return n;
}
function fillerMove(b) {
  const s = b.side, ms = b.genMoves();
  let best = ms[0], bv = -Infinity;
  for (const m of ms) {
    const a = C.mArrow(m);
    b.play(m);
    let free = 0; for (const x of C.NEIGH[a]) if (b.cells[x] === C.EMPTY) free++;
    const v = reachable(b, s) * 10 - free;
    b.undo();
    if (v > bv) { bv = v; best = m; }
  }
  return best;
}

function maybeFiller(b, o, t0) {
  if (o.filler === false) return null;
  if (!E.separated(b)) return null;
  return { move: fillerMove(b), reason: 'filler', score: E.evaluate(b, o.weights ||
undefined), timeMs: Date.now() - t0 };
}

class RandomAgent {
  constructor(o = {}) { this.rng = C.mulberry32(o.seed || 1); this.o = o; }
  think(b) {
    const t0 = Date.now();
    const ms = b.genMoves();
    return { move: ms[(this.rng() * ms.length) | 0], reason: 'random', timeMs: Date.now() -
t0 };
  }
}

/** 一步贪心：枚举全部合法走法，取走完后静态评估最好的一步*/
class GreedyAgent {
  constructor(o = {}) { this.o = o; this.w = o.weights || E.DEFAULT_WEIGHTS; }
  think(b0) {
    const t0 = Date.now();
    const f = maybeFiller(b0, this.o, t0); if (f) return f;
    const b = b0.clone();
    let best = -1, bv = -Infinity;
    for (const m of b.genMoves()) { b.play(m); const v = -E.evaluate(b, this.w); b.undo(); if
(v > bv) { bv = v; best = m; } }
    return { move: best, reason: 'greedy', score: bv, timeMs: Date.now() - t0 };
```

```
  }
}

class ABAgent {
  constructor(o = {}) { this.o = o; this.engine = new AlphaBeta(o); }
  think(b) { const f = maybeFiller(b, this.o, Date.now()); return f || this.engine.search(b);
}
}
class MCTSAgent {
  constructor(o = {}) { this.o = o; this.engine = new MCTS(o); }
  think(b) { const f = maybeFiller(b, this.o, Date.now()); return f || this.engine.search(b);
}
}

// 预设：实验与界面共用
const PRESETS = {
  'random': { kind: 'random' },
  'greedy': { kind: 'greedy' },
  // α-β 完整版与消融
  'ab': { kind: 'ab' },
  'ab-noTT': { kind: 'ab', useTT: false },
  'ab-noKiller': { kind: 'ab', useKillers: false, useHistory: false },
  'ab-noPVS': { kind: 'ab', usePVS: false },
  'ab-cheapOrder': { kind: 'ab', order: 'cheap' },
  'ab-sepLeaf': { kind: 'ab', leafOrder: 'sep' },
  'ab-wide': { kind: 'ab', width: 16, preWidth: 48, rootPre: 120, rootWidth: 60 },
  'ab-narrow': { kind: 'ab', width: 4, preWidth: 12, rootPre: 30, rootWidth: 15 },
  'ab-simpleEval': { kind: 'ab', weights: [1, 0, 0, 0, 0, 1, 0, 0, 0, 0] },
  'ab-tuned': { kind: 'ab', weights: 'tuned' },
  'ab-plain': { kind: 'ab', useTT: false, useKillers: false, useHistory: false, usePVS: false
},
  // MCTS 系列
  'mcts-pure': { kind: 'mcts', selection: 'uct', order: 'random', pw: false, rollout:
'random', sepCheck: false, winCheck: false, filler: false },
  'mcts-uct': { kind: 'mcts', selection: 'uct', order: 'random', pw: false, rollout: 'cutoff'
},
  'mcts-pw': { kind: 'mcts', selection: 'uct', order: 'heuristic', pw: true, rollout:
'random' },
  'mcts-cutoff': { kind: 'mcts', selection: 'uct', order: 'heuristic', pw: true, rollout:
'cutoff' },
  'mcts-sep': { kind: 'mcts', selection: 'uct', order: 'sep', pw: true, rollout: 'cutoff',
maxCand: 30 },
  'mcts-puct': { kind: 'mcts', selection: 'puct', order: 'sep', pw: true, rollout: 'cutoff',
maxCand: 30 },
  'mcts-learned': { kind: 'mcts', selection: 'puct', order: 'learned', pw: true, rollout:
'cutoff', maxCand: 30 },
};

let POLICY_THETA = null;
let TUNED_WEIGHTS = null;
function setPolicyTheta(theta) { POLICY_THETA = theta; }
function setTunedWeights(w) { TUNED_WEIGHTS = w; }

function makeAgent(name, overrides = {}) {
  const base = PRESETS[name];
  if (!base) throw new Error('unknown agent ' + name);
  const o = Object.assign({}, base, overrides);
  if (o.weights === 'tuned') {
    if (!TUNED_WEIGHTS) throw new Error('tuned weights not loaded');
    o.weights = TUNED_WEIGHTS;
  }
  if (o.order === 'learned' && !o.policyModel) {
    if (!POLICY_THETA) throw new Error('policy weights not loaded');
    o.policyModel = new LinearPolicy(POLICY_THETA);
  }
  switch (o.kind) {
    case 'random': return new RandomAgent(o);
    case 'greedy': return new GreedyAgent(o);
    case 'ab': return new ABAgent(o);
```

```
    case 'mcts': return new MCTSAgent(o);
  }
  throw new Error('bad kind');
}

module.exports = { RandomAgent, GreedyAgent, ABAgent, MCTSAgent, PRESETS, makeAgent,
setPolicyTheta, setTunedWeights, fillerMove };
```

engine/index.js （10 行）

```
'use strict';
// 引擎入口：汇总导出（Node.js 与浏览器打包共用）
const C = require('./core.js');
const E = require('./eval.js');
const M = require('./movegen.js');
const S = require('./search.js');
const T = require('./mcts.js');
const A = require('./agents.js');
const P = require('./policy.js');
module.exports = Object.assign({}, C, { Eval: E, MoveGen: M, Search: S, MCTSMod: T, Policy: P
}, A, { evaluate: E.evaluate, territoryMap: E.territoryMap });
```

game/src/app.js （566 行）

```
/*
 * app.js — 亚马逊棋AI 实验台的界面逻辑
 * 引擎（AmazonsAI）在主线程中只用于规则判定与领地计算，搜索在Web Worker 中执行。
 * 走一步分三次点击：选择己方的后 → 选择目的地 → 选择射箭位置。
 */
/* global AmazonsAI, POLICY_THETA, TUNED_WEIGHTS, OPENINGS, AI_LIST_CFG, WIN_K */
(function () {
  'use strict';
  const G = AmazonsAI;
  const N = G.N;
  const $ = id => document.getElementById(id);
  const hasPolicy = typeof POLICY_THETA !== 'undefined' && POLICY_THETA;
  const hasTuned = typeof TUNED_WEIGHTS !== 'undefined' && TUNED_WEIGHTS;
  if (hasPolicy) G.setPolicyTheta(POLICY_THETA);
  if (hasTuned) G.setTunedWeights(TUNED_WEIGHTS);
  const K = typeof WIN_K !== 'undefined' && WIN_K ? WIN_K : 0.15;   // 评估值→胜率 的换算系数

  // ---------------------------------------------------------------- 配置
  const AI_LIST = (typeof AI_LIST_CFG !== 'undefined' ? AI_LIST_CFG : [
    { id: 'ab', name: 'α-β 搜索（最强）', short: 'α-β' },
    { id: 'mcts-puct', name: 'MCTS · PUCT + 评估截断', short: 'MCTS-PUCT' },
    { id: 'mcts-cutoff', name: 'MCTS · UCT + 评估截断', short: 'MCTS-截断' },
    { id: 'mcts-pure', name: 'MCTS · 纯随机模拟', short: 'MCTS-纯随机' },
    { id: 'greedy', name: '贪心 · 一步评估', short: '贪心' },
    { id: 'random', name: '随机走子', short: '随机' },
  ]).filter(a => (!a.needPolicy || hasPolicy) && (!a.needTuned || hasTuned));
  const THINK_STEPS = [100, 250, 500, 1000, 2000, 3000, 5000];
  const MODE_DESC = { hw: ['玩家', 'AI'], hb: ['AI', '玩家'], aa: ['AI', 'AI'], hh: ['玩家',
'玩家'] };

  // ---------------------------------------------------------------- 状态
  let board = new G.Board();
  let evalHist = [0.5];          // evalHist[k] = 第k 手后白方胜率估计（null 表示无）
  let result = null;             // 最近一次搜索结果{ key, side, agent, data }
  let live = null;               // 搜索进行中的中间结果
  let hint = null;               // 提示着法（字符串）
  let sel = { from: -1, to: -1 };   // 人类走子的中间状态
  let legal = null;              // 当前局面全部合法走法（整数）
  let worker = null, jobId = 0, busy = false, paused = false, gameOverShown = false;

  // ---------------------------------------------------------------- Worker
```

```
  function makeWorker() {
    if (worker) worker.terminate();
    const src = $('workerSrc').textContent;
    const url = URL.createObjectURL(new Blob([src], { type: 'text/javascript' }));
    worker = new Worker(url);
    worker.onmessage = onWorkerMsg;
    worker.onerror = e => { setStatus('Worker 出错: ' + e.message, false); busy = false; };
  }
  function cancelSearch() {
    if (busy) { makeWorker(); busy = false; }
    live = null; showThinking(false);
  }
  function posKey(b) { return b.history.join(','); }

  function requestSearch(purpose) {
    const side = board.side;
    const agent = side === G.WHITE ? $('aiWhite').value : $('aiBlack').value;
    const timeMs = THINK_STEPS[+$('think').value];
    const id = ++jobId;
    busy = true; live = null;
    showThinking(true, purpose === 'hint' ? '正在为你分析…' : `${aiName(agent)} 思考中…`);
    setStatus(purpose === 'hint' ? '分析中' : 'AI 思考中', true);
    worker.postMessage({ type: 'think', id, moves: board.history.map(G.moveToStr), agent,
opts: { timeMs, seed: 1 + board.history.length } });
    worker.pending = { id, purpose, key: posKey(board), side, agent, t0: performance.now() };
  }

  function onWorkerMsg(ev) {
    const msg = ev.data, p = worker.pending;
    if (!p || msg.id !== p.id) return;
    if (msg.type === 'progress') {
      live = { key: p.key, side: p.side, agent: p.agent, data: msg.data };
      renderSearchPanel(live, true);
      draw();
      return;
    }
    busy = false; showThinking(false);
    if (msg.type === 'error') { setStatus('搜索出错', false); console.error(msg.message);
return; }
    const data = msg.data;
    result = { key: p.key, side: p.side, agent: p.agent, data };
    live = null;
    renderSearchPanel(result, false);
    if (p.key !== posKey(board)) { draw(); return; }
    if (p.purpose === 'hint') { hint = data.move; setStatus('提示: ' + data.move, false);
draw(); return; }
    setStatus('准备就绪', false);
    const wr = winrateWhite(result);
    const wait = Math.max(0, 250 - (performance.now() - p.t0));   // 至少展示250 ms 的思考过程
    setTimeout(() => { if (p.key === posKey(board)) playMove(G.parseMove(data.move), wr); },
wait);
  }

  // ---------------------------------------------------------------- 规则与流程
  function isAI(side) {
    const m = $('mode').value;
    if (m === 'aa') return true;
    if (m === 'hw') return side === G.BLACK;
    if (m === 'hb') return side === G.WHITE;
    return false;
  }
  function playMove(m, wr = null) {
    if (board.isOver() || !board.isLegal(m)) return;
    board.play(m);
    hint = null; sel = { from: -1, to: -1 };
    const k = board.history.length;
    evalHist.length = k;
    evalHist[k] = wr != null ? wr : (k > 0 ? evalHist[k - 1] ?? 0.5 : 0.5);
    afterMove();
```

```
  }
  function afterMove() {
    legal = board.isOver() ? [] : board.genMoves();
    draw(); renderRecord(); renderPlayers(); drawCurve(); renderTerritory();
    $('moveNo').textContent = board.history.length;
    if (board.isOver()) { onGameOver(); return; }
    maybeAI();
  }
  function maybeAI() {
    if (board.isOver() || busy || paused) return;
    if (isAI(board.side)) {
      const delay = $('mode').value === 'aa' ? 200 : 0;
      setTimeout(() => { if (!busy && !paused && isAI(board.side) && !board.isOver())
requestSearch('move'); }, delay);
    }
  }
  function onGameOver() {
    if (gameOverShown) return;
    gameOverShown = true;
    const w = board.winner, who = w === G.WHITE ? '白方' : '黑方', loser = w === G.WHITE ? '黑
方' : '白方';
    const ctrl = isAI(w) ? aiName(w === G.WHITE ? $('aiWhite').value : $('aiBlack').value) :
'玩家';
    $('modalTitle').textContent = `${who}获胜`;
    $('modalText').textContent = `第${board.history.length} 手后，${loser}的四个后都已无路可走，$
{who}（${ctrl}）获胜。`;
    setTimeout(() => { $('modal').hidden = false; }, 350);
    setStatus(`${who}获胜`, false);
  }
  function newGame() {
    cancelSearch();
    board = new G.Board(); evalHist = [0.5]; result = null; hint = null; gameOverShown =
false; paused = false; sel = { from: -1, to: -1 };
    $('btnPause').textContent = '暂停';
    $('modal').hidden = true;
    if ($('opening').value === 'random' && typeof OPENINGS !== 'undefined') {
      const op = OPENINGS[Math.floor(Math.random() * OPENINGS.length)];
      for (const s of op) { board.play(G.parseMove(s)); evalHist.push(0.5); }
    }
    renderSearchPanel(null); afterMove();
  }
  function undo() {
    cancelSearch();
    if (!board.history.length) return;
    const m = $('mode').value;
    let n = 1;
    if ((m === 'hw' || m === 'hb') && board.history.length >= 2) n = isAI(board.side) ? 1 :
2;   // 退回到玩家行棋
    for (let i = 0; i < n && board.history.length; i++) board.undo();
    evalHist.length = board.history.length + 1;
    gameOverShown = false; result = null; hint = null; sel = { from: -1, to: -1 }; $
('modal').hidden = true;
    if (m === 'aa') { paused = true; $('btnPause').textContent = '继续'; }
    afterMove();
  }

  // ---------------------------------------------------------------- 胜率换算（白方视角）
  function winrateWhite(res) {
    if (!res) return null;
    const d = res.data;
    let w = null;
    if (d.reason === 'win') w = 1;
    else if (d.winrate != null) w = d.winrate;
    else if (d.score != null) {
      if (d.score > G.Search.WIN_BOUND) w = 1; else if (d.score < -G.Search.WIN_BOUND) w = 0;
      else w = 1 / (1 + Math.exp(-K * d.score));
    }
    if (w == null) return null;
    return res.side === G.WHITE ? w : 1 - w;
```

```
  }

  // ---------------------------------------------------------------- 绘制棋盘
  const cv = $('board'), ctx = cv.getContext('2d');
  let geo = null;
  function resize() {
    const wrap = $('boardWrap');
    const size = Math.floor(wrap.clientWidth);
    const dpr = window.devicePixelRatio || 1;
    cv.width = size * dpr; cv.height = size * dpr;
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    const off = size * 0.06, cell = (size - 2 * off) / N;
    geo = { size, cell, off };
    draw();
  }
  const X = c => geo.off + c * geo.cell;          // 格子左上角
  const Y = r => geo.off + r * geo.cell;
  const CX = i => X(i % N) + geo.cell / 2, CY = i => Y((i / N) | 0) + geo.cell / 2;

  function draw() {
    if (!geo) return;
    const { size, cell, off } = geo;
    ctx.fillStyle = '#2b3440'; ctx.fillRect(0, 0, size, size);
    const key = posKey(board);
    const terr = $('lyTerr').checked ? G.territoryMap(board) : null;
    for (let i = 0; i < N * N; i++) {
      const r = (i / N) | 0, c = i % N;
      ctx.fillStyle = (r + c) % 2 ? '#e2d3b3' : '#f1e8d4';
      ctx.fillRect(X(c), Y(r), cell, cell);
      if (terr && board.cells[i] === G.EMPTY && terr.owner[i] !== null) {
        const o = terr.owner[i];
        ctx.fillStyle = o === 1 ? 'rgba(66,133,244,.32)' : o === -1 ? 'rgba(219,68,55,.30)' :
'rgba(120,120,120,.16)';
        ctx.fillRect(X(c), Y(r), cell, cell);
      }
    }
    // 坐标
    ctx.fillStyle = '#d7dde5'; ctx.font = `${Math.max(10, cell * 0.28)}px sans-serif`;
ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    for (let i = 0; i < N; i++) {
      ctx.fillText('abcdefghij'[i], X(i) + cell / 2, size - off / 2);
      ctx.fillText('abcdefghij'[i], X(i) + cell / 2, off / 2);
      ctx.fillText(String(N - i), off / 2, Y(i) + cell / 2);
      ctx.fillText(String(N - i), size - off / 2, Y(i) + cell / 2);
    }
    // 上一手
    const lm = board.lastMove();
    if (lm >= 0) {
      for (const [sq, col] of [[G.mFrom(lm), 'rgba(255,214,0,.35)'], [G.mTo(lm),
'rgba(255,214,0,.55)']]) { ctx.fillStyle = col; ctx.fillRect(X(sq % N), Y((sq / N) | 0),
cell, cell); }
    }
    // 当前局面的搜索结果；若没有，则显示“上一手AI 着法”所依据的搜索结果
    let cur = live && live.key === key ? live : (result && result.key === key ? result :
null);
    let shift = 0;
    if (!cur && result && board.history.length && result.key === board.history.slice(0, -
1).join(',') && result.data.move === G.moveToStr(lm)) { cur = result; shift = 1; }
    if ($('lySearch').checked && cur && !shift && !board.isOver()) drawSearch(cur);
    // 箭与后
    const showNum = $('lyNum').checked;
    const arrowNo = new Map(); board.history.forEach((m, k) => arrowNo.set(G.mArrow(m), k +
1));
    for (let i = 0; i < N * N; i++) {
      const v = board.cells[i];
      if (v === G.ARROW) drawArrowCell(i, showNum ? arrowNo.get(i) : null, lm >= 0 &&
G.mArrow(lm) === i);
      else if (v === G.WHITE || v === G.BLACK) drawQueen(i, v, 1);
    }
```

```
    if (terr && $('lyDist').checked) drawDist(terr);
    if ($('lyPV').checked && cur && !board.isOver()) drawPV(cur, shift);
    if (hint) drawMoveArrow(G.parseMove(hint), '#1f4e79', 3, true);
    drawSelection();
  }

  function drawQueen(i, side, alpha) {
    const x = CX(i), y = CY(i), rad = geo.cell * 0.38;
    ctx.save(); ctx.globalAlpha = alpha;
    if (alpha === 1) { ctx.shadowColor = 'rgba(0,0,0,.35)'; ctx.shadowBlur = rad * 0.35;
ctx.shadowOffsetY = rad * 0.1; }
    const gr = ctx.createRadialGradient(x - rad * 0.35, y - rad * 0.4, rad * 0.1, x, y, rad);
    if (side === G.BLACK) { gr.addColorStop(0, '#6a6a6a'); gr.addColorStop(1, '#0b0b0b'); }
else { gr.addColorStop(0, '#ffffff'); gr.addColorStop(1, '#cfcfcf'); }
    ctx.fillStyle = gr; ctx.beginPath(); ctx.arc(x, y, rad, 0, 7); ctx.fill();
    ctx.restore();
    ctx.save(); ctx.globalAlpha = alpha;
    ctx.fillStyle = side === G.BLACK ? '#f2d16b' : '#7a5a10';
    ctx.font = `${geo.cell * 0.46}px "Segoe UI Symbol", "Noto Sans Symbols2", sans-serif`;
ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillText('♛', x, y + geo.cell * 0.02);
    ctx.restore();
  }
  function drawArrowCell(i, num, last) {
    const x = X(i % N), y = Y((i / N) | 0), c = geo.cell, p = c * 0.12;
    ctx.fillStyle = last ? '#8e3b2c' : '#5d4a36'; ctx.fillRect(x + p, y + p, c - 2 * p, c - 2
* p);
    ctx.strokeStyle = 'rgba(255,255,255,.55)'; ctx.lineWidth = 2;
    if (num != null) {
      ctx.fillStyle = '#fff'; ctx.font = `600 ${c * 0.3}px sans-serif`; ctx.textAlign =
'center'; ctx.textBaseline = 'middle'; ctx.fillText(String(num), x + c / 2, y + c / 2 + 1);
    } else {
      const q = c * 0.3;
      ctx.beginPath(); ctx.moveTo(x + q, y + q); ctx.lineTo(x + c - q, y + c - q);
ctx.moveTo(x + c - q, y + q); ctx.lineTo(x + q, y + c - q); ctx.stroke();
    }
  }
  // 用带箭头的线段画一个走法：后 from → to，再从 to 射箭到 arrow（虚线）
  function drawMoveArrow(m, color, width, dashedArrow, label) {
    const f = G.mFrom(m), t = G.mTo(m), a = G.mArrow(m);
    const line = (p, q, dash) => {
      const x1 = CX(p), y1 = CY(p), x2 = CX(q), y2 = CY(q);
      ctx.save(); ctx.strokeStyle = color; ctx.fillStyle = color; ctx.lineWidth = width;
ctx.lineCap = 'round';
      if (dash) ctx.setLineDash([width * 2, width * 1.6]);
      ctx.beginPath(); ctx.moveTo(x1, y1); ctx.lineTo(x2, y2); ctx.stroke();
ctx.setLineDash([]);
      const ang = Math.atan2(y2 - y1, x2 - x1), h = width * 3.2;
      ctx.beginPath(); ctx.moveTo(x2, y2); ctx.lineTo(x2 - h * Math.cos(ang - 0.45), y2 - h *
Math.sin(ang - 0.45)); ctx.lineTo(x2 - h * Math.cos(ang + 0.45), y2 - h * Math.sin(ang +
0.45)); ctx.closePath(); ctx.fill();
      ctx.restore();
    };
    line(f, t, false);
    line(t, a, dashedArrow);
    ctx.save(); ctx.strokeStyle = color; ctx.lineWidth = width; const s = geo.cell * 0.22;
    ctx.beginPath(); ctx.moveTo(CX(a) - s, CY(a) - s); ctx.lineTo(CX(a) + s, CY(a) + s);
ctx.moveTo(CX(a) + s, CY(a) - s); ctx.lineTo(CX(a) - s, CY(a) + s); ctx.stroke();
    ctx.restore();
    if (label != null) {
      ctx.save(); ctx.fillStyle = color; ctx.beginPath(); ctx.arc(CX(t) + geo.cell * 0.3,
CY(t) - geo.cell * 0.3, geo.cell * 0.17, 0, 7); ctx.fill();
      ctx.fillStyle = '#fff'; ctx.font = `700 ${geo.cell * 0.2}px sans-serif`; ctx.textAlign
= 'center'; ctx.textBaseline = 'middle';
      ctx.fillText(String(label), CX(t) + geo.cell * 0.3, CY(t) - geo.cell * 0.3 + 1);
ctx.restore();
    }
  }
  function drawSearch(cur) {
```

```
    const d = cur.data;
    const list = (d.rootMoves || []).slice(0, 5);
    if (!list.length && d.kind === 'ab' && d.iteration) {
drawMoveArrow(G.parseMove(d.iteration.move), 'rgba(192,57,43,.85)', 3, true); return; }
    if (!list.length && d.kind === 'mcts' && d.best) { drawMoveArrow(G.parseMove(d.best),
'rgba(192,57,43,.85)', 3, true); return; }
    for (let k = list.length - 1; k >= 0; k--) {
      const m = G.parseMove(list[k].move);
      drawMoveArrow(m, k === 0 ? 'rgba(192,57,43,.9)' : `rgba(31,78,121,${0.75 - k * 0.12})`,
k === 0 ? 3.2 : 2, true, k + 1);
    }
  }
  function drawPV(cur, shift = 0) {
    const d = cur.data;
    const pv = (d.pv || (d.iteration && d.iteration.pv) || []).slice(shift + (d.rootMoves &&
d.rootMoves.length && !shift ? 1 : 0), shift + 4);
    if (!pv.length) return;
    // 在一个副本上依次走主变例，用半透明画出后续各手（编号从 2 开始，因为第 1 手已由“搜索结果”层画出）
    const b = board.clone(); let start = shift + (d.rootMoves && d.rootMoves.length && !shift
? 1 : 0);
    if (shift === 0 && start === 1) { const first = (d.pv || [])[0]; if (!first) return;
const m0 = G.parseMove(first); if (!b.isLegal(m0)) return; b.play(m0); }
    pv.forEach((s, k) => {
      const m = G.parseMove(s); if (!b.isLegal(m)) return;
      drawMoveArrow(m, b.side === G.WHITE ? 'rgba(255,255,255,.85)' : 'rgba(20,20,20,.75)',
2, true, k + 1 + start);
      b.play(m);
    });
  }
  function drawDist(terr) {
    ctx.font = `600 ${geo.cell * 0.2}px sans-serif`; ctx.textBaseline = 'middle';
    for (let i = 0; i < N * N; i++) {
      if (board.cells[i] !== G.EMPTY) continue;
      const a = terr.dW[i], c = terr.dB[i];
      ctx.textAlign = 'left'; ctx.fillStyle = '#1a4fa0'; ctx.fillText(a >= G.INF ? '∞' :
String(a), X(i % N) + geo.cell * 0.08, Y((i / N) | 0) + geo.cell * 0.2);
      ctx.textAlign = 'right'; ctx.fillStyle = '#a3261b'; ctx.fillText(c >= G.INF ? '∞' :
String(c), X(i % N) + geo.cell * 0.92, Y((i / N) | 0) + geo.cell * 0.8);
    }
  }
  function drawSelection() {
    if (sel.from < 0) return;
    const c = geo.cell;
    ctx.strokeStyle = '#ffb300'; ctx.lineWidth = 3; ctx.strokeRect(X(sel.from % N) + 2,
Y((sel.from / N) | 0) + 2, c - 4, c - 4);
    const dots = new Set();
    for (const m of legal) {
      if (G.mFrom(m) !== sel.from) continue;
      if (sel.to < 0) dots.add(G.mTo(m)); else if (G.mTo(m) === sel.to)
dots.add(G.mArrow(m));
    }
    if (sel.to >= 0) {
      ctx.fillStyle = (Math.floor(sel.from / N) + sel.from % N) % 2 ? '#e2d3b3' : '#f1e8d4';
      ctx.fillRect(X(sel.from % N), Y((sel.from / N) | 0), c, c);
      drawQueen(sel.to, board.side, 0.85);
      ctx.strokeStyle = '#ffb300'; ctx.strokeRect(X(sel.to % N) + 2, Y((sel.to / N) | 0) + 2,
c - 4, c - 4);
    }
    for (const d of dots) {
      ctx.fillStyle = sel.to < 0 ? 'rgba(46,125,50,.75)' : 'rgba(192,57,43,.8)';
      ctx.beginPath(); ctx.arc(CX(d), CY(d), c * 0.13, 0, 7); ctx.fill();
    }
  }

  // ---------------------------------------------------------------- 面板
  function aiName(id) { const a = AI_LIST.find(x => x.id === id); return a ? a.short : id; }
  function fmtScore(v, short) {
    if (v == null) return '–';
    if (v > G.Search.WIN_BOUND) return short ? '胜' : '必胜';
```

```
    if (v < -G.Search.WIN_BOUND) return short ? '负' : '必败';
    return (v > 0 ? '+' : '') + v.toFixed(short ? 1 : 2);
  }
  function fmtNum(n) { if (n == null) return '–'; if (n >= 1e6) return (n / 1e6).toFixed(2) +
'M'; if (n >= 1e3) return (n / 1e3).toFixed(1) + 'k'; return String(n); }
  const REASONS = {
    search: '迭代加深 α-β 搜索：按搜索值选择着法。箭头为根结点得分最高的 5 个候选（虚线为射箭方向）。',
    mcts: '蒙特卡洛树搜索：选择访问次数最多的子结点。箭头为访问次数最多的 5 个候选。',
    win: '一步制胜：走完后对方四个后都无路可走。', random: '随机走子。', greedy: '一步贪心：选择走完后静
态评估最高的走法。',
    filler: '双方已完全分隔：进入填格阶段，使用共用的填格策略（尽量保留己方可走的格子）。', fallback: '时
间太短，按启发式排序走第一步。',
  };

  function renderSearchPanel(cur, isLive) {
    const thead = $('searchTable').querySelector('thead'), tbody = $
('searchTable').querySelector('tbody');
    if (!cur) {
      ['kAlgo', 'kDepth', 'kNodes', 'kNps', 'kTime', 'kEval'].forEach(k => $(k).textContent =
'–');
      $('kReason').textContent = '尚未开始搜索。'; $('kReason').className = 'reason'; $
('kPV').textContent = '–';
      thead.innerHTML = ''; tbody.innerHTML = ''; return;
    }
    const d = cur.data;
    $('kAlgo').textContent = aiName(cur.agent);
    if (d.kind === 'ab' && d.iteration) {
      const it = d.iteration;
      $('kDepth').textContent = `${it.depth} 层`; $('kNodes').textContent = fmtNum(it.nodes);
      $('kNps').textContent = fmtNum(Math.round(it.nodes / Math.max(1, it.ms) * 1000)) +
'/s';
      $('kTime').textContent = it.ms + ' ms'; $('kEval').textContent = fmtScore(it.score);
      $('kPV').textContent = (it.pv || []).join('  ');
      $('kReason').textContent = `迭代加深进行中：第 ${it.depth} 层完成，当前最佳 ${it.move}。`; $
('kReason').className = 'reason';
      appendIterRow(it);
      return;
    }
    if (d.kind === 'mcts' || d.simulations != null) {
      $('kDepth').textContent = `${fmtNum(d.simulations)} 次`;
      $('kNodes').textContent = `树深 ${d.selDepth ?? d.maxDepth ?? '–'}`;
      const ms = d.timeMs ?? d.ms;
      $('kNps').textContent = ms ? fmtNum(Math.round(d.simulations / ms * 1000)) + ' 次/s' :
'–';
      $('kTime').textContent = (ms ?? 0) + ' ms';
      $('kEval').textContent = d.winrate != null ? (d.winrate * 100).toFixed(1) + '%' : '–';
      $('kPV').textContent = (d.pv || []).join('  ') || (d.best || '–');
      $('kReason').textContent = isLive ? `模拟进行中：已完成 ${d.simulations} 次模拟，当前最佳 $
{d.best}。` : (REASONS[d.reason] || d.reason);
      $('kReason').className = 'reason' + (d.reason === 'win' ? ' win' : '');
      if (!isLive) {
        thead.innerHTML = '<tr><th>#</th><th>着法</th><th>访问 N</th><th>胜率 Q</th><th>先验
P</th><th></th></tr>';
        const rm = d.rootMoves || [], maxN = Math.max(1, ...rm.map(c => c.visits));
        tbody.innerHTML = rm.map((c, k) => `<tr><td>${k + 1}</td><td>${c.move}</td><td>$
{c.visits}</td><td>${(c.winrate * 100).toFixed(1)}%</td><td>${c.prior != null ? (c.prior *
100).toFixed(1) + '%' : '–'}</td><td><span class="bar" style="width:${Math.round(50 *
c.visits / maxN)}px"></span></td></tr>`).join('');
      }
      return;
    }
    $('kDepth').textContent = d.depth != null ? `${d.depth} 层` : '1 层';
    $('kNodes').textContent = fmtNum(d.nodes);
    $('kNps').textContent = d.nodes && d.timeMs ? fmtNum(Math.round(d.nodes / Math.max(1,
d.timeMs) * 1000)) + '/s' : '–';
    $('kTime').textContent = (d.timeMs ?? 0) + ' ms';
    $('kEval').textContent = fmtScore(d.score);
    $('kPV').textContent = (d.pv || []).join('  ') || '–';
    let reason = REASONS[d.reason] || d.reason;
```

```
    if (d.partialUsed) reason += ' 最后一层未搜完，采用了该层已搜完部分中的最佳着法。';
    $('kReason').textContent = reason;
    $('kReason').className = 'reason' + (d.score > G.Search.WIN_BOUND ? ' win' : '');
    if (d.rootMoves && d.rootMoves.length) {
      const its = (d.iterations || []).map(it => `${it.depth}层:${it.move}(${fmtScore(it.score, true)})`).join(' → ');
      thead.innerHTML = '<tr><th>#</th><th>候选着法</th><th>搜索值</th><th></th></tr>';
      const mx = Math.max(...d.rootMoves.map(r => r.score)), mn = Math.min(...d.rootMoves.map(r => r.score));
      tbody.innerHTML = d.rootMoves.map((r, k) => `<tr><td>${k + 1}</td><td>${r.move}</td><td>${fmtScore(r.score, true)}</td><td><span class="bar" style="width:${Math.round(6 + 44 * (mx > mn ? (r.score - mn) / (mx - mn) : 1))}px"></span></td></tr>`).join('') +
        (its ? `<tr><td colspan="4" class="iters">迭代过程: ${its}</td></tr>` : '');
    } else { thead.innerHTML = ''; tbody.innerHTML = ''; }
  }
  function appendIterRow(it) {
    const thead = $('searchTable').querySelector('thead'), tbody = $('searchTable').querySelector('tbody');
    if (it.depth === 1 || !thead.innerHTML.includes('深度')) { thead.innerHTML = '<tr><th>深度</th><th>最佳</th><th>评估</th><th>结点</th><th>用时</th></tr>'; tbody.innerHTML = ''; }
    tbody.insertAdjacentHTML('beforeend', `<tr><td>${it.depth}</td><td>${it.move}</td><td>${fmtScore(it.score, true)}</td><td>${fmtNum(it.nodes)}</td><td>${it.ms} ms</td></tr>`);
  }

  function renderPlayers() {
    const m = $('mode').value;
    const [wn, bn] = MODE_DESC[m];
    $('nameWhite').textContent = wn === 'AI' ? aiName($('aiWhite').value) : '玩家';
    $('nameBlack').textContent = bn === 'AI' ? aiName($('aiBlack').value) : '玩家';
    $('plWhite').classList.toggle('turn', !board.isOver() && board.side === G.WHITE);
    $('plBlack').classList.toggle('turn', !board.isOver() && board.side === G.BLACK);
    $('aiWhite').disabled = !isAI(G.WHITE) && m !== 'hh';
    $('aiBlack').disabled = !isAI(G.BLACK) && m !== 'hh';
    $('btnPause').disabled = m !== 'aa';
  }
  function renderRecord() {
    const ol = $('record');
    ol.innerHTML = board.history.map((m, i) => `<li class="${i % 2 ? 'b' : 'w'}${i === board.history.length - 1 ? ' cur' : ''}">${G.moveToStr(m)}</li>`).join('');
    ol.scrollTop = ol.scrollHeight;
  }
  function renderTerritory() {
    const t = G.territoryMap(board), f = G.Eval.texelFeatures(board);
    let w = 0, b = 0, eq = 0, none = 0;
    for (let i = 0; i < N * N; i++) { const o = t.owner[i]; if (board.cells[i] !== G.EMPTY) continue; if (o === 1) w++; else if (o === -1) b++; else if (o === 0) eq++; else none++; }
    const mob = s => board.queens[s].map(q => G.Eval.queenMobility(board.cells, q));
    const mw = mob(G.WHITE), mb = mob(G.BLACK);
    const sep = G.Eval.separated(board);
    $('terrStats').innerHTML = `
      <div class="kpis">
        <div><small>白方领地（后距离更近）</small><b style="color:#2a62c9">${w} 格</b></div>
        <div><small>黑方领地</small><b style="color:#b8372a">${b} 格</b></div>
        <div><small>均势 / 无人可达</small><b>${eq} / ${none}</b></div>
        <div><small>剩余空格</small><b>${100 - 8 - board.history.length}</b></div>
        <div><small>阶段系数 p</small><b>${f.p.toFixed(2)}</b></div>
        <div><small>局面状态</small><b>${board.isOver() ? '已结束' : sep ? '已分隔' : '争夺中'}</b></div>
      </div>
      <table class="mob"><thead><tr><th>后</th><th>位置</th><th>一步可达格数</th></tr></thead><tbody>
      ${board.queens[G.WHITE].map((q, k) => `<tr><td>白 ${k + 1}</td><td>${G.sqToStr(q)}</td><td>${mw[k]}${mw[k] === 0 ? ' （被困）' : ''}</td></tr>`).join('')}
      ${board.queens[G.BLACK].map((q, k) => `<tr><td>黑 ${k + 1}</td><td>${G.sqToStr(q)}</td><td>${mb[k]}${mb[k] === 0 ? ' （被困）' : ''}</td></tr>`).join('')}
      </tbody></table>`;
  }
```

```
  // ---------------------------------------------------------------- 胜率曲线
  function drawCurve() {
    const c = $('curve'), x = c.getContext('2d');
    const w = c.clientWidth || 360, h = 190, dpr = window.devicePixelRatio || 1;
    c.width = w * dpr; c.height = h * dpr; x.setTransform(dpr, 0, 0, dpr, 0, 0);
    x.clearRect(0, 0, w, h);
    const L = 36, R = 10, T = 10, B = 24;
    const n = Math.max(10, board.history.length);
    const px = k => L + (w - L - R) * k / n, py = v => T + (h - T - B) * (1 - v);
    x.strokeStyle = '#e5e0d5'; x.fillStyle = '#6b7280'; x.font = '11px sans-serif';
x.textAlign = 'right'; x.textBaseline = 'middle';
    for (const v of [0, 0.25, 0.5, 0.75, 1]) { x.beginPath(); x.moveTo(L, py(v)); x.lineTo(w
- R, py(v)); x.stroke(); x.fillText(Math.round(v * 100) + '%', L - 4, py(v)); }
    x.textAlign = 'center'; x.textBaseline = 'top';
    for (let k = 0; k <= n; k += Math.ceil(n / 8)) x.fillText(String(k), px(k), h - B + 6);
    x.strokeStyle = '#1f4e79'; x.lineWidth = 2; x.beginPath();
    let started = false;
    for (let k = 0; k <= board.history.length; k++) {
      const v = evalHist[k]; if (v == null) continue;
      if (!started) { x.moveTo(px(k), py(v)); started = true; } else x.lineTo(px(k), py(v));
    }
    x.stroke();
    x.fillStyle = 'rgba(66,133,244,.07)'; x.fillRect(L, T, w - L - R, (h - T - B) / 2);
  }

  // ---------------------------------------------------------------- 交互
  function setStatus(text, isBusy) { $('statusText').textContent = text; $
('status').classList.toggle('busy', !!isBusy); }
  function showThinking(on, text) { $('thinking').hidden = !on; if (text) $
('thinkingText').textContent = text; }

  function clickCell(i) {
    if (board.isOver() || isAI(board.side)) return;
    if (busy && worker.pending && worker.pending.purpose !== 'hint') return;
    const v = board.cells[i];
    if (sel.from < 0 || (sel.to < 0 && v === board.side)) {
      if (v === board.side && legal.some(m => G.mFrom(m) === i)) { sel = { from: i, to: -1 };
setStatus('选择目的地', false); }
      draw(); return;
    }
    if (sel.to < 0) {
      if (legal.some(m => G.mFrom(m) === sel.from && G.mTo(m) === i)) { sel.to = i;
setStatus('选择射箭位置', false); }
      else sel = { from: -1, to: -1 };
      draw(); return;
    }
    const m = G.mk(sel.from, sel.to, i);
    if (board.isLegal(m)) { cancelSearch(); setStatus('准备就绪', false); playMove(m); }
    else { sel = { from: -1, to: -1 }; draw(); }
  }
  cv.addEventListener('click', ev => {
    if (!geo) return;
    const rect = cv.getBoundingClientRect();
    const c = Math.floor((ev.clientX - rect.left - geo.off) / geo.cell), r =
Math.floor((ev.clientY - rect.top - geo.off) / geo.cell);
    if (r < 0 || r >= N || c < 0 || c >= N) { sel = { from: -1, to: -1 }; draw(); return; }
    clickCell(r * N + c);
  });
  cv.addEventListener('contextmenu', ev => { ev.preventDefault(); sel = { from: -1, to: -1 };
draw(); });

  function fillAISelect(s, def) { s.innerHTML = AI_LIST.map(a => `<option value="${a.id}">$
{a.name}</option>`).join(''); s.value = def; }
  fillAISelect($('aiWhite'), AI_LIST[0].id);
  fillAISelect($('aiBlack'), AI_LIST[1] ? AI_LIST[1].id : AI_LIST[0].id);
  const syncThink = () => { const v = THINK_STEPS[+$('think').value]; $
('thinkOut').textContent = (v / 1000).toFixed(v < 1000 ? 2 : 1) + ' 秒'; };
  $('think').addEventListener('input', syncThink); syncThink();
  $('mode').addEventListener('change', () => { cancelSearch(); paused = false; $
```

```
('btnPause').textContent = '暂停'; sel = { from: -1, to: -1 }; renderPlayers(); draw();
maybeAI(); });
  ['aiWhite', 'aiBlack'].forEach(id => $(id).addEventListener('change', renderPlayers));
  ['lyTerr', 'lySearch', 'lyPV', 'lyDist', 'lyNum'].forEach(id => $
(id).addEventListener('change', draw));
  $('btnNew').addEventListener('click', newGame);
  $('modalNew').addEventListener('click', newGame);
  $('modalClose').addEventListener('click', () => { $('modal').hidden = true; });
  $('btnUndo').addEventListener('click', undo);
  $('btnHint').addEventListener('click', () => { if (!board.isOver() && !busy)
requestSearch('hint'); });
  $('btnPause').addEventListener('click', () => {
    paused = !paused; $('btnPause').textContent = paused ? '继续' : '暂停';
    if (paused) cancelSearch(); else maybeAI();
  });
  document.querySelectorAll('.tab').forEach(t => t.addEventListener('click', () => {
    document.querySelectorAll('.tab').forEach(x => x.classList.toggle('active', x === t));
    document.querySelectorAll('.tabpane').forEach(p => p.classList.toggle('active', p.id ===
t.dataset.tab));
    if (t.dataset.tab === 'tCurve') drawCurve();
  }));
  $('btnExport').addEventListener('click', () => {
    const rec = { format: 'amazons-ai-lab/1', board: '10x10', date: new Date().toISOString(),
      white: $('nameWhite').textContent, black: $('nameBlack').textContent,
      result: board.isOver() ? (board.winner === G.WHITE ? 'white' : 'black') : 'unfinished',
      moves: board.history.map(G.moveToStr) };
    const a = document.createElement('a');
    a.href = URL.createObjectURL(new Blob([JSON.stringify(rec, null, 2)], { type:
'application/json' }));
    a.download = `amazons_${Date.now()}.json`; a.click();
  });
  $('btnImport').addEventListener('click', () => {
    const box = $('importBox');
    if (box.hidden) { box.hidden = false; box.focus(); return; }
    const txt = box.value.trim();
    if (!txt) { box.hidden = true; return; }
    let moves;
    try { moves = JSON.parse(txt).moves; } catch (e) { moves = txt.split(/[\s,, ]
+/).filter(Boolean); }
    try {
      cancelSearch();
      const b = G.Board.fromMoves(moves);
      board = b; evalHist = new Array(b.history.length + 1).fill(null); evalHist[0] = 0.5;
      result = null; gameOverShown = false; box.hidden = true; box.value = ''; sel = { from:
-1, to: -1 };
      $('mode').value = 'hh'; paused = false; afterMove();
      setStatus(`已导入 ${moves.length} 手`, false);
    } catch (e) { setStatus('棋谱格式有误: ' + e.message, false); }
  });
  $('btnCopy').addEventListener('click', () => {
    const s = board.history.map(G.moveToStr).join(' ');
    if (navigator.clipboard) navigator.clipboard.writeText(s).then(() => setStatus('已复制棋
谱', false), () => setStatus('复制失败', false));
  });
  document.addEventListener('keydown', e => {
    if ((e.ctrlKey || e.metaKey) && e.key === 'z') { e.preventDefault(); undo(); }
    if (e.key === 'Escape') { sel = { from: -1, to: -1 }; draw(); }
  });
  window.addEventListener('resize', () => { resize(); drawCurve(); });

  // 供自动化测试使用的只读接口
  window.__amazons = { get board() { return board; }, get busy() { return busy; }, get
result() { return result; }, get sel() { return sel; }, G, playMove: s =>
playMove(G.parseMove(s)), clickCell, newGame };

  makeWorker();
  resize();
  newGame();
})();
```

game/src/worker.js （37 行）

```
/*
 * worker.js — 在Web Worker 中运行AI 搜索，避免阻塞界面。
 * 打包时build.js 会把引擎代码拼接在本文件之前，全局对象AmazonsAI 可直接使用。
 * 消息协议：
 *   主线程 →worker: { type: 'think', id, moves: [着法字符串...], agent, opts }
 *   worker → 主线程: { type: 'progress', id, data } / { type: 'result', id, data } / { type:
'error', id, message }
 */
/* global AmazonsAI, POLICY_THETA, TUNED_WEIGHTS */
(function () {
  const G = AmazonsAI;
  if (typeof POLICY_THETA !== 'undefined' && POLICY_THETA) G.setPolicyTheta(POLICY_THETA);
  if (typeof TUNED_WEIGHTS !== 'undefined' && TUNED_WEIGHTS)
G.setTunedWeights(TUNED_WEIGHTS);

  self.onmessage = function (ev) {
    const msg = ev.data;
    if (msg.type !== 'think') return;
    const id = msg.id;
    try {
      const b = G.Board.fromMoves(msg.moves);
      const opts = Object.assign({}, msg.opts || {});
      if (msg.agent.startsWith('ab')) opts.onIteration = it => self.postMessage({ type:
'progress', id, data: { kind: 'ab', iteration: it } });
      else if (msg.agent.startsWith('mcts')) opts.onProgress = rep => self.postMessage({
type: 'progress', id, data: Object.assign({ kind: 'mcts' }, rep) });
      const agent = G.makeAgent(msg.agent, opts);
      const t0 = Date.now();
      const r = agent.think(b);
      const out = {
        move: G.moveToStr(r.move), reason: r.reason, score: r.score, depth: r.depth,
selDepth: r.selDepth, nodes: r.nodes,
        timeMs: r.timeMs, wallMs: Date.now() - t0, pv: r.pv, iterations: r.iterations,
partialUsed: r.partialUsed,
        simulations: r.simulations, winrate: r.winrate, avgDepth: r.avgDepth, rootChildren:
r.rootChildren,
        rootMoves: (r.rootMoves || []).map(x => Object.assign({}, x, { move:
G.moveToStr(x.move) })),
      };
      self.postMessage({ type: 'result', id, data: out });
    } catch (e) {
      self.postMessage({ type: 'error', id, message: String(e && e.stack || e) });
    }
  };
})();
```

game/build.js （39 行）

```
'use strict';
/*
 * 打包脚本：把引擎（CommonJS 模块）、模型参数、Worker 与界面代码内联为单个HTML 文件。
 *   node game/build.js   →   game/dist/amazons.html
 * 生成的文件双击即可在浏览器中打开（file:// 协议），无需服务器与网络。
 */
const fs = require('fs');
const path = require('path');
const root = path.join(__dirname, '..');
const rd = p => fs.readFileSync(path.join(root, p), 'utf8');

const files = ['core.js', 'eval.js', 'movegen.js', 'search.js', 'mcts.js', 'policy.js',
'agents.js', 'index.js'];
let engine = '/* AmazonsAI engine bundle — generated by game/build.js */\n(function (global)
{\n"use strict";\nconst __mods = {};\nconst __cache = {};\n' +
  'function require(name) { const k = name.replace(/^\\.\\//,
```

```
"").replace(/^\\.\\.\\/engine\\//, ""); if (__cache[k]) return __cache[k].exports; const m =
{ exports: {} }; __cache[k] = m; __mods[k](m, m.exports, require); return m.exports; }\n';
for (const f of files) engine += `__mods[${JSON.stringify(f)}] = function (module, exports,
require) {\n${rd('engine/' + f)}\n};\n`;
engine += 'global.AmazonsAI = require("index.js");\n})(typeof self !== "undefined" ? self :
this);\n';

const readJSON = p => fs.existsSync(path.join(root, p)) ? JSON.parse(rd(p)) : null;
const pw = readJSON('models/policy_weights.json'), tw =
readJSON('models/eval_weights_tuned.json'), ui = readJSON('models/ui_config.json');
let models = '';
models += `var POLICY_THETA = ${pw ? JSON.stringify(pw.theta) : 'null'};\n`;
models += `var TUNED_WEIGHTS = ${tw ? JSON.stringify(tw.weights) : 'null'};\n`;
models += `var WIN_K = ${tw && (tw.K_hand || tw.K) ? (tw.K_hand || tw.K) : "null"};\n`;
models += `var AI_LIST_CFG = ${ui ? JSON.stringify(ui.ai_list) : 'undefined'};\n`;
const op = JSON.parse(rd('data/openings.json')).openings.map(o => o.moves);
models += `var OPENINGS = ${JSON.stringify(op)};\n`;

const worker = engine + '\n' + models + '\n' + rd('game/src/worker.js');
let html = rd('game/src/index.html');
const put = (tag, s) => { if (!html.includes(tag)) throw new Error(tag); html =
html.split(tag).join(s.replace(/<\/script/gi, '<\\/script')); };
put('/*STYLE*/', rd('game/src/style.css'));
put('/*ENGINE*/', engine);
put('/*MODELS*/', models);
put('/*WORKER*/', worker);
put('/*APP*/', rd('game/src/app.js'));
fs.mkdirSync(path.join(__dirname, 'dist'), { recursive: true });
fs.writeFileSync(path.join(__dirname, 'dist/amazons.html'), html);
fs.writeFileSync(path.join(__dirname, 'dist/amazons-engine.js'), engine);
console.log('built game/dist/amazons.html', (html.length / 1024).toFixed(1) + ' KB');
```

experiments/arena.js （28 行）

```
'use strict';
// 对局器：给定开局与双方智能体，完整下完一局并记录每步诊断信息（亚马逊棋白方先走）
const G = require('../engine');
function playGame(whiteAgent, blackAgent, opening = [], opts = {}) {
  const b = G.Board.fromMoves(opening);
  const log = [];
  const maxMoves = opts.maxMoves || 100;            // 10×10 棋盘最多92 手，必然在此之前结束
  while (!b.isOver() && b.history.length < maxMoves) {
    const ag = b.side === G.WHITE ? whiteAgent : blackAgent;
    const t0 = process.hrtime.bigint();
    const r = ag.think(b);
    const ms = Number(process.hrtime.bigint() - t0) / 1e6;
    if (!b.isLegal(r.move)) throw new Error('agent returned illegal move ' + r.move);
    log.push({ side: b.side, move: r.move, ms, reason: r.reason, score: r.score, depth:
r.depth, nodes: r.nodes, sims: r.simulations, winrate: r.winrate, selDepth: r.selDepth });
    b.play(r.move);
  }
  return { winner: b.winner, moves: b.history.slice(), opening: opening.slice(), log, length:
b.history.length };
}
module.exports = { playGame };
if (require.main === module) {
  const [a, bb, t] = process.argv.slice(2);
  const A = G.makeAgent(a, { timeMs: +t || 200 }), B = G.makeAgent(bb, { timeMs: +t || 200,
seed: 2 });
  const t0 = Date.now();
  const g = playGame(A, B, []);
  const bd = G.Board.fromMoves(g.moves); console.log(bd.toString());
  console.log('winner', g.winner === 1 ? 'white ' + a : 'black ' + bb, 'len', g.length,
'sec', (Date.now() - t0) / 1000);
  console.log(g.log.map(x => `${G.moveToStr(x.move)}:${x.reason[0]}:${x.depth ?? x.sims}:$
{x.score != null ? (+x.score).toFixed(1) : (+x.winrate).toFixed(2)}`).join(' '));
}
```

experiments/runner.js （59 行）

```
'use strict';
/*
 * 并行对局调度器
 *   node experiments/runner.js jobs.json out.jsonl [workers]
 * jobs.json: [{ id, tag, white: {agent, opts}, black: {agent, opts}, opening: [...着法字符串],
 seed }]（白方先走）
 * 每局结果写一行JSON；已完成的id 会被跳过（支持断点续跑）。
 */
const fs = require('fs');
const path = require('path');
const { fork } = require('child_process');

if (process.argv[2] === '--worker') {
  const G = require('../engine');
  const { playGame } = require('./arena.js');
  const root = path.join(__dirname, '..');
  const pw = path.join(root, 'models', 'policy_weights.json');
  if (fs.existsSync(pw)) G.setPolicyTheta(JSON.parse(fs.readFileSync(pw)).theta);
  const tw = path.join(root, 'models', 'eval_weights_tuned.json');
  if (fs.existsSync(tw)) G.setTunedWeights(JSON.parse(fs.readFileSync(tw)).weights);
  process.on('message', job => {
    const mk = s => G.makeAgent(s.agent, Object.assign({ seed: (job.seed || 1) * 7919 + (s
=== job.white ? 1 : 2) }, s.opts || {}));
    const t0 = Date.now();
    const g = playGame(mk(job.white), mk(job.black), job.opening.map(G.parseMove));
    const out = { id: job.id, tag: job.tag, white: job.white, black: job.black, opening:
job.opening, winner: g.winner, length: g.length,
      moves: g.moves.map(G.moveToStr), wallMs: Date.now() - t0 };
    out.log = g.log.map(x => ({ s: x.side, m: G.moveToStr(x.move), ms: +x.ms.toFixed(1), r:
x.reason, sc: x.score, d: x.depth, n: x.nodes, sims: x.sims, wr: x.winrate != null ? +
(+x.winrate).toFixed(4) : undefined, sd: x.selDepth }));
    process.send(out);
  });
  process.send({ ready: true });
  return;
}

const [jobsFile, outFile, nw] = process.argv.slice(2);
const jobs = JSON.parse(fs.readFileSync(jobsFile));
const done = new Set();
if (fs.existsSync(outFile)) for (const line of fs.readFileSync(outFile, 'utf8').split('\n'))
if (line.trim()) done.add(JSON.parse(line).id);
const queue = jobs.filter(j => !done.has(j.id));
const W = +(nw || 3);
const TOTAL = queue.length;
console.log(`${jobs.length} jobs, ${done.size} done, ${queue.length} to run, ${W} workers`);
const out = fs.openSync(outFile, 'a');
let finished = 0, active = 0;
const t0 = Date.now();
for (let i = 0; i < Math.min(W, queue.length); i++) {
  const w = fork(__filename, ['--worker']);
  active++;
  w.on('message', msg => {
    if (!msg.ready) {
      fs.writeSync(out, JSON.stringify(msg) + '\n');
      finished++;
      if (finished % 20 === 0 || finished === TOTAL) {
        const el = (Date.now() - t0) / 1000;
        console.log(`${finished}/${TOTAL}  ${el.toFixed(0)}s  eta ${(el / finished * (TOTAL -
finished)).toFixed(0)}s`);
      }
    }
    const j = queue.shift();
    if (j) w.send(j); else { w.kill(); if (--active === 0) fs.closeSync(out); }
  });
}
```

experiments/make_jobs.js （81 行）

```
'use strict';
// 生成各实验的对局任务清单（data/jobs/*.json）
const G = require('../engine');
const fs = require('fs');
const path = require('path');
const OP = JSON.parse(fs.readFileSync(path.join(__dirname,
'../data/openings.json'))).openings.map(o => o.moves);
const dir = path.join(__dirname, '../data/jobs'); fs.mkdirSync(dir, { recursive: true });
const save = (name, jobs) => { fs.writeFileSync(path.join(dir, name + '.json'),
JSON.stringify(jobs)); console.log(name, jobs.length); };

// 配对对局：每个开局两局，交换先后手（A 执白一局、执黑一局）
function pairJobs(tag, A, B, openings, seed0 = 1) {
  const jobs = [];
  openings.forEach((op, i) => {
    jobs.push({ id: `${tag}|${i}|AB`, tag, white: A, black: B, opening: op, seed: seed0 + i *
2 });
    jobs.push({ id: `${tag}|${i}|BA`, tag, white: B, black: A, opening: op, seed: seed0 + i *
2 + 1 });
  });
  return jobs;
}
module.exports = { pairJobs, OP };

if (require.main === module && process.argv[2] === 'selfplay') {
  // 学习数据：随机2–4 手开局（每手从静态评估前40 名中随机选取），双方为200 ms 的α-β
  const rng = G.mulberry32(777);
  const jobs = [];
  for (let i = 0; i < 400; i++) {
    const b = new G.Board();
    const k = 2 + ((rng() * 3) | 0);
    for (let j = 0; j < k; j++) {
      const ms = G.MoveGen.sepTopMoves(b, 40).moves;
      b.play(ms[(rng() * ms.length) | 0]);
    }
    const A = { agent: 'ab', opts: { timeMs: 200 } };
    jobs.push({ id: `sp|${i}`, tag: 'selfplay', white: A, black: A, opening:
b.history.map(G.moveToStr), seed: i });
  }
  save('selfplay', jobs);
}

if (require.main === module && process.argv[2] === 'main') {
  const T = 200;                          // 标准思考时间（毫秒/步）
  const op16 = OP.slice(0, 16);           // 16 个均衡开局× 交换先后= 每组32 局（开局16–31 只用于参数
试调，不用于正式实验）
  const A = (agent, opts = {}) => ({ agent, opts: Object.assign({ timeMs: T }, opts) });
  let all = [];
  // ---------------- 实验T：循环赛（8 个智能体，两两32 局）
  const pool = ['random', 'greedy', 'ab', 'ab-tuned', 'mcts-pure', 'mcts-cutoff', 'mcts-
puct', 'mcts-learned'];
  for (let i = 0; i < pool.length; i++) for (let j = i + 1; j < pool.length; j++)
    all.push(...pairJobs(`T:${pool[i]}~${pool[j]}`, A(pool[i]), A(pool[j]), op16, 1000 + i *
100 + j));
  // ---------------- 实验A：α-β 消融（均与完整α-β 对弈）
  const abVar = {
    'noTT': A('ab-noTT'), 'noKiller': A('ab-noKiller'), 'noPVS': A('ab-noPVS'), 'plain':
A('ab-plain'),
    'cheapOrder': A('ab-cheapOrder'), 'sepLeaf': A('ab-sepLeaf'), 'wide': A('ab-wide'),
'narrow': A('ab-narrow'),
    'simpleEval': A('ab-simpleEval'), 'noMob': A('ab', { weights: [0.2, 0.2, 0.3, 0.3, 0, 1,
0.2, 0.05, 0.05, 0] }),
    'noPhase': A('ab', { weights: [0.6, 0.2, 0.175, 0.175, 0.15, 0.6, 0.2, 0.175, 0.175,
0.15] }),
  };
  for (const [k, v] of Object.entries(abVar)) all.push(...pairJobs(`A:${k}`, v, A('ab'),
```

```
op16, 3000));
  // ---------------- 实验M：MCTS 消融（均与mcts-puct 对弈）
  const mv = {
    'uctSel': A('mcts-sep'), 'cheapOrder': A('mcts-puct', { order: 'heuristic', maxCand:
60 }), 'randomOrder': A('mcts-puct', { order: 'random', pw: false, selection: 'uct' }),
    'noPW': A('mcts-puct', { pw: false }), 'cand10': A('mcts-puct', { maxCand: 10 }),
'cand60': A('mcts-puct', { maxCand: 60 }),
    'rollEval': A('mcts-puct', { rollout: 'eval' }), 'cut2': A('mcts-puct', { cutoff: 2 }),
'cut8': A('mcts-puct', { cutoff: 8 }), 'rollRandom': A('mcts-puct', { rollout: 'random' }),
    'c0.5': A('mcts-puct', { cPuct: 0.5 }), 'c4': A('mcts-puct', { cPuct: 4 }), 'tau1':
A('mcts-puct', { tau: 1 }), 'tau16': A('mcts-puct', { tau: 16 }), 'learned': A('mcts-
learned'),
  };
  for (const [k, v] of Object.entries(mv)) all.push(...pairJobs(`M:${k}`, v, A('mcts-puct'),
op16, 5000));
  // ---------------- 实验S：算力缩放（与同一算法200 ms 版本对弈）
  for (const t of [50, 100, 400, 800]) {
    all.push(...pairJobs(`S:ab@${t}`, A('ab', { timeMs: t }), A('ab'), op16, 7000));
    all.push(...pairJobs(`S:mcts@${t}`, A('mcts-puct', { timeMs: t }), A('mcts-puct'), op16,
7000));
  }
  // ---------------- 实验X：相同预算下α-β 对MCTS
  for (const t of [50, 100, 200, 400, 800]) all.push(...pairJobs(`X:ab~mcts@${t}`, A('ab', {
timeMs: t }), A('mcts-puct', { timeMs: t }), op16, 7500));
  save('main', all);
}

if (require.main === module && process.argv[2] === 'replicate') {
  // 重复实验：关键对比在同样16 个开局上用3 组新的随机种子各再下32 局（α-β 是确定性的，但受计时影响；MCTS
受种子影响）
  const A = (agent, opts = {}) => ({ agent, opts: Object.assign({ timeMs: 200 }, opts) });
  const op16 = OP.slice(0, 16);
  const pairs = [['ab', 'mcts-puct'], ['mcts-learned', 'mcts-puct'], ['ab-tuned', 'ab'],
['mcts-puct', 'mcts-cutoff']];
  let all = [];
  for (let r = 1; r <= 3; r++) for (const [a, b] of pairs) all.push(...pairJobs(`R${r}:${a}~$
{b}`, A(a), A(b), op16, 20000 + r * 1000));
  save('replicate', all);
}
```